Informatica® Cloud Data Integration for PowerCenter

October 2026

Developer Tool Guide

Informatica Cloud Data Integration for PowerCenter Developer Tool Guide
October 2026
April 2026

© Copyright Informatica LLC 2024, 2026

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices and AI assistant acknowledgements are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2026-10-14

Table of Contents

Preface 12

Chapter 1: Informatica Developer 13

Informatica Developer Overview 13
Start Informatica Developer 13
- Starting the Developer Tool on a Local Machine 13
- Starting the Developer Tool on a Remote Machine 14

Informatica Developer Interface 14
- Informatica Developer Views 15

Setting Up Informatica Developer 17
- Step 1. Add a Domain 17
- Step 2. Add a Model Repository 17
- Step 3. Select a Default Data Integration Service 18

Domains 18
Projects 18
- Creating a Project 19
- Filter Projects 19

Project Permissions 20
- Permissions for External Objects 20
- Permissions for Dependent Object Instances 21
- Parent Object Access 21
- Assigning Permissions 22

Folders 22
- Creating a Folder 23

Copy Object Operations 23
- Copying an Object 24
- Saving a Copy of an Object 24

Tags 24
- Creating a Tag 25
- Assigning a Tag 25
- Viewing Tags 25

Chapter 2: The Model Repository 26

Model Repository Overview 26
Objects in Informatica Developer 27
- Object Properties 29

Repository Object Locks 29
- Lock Management 29
- Rules and Guidelines for Lock Management 30

Team-based Development with Versioned Objects 31

Versioned Object Management. . . . 31
Historical Versions of Objects. . . . 33
Checked Out Objects View. . . . 34
Version History View. . . . 34
Troubleshooting Team-based Development. . . . 35
Connecting to a Model Repository. . . . 35
Model Repository Service Refresh. . . . 35

Chapter 3: Searches in Informatica Developer. . . . 36

Searches in Informatica Developer Overview. . . . 36
Domain Search. . . . 36
Searching for Objects and Properties. . . . 37
Editor Search. . . . 38

Chapter 4: Connections. . . . 39

Connections Overview. . . . 39
Connection Explorer View. . . . 41
Connection Management. . . . 41
Creating a Connection 41
Editing a Connection. . . . 42
Copying a Connection. . . . 43
Deleting a Connection. . . . 43
Refreshing the Connections List. . . . 43
Connection Switching. . . . 44
Before You Switch a Connection. . . . 44
Switching a Connection. . . . 45
After You Switch a Connection. . . . 46
Third-Party JDBC Drivers. . . . 47
Environment SQL. . . . 47
Connection Environment SQL. . . . 48
Transaction Environment SQL. . . . 48
Guidelines for Configuring Environment SQL. . . . 48

Chapter 5: Physical Data Objects. . . . 49

Physical Data Objects Overview. . . . 49
Physical Data Object Types. . . . 50
Relational Data Objects. . . . 50
Importing a Relational Data Object. . . . 52
Key Relationships. . . . 52
Customized Data Objects. . . . 54
Key Relationships. . . . 55
Customized Data Object Write Properties. . . . 56
Creating a Customized Data Object. . . . 57

Adding Relational Resources to a Customized Data Object. 57
Adding Relational Data Objects to a Customized Data Object. 58
Creating Keys in a Customized Data Object. 58
Creating Relationships within a Customized Data Object. 59
Create or Replace Target Tables. 59
Rules and Guidelines to Create or Replace Target Tables. 60
Generating and Executing DDL at Design-Time. 60
Generating and Executing DDL at Run Time 62
DDL Generation Errors. 62
Custom Queries. 62
Custom Query Optimization. 63
Default Query. 63
Hints. 64
Select Distinct. 66
Filters. 66
Sorted Ports. 67
User-Defined Joins. 68
Outer Join Support. 69
Informatica Join Syntax. 69
PreSQL and PostSQL Commands. 73
Creating a Custom Query. 74
Nonrelational Data Objects. 75
Importing Nonrelational Data Objects. 75
Creating a Read, Write, or Lookup Transformation from Nonrelational Data Operations. 76
WSDL Data Object. 76
WSDL Data Object Overview View. 77
WSDL Data Object Advanced View. 77
Importing a WSDL Data Object. 77
WSDL Synchronization. 78
Certificate Management. 79
Synchronization. 80
Synchronizing a Flat File Data Object in Informatica Developer. 80
Synchronizing a Relational Data Object in Informatica Developer. 82
Troubleshooting Physical Data Objects. 83

Chapter 6: Flat File Data Objects. 84

Flat File Data Objects Overview. 84
Generate the Source File Name. 85
Flat File Data Object Overview Properties. 85
Flat File Data Object Advanced Properties. 86
Format Properties. 87
Column Format: Delimited Properties. 88
Column Format: Fixed-width Properties. 89

Run-time: Read Properties. . . . 90
Run-time: Write Properties. . . . 92
Control File. . . . 93
Update Columns at Run Time. . . . 94
Generate Run-time Column Names Automatically. . . . 94
Generate Run-time Column Names From Data File Header. . . . 95
Generate Column Metadata from Control Files. . . . 95
Control File Formats. . . . 96
Parameterization of Run-time Properties. . . . 96
Run-time Processing of Control Files. . . . 97
Rules and Guidelines for Control Files. . . . 97
Copying from Excel to a Flat File Data Object. . . . 98
Editing Flat File Data Objects in Excel. . . . 98
Copying Metadata to a Flat File Data Object. . . . 98
Example, Editing Data Object in Excel. . . . 98
Create a Flat File Data Object. . . . 99
Creating an Empty Flat File Data Object. . . . 99
Creating a Flat File Data Object from an Existing Flat File. . . . 100
Creating a Flat File Data Object from a Control File. . . . 102

Chapter 7: Logical View of Data. . . . 103

Logical View of Data Overview. . . . 103
Logical Data Object Model Example. . . . 104
Developing a Logical View of Data. . . . 104
Logical Data Object Models. . . . 105
Creating a Logical Data Object Model. . . . 105
Importing a Logical Data Object Model from a Modeling Tool. . . . 106
Logical Data Object Model Properties. . . . 106
CA ERwin Data Modeler Import Properties. . . . 107
IBM Cognos Business Intelligence Reporting - Framework Manager Import Properties. . . . 108
SAP BusinessObjects Designer Import Properties. . . . 109
SAP PowerDesigner CDM Import Properties. . . . 110
SAP PowerDesigner PDM Import Properties. . . . 111
XSD Import Properties. . . . 112
Logical Data Objects. . . . 113
Logical Data Object Properties. . . . 113
Attribute Relationships. . . . 114
Creating a Logical Data Object. . . . 114
Logical Data Object Mappings. . . . 117
Logical Data Object Read Mappings. . . . 118
Logical Data Object Write Mappings. . . . 118
Creating a Logical Data Object Mapping. . . . 118

Chapter 8: Viewing Data. 120

Viewing Data Overview. 120
Configurations. 121
Configuration Properties. 121
Data Viewer Configurations. 124
Mapping Configurations. 126
Web Service Configurations. 127
Updating the Default Configuration Properties. 127
Troubleshooting Configurations. 128
Exporting Data. 128
Object Dependencies. 129
View Object Dependencies. 129
Viewing Object Dependencies. 129
Filtering Object Dependencies. 130
Logs. 130
Log File Format. 131
Validation Preferences. 131
Grouping Error Messages. 131
Limiting Error Messages. 131
Monitoring Jobs from the Developer Tool. 132

Chapter 9: Application Deployment. 133

Application Deployment Overview. 133
Application Creation. 134
Full Application. 134
Incremental Application. 135
Application Properties. 135
Deployment to an Application. 137
Deployment to an Archive File. 139
Deployment with Resource Parameters. 139
Viewing Run-time Objects. 141
Application Redeployment. 141
Application State Information. 142
How to Create, Deploy, and Update an Application. 143
Creating an Application. 144
Deploying an Application. 146
Deploying an Object. 147
Importing Application Archives. 148
Connecting to a Run-time Application. 148
Editing an Application. 149
Redeploying an Application to a Data Integration Service. 149

Chapter 10: Application Patch Deployment 151

Application Patch Deployment Overview 151
Application Object Instances 152
Application Object Dependencies 152
Incremental Deployment Wizard 155
Select Applications 156
Source Object Selection 157
Patch Deployment Strategy 159
Choose the Application Patch Type 159
Inheriting Only Direct Dependencies 161
Inheriting Direct and Indirect Dependencies 162
Inheriting Direct, Indirect, and Remote Dependencies 163
Determine Application Object Impact 165
Preview Object Impact 166
Identify Affected Objects 166
Identify the Scope of Object Impact 169
Specify a Patch Deployment Strategy 169
Patch Deployment Strategy Properties 170
Deploying an Application Patch 171
Rules and Guidelines for Application Patch Deployment 173

Chapter 11: Application Patch Deployment Example 174

Application Patch Deployment Example Overview 174
Patch Type: Direct 176
Patch Type: Direct and Indirect 178
Patch Type: Direct, Indirect, and Remote 180
Conclusion 182

Chapter 12: Continuous Integration and Continuous Delivery (CI/CD) 183

CI/CD Overview 183
REST API and infacmd Quick Reference 184
Using the REST API or infacmd 186
Using the REST API to Deploy and Redeploy an Application 186
Continuous Integration 187
Deploy Objects 187
Assign Objects 187
Resolve Object Conflicts 188
Continuous Delivery 188
Sample Automated Integration Process 189
Step 1. Prepare Mappings for Deployment 189
Step 2. Deploy Mappings to a Data Integration Service 190
Step 3. Run Sanity Tests 190

Step 4. Prepare Mappings for Editing. . . . 191
Sample Automated Delivery Process. . . . 191

Chapter 13: Object Import and Export. . . . 193

Object Import and Export Overview. . . . 193
Import and Export Objects. . . . 194
Object Export. . . . 195
Exporting Objects. . . . 195
Object Import. . . . 196
Importing Projects. . . . 196
Importing Objects. . . . 197
Importing Objects from a Previous Informatica Release. . . . 199

Appendix A: Data Type Reference. . . . 200

Data Type Reference Overview. . . . 200
Transformation Data Types. . . . 201
Integer Data Types. . . . 203
Binary Data Type. . . . 205
Date/Time Data Type. . . . 205
Decimal and Double Data Types. . . . 207
String Data Types. . . . 209
Flat File and Transformation Data Types. . . . 210
DB2 for LUW and Transformation Data Types. . . . 211
DB2 for i5/OS, DB2 for z/OS, and Transformation Datatypes. . . . 213
Unsupported DB2 for i5/OS and DB2 for z/OS Datatypes. . . . 214
JDBC and Transformation Datatypes. . . . 214
Microsoft SQL Server and Transformation Data Types. . . . 215
Uniqueidentifier Data Type. . . . 217
Nonrelational and Transformation Datatypes. . . . 218
ODBC and Transformation Data Types. . . . 220
Oracle and Transformation Data Types. . . . 221
Number(P,S) Data Type. . . . 223
Char, Varchar, Clob Data Types . . . 223
Unsupported Oracle Data Types. . . . 224
SAP HANA and Transformation Datatypes. . . . 224
XML and Transformation Datatypes. . . . 226
Converting Data. . . . 228
Port-to-Port Data Conversion. . . . 228

Appendix B: Keyboard Shortcuts. . . . 230

Keyboard Shortcuts for Advanced Mapping Options. . . . 230
Keyboard Shortcuts for Objects. . . . 231
Keyboard Shortcuts for Ports. . . . 232

Keyboard Shortcuts for the Transformation Palette. 233
Keyboard Shortcuts for the Workbench. 233

Appendix C: Connection Properties. 235

Connection Properties Overview. 236
Amazon Redshift Connection Properties. 236
Amazon S3 Connection Properties. 239
Greenplum Connection Properties. 242
Google BigQuery Connection Properties. 243
    Connection Modes. 244
Google Cloud Storage Connection Properties. 245
IBM DB2 Connection Properties. 246
IBM DB2 for i5/OS Connection Properties. 249
IBM DB2 for z/OS Connection Properties. 252
JDBC Connection Properties. 255
JDBC V2 Connection Properties. 257
JD Edwards EnterpriseOne Connection Properties. 259
LDAP Connection Properties. 260
Microsoft Azure Blob Storage Connection Properties. 261
Microsoft Azure Cosmos DB SQL API Connection Properties. 263
Microsoft Azure Data Lake Storage Gen2 Connection Properties. 263
MS SQL Server Connection Properties. 265
Netezza Connection Properties. 269
ODBC Connection Properties. 270
Oracle Connection Properties. 271
Salesforce Connection Properties. 274
Salesforce Marketing Cloud Connection Properties. 275
SAP Connection Properties. 276
Snowflake Connection Properties. 279
Teradata Parallel Transporter Connection Properties. 280
Tableau Connection Properties. 283
Identifier Properties in Database Connections. 283
    Regular Identifiers. 284
    Delimited Identifiers. 284
    Identifier Properties. 284
PowerExchange for PostgreSQL Connection Properties. 286
Microsoft Dynamics 365 for Sales Connection Properties. 287
PowerExchange for Oracle E-Business Suite Connection Properties. 289
Siebel Application Connections for Sources, Targets, and EIM Invoker Transformations. 289
Microsoft Dynamics CRM Connection. 290
PowerExchange for Essbase Connections. 291
Vertica Relational Connection Properties. 292
PowerExchange for Db2 Warehouse Connections. 293

PowerExchange for HANA Connections. 293

Index. 295

Preface

Use the *Informatica® Developer Tool Guide* to learn about the user interface in the Developer tool. Learn how to connect to a Model repository, create connections and data objects, and deploy applications.

Note: The Data Integration Service, the Model Repository Service, and the monitoring Model Repository Service apply only to Data Engineering Integration modernization and Informatica Data Quality modernization in the native environment.

CHAPTER 1

Informatica Developer

This chapter includes the following topics:

- Informatica Developer Overview, 13
- Start Informatica Developer, 13
- Informatica Developer Interface, 14
- Setting Up Informatica Developer, 17
- Domains, 18
- Projects, 18
- Project Permissions, 20
- Folders, 22
- Copy Object Operations, 23
- Tags, 24

Informatica Developer Overview

The Developer tool is an application that you use to design and implement data integrationand data services solutions.

You can use the Developer tool to import metadata, create connections, and create data objects. You can also use the Developer tool to create and run mappings and workflows.

Start Informatica Developer

If the Developer tool is installed on a local machine, use the Windows Start menu to start the tool. If the Developer tool is installed on a remote machine, use the command line to start the tool.

Starting the Developer Tool on a Local Machine

Use the Windows Start menu to start the Developer tool installed on a local machine.

1. From the Windows Start menu, click **All Programs Informatica CDI-PC Informatica Developer**.

2. Click **Workbench**.

   The first time you start the Developer tool, you must add a domain and a Model repository. If the domain includes more than one Data Integration Service, you must also select a default service.

Starting the Developer Tool on a Remote Machine

Use the command line to start the Developer tool installed on a remote machine.

When the Developer tool is installed on a remote machine, you might not have write access to the installation directory. You must specify a workspace directory on your local machine where the Developer tool can write temporary files. An administrator can configure the default local workspace directory for all users. You can override the default directory when you start the Developer tool.

If the configured local workspace directory does not exist, the Developer tool creates the directory when it writes temporary files.

1. Open a command prompt.
2. Enter the command to start the Developer tool. You can use the default local workspace directory or override the default directory.
   - To use the default local workspace directory, enter the following command:

     ```
     \\<remote installation directory>\developer.exe
     ```
   - To override the default local workspace directory, enter the following command:

     ```
     \\<remote installation directory>\developer.exe -data <local workspace directory>
     ```

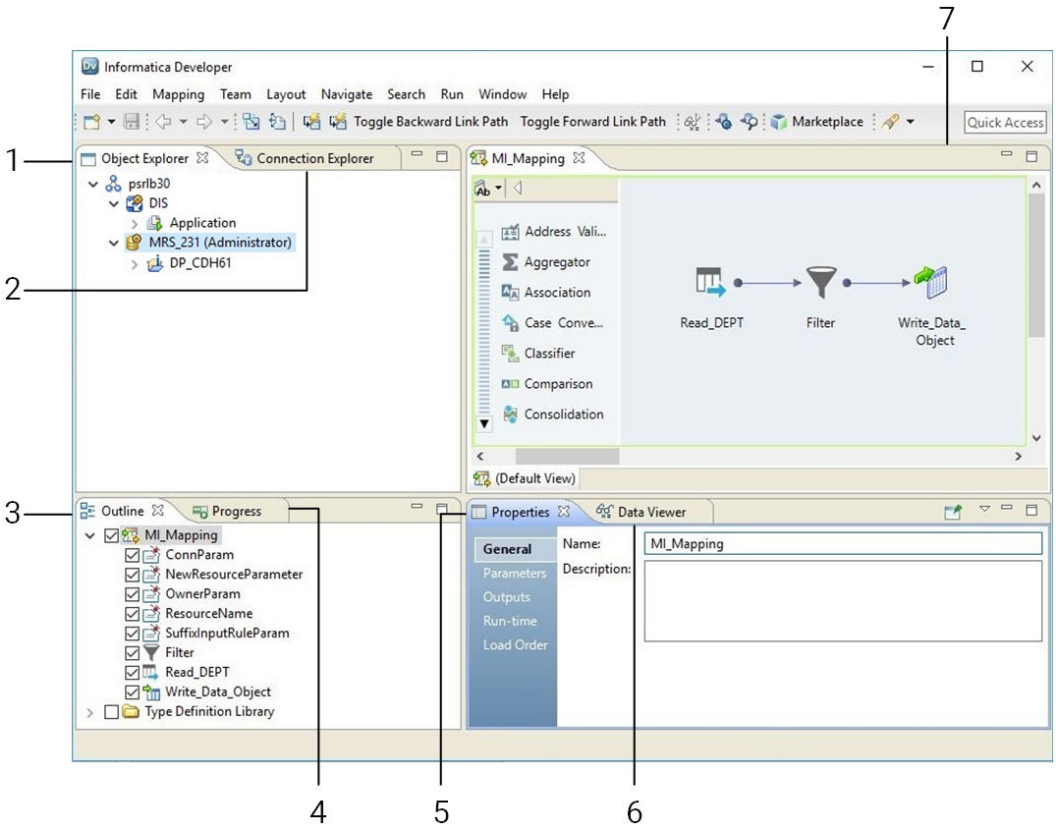
     Folder names in the local workspace directory cannot contain the number sign (#) character. If folder names in the local workspace directory contain spaces, enclose the full directory in double quotes.
3. Click **Workbench**.

   The first time you start the Developer tool, you must add a domain and a Model repository. If the domain contains more than one Data Integration Service, you must also select a default Data Integration Service.

Informatica Developer Interface

You can work on multiple tasks in the Developer tool at the same time. You can also work in multiple folders and projects at the same time. To work in the Developer tool, you access the Developer tool workbench.

The following figure shows the Developer tool workbench:



1. Object Explorer view
2. Connection Explorer view
3. Outline view
4. Progress view
5. Properties view
6. Data Viewer view
7. Editor

The recommended screen resolution for the Developer tool is 1920 * 1080 on Windows.

Informatica Developer Views

The Developer tool workbench includes an editor and views. You edit objects, such as mappings, in the editor. The Developer tool displays views based on which object is selected in the editor.

You can select additional views, hide views, and move views to another location in the Developer tool workbench.

To select the views you want to display, click **Window** > **Show View**.

Default Views

The Developer tool displays the following views by default:

Connection Explorer view

Displays connections to relational databases.

Data Viewer view

Displays source data and previews the output of a transformation.

Object Explorer view

Displays the domain and the design-time and run-time objects in the domain. Design-time objects are stored in projects and folders in the Model repository. Run-time objects are stored as part of a run-time application on a Data Integration Service.

Outline view

Displays objects that are dependent on an object selected in the **Object Explorer** view.

Progress view

Displays the progress of operations in the Developer tool, such as a mapping run.

Properties view

Displays the properties for an object that is selected in the editor.

Additional Views

The Developer tool workbench also displays the following views:

Alerts view

Displays connection status alerts.

Audit Viewer view

Displays audit results for rules and conditions after a mapping runs with an audit.

Cheat Sheets view

Displays the cheat sheet that you open. To open a cheat sheet, click **Help** > **Cheat Sheets** and select a cheat sheet.

Checked Out Objects view

Displays all objects that you have checked out.

Data Processor Events view

Displays information about initialization, execution, and summary events that occur when you run a Data Processor transformation in the Developer tool.

Data Processor Hex Source view

Displays an input document in hexadecimal format.

Help view

Displays context-sensitive online help.

Object Dependencies view

Displays object dependencies when you view, modify, or delete an object.

Search view

Displays the search results. You can also launch the search options dialog box.

Tags view

Displays tags that define an object in the Model repository based on business usage.

Notifications view

Displays options to notify users or groups when all work in the Human task is complete.

Validation Log view

Displays object validation errors.

Version History view

Displays the version history of selected objects. You can read check-in comments and view user information about object check-ins.

Setting Up Informatica Developer

Set up Informatica Developer to access Model repository objects.

To set up the Developer tool, complete the following tasks:

1. Add a domain.
2. Add a Model repository.
3. If the domain includes more than one Data Integration Service, select a default service.

After you set up the Developer tool, you can create projects and folders to store your work.

Step 1. Add a Domain

Add a domain in the Developer tool to access services that run on the domain.

Before you add a domain, verify that you have a domain name, host name, and port number to connect to a domain. You can get this information from an administrator.

1. Click **Window** > **Preferences**.

   The **Preferences** dialog box appears.
2. Select **Informatica** > **Domains**.
3. Click **Add**.

   The **New Domain** dialog box appears.
4. Enter the domain name, host name, and port number.
5. Click **Finish**.
6. Click **OK**.

Step 2. Add a Model Repository

Add a Model repository to access projects and folders.

Before you add a Model repository, verify the following prerequisites:

- An administrator has configured a Model Repository Service in the Administrator tool.
- You have a user name and password to access the Model Repository Service. You can get this information from an administrator.

1. Click **File** > **Connect to Repository**.

   The **Connect to Repository** dialog box appears.
2. Click **Browse** to select a Model Repository Service.
3. Click **OK**.

4. Click **Next**.
5. Enter your user name and password.
6. Click **Next**.

   The **Open Project** dialog box appears.
7. To filter the list of projects that appear in the **Object Explorer** view, clear the projects that you do not want to open.
8. Click **Finish**.

   The Model Repository Service appears in the **Object Explorer** view and displays the projects that you chose to open. The **Object Explorer** view also shows the domain that contains the service.

Step 3. Select a Default Data Integration Service

The Data Integration Service performs data integration tasks in the Developer tool. If the domain includes more than one Data Integration Service, select a default service. You can override the default Data Integration Service when you run a mapping.

Note: If the domain contains only one Data Integration Service, this step is optional.

Add a domain before you select a Data Integration Service.

1. Click **Window** > **Preferences**.

   The **Preferences** dialog box appears.
2. Select **Informatica** > **Data Integration Services**.
3. Expand the domain.
4. Select a Data Integration Service.
5. Click **Set as Default**.
6. Click **OK**.

Domains

The Informatica domain is a collection of nodes and services that define the Informatica environment.

You add a domain in the Developer tool. You can also edit the domain information or remove a domain. You manage domain information in the Developer tool preferences.

Projects

A project is the top-level container that you use to store folders and objects in the Developer tool.

Use projects to organize and manage the objects that you want to use for data service solutions.

You manage and view projects in the **Object Explorer** view. When you create a project, the Developer tool stores the project in the Model repository.

The following table describes the tasks that you can perform on a project:

Task	Description
Manage projects	Manage project contents. You can create, duplicate, rename, and delete a project. You can view project contents.
Filter projects	Filter the list of projects that appear in the **Object Explorer** view.
Manage folders	Organize project contents in folders. You can create, duplicate, rename, and move folders within projects.
Manage objects	View object contents, duplicate, rename, move, and delete objects in a project or in a folder within a project.
Search projects	Search for folders or objects in projects. You can view search results and select an object from the results to view its contents.
Assign permissions	Select the users and groups that can view and edit objects in the project. Specify which users and groups can assign permissions to other users and groups.

Creating a Project

Create a project to store objects and folders.

1. Select a Model Repository Service in the **Object Explorer** view.
2. Click **File** > **New** > **Project**.

   The **New Project** dialog box appears.
3. Enter a name for the project.
4. Click **Next**.

   The **Project Permissions** page of the **New Project** dialog box appears.
5. Optionally, select a user or group and assign permissions.
6. Click **Finish**.

   The project appears under the Model Repository Service in the **Object Explorer** view.

Filter Projects

You can filter the list of projects that appear in the **Object Explorer** view. You might want to filter projects if you have access to a large number of projects but need to manage only some of them.

The Developer tool retains the list of projects that you filter the next time that you connect to the repository.

You can filter projects at the following times:

Before you connect to the repository

When you filter projects before you connect to the repository, you can decrease the amount of time that the Developer tool takes to connect to the repository.

Select **File** > **Connect to Repository**. After you select the repository and enter your user name and password, click **Next**. The **Open Project** dialog box displays all projects to which you have access. Select the projects that you want to open in the repository and then click **Finish**.

After you connect to the repository

If you are connected to the repository, click **File** > **Close Projects** to filter projects out of the **Object Explorer** view. The **Close Project** dialog box displays all projects that are currently open in the **Object Explorer** view. Select the projects that you want to filter out and then click **Finish**.

To open projects that you filtered, click **File** > **Open Projects**.

Project Permissions

Assign project permissions to users or groups. Project permissions determine whether a user or group can view objects, edit objects, or assign permissions to others.

You can assign the following permissions:

Read

The user or group can open, preview, export, validate, and deploy all objects in the project. The user or group can also view project details.

Write

The user or group has read permission on all objects in the project. Additionally, the user or group can edit all objects in the project, edit project details, delete all objects in the project, and delete the project.

Grant

The user or group has read permission on all objects in the project. Additionally, the user or group can assign permissions to other users or groups.

Users assigned the Administrator role for a Model Repository Service inherit all permissions on all projects in the Model Repository Service. Users assigned to a group inherit the group permissions.

Permissions for External Objects

Permissions apply to objects within a project. The Developer tool does not extend permissions to dependent objects when the dependent objects exist in other projects.

Dependent objects are objects that are used by other objects. For example, you create a mapplet that contains a non-reusable Expression transformation. The mapplet is the parent object. The Expression transformation is a dependent object of the mapplet.

The Developer tool creates instances of objects when you use reusable objects within a parent object. For example, you create a mapping with a reusable Lookup transformation. The mapping is the parent object. It contains an instance of the Lookup transformation.

An object can contain instances of dependent objects that exist in other projects. To view dependent object instances from other projects, you must have read permission on the other projects. To edit dependent object instances from other projects, you must have write permission on the parent object project and read permission on the other projects.

Permissions for Dependent Object Instances

You might need to access an object that contains dependent object instances from another project. If you do not have read permission on the other project, the Developer tool gives you different options based on how you access the parent object.

When you try to access a parent object that contains dependent object instances that you cannot view, the Developer tool displays a warning message. If you continue the operation, the Developer tool produces results that vary by operation type.

The following table lists the results of the operations that you can perform on the parent object:

Operation	Result
Open the parent object.	The Developer tool prompts you to determine how to open the parent object: - Open a Copy. The Developer tool creates a copy of the parent object. The copy does not contain the dependent object instances that you cannot view. - Open. The Developer tool opens the object, but it removes the dependent object instances that you cannot view. If you save the parent object, the Developer tool removes the dependent object instances from the parent object. The Developer tool does not remove the dependent objects from the repository. - Cancel. The Developer tool does not open the parent object.
Export the parent object to an XML file for use in the Developer tool.	The Developer tool creates the export file without the dependent object instances.
Export the parent object to PowerCenter.	You cannot export the parent object.
Validate the parent object.	The Developer tool validates the parent object as if the dependent objects were not part of the parent object.
Deploy the parent object.	You cannot deploy the parent object.
Copy and paste the parent object.	The Developer tool creates the new object without the dependent object instances.

Security Details

When you access an object that contains dependent object instances that you cannot view, the Developer tool displays a warning message. The warning message allows you to view details about the dependent objects.

To view details about the dependent objects, click the **Details** button in the warning message. If you have the Show Security Details Model Repository Service privilege, the Developer tool lists the projects that contain the objects that you cannot view. If you do not have the Show Security Details privilege, the Developer tool indicates that you that you do not have sufficient privileges to view the project names.

Parent Object Access

If you create parent objects that use dependent object instances from other projects, users might not be able to edit the parent objects. If you want users to be able to edit the parent object and preserve the parent object functionality, you can create instances of the dependent objects in a mapplet.

For example, you create a mapping that contains a reusable Lookup transformation from another project. You want the users of your project to be able to edit the mapping, but not the Lookup transformation.

If you place the Lookup transformation in the mapping, users that do not have read permission on the other project get a warning message when they open the mapping. They can open a copy of the mapping or open the mapping, but the Developer tool removes the Lookup transformation instance.

To allow users to edit the mapping, perform the following tasks:

1. Create a mapplet in your project. Add an Input transformation, the reusable Lookup transformation, and an Output transformation to the mapplet.
2. Edit the mapping, and replace the Lookup transformation with the mapplet.
3. Save the mapping.

When users of your project open the mapping, they see the mapplet instead of the Lookup transformation. The users can edit any part of the mapping except the mapplet.

If users export the mapping, the Developer tool does not include the Lookup transformation in the export file.

Assigning Permissions

You can add users and groups to a project and assign permissions for the users and groups. Assign permissions to determine the tasks that users can complete on objects in the project.

1. Select a project in the **Object Explorer** view.
2. Click **File** > **Properties**.

   The **Properties** window appears.
3. Select **Permissions**.
4. Click **Add** to add a user and assign permissions for the user.

   The **Domain Users and Groups** dialog box appears.
5. To filter the list of users and groups, enter a name or string.

   Optionally, use the wildcard characters in the filter.
6. To filter by security domain, click the **Filter by Security Domains** button.
7. Select **Native** to show users and groups in the native security domain. Or, select **All** to show all users and groups.
8. Select a user or group, and click **OK**.

   The user or group appears in the **Project Permissions** page of the **New Project** dialog box.
9. Select read, write, or grant permission for the user or group.
10. Click **OK**.

Folders

Use folders to organize objects in a project. Create folders to group objects based on business needs. For example, you can create a folder to group objects for a particular task in a project. You can create a folder in a project or in another folder.

Folders appear within projects in the **Object Explorer** view. A folder can contain other folders, data objects, and object types.

You can perform the following tasks on a folder:

- Create a folder.

- View a folder.
- Rename a folder.
- Duplicate a folder.
- Move a folder.
- Delete a folder.

Creating a Folder

Create a folder to store related objects in a project. You must create the folder in a project or another folder.

1. In the **Object Explorer** view, select the project or folder where you want to create a folder.
2. Click **File** > **New** > **Folder**.

   The **New Folder** dialog box appears.
3. Enter a name for the folder.
4. Click **Finish**.

   The folder appears under the project or parent folder.

Copy Object Operations

You can copy objects within a project or to a different project. You can also copy objects to folders in the same project or to folders in a different project.

You can save a copy of an object with a different name. You can also copy an object to provide a link to the object in another medium, such as an email message.

You can copy the following objects to another project or folder, save copies of the objects with different names, or copy the objects as links:

- Application
- Data service
- Logical data object model
- Mapping
- Mapplet
- Physical data object
- Reference table
- Reusable transformation
- Rule
- Scorecard
- Virtual stored procedure
- Workflow

Use the following guidelines when you copy objects:

- You can copy segments of mappings, mapplets, rules, and virtual stored procedures.
- You can copy a folder to another project.

- You can copy a logical data object as a link.
- You can paste an object multiple times after you copy it.
- If the project or folder contains an object with the same name, you can rename or replace the object.

Copying an Object

Copy an object to make it available in another project or folder.

1. Select an object in a project or folder.
2. Click **Edit** > **Copy**.
3. Select the project or folder that you want to copy the object to.
4. Click **Edit** > **Paste**.

Saving a Copy of an Object

Save a copy of an object to save the object with a different name.

When you save a copy of a reference data object that specifies reference data values in a location outside the Model repository, the Model repository marks the object as not valid. To create a copy, select the object in the **Object Explorer** view, click **Edit** > **Copy**, and then paste the object to the location you want.

1. Open an object in the editor.
2. Click **File** > **Save a Copy As**.
3. Enter a name for the copy of the object.
4. Click **Browse** to select the project or folder that you want to copy the object to.
5. Click **Finish**.

Tags

A tag is metadata that defines an object in the domain based on business usage. Create tags to group objects according to their business usage.

After you create a tag, you can associate the tag with one or more objects. You can also remove the association between a tag and an object. You can use the tag to search the domain for objects associated with the tag. The Developer tool displays a glossary of all tags.

For example, you create a tag named XYZCorp_CustomerOrders and assign it to tables that contain information for the customer orders from the XYZ Corporation. Users can search by the XYZCorp_CustomerOrders tag to identify the tables associated with the tag.

When you deploy a mapping that is associated with a tag, the tag is propagated to the run-time version of the mapping on the Data Integration Service. If you update the deployed mapping using an application patch, the name of the patch is associated as a tag with the run-time version of the mapping.

Creating a Tag

Create a tag to add metadata that defines an object based on business usage.

1. Use one of the following methods to create a tag:
   - Click **Window** > **Preferences**. In the **Preferences** dialog box, select **Informatica** > **Tags**. Select a Model Repository Service and click **Add**.
   - Open an object in the editor. In the **Tags** view, click **Edit**. In the **Assign Tags for Object** dialog box, click **New**.
2. Enter a name for the tag.
3. Optionally, enter a description.
4. Click **OK**.

Assigning a Tag

Assign a tag to an object to associate the object with the metadata definition.

1. Open an object in the editor.
2. In the **Tags** view, click **Edit**.

   The **Assign Tags for Object** dialog box appears. The **Available Tags** area displays all tags defined in the repository. You can search for a tag by name or description. The **Assign Tags** area displays the opened object and any tags assigned to the object.
3. In the **Available Tags** area, select a tag.
4. In the **Assign Tags** area, select the object.
5. Click **Assign**.
6. To remove a tag from an object, select the tag in the **Available Tags** area and the object in the **Assign Tags** area, and then click **Remove**.

Viewing Tags

You can view all tags assigned to an object, or you can view all tags defined in the Model repository.

1. To view tags assigned to an object, open the object in the editor.
2. Select the **Tags** view.

   The **Tags** view displays all tags assigned to the object.
3. To view all the tags defined in the Model repository, click **Window** > **Preferences**.

   The **Preferences** dialog box appears.
4. Select **Informatica** > **Tags**.

   The **Tags** area displays all tags defined in the Model repository. You can search for a tag by name or description.

CHAPTER 2

The Model Repository

This chapter includes the following topics:

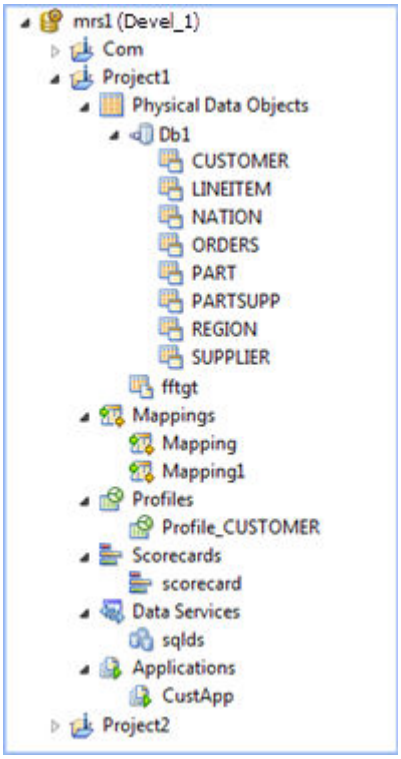
- Model Repository Overview, 26
- Objects in Informatica Developer, 27
- Repository Object Locks, 29
- Team-based Development with Versioned Objects, 31
- Connecting to a Model Repository, 35
- Model Repository Service Refresh, 35

Model Repository Overview

The Model repository is a relational database that stores the metadata for projects and folders.

Connect to the Model repository to create and edit physical data objects, mapping, and other objects. Include objects in an application, and then deploy the application to make the objects available for access by end users and third-party tools.

The following image shows an open Model repository named mrs1 in the **Object Explorer** view:



The Model Repository Service manages the Model repository. All client applications and application services that access the Model repository connect through the Model Repository Service. Client applications include the Developer tool. Informatica services that access the Model repository include the Model Repository Service and the Data Integration Service.

When you set up the Developer tool, you must add a Model repository. Each time you open the Developer tool, you connect to the Model repository to access projects and folders.

When you edit an object, the Model repository locks the object for your exclusive editing. You can also integrate the Model repository with a third-party version control system. With version control system integration, you can check objects out and in, undo the checkout of objects, and view and retrieve historical versions of objects.

Objects in Informatica Developer

You can create, manage, or view certain objects in the **Object Explorer** view in the Developer tool.

You can create the following Model repository objects in the Developer tool:

Application

A deployable object that can contain data objects, mappings, SQL data services, web services, and workflows. You can create, edit, and delete applications.

Data service

A collection of reusable operations that you can run to access and transform data. A data service provides a unified model of data that you can access through a web service or run an SQL query against. You can create, edit, and delete data services.

Data object operation

Repository object that contains properties required to perform certain run-time operations on sources or targets. Required for some PowerExchange adapter data sources.

Folder

A container for objects in the Model repository. Use folders to organize objects in a project and create folders to group objects based on business needs. You can create, edit, and delete folders.

Logical data object

An object in a logical data object model that describes a logical entity in an enterprise. It has attributes and keys, and it describes relationships between attributes. You can create, edit, and delete logical data objects in a logical data object model.

Logical data object mapping

A mapping that links a logical data object to one or more physical data objects. It can include transformation logic. You can create, edit, and delete logical data object mappings for a logical data object.

Logical data object model

A data model that contains logical data objects and defines relationships between them. You can create, edit, and delete logical data object models.

Mapping

A set of inputs and outputs linked by transformation objects that define the rules for data transformation. You can create, edit, and delete mappings.

Mapplet

A reusable object that contains a set of transformations that you can use in multiple mappings or validate as a rule. You can create, edit, and delete mapplets.

Operation mapping

A mapping that performs the web service operation for the web service client. An operation mapping can contain an Input transformation, an Output transformation, and multiple Fault transformations. You can create, edit, and delete operation mappings in a web service.

Physical data object

A physical representation of data that is used to read from, look up, or write to resources. You can create, edit, and delete physical data objects.

Reference table

Contains the standard versions of a set of data values and any alternative version of the values that you may want to find. You can view and delete reference tables.

Transformation

A repository object in a mapping that generates, modifies, or passes data. Each transformation performs a different function. A transformation can be reusable or non-reusable. You can create, edit, and delete transformations.

Virtual schema

A schema in a virtual database that defines the database structure. You can create, edit, and delete virtual schemas in an SQL data service.

Virtual stored procedure

A set of procedural or data flow instructions in an SQL data service. You can create, edit, and delete virtual stored procedures in a virtual schema.

Virtual table

A table in a virtual database. You can create, edit, and delete virtual tables in a virtual schema.

Virtual table mapping

A mapping that contains a virtual table as a target. You can create, edit, and delete virtual table mappings for a virtual table.

Workflow

A graphical representation of a set of events, tasks, and decisions that define a business process. You can create, edit, and delete workflows.

Object Properties

You can view the properties of a project, folder, or any other object in the Model repository.

The **General** tab in the **Properties** view shows the object properties. Object properties include the name, description, and location of the object in the repository. Object properties also include the user who created and last updated the object and the time the event occurred.

To access the object properties, select the object in the **Object Explorer** view and click **File** > **Properties**.

Repository Object Locks

When you begin to edit an object, the Model repository locks the object so that other users cannot save changes to it. When you save the object, you retain the lock. When you close the object, the repository releases the lock.

Object locking is part of continuous integration in a CI/CD pipeline. For more information about CI/CD, see "CI/CD Overview" on page 183.

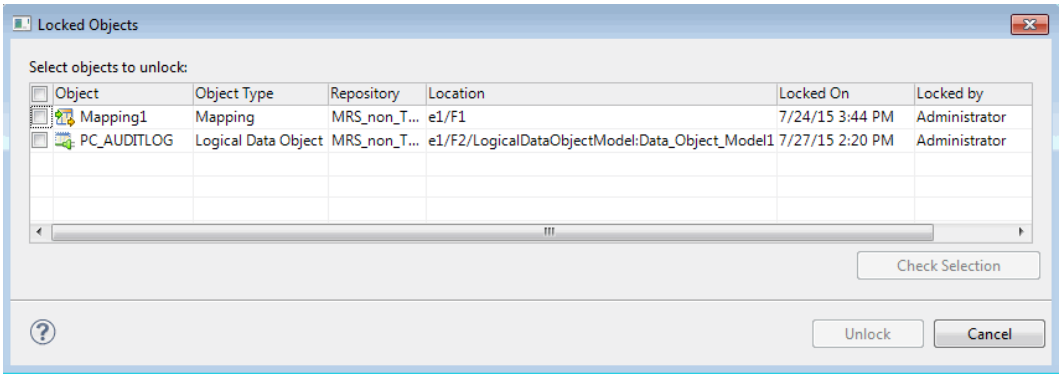
Lock Management

The Model repository retains object locks if the Developer tool stops unexpectedly. When you connect to the Model repository again, you can view the objects that you have locked. You can continue to edit the objects, or you can unlock the objects.

You might want to unlock objects if the developer who locked the object is unavailable and another developer is assigned to develop the object. You view and unlock locked objects through the **Locked Objects** dialog box.

To view the **Locked Objects** dialog box, click **File** > **Locked Objects**.

The following image shows the **Locked Objects** dialog box:



You can unlock one or more objects in the **Locked Objects** dialog box. Select objects to unlock by performing one of the following actions:

- Use the check box to select one or more objects.
- Select one or more rows and click **Check Selection** to enable the check box for each object that you selected.

Then you can click **Unlock** to unlock all the selected objects.

Tip: If your work is interrupted, you can view the objects that you have locked to identify the objects you were working on.

Rules and Guidelines for Lock Management

Consider the following rules and guidelines when you manage object locks:

- The Model repository does not lock the object when you open it. The Model repository locks the object only after you begin to edit it. For example, the Model repository locks a mapping when you insert a cursor in an editable field or connect mapping objects.
- You can use more than one client tool to develop an object. For example, you can edit an object on one machine and then open the object on another machine and continue to edit it. When you return to the first machine, you must close the object editor and reopen it to regain the lock. The same principle applies when a user with administrative privileges unlocks an object that you had open.
- If the Model Repository Service restarts while you have an object open for editing, you lose the lock on the object. Until you regain a lock on the object, another user can open and edit it. To regain a lock on the object, save changes to the object and close it, then reopen the object for editing.
- When you delete a folder that contains objects, and you are not allowed to delete any one of the objects, then you cannot delete the folder. For example, if you cannot delete an object because you do not own the lock on the object, the object and the folder remain.
- More than one developer can open and edit the contents of an SQL data service object at the same time. For example, UserA can open and begin to edit an SQL data service, and then UserB can open and begin to edit the same object. If UserB saves and closes the object before UserA, the Model repository does not tell UserA of the potential conflict until UserA saves the object. In this case, UserA can save changes by saving the SQL data service with another name.
- An administrator can revoke your write permission on an object you have locked, or reassign the lock to another user. In this case, you cannot edit or save the object. You can save the object with another name.

Team-based Development with Versioned Objects

Team-based development is the integration of the Model repository with a third-party version control system. The version control system saves multiple versions of objects and assigns each version a version number.

You manage object versions through the Developer tool. You can perform actions such as checking objects out and in, viewing and retrieving historical versions of objects, and undoing a checkout.

The Model repository protects objects from being overwritten by other members of the development team. If you open an object that another user has checked out, you receive a notification that identifies the user who checked it out. You can open a checked out object in read-only mode, or you can save it with a different name.

When the connection to the version control system is active, the Model repository has the latest version of each object.

The Model repository maintains the state of checked-out objects if it loses the connection to the version control system. While the connection with the version control system is down, you can continue to open, edit, save, and close objects. The Model repository tracks and maintains object states.

When the connection is restored, you can resume version control system-related actions, such as checking in or undoing the checkout of objects. If you opened and edited an object during the time when the connection was down, the Model repository checks out the object to you.

Note: SQL data service objects are not included in version control.

Object versioning is part of continuous integration in a CI/CD pipeline. For more information about CI/CD, see "CI/CD Overview" on page 183.

Versioned Object Management

When the Model repository is integrated with a version control system, you can manage versions of objects. For example, you can check out and check in objects, undo checkouts, and view objects that you have checked out.

You can perform the following actions:

Check out an object.

When you check out an object, it maintains a checked-out state until you check it in or undo the checkout. You can view objects that you have checked out in the **Checked Out Objects** view. To check out an object, you can begin to edit it, or you can right-click the object in the Object Explorer and select Check Out.

Undo a checkout of an object.

When you undo a checkout, you check in the object without changes and without incrementing the version number or version history. All changes that you made to the object after you checked it out are lost. To undo a checkout, you can use the **Checked Out Objects** view or the object right-click menu. For example, you might want to undo a checkout to delete changes to an object.

Note: If a user moved an object while it was checked out and you undo the checkout, the object remains in its current location, and its version history restarts. Undoing the checkout does not restore it to its pre-checkout location.

Check in an object.

When you check in an object, the version control system updates the version history and increments the version number. You can add check-in comments up to a 4 KB limit. To check in an object, you can use the **Checked Out Objects** view or the object right-click menu.

Note: The Perforce and SVN version control systems limit file path lengths due to a Windows restriction that restricts path lengths to 260 bytes. If you attempt to check in a file with a path that exceeds the limitation, Perforce returns an error. A 260 byte path length results in varying path lengths depending on the character set of your system.

Delete an object.

A versioned object must be checked out before you can delete it. If it is not checked out when you perform the delete action, the Model repository checks the object out to you and marks it for delete. To complete the delete action, you must check in the object.

When you delete a versioned object, the version control system deletes all versions.

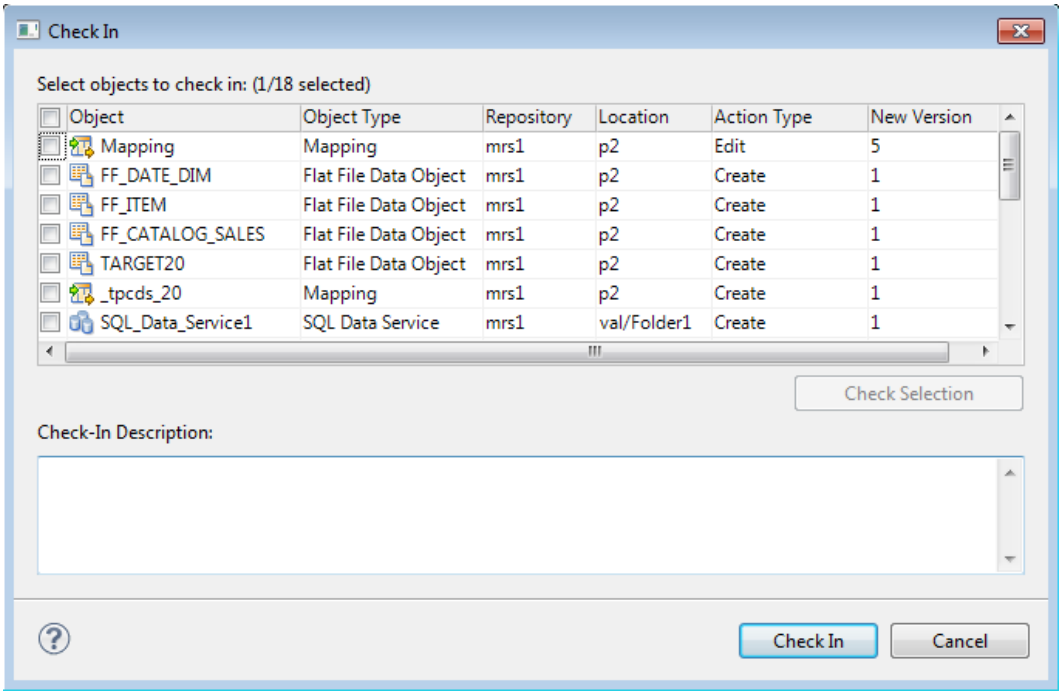
To delete an object, you can use the **Checked Out Objects** view or the object right-click menu.

Check In Dialog Box

The **Check In** dialog box displays all objects you have checked out.

Use the **Check In** dialog box to check in objects. You can also use it to delete objects that you marked for delete.

The following image shows the **Check In** dialog box:



The **Action Type** column shows the action that the Model Repository Service performs on each item when you check it in. The Model Repository Service performs one of the following action types:

Action Type	Description
Create	The object is new and has never been checked in. When you select it and click **Check In**, the Model Repository Service checks it in to the version control system and assigns it a version number of 1.
Delete	The object is marked for delete. When you select it and click **Check In**, the Model Repository Service deletes the object.
Edit	The object is marked for edit. When you select it and click **Check In**, the Model Repository Service replaces the version in the version control system with the new version.
Move	After you checked out the object, you moved it from one Model repository project or folder to another. When you select it and click **Check In**, the Model Repository Service checks it in to the version control system in the new location.

The **Location** column shows the current location of the object. If you moved the object, the **Action Type** is Move.

Tip: You can see previous locations of a moved object in the **Version History** view.

The **New Version** column shows the version number of the object after you complete the checkin.

You can check in one or more objects in the **Check In** dialog box. Select objects to check in by performing one of the following actions:

- Use the check box to select one or more objects.
- Select one or more rows and click **Check Selection** to enable the check box for each object that you selected.

Optionally, you can add a description in the **Check-In Description** text pane. Then you can click **Check In** to check in all the selected objects.

Deleting a Versioned Object

When you want to delete a versioned object, you mark it for deletion, and then check it in.

1. Right-click an object in the **Object Explorer** or the **Checked Out Objects** view, and select **Delete**.

   The Action attribute changes from **Edit** to **Delete**.
2. Select the object and select **Check In**.

   The Model repository deletes the object.

Historical Versions of Objects

The version control system saves a copy of an object every time you check it in.

The version control system saves a version of the object and assigns a version number to it. The highest number designates the latest version.

Use the **Version History** view to view the version history of objects, open historical versions in read-only view, and restore historical versions of objects. You might want to restore a historical version of an object to restore previous functionality.

When you restore a historical version of an object, that version becomes the latest version. The Model repository replaces the current version of the object with the historical version and checks the object out to you. If the object is open in an editor, the Developer tool refreshes the object to the restored version.

Restoring a Historical Version of an Object

You can view or get a historical version of an object from the version control system.

1. Right-click an object in the **Object Explorer** and select **View Version History**.

   The **Revision History** view opens.

2. To view a historical version, right-click one of the versions listed in the **Revision History** view and select **View Version**.

   The Developer tool opens a read-only version of the object. The Developer tool lists the version number in the title bar of the editor window.

3. To restore a historical version, right-click one of the versions listed in the **Revision History** view and select **Restore Version**.

   The object opens in the editor. You can edit the object, or check it in without editing it.

4. Check in the object.

   The restored version of the object becomes the latest version in the Model repository and the version control system.

Checked Out Objects View

The **Checked Out Objects** view lists all the objects that you have checked out.

You can perform the following actions in the **Checked Out Objects** view:

- Undo a checkout of an object
- Check in an object

To perform one of these actions, right-click the object and select the action.

To delete an object that is marked for delete in the **Action Type** column, right-click the object and select **Check In**.

Version History View

The **Version History** view displays the version history of selected objects. You can read check-in comments and view user information about object check-ins.

The view displays the following information:

- Version. Integer showing the order in which versions were saved. The version control system assigns the highest number to the most recent checked-in version.
- Name. Name of the object.
- Location. Path to the object in the Model repository.
- Action type. The action that the user took when checking in the object. You can add, edit, or delete and object when you check it in.
- User. User ID of the team member who checked in the object.
- Version date. Date and time that a user checked in the object.
- Version description. Check-in comments for the selected version.

To view version history, right-click an object in the **Object Explorer** and select **View Version History**.

Troubleshooting Team-based Development

Consider the following troubleshooting tips when you use features related to team-based development:

The Perforce version control system fails to check in some objects, with an error about excessively long object path names.

Due to Windows OS limitations on the number of characters in a file path, Model repository objects with long path and file names fail when you try to check them in. The Perforce error message reads "Submit aborted" and says the file path exceeds the internal length limit.

To work around this problem, limit the length of project, folder, and object names in the Model repository. Shorter names in all instances help limit the total number of characters in the object path name.

Connecting to a Model Repository

Each time you open the Developer tool, you connect to a Model repository to access projects and folders. When you connect to a Model repository, you enter connection information to access the domain that includes the Model Repository Service that manages the Model repository.

1. In the **Object Explorer** view, right-click a Model repository and click **Connect**.

   The **Connect to Repository** dialog box appears.
2. Enter the domain user name and password.
3. Select a namespace.
4. Click **OK**.

   The Developer tool connects to the Model repository. The Developer tool displays the projects in the repository.

Model Repository Service Refresh

You can refresh the Model Repository Service to see new and updated objects in the Model repository.

To refresh the Model Repository Service, right-click the Model Repository Service in the **Object Explorer** view and select **Refresh.**

CHAPTER 3

Searches in Informatica Developer

This chapter includes the following topics:

- Searches in Informatica Developer Overview, 36
- Domain Search, 36
- Editor Search, 38

Searches in Informatica Developer Overview

You can perform searches in Informatica Developer to search for objects.

You can search the domains that appear in the **Object Explorer** view to find the latest versions of objects and object properties.

Domain Search

You can search for objects and object properties in a domain.

You can create a search query and then filter the search results. You can view search results and select an object from the results to view its contents. Search results appear on the **Search** view. The search cannot display results if it finds more than 2048 objects. If search fails because the results contain more than 2048 objects, change the search options so that fewer objects match the search criteria.

The following table lists the search options that you can use to search for objects:

Search Option	Description
Containing text	Object or property that you want to search for. Enter an exact string or use a wildcard. Not case sensitive.
Names	One or more objects that contain the name. Enter an exact string or use a wildcard. Not case sensitive.
Tags	One or more objects that use a tag. Enter an exact string or use a wildcard. Not case sensitive.

Search Option	Description
Search for	One or more object types to search for.
Scope	Search the workspace or an object that you selected.

A search engine indexes the metadata in the domain. To correctly index the metadata, the search engine uses a search analyzer appropriate for the language of the metadata that you are indexing. The Developer tool uses the search engine to perform searches on design-time and run-time objects.

You can search in different languages. To search in a different language, an administrator must change the search analyzer and configure the following Model repositories to use the search analyzer:

- Model repository that stores the design-time objects.
- Model repository that is associated with the Data Integration Service where the run-time objects are deployed.

Note: To search for an object or an object property, you must have connected to the object in the **Object Explorer** view. To expand the search to additional objects, connect to the Model repository or to the run-time application that contains the objects.

Guidelines for Searches

Refer to the following guidelines when you search a domain:

- You must save a design-time object before you can search on it.
- A search returns ports and dynamic ports but does not return generated ports.
- When you search for a mapping, mapplets with a name that is similar to the search pattern appear in the search results.
- When you refresh a Model Repository Service or a run-time application, the search results in Search view no longer appear. You must search the domain again to get results based on the refreshed Model Repository Service or the refreshed run-time application.

Searching for Objects and Properties

Search for objects and properties in a domain.

1. Click **Search** > **Object Search**.

   The **Search** dialog box appears.
2. Enter the object or property you want to search for. Optionally, include wildcard characters.
3. If you want to search for a property in an object, optionally enter one or more names or tags separated by a comma.
4. Optionally, choose the object types that you want to search for.
5. Choose to search the workspace or the object that you selected.
6. Click **Search**.

   The search results appear in the **Search** view.
7. In the **Search** view, double-click an object to open it in the **Editor**.

   If the object is part of a run-time application, the object is read-only.

Editor Search

You can find objects, ports, groups, expressions, and attributes that are open in the editor. The Developer tool highlights the objects within the open editor. The objects do not need to be in the Model repository.

To display the find fields below the editor, select **Edit** > **Find/Replace**. To find an object, enter a search string and the types of objects to find. The object types that you can find varies by editor. If you do not specify any type of object, the Developer tool finds the search string in transformations.

When you search for ports, columns, or attributes, you can also select the data type. For example, you can find integer or bigint ports with names that contain the string "_ID."

The following table lists the types of objects that you can find in each editor:

Editor	Object types
Mapping	Mapping objects, expressions, groups, ports, and complex data type definitions
Mapplet	Mapplet objects, expressions, groups, ports, and complex data type definitions
Logical data object model	Logical data objects and attributes
Physical data object read or write mapping	Mapping objects and columns
SQL data service	Virtual tables and attributes
Virtual stored procedure	Transformations, expressions, groups, and ports
Virtual table mapping	Virtual table mapping objects, expressions, groups, and ports
Web service operation mapping	Web service operation mapping objects, expressions, groups, and ports
Workflow	Workflow objects
Note: The Model repository does not store the names of generated ports, and you cannot search for them in the editor.	

When the Developer tool finds the search string, it displays the object locations. It also highlights the object in which the search string occurs. If the search string occurs in an iconized transformation in the mapping editor, the Developer tool highlights the iconized transformation.

You can select the following options to navigate the results of a find:

- Next Match. Finds the next occurrence of the search string.
- Previous Match. Finds the previous occurrence of the search string.
- Highlight All. Highlights all occurrences of the search string.
- Expand Iconized Transformations. Expands all iconized transformations in which the search string occurs.

CHAPTER 4

Connections

This chapter includes the following topics:

- Connections Overview, 39
- Connection Explorer View, 41
- Connection Management, 41
- Connection Switching, 44
- Third-Party JDBC Drivers, 47
- Environment SQL, 47

Connections Overview

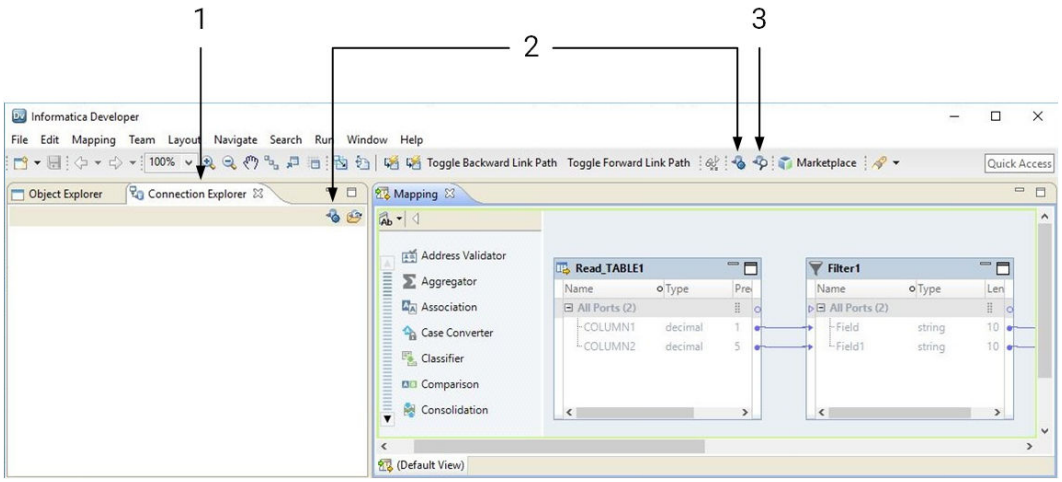
A connection is a repository object that defines a connection in the domain configuration repository.

Create a connection to import data objects, preview data, and run mappings. The Developer tool uses the connection when you import a data object. The Data Integration Service uses a connection when you run mappings, preview data, or consume web services.

Note: The Developer tool does not use connections to import flat file data objects or preview, read, or write flat file data.

The Developer tool stores connections in the domain configuration repository. Any connection that you create in the Developer tool is available in the Administrator tool.

Create and manage connections in the **Preferences** dialog box or the **Connection Explorer** view.



1. Connection Explorer view
2. Create Connection
3. Show Connections

After you create a connection, you can perform the following actions on the connection:

Edit the connection.

You can change the connection name and the description. You can also edit connection details such as the user name, password, and connection strings.

The Data Integration Service identifies connections by the connection ID. Therefore, you can change the connection name. When you rename a connection, the Developer tool updates the objects that use the connection.

Deployed applications and parameter files identify a connection by name, not by connection ID. Therefore, when you rename a connection, you must redeploy all applications that use the connection. You must also update all parameter files that use the connection parameter.

Copy the connection.

Copy a connection to create a connection that is similar to another connection. For example, you might create two Oracle connections that differ only in user name and password.

Delete the connection.

When you delete a connection, objects that use the connection are no longer valid. If you accidentally delete a connection, you can re-create it by creating another connection with the same connection ID as the deleted connection.

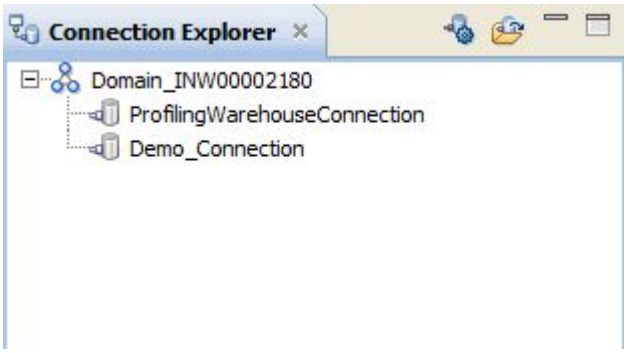
Refresh the connections list.

You can refresh the connections list to see the latest list of connections for the domain. Refresh the connections list after a user adds, deletes, or renames a connection in the Administrator tool.

Connection Explorer View

Use the **Connection Explorer** view to view relational database connections and to create relational data objects.

The following figure shows the **Connection Explorer** view in the Developer tool:



You can complete the following tasks in the **Connection Explorer** view:

- Add a connection to the view. Click the **Select Connection** button to choose one or more connections to add to the **Connection Explorer** view.
- Connect to a relational database. Right-click the database and click **Connect**.
- Disconnect from a relational database. Right-click the database and click **Disconnect**.
- Create a relational data object. After you connect to a relational database, expand the database to view tables. Right-click a table, and click **Add to Project** to open the **New Relational Data Object** dialog box.
- Refresh a connection. Right-click a connection and click **Refresh**.
- Show only the default schema. Right-click a connection and click **Show Default Schema Only**. Default is enabled.
- Delete a connection from the **Connection Explorer** view. The connection remains in the Model repository. Right-click a connection and click **Delete**.

Connection Management

Create and manage connections in the **Preferences** dialog box or the **Connection Explorer** view.

Creating a Connection

Create a connection to access a compute cluster, database, enterprise application, file system, nonrelational database, NoSQL database, social media application, or web service. Create the connection before you import physical data objects, preview data, or run mappings.

1. Click **Window** > **Preferences**.
2. Select the type of connection that you want to create:
   - To select a non-web services connection, select **Informatica** > **Connections**.
   - To select a web services connection, select **Informatica** > **Web Services** > **Connections**.

3. Expand the domain in the **Available Connections** list.
4. Select a connection type in the **Available Connections** list, and click **Add**.

   The **New <Connection Type> Connection** dialog box appears.
5. Enter the following information:

Property	Description
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. It cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Optional description for the connection.
Location	Domain in which the connection exists.
Type	Specific connection type, such as Oracle, Twitter, or Web Services.

6. Click **Next**.
7. Configure the connection properties.
8. Click **Test Connection** to verify that you entered the connection properties correctly and that you can connect to the database, application, file system, or URI.
9. Click **Finish**.

After you create a connection, you can add it to the **Connection Explorer** view.

Editing a Connection

You can edit the connection name, description, and connection properties.

1. Click **Window** > **Preferences**.
2. Select the type of connection that you want to edit.
   - To select a non-web services connection, select **Informatica** > **Connections**.
   - To select a web services connection, select **Informatica** > **Web Services** > **Connections**.
3. Expand the domain in the **Available Connections** list.
4. Select the connection in **Available Connections**, and click **Edit**.

   The **Edit Connection** dialog box appears.
5. Optionally, edit the connection name and description.

   Note: If you change a connection name, you must redeploy all applications that use the connection. You must also update all parameter files that use the connection parameter.
6. Click **Next**.
7. Optionally, edit the connection properties.
8. Click **Test Connection** to verify that you entered the connection properties correctly and that you can connect to the database.
9. Click **OK** to close the **Edit Connection** dialog box.
10. Click **OK** to close the **Preferences** dialog box.

Copying a Connection

You can copy a connection within a domain or into another domain.

1. Click **Window** > **Preferences**.
2. Select the type of connection that you want to copy.
   - To select a non-web services connection, select **Informatica** > **Connections**.
   - To select a web services connection, select **Informatica** > **Web Services** > **Connections**.
3. Expand the domain in the **Available Connections** list.
4. Select the connection in **Available Connections**, and click **Copy**.

   The **Copy Connection** dialog box appears.
5. Enter the connection name and ID, and select the domain.

   The name and ID must be unique in the domain.
6. Click **OK** to close the **Copy Connection** dialog box.
7. Click **OK** to close the **Preferences** dialog box.

Deleting a Connection

When you delete a connection through the **Preferences** dialog box, the Developer tool removes the connection from the Model repository.

1. Click **Window** > **Preferences**.
2. Select the type of connection that you want to delete.
   - To select a non-web services connection, select **Informatica** > **Connections**.
   - To select a web services connection, select **Informatica** > **Web Services** > **Connections**.
3. Expand the domain in the **Available Connections** list.
4. Select the connection in **Available Connections**, and click **Remove**.
5. Click **OK** to close the **Preferences** dialog box.

Refreshing the Connections List

Refresh the connections list to see the latest list of connections in the domain.

1. Click **Window** > **Preferences**.
2. Select the type of connection that you want to refresh.
   - To select a non-web services connection, select **Informatica** > **Connections**.
   - To select a web services connection, select **Informatica** > **Web Services** > **Connections**.
3. Select the domain in the **Available Connections** list.
4. Click **Refresh**.
5. Expand the domain in the **Available Connections** list to see the latest list of connections.
6. Click **OK** to close the **Preferences** dialog box.

Connection Switching

You can switch the connection of a relational data object or customized data object to use a different relational database connection. You can also simultaneously switch the connections for multiple data objects. Switching the connection saves time and effort because you do not have to update each mapping to use the new connection.

After you switch the connection, the Developer tool updates the connection details for the data object in all Read, Write, and Lookup transformations that are based on the data object. The Developer tool also updates the database type for the data object based on the database that the new connection points to.

You can switch a connection to one of the following connection types:

- IBM DB2
- Microsoft SQL Server
- ODBC
- Oracle

When the Developer tool switches a connection, it does not validate the metadata compatibility. Therefore, before you switch the connection, you must ensure that the database that the new connection points to contains a table with the same columns and metadata as the data object for which you are switching the connection. Otherwise, data loss or inconsistencies can occur.

Example

You have created an Oracle relational data object in the Developer tool and added it as a Write transformation in multiple mappings.

You migrate the Oracle database to an IBM DB2 database. You want to update the existing mappings in the Developer tool to write data to the IBM DB2 database.

Instead of replacing the data object that the Write transformation is based on in each mapping, you can switch the connection of the Oracle data object to an IBM DB2 connection. The new connection points to the IBM DB2 database to which you want to write data. When you run mappings that contain the Oracle relational data object, the Data Integration Service uses the new IBM DB2 connection to run the mappings.

Before You Switch a Connection

Before you switch a connection, verify that the following requirements are met:

- You have write permissions on the data object for which you want to switch the connection.
- The database that the new connection points to contains a table with the same columns and metadata as the data object for which you want to switch the connection. If the precision and scale of columns in the table are lower than that of the data object, data loss or inconsistencies can occur.
- The data object for which you want to switch the connection does not contain unsaved changes. If it contains unsaved changes, the Developer tool does not switch the connection.
- The customized data object for which you want to switch the connection does not contain relational data objects of different database types. If it does, the Developer tool does not display an error when you switch the connection. However, when you run a mapping with the customized data object, the mapping fails.
- The table names that the original connection and new connection point to match exactly, including the case. If the connection supports quoted identifiers and mixed-case table names, the Developer tool treats the table names as case sensitive.

- The data object for which you want to switch the connection is checked out if the object is stored in a versioned repository.

Switching a Connection

You can switch the connection for a relational data object or customized data object to use a different relational database connection and simultaneously update the existing mappings to use the new connection.

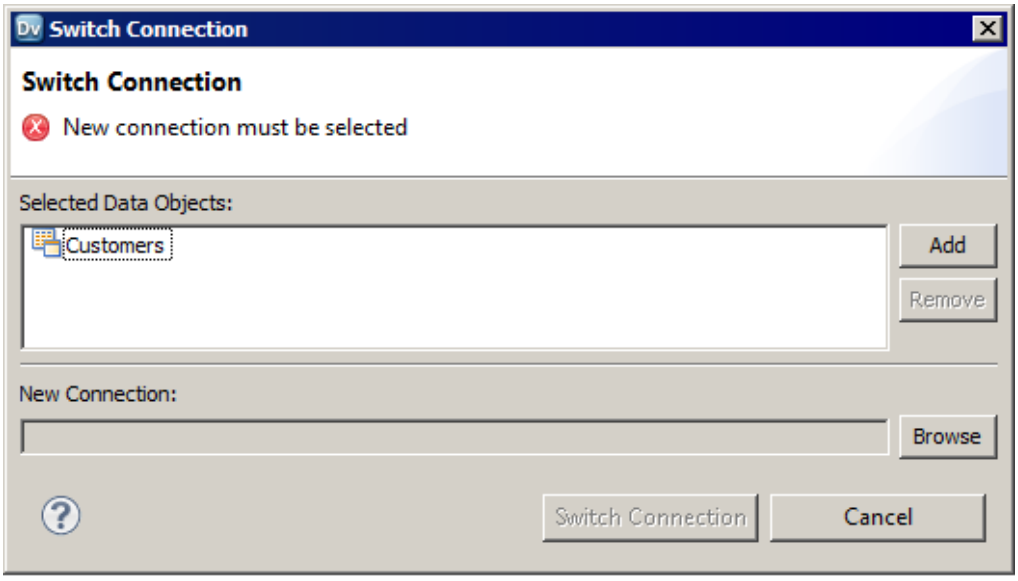
1. In the **Object Explorer** view, right-click the data object for which you want to switch the connection.

   You can select multiple data objects under different folders of a project or across projects.

   You can also select a connection and simultaneously switch the connection for all relational data objects and customized data objects that use the connection.

2. Click **Switch Connection**.

   The **Switch Connection** dialog box appears and displays the selected data objects.



3. Click **Add** to update the list of data objects for which you want to switch the connection.

   To remove an object, select the object and click **Remove**.

4. Click **Browse** next to the **New Connection** field.

   The Developer tool displays the connections that you can use for the data object.

5. Select the new connection that you want to use for the data object and click **OK**.

6. Click **Switch Connection**.

   A message appears prompting you to ensure that the database that the new connection points to contains a table with the same columns and metadata as the data object for which you are switching the connection.

7. Click **OK** to switch the connection for the data object.

   A message appears stating that the connection was switched successfully. The Developer tool updates the connection details and the database type for the data object in all Developer tool objects associated with the data object. When you run mappings that contain the data object, the Data Integration Service uses the new connection.

After You Switch a Connection

After you switch a connection, verify the data object properties and manually edit the properties, if required.

Perform the following tasks after you switch a connection:

- Verify the data type mapping.
- Verify the table owner name.
- Verify Lookup transformations.
- Reconfigure hints.
- Synchronize data objects.

Verify the Data Type Mapping

When you switch a connection, the Developer tool identifies the best data type match between the databases that the original connection and the new connection point to. It sets the data types for the data object and the transformations in the mapping that are based on the data object accordingly. Before you run a mapping, verify the data types and manually update them, if required.

For example, you switch a connection from Oracle to Microsoft SQL Server. For data types with a fixed precision, by default, the Developer tool sets the precision based on the precision of the Microsoft SQL Server database. However, for data types with a variable precision, the Developer tool sets the precision and scale based on the Oracle database. For data types such as Timestamp with Time Zone, the Developer tool sets the data type as Varchar (0,0) or an equivalent data type that the Microsoft SQL Server database supports for Varchar.

Verify the Table Owner Name

When you switch a connection, the Developer tool retains the table owner name of the data object. You can manually edit the table owner name in the data object properties, if required.

If you set the table owner name to blank and switch the connection, you can successfully preview the data only if the table exists in the default schema or public schema of the database that the new connection points to. Otherwise, the data preview fails. You must manually update the table owner name in the run-time properties of the data object to successfully preview the data.

Verify Lookup Transformations

After you switch a connection, rebuild the lookup cache and verify the lookup conditions for Lookup transformations based on the data object, if required.

Perform the following tasks:

Rebuild the lookup cache

If you configure the Data Integration Service to persist the lookup cache and switch the connection for the associated data object, you must update the Lookup transformation to rebuild the lookup cache based on the new connection. Otherwise, when you run the mapping, an error occurs stating that the cache file was created with a different database connection.

Verify the lookup conditions

When you switch a connection, the Developer tool identifies the best data type match between the databases that the original connection and the new connection point to, and sets the data types accordingly. After you switch a connection, the lookup conditions might not be valid because of the data type change. You must verify the lookup conditions and manually update them.

For example, you create a mapping that contains an IBM DB2 source table, lookup table, and target table. You configure the lookup condition on an Integer column of the source table and lookup table. If you switch the connection of the lookup table from IBM DB2 to Oracle, the transformation data type of the Integer column in the lookup table changes to Decimal. The lookup condition is not valid because you cannot compare an Integer column with a Decimal column.

Reconfigure Hints

When you switch the connection of a customized data object, the Developer tool does not retain the hints that you had configured for the customized data object. You must manually reconfigure the hints.

Synchronize Data Objects

After you switch the connection, the Developer tool retains only the active reference key constraints.

When you switch the connection for multiple data objects simultaneously, you must synchronize the data objects to ensure that the key relationships are accurate.

If the data objects contain multiple tables with cyclic reference key constraints between them and you switch the connection for a subset of tables to a different database type, the Developer table switches the connection without displaying any error. However, when you view the key relationships for the data objects, the Developer tool displays the key relationships with reference to the original database. To update the key relationships and point them to the new database, you must synchronize the data objects.

Third-Party JDBC Drivers

If you want to connect to sources, targets, and lookups through JDBC, install and configure a JDBC Type 4 driver from a third-party vendor.

To import metadata in the Developer tool, copy the third-party JDBC driver jar file to the following location:

```
<Informatica installation directory>\clients\externaljdbcjars
```

To run data previews or mappings, copy the JDBC driver jar file to the following location:

```
<Informatica installation directory>\externaljdbcjars
```

Environment SQL

The Integration Service runs environment SQL in auto-commit mode and closes the transaction after it issues the SQL. Use SQL commands that do not depend on a transaction being open during the entire read or write process. For example, if a source database is set to read only mode and you create an environment SQL statement in the source connection to set the transaction to read only, the Integration Service issues a commit after it runs the SQL and cannot read the source in read only mode.

You can configure connection environment SQL or transaction environment SQL.

Use environment SQL for source, target, lookup, and stored procedure connections. If the SQL syntax is not valid, the Integration Service does not connect to the database, and the session fails.

Note: When a connection object has “environment SQL,” the connection uses “connection environment SQL.”

Connection Environment SQL

This custom SQL string sets up the environment for subsequent transactions. The Integration Service runs the connection environment SQL each time it connects to the database. If you configure connection environment SQL in a target connection, and you configure three partitions for the pipeline, the Integration Service runs the SQL three times, once for each connection to the target database. Use SQL commands that do not depend on a transaction being open during the entire read or write process.

For example, use the following SQL statement to set the quoted identifier parameter for the duration of the connection:

```
SET QUOTED_IDENTIFIER ON
```

Use the SQL statement in the following situations:

- You want to set up the connection environment so that double quotation marks are object identifiers.
- You configure the target load type to Normal and the Microsoft SQL Server target name includes spaces.

Transaction Environment SQL

This custom SQL string also sets up the environment, but the Integration Service runs the transaction environment SQL at the beginning of each transaction.

Use SQL commands that depend on a transaction being open during the entire read or write process. For example, you might use the following statement as transaction environment SQL to modify how the session handles characters:

```
ALTER SESSION SET NLS_LENGTH_SEMANTICS=CHAR
```

This command must be run before each transaction. The command is not appropriate for connection environment SQL because setting the parameter once for each connection is not sufficient.

Guidelines for Configuring Environment SQL

Consider the following guidelines when creating the SQL statements:

- You can enter any SQL command that is valid in the database associated with the connection object. The Integration Service does not allow nested comments, even though the database might.
- When you enter SQL in the SQL Editor, you type the SQL statements.
- Use a semicolon (;) to separate multiple statements.
- The Integration Service ignores semicolons within /*...*/.
- If you need to use a semicolon outside of comments, you can escape it with a backslash (\).
- You can use parameters and variables in the environment SQL. Use any parameter or variable type that you can define in the parameter file. You can enter a parameter or variable within the SQL statement, or you can use a parameter or variable as the environment SQL. For example, you can use a session parameter, $ParamMyEnvSQL, as the connection or transaction environment SQL, and set $ParamMyEnvSQL to the SQL statement in a parameter file.
- You can configure the table owner name using sqlid in the connection environment SQL for a DB2 connection. However, the table owner name in the target instance overrides the SET sqlid statement in environment SQL. To use the table owner name specified in the SET sqlid statement, do not enter a name in the target name prefix.

CHAPTER 5

Physical Data Objects

This chapter includes the following topics:

- Physical Data Objects Overview, 49
- Physical Data Object Types, 50
- Relational Data Objects, 50
- Customized Data Objects, 54
- Create or Replace Target Tables, 59
- Custom Queries, 62
- Nonrelational Data Objects, 75
- WSDL Data Object, 76
- Synchronization, 80
- Troubleshooting Physical Data Objects, 83

Physical Data Objects Overview

A physical data object is the physical representation of data that is used to read from, look up, or write to data sources. If the data object source changes, you can synchronize the physical data object. When you synchronize a physical data object, the Developer tool reimports the object metadata.

You can create any physical data object in a project or folder. Physical data objects in projects and folders are reusable objects. You can use them in any type of mapping or mapplet, but you cannot change the data object within the mapping or mapplet. To update the physical data object, you must edit the object within the project or folder.

You can include a physical data object in a mapping or mapplet. You can add a physical data object to a mapping or mapplet as a read, write, or lookup transformation. You can add a physical data object to a logical data object mapping to map logical data objects.

You can also include a physical data object in a virtual table mapping when you define an SQL data service. You can include a physical data object in an operation mapping when you define a web service.

Physical Data Object Types

You can create different types of physical data objects based on the type of data source that you want to read data from or write data to.

Physical data objects include the following types:

Relational data object

A physical data object that uses a relational table, view, or synonym as a source. For example, you can create a relational data object from an Oracle view.

Depending on the object type, you can add a relational data object to a mapping or mapplet as a source, a target, or a Lookup transformation.

Customized data object

A physical data object that uses one or multiple related relational resources or relational data objects as sources. Relational resources include tables, views, and synonyms. For example, you can create a customized data object from two Microsoft SQL Server tables that have a primary key-foreign key relationship.

Create a customized data object if you want to perform operations such as joining data, filtering rows, sorting ports, or running custom queries in a reusable data object.

Nonrelational data object

A physical data object that uses a nonrelational database resource as a source. For example, you can create a nonrelational data object from a VSAM source.

Flat file data object

A physical data object that uses a flat file as a source. You can create a flat file data object from a delimited or fixed-width flat file.

WSDL data object

A physical data object that uses a WSDL file as a source.

Informatica PowerExchange® adapters also provide access to many data sources that you can use to create physical data objects, such as SAP, Salesforce, and Netezza.

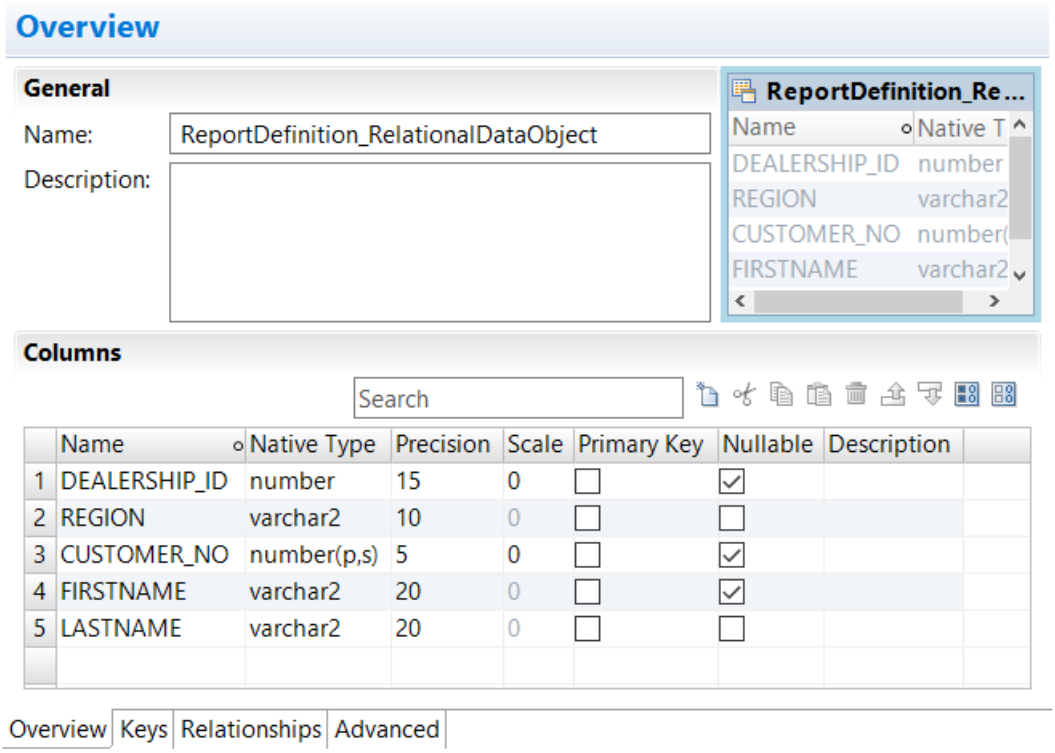
Relational Data Objects

A relational data object is a physical data object that has a relational table, view, or synonym as a source. You can include a relational data object in a mapping or mapplet.

The relational data object describes a resource in a database. When you add a relational data object to the repository, you import it from a source database. You can change the relational data object definition after you import it to the repository. You can add and remove ports, define primary keys, and configure relationships between multiple relational data objects in the repository. You can change or parameterize the default connection, the database owner, and the resource name. You can also select or clear all nullable values of the columns using Enable All Nullable or Disable All Nullable icons.

When you add the relational data object to a mapping or mapplet, you indicate whether the you want to create a read or a write object. You can configure different run-time properties based on whether the object is a source, a target, or a lookup table.

The following image shows a sample relational data object in the editor:



You can create primary key-foreign key relationships between relational data objects whether or not the relationships exist in the source database.

You can add multiple relational data objects to a mapping or mapplet as sources. When you add multiple relational data objects at the same time, the Developer tool prompts you to add the objects in either of the following ways:

- As related data objects. The Developer tool creates one read transformation with multiple relational resources. The read transformation has the same capabilities as a customized data object.
- As independent data objects. The Developer tool creates one read transformation for each relational data object. The read transformations have the same capabilities as relational data objects.

You can import the following types of relational data objects:

- DB2 for i5/OS
- DB2 for z/OS
- HAWQ
- IBM DB2
- JDBC
- Microsoft SQL Server
- Netezza
- ODBC
- Oracle
- SAP HANA

Importing a Relational Data Object

Import a relational data object to add to a mapping or mapplet.

Before you import a relational data object, you must configure a connection to the database.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Data Object**.

   The **New** dialog box appears.
3. Select **Relational Data Object** and click **Next**.

   The **New Relational Data Object** dialog box appears.
4. Click **Browse** next to the Connection option and select a connection to the database.
5. Click **Create data object from existing resource**.
6. Click **Browse** next to the **Resource** option.

   The **Select a Resource** dialog box appears.
7. Select the table, view, or synonym that you want to import.
8. To filter data objects, enter a name in the **Filter** section and click **Search**.

   You can use a question mark (?) as a single-character wild card, or an asterisk (*) as a multiple-character wild card in a search query.

   Note: The search displays a maximum of 20K results.
9. If an physical data object with the same name as the resource you are importing already exists, the **Import Conflict Resolution** dialog box appears. Choose how to resolve the import conflict. You can create a physical data object with a different name, reuse the existing object, or replace the existing object.

   Select **Apply to all conflicts** to apply the same solution to all import conflicts.
10. Enter a name for the physical data object.
11. Click **Browse** next to the **Location** option and select the project where you want to import the relational data object.
12. Click **Finish**.

    The data object appears under **Physical Data Objects** in the project or folder in the **Object Explorer** view.

Key Relationships

You can create key relationships between relational data objects. Key relationships allow you to join relational data objects when you use them as sources in a customized data object or as read transformations in a mapping or mapplet.

When you import relational data objects, the Developer tool retains the primary key information defined in the database. When you import related relational data objects at the same time, the Developer tool also retains foreign keys and key relationships. However, if you import related relational data objects separately, you must re-create the key relationships after you import the objects.

To create key relationships between relational data objects, first create a primary key in the referenced object. Then create the relationship in the relational data object that contains the foreign key.

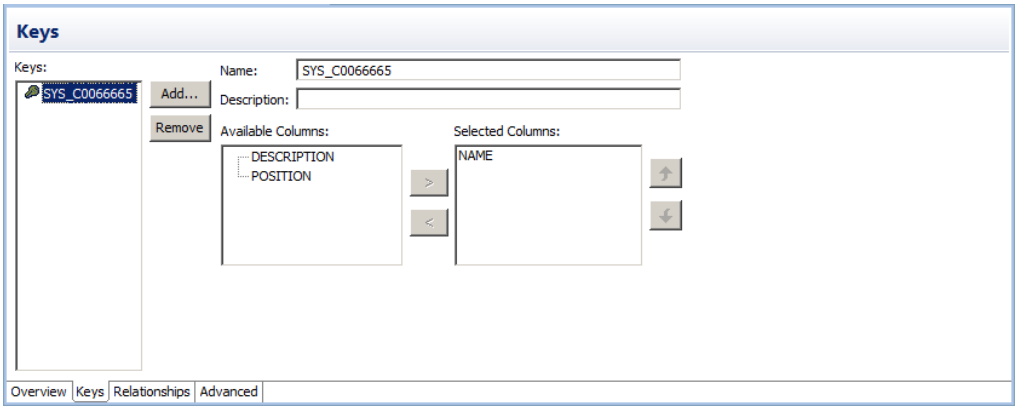
The key relationships that you create exist in the relational data object metadata. You do not need to alter the source relational resources.

Creating Keys in a Relational Data Object

Create key columns to identify each row in a relational data object. You can create one primary key in each relational data object.

1. Open the relational data object.
2. Select the **Keys** view.

   The following figure shows the **Keys** view for a sample relational data object that is open in the editor:



3. Click **Add**.

   The **New Key** dialog box appears.
4. Enter a key name.
5. If the key is a primary key, select **Primary Key**.
6. Select the key columns.
7. Click **OK**.
8. Save the relational data object.

Creating Relationships between Relational Data Objects

You can create key relationships between relational data objects. You cannot create key relationships between a relational data object and a customized data object.

The relational data object that you reference must have a primary key.

1. Open the relational data object where you want to create a foreign key.
2. Select the **Relationships** view.
3. Click **Add**.

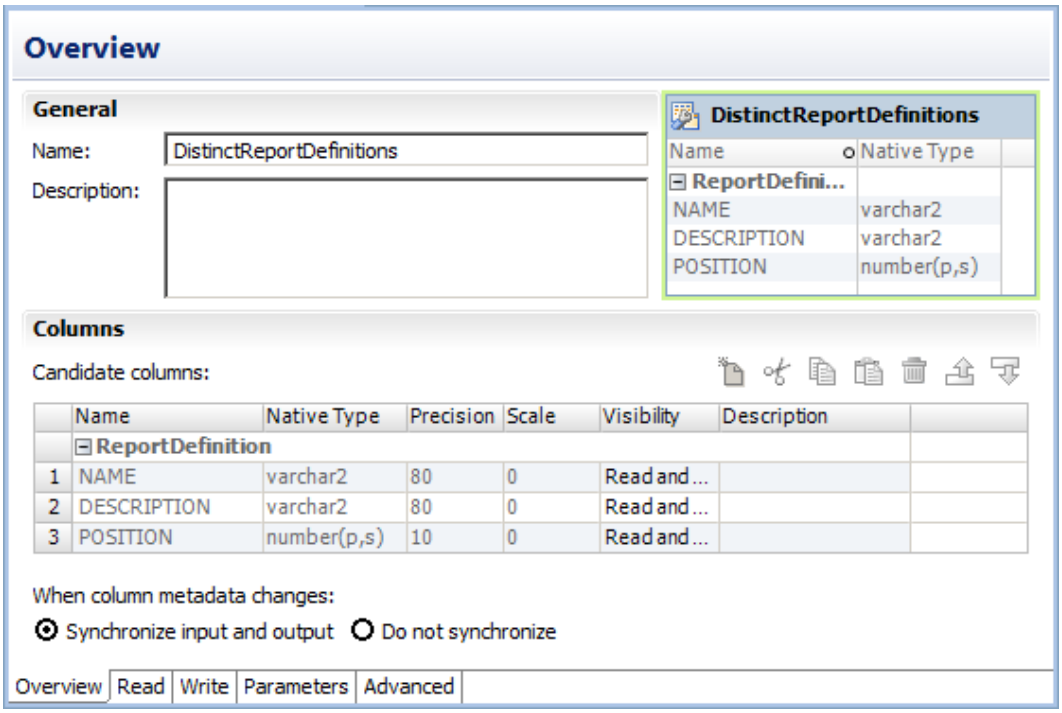
   The **New Relationship** dialog box appears.
4. Enter a name for the foreign key.
5. Select a primary key from the referenced relational data object.
6. Click **OK**.
7. In the **Relationships** properties, select the foreign key columns.
8. Save the relational data object.

Customized Data Objects

Customized data objects are physical data objects with one or more relational resources. Create a customized data object if you want to perform operations such as joining data, filtering rows, sorting ports, or running custom queries when the Data Integration Service reads source data. You can reuse a customized data object in a mapping or mapplet.

You can create customized data objects in projects and folders. You cannot change the customized data object from within a mapping or mapplet. If you change a customized data object in a project or folder, the Developer tool updates the object in all mappings or mapplets that use the object.

The following figure shows a sample customized data object that is open in the editor:



Create a customized data object to perform the following tasks:

- Create a custom query to replace the default query that the Data Integration Service runs to read the source data. The default query is a SELECT statement that references each column that the Data Integration Service reads from the source.
- Define parameters for the data object. You can define and assign parameters in a customized data object to represent connections. You can define parameters for the connection name, table owner, and table name. When you run a mapping that uses the customized data object, you can define different values for the connection parameters at runtime.
- Join source data that originates from the same source database. You can join multiple tables with primary key-foreign key relationships whether or not the relationships exist in the database.
- Retain key relationships when you synchronize the object with the sources. If you create a customized data object that contains multiple tables, and you define key relationships that do not exist in the database, you can retain the relationships when you synchronize the data object.
- Select distinct values from the source. If you choose Select Distinct, the Data Integration Service adds a SELECT DISTINCT statement to the default SQL query.

- Filter rows when the Data Integration Service reads source data. If you include a filter condition, the Data Integration Service adds a WHERE clause to the default query.
- Specify sorted ports. If you specify a number for sorted ports, the Data Integration Service adds an ORDER BY clause to the default SQL query.
- Specify an outer join instead of the default inner join. If you include a user-defined join, the Data Integration Service replaces the join information specified by the metadata in the SQL query.
- Add pre- and post-mapping SQL commands. The Data Integration Service runs pre-mapping SQL commands against the source database before it reads the source. It runs post-mapping SQL commands against the source database after it writes to the target.

You can create customized data objects from the following types of connections and objects:

- DB2 i5/OS connections
- DB2 z/OS connections
- IBM DB2 connections
- JDBC connections
- Microsoft SQL Server connections
- ODBC connections
- Oracle connections
- Relational data objects

You can also add sources to a customized data object through a custom SQL query.

Key Relationships

You can create key relationships between sources in a customized data object when the sources are relational resources. Key relationships allow you to join the sources within the customized data object.

Note: If a customized data object uses relational data objects as sources, you cannot create key relationships within the customized data object. You must create key relationships between the relational data objects instead.

When you import relational resources into a customized data object, the Developer tool retains the primary key information defined in the database. When you import related relational resources into a customized data object at the same time, the Developer tool also retains key relationship information. However, if you import related relational resources separately, you must re-create the key relationships after you import the objects into the customized data object.

When key relationships exist between sources in a customized data object, the Data Integration Service joins the sources based on the related keys in each source. The default join is an inner equijoin that uses the following syntax in the WHERE clause:

```
Source1.column_name = Source2.column_name
```

You can override the default join by entering a user-defined join or by creating a custom query.

To create key relationships in a customized data object, first create a primary key in the referenced source transformation. Then create the relationship in the source transformation that contains the foreign key.

The key relationships that you create exist in the customized data object metadata. You do not need to alter the source relational resources.

Customized Data Object Write Properties

The Data Integration Service uses write properties when it writes data to relational resources. To edit write properties, select the Input transformation in the **Write** view, and then select the **Advanced** properties.

The following table describes the write properties that you configure for customized data objects:

Property	Description
Truncate Hive Target Partition	Overwrites the partition in the Hive target in which the data is being inserted. To enable this option, you must also select the option to truncate target tables. You can truncate external partitioned tables and non-partitioned tables. Default is disabled.
Load type	Type of target loading. Select Normal or Bulk. If you select Normal, the Data Integration Service loads targets normally. You can choose Bulk when you load to DB2, Sybase, Oracle, or Microsoft SQL Server. If you specify Bulk for other database types, the Data Integration Service reverts to a normal load. Bulk loading can increase mapping performance, but it limits the ability to recover because no database logging occurs. When you write to an Oracle target with bulk loading, you can optimize performance by disabling constraints in the Oracle database. Choose Normal mode if the mapping contains an Update Strategy transformation. If you choose Normal and the Microsoft SQL Server target name includes spaces, configure the following environment SQL in the connection object: `SET QUOTED_IDENTIFIER ON`
Update override	Overrides the default UPDATE statement for the target.
Delete	Deletes all rows flagged for delete. Default is enabled.
Insert	Inserts all rows flagged for insert. Default is enabled.
Target Schema Strategy	Type of target schema strategy for the relational or Hive target table. You can select one of the following target schema strategies: - RETAIN - Retain existing target schema. The Data Integration Service retains the existing target schema. - CREATE - Create or replace table at run time. The Data Integration Service drops the target table at run time and replaces it with a table based on a target table that you identify. - Assign Parameter. You can assign a parameter to represent the value for the target schema strategy and then change the parameter at run time.
DDL query for create or replace	The DDL query based on which the Data Integration Service creates or replaces the target table. This option is applicable when you select the **CREATE - Create or replace table at run time** target schema strategy.
Truncate target table	Truncates the target before it loads data. Default is disabled.

Property	Description
Update strategy	Update strategy for existing rows. You can select one of the following strategies: - Update as update. The Data Integration Service updates all rows flagged for update. - Update as insert. The Data Integration Service inserts all rows flagged for update. You must also select the **Insert** target option. - Update else insert. The Data Integration Service updates rows flagged for update if they exist in the target and then inserts any remaining rows marked for insert. You must also select the **Insert** target option.
PreSQL	SQL command the Data Integration Service runs against the target database before it reads the source. The Developer tool does not validate the SQL.
PostSQL	SQL command the Data Integration Service runs against the target database after it writes to the target. The Developer tool does not validate the SQL.

Creating a Customized Data Object

Create a customized data object to add to a mapping or mapplet. After you create a customized data object, add sources to it.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Data Object**.

   The **New** dialog box appears.
3. Select **Relational Data Object** and click **Next**.

   The **New Relational Data Object** dialog box appears.
4. Click **Browse** next to the Connection option and select a connection to the database.
5. Click **Create customized data object**.
6. Enter a name for the customized data object.
7. Click **Browse** next to the Location option and select the project where you want to create the customized data object.
8. Click **Finish**.

   The customized data object appears under Physical Data Objects in the project or folder in the **Object Explorer** view.

Add sources to the customized data object. You can add relational resources or relational data objects as sources. You can also use a custom SQL query to add sources.

Adding Relational Resources to a Customized Data Object

After you create a customized data object, add sources to it. You can use relational resources as sources.

Before you add relational resources to a customized data object, you must configure a connection to the database.

1. In the **Connection Explorer** view, select one or more relational resources in the same relational connection.
2. Right-click in the **Connection Explorer** view and select **Add to project**.

   The **Add to Project** dialog box appears.
3. Select **Add as related resource(s) to existing customized data object** and click **OK**.

The **Add to Data Object** dialog box appears.

4. Select the customized data object and click **OK**.
5. If you add multiple resources to the customized data object, the Developer tool prompts you to select the resource to write to. Select the resource and click **OK**.

   If you use the customized data object in a mapping as a write transformation, the Developer tool writes data to this resource.

The Developer tool adds the resources to the customized data object.

Adding Relational Data Objects to a Customized Data Object

After you create a customized data object, add sources to it. You can use relational data objects as sources.

1. Open the customized data object.
2. Select the **Read** view.
3. In the **Object Explorer** view, select one or more relational data objects in the same relational connection.
4. Drag the objects from the **Object Explorer** view to the customized data object **Read** view.
5. If you add multiple relational data objects to the customized data object, the Developer tool prompts you to select the object to write to. Select the object and click **OK**.

   If you use the customized data object in a mapping as a write transformation, the Developer tool writes data to this relational data object.

The Developer tool adds the relational data objects to the customized data object.

Creating Keys in a Customized Data Object

Create key columns to identify each row in a source transformation. You can create one primary key in each source transformation.

1. Open the customized data object.
2. Select the **Read** view.
3. Select the source transformation where you want to create a key.

   The source must be a relational resource, not a relational data object. If the source is a relational data object, you must create keys in the relational data object.
4. Select the **Keys** properties.
5. Click **Add**.

   The **New Key** dialog box appears.
6. Enter a key name.
7. If the key is a primary key, select **Primary Key**.
8. Select the key columns.
9. Click **OK**.
10. Save the customized data object.

Creating Relationships within a Customized Data Object

You can create key relationships between sources in a customized data object.

The source transformation that you reference must have a primary key.

1. Open the customized data object.
2. Select the **Read** view.
3. Select the source transformation where you want to create a foreign key.

   The source must be a relational resource, not a relational data object. If the source is a relational data object, you must create relationships in the relational data object.
4. Select the **Relationships** properties.
5. Click **Add**.

   The **New Relationship** dialog box appears.
6. Enter a name for the foreign key.
7. Select a primary key from the referenced source transformation.
8. Click **OK**.
9. In the **Relationships** properties, select the foreign key columns.
10. Save the customized data object.

Create or Replace Target Tables

In the Developer tool, you can generate a DDL script for one or more relational data objects in the Model repository, and run the DDL script to create or replace tables in the target database. If a target already exists in that database, you can drop the target and re-create it.

You can create or replace the target table at design-time or at run time. To create or replace tables at design-time, you must create and execute the DDL script before you run a mapping. You can configure a target schema strategy in the Write transformation in a mapping to create or replace a relational target at run time.

The Developer tool generates a database-specific version of the DDL script for all supported connection types. If you select a JDBC or ODBC target, the Developer tool generates the ANSI SQL-92 generic data type format. You can generate DDL scripts for the following database types:

- IBM DB2
- Greenplum
- Hive
- JDBC
- Microsoft SQL Server
- Netezza
- ODBC
- Oracle
- Teradata

Note: You cannot generate a DDL script if you use an OLE DB SQL Server connection.

Rules and Guidelines to Create or Replace Target Tables

Consider the following rules and guidelines when you generate and execute a DDL script:

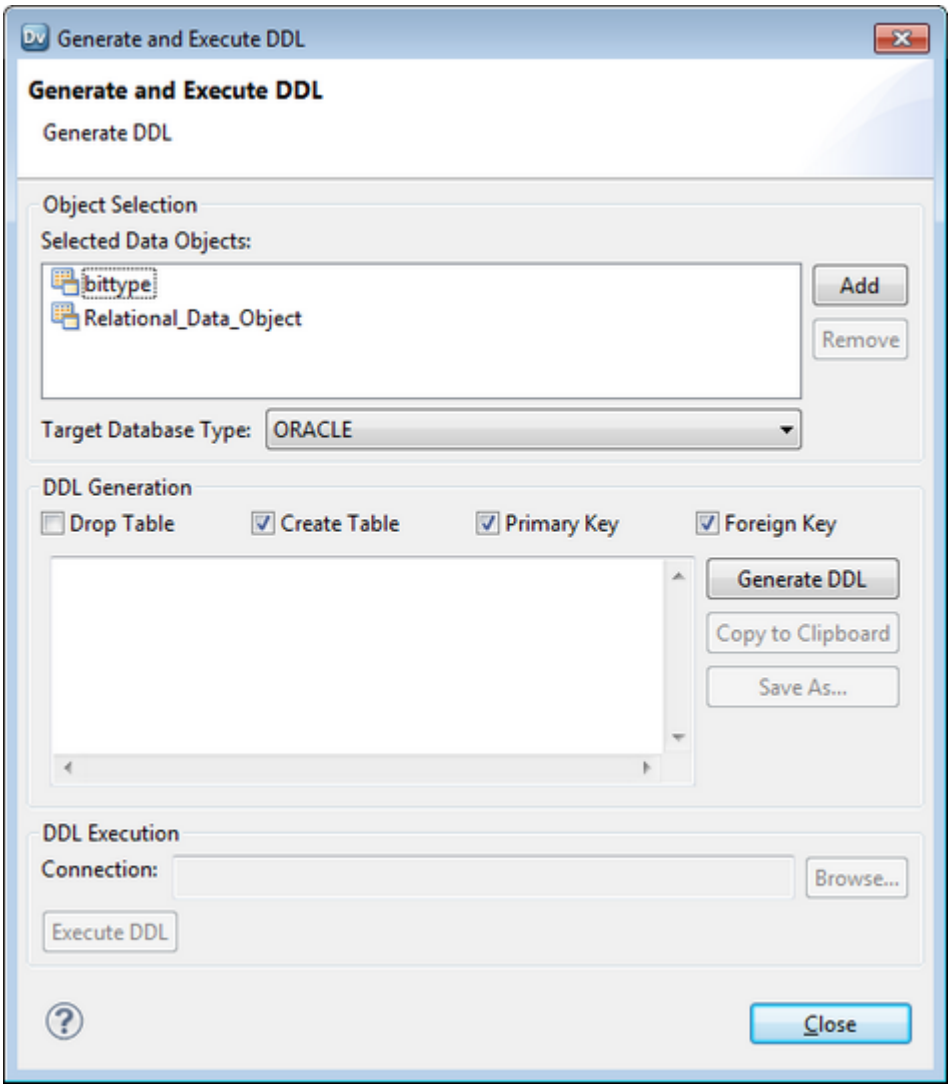
- Avoid selecting multiple objects with the same source table name. When you select multiple objects with the same source table name, the DDL code fails. If you select the Drop Table and the Create Table options for three objects with the same source table name, the DDL code succeeds for the first Drop Table and Create Table commands, and fails for the subsequent commands.
- Before you run a mapping, verify the data types and manually update them, if required. The char and byte semantics in Oracle are ignored when you generate the DDL script. When you create a table that contains char and varchar2 columns in an Oracle database, you can define the storage for char and byte semantics. When you import the metadata for the Oracle table into the Developer tool, the char and byte semantics are ignored. If you generate the DDL script for the Oracle table, the Developer tool defines the data type as Char.
- If you generate DDL for ODBC connections, the Developer tool creates the DDL script in ANSI SQL-92 generic data type format. The ANSI SQL-92 format might not run on all databases as the target database might not support the data type or data length.
- If you create a DDL script to migrate database tables from Greenplum to Netezza, you can incorporate only 16000 characters in the NVARCHAR column because the NVARCHAR data type supports only up to 16000 characters in a Netezza database.
- When you generate the DDL script, the Developer tool identifies the best data type match between the databases that the original connection and the new connection point to. The precision and scale for data types vary between databases. In an Oracle database, the default precision and scale for the Timestamp data type is (29, 9). When you generate the DDL script from Oracle to Microsoft SQL Server, the precision and scale for the Timestamp data type reduce to (26, 6). When you generate the DDL script from Oracle to DB2, the precision and scale for the Timestamp data type reduce to (27, 7).

Generating and Executing DDL at Design-Time

Before you generate and run the DDL script, ensure that the user has the appropriate read and write permissions to access the target database.

1. In the **Object Explorer** view of the Developer tool, select the relational data object for which you want to create a table in the target database. If you want to create multiple tables, hold down the **Shift** and **Ctrl** keys to select multiple data objects.
2. Right-click a selected data object and select **Generate and Execute DDL**.

   The Generate and Execute DDL dialog box appears.



3. In the Object Selection area, select the **Target Database Type** for which you want to generate DDL.
4. In the DDL Generation area, you can select the following options:
   - **Drop Table**. Drop a table in the database before you create it. Select this option to replace a table.
   - **Create Table**. Creates a table on the target database.
   - **Primary Key**. Creates primary keys based on the selected data objects.
   - **Foreign Key**. Creates foreign keys based on the selected data objects.
   - **Copy to Clipboard**. Copies the DDL script to the clipboard.
   - **Save As**. Saves the DDL script to a file.
5. In the DDL Generation area, click **Generate DDL**.

   The DDL generated for the selected data objects appears in the DDL Generation area.

   Warning: If you edit the DDL script generated in the DDL Generation area, you might encounter errors when you run the DDL script.
6. In the DDL Execution area, click **Browse** and choose a target database connection.

The **Choose Connection** dialog box lists the target database connections based on the target database type that you choose in the Object Selection area. For example, if you choose Oracle as the target database type, the Developer tool displays Oracle connections.

7. Click **Execute DDL**.
8. Click **Close**.

Generating and Executing DDL at Run Time

When you select a target schema strategy in a Write transformation in a mapping to create or replace a relational or Hive target at run time, you can define a DDL query.

The Data Integration Service creates or replaces relational and Hive target tables based on the DDL query that you define.

You can define a DDL query to customize the table or you can specify additional parameters such as partitions, based on which the Data Integration Service must create or replace the target table. The table contains the columns that you define in the DDL query. You can also enter placeholders in the DDL query. The Data Integration Service substitutes the placeholders with the actual values at run time.

For more information, see the *Informatica Developer Mapping Guide*.

DDL Generation Errors

When you generate and run a DDL script, you might encounter an error. The error can occur due to one of the following reasons:

- If the target database does not support the data type that you selected.
- If you select a physical data object that contains a cyclic dependency.
- If you select a physical data object that has no columns.

Custom Queries

A custom SQL query is a SELECT statement that overrides the default SQL query in a customized data object.

A custom query overrides the default SQL query that the Data Integration Service uses to read data from a relational source. The custom query also overrides the simple query settings that you define when you enter a source filter, use sorted ports, enter a user-defined join, or select distinct ports.

You can create a custom query to perform SQL operations that are valid in the database language but not available in the transformation language. When you define a custom query in a customized data object, you can reuse the object in multiple mappings.

Use the following guidelines when you create a custom query in a customized data object:

- In the SELECT statement, list the column names in the order in which they appear in the source transformation.
- Enclose all database reserved words in quotes.
- Add an escape character before a dollar sign ($). If the $ has preceding forward slash (\) characters, add an escape character (\) to both the forward slash and dollar sign characters, for example, enter $ as \$ and \$ as \\\$.

If you use a customized data object to perform a self-join, you must enter a custom SQL query that includes the self-join. You can use a customized data object with a custom query as a source in a mapping. The source database executes the query before it passes data to the Data Integration Service. You can create a custom query to add sources to an empty customized data object. You can also use a custom query to override the default SQL query.

Custom Query Optimization

The Data Integration Service can push a custom query to run in a relational data object for increased performance. Choose to push a custom query if the query forms a valid subquery for the database.

When you use a custom query to read data in a relational data object, the Data Integration Service can optimize the query by pushing the query to run in the database. The Data Integration Service can push the custom query if the query forms a valid subquery for the database. If the SQL syntax for the custom query is not valid in a subquery for the database, the resulting query fails to run.

If you push a custom query to a relational database other than IBM DB2, you must specify an alias for each expression in the select list that is not a column reference. The aliases allow the Data Integration Service to refer to the expressions in the select list.

See the database documentation for information about valid SQL syntax for aliases and subqueries.

Default Query

The Data Integration Service generates a default SQL query that it uses to read data from relational sources. You can override the default query in a customized data object or an instance of a relational data object.

You can override the default query through the simple or advanced query. Use the simple query to select distinct values, enter a source filter, sort ports, or enter a user-defined join. Use the advanced query to create a custom SQL query for reading data from the sources. The custom query overrides the default and simple queries.

If any table name or column name contains a database reserved word, you can create and maintain a reserved words file, reswords.txt. Create the reswords.txt file on any machine the Data Integration Service can access.

When the Data Integration Service runs a mapping, it searches for the reswords.txt file. If the file exists, the Data Integration Service places quotes around matching reserved words when it executes SQL against the database. If you override the default query, you must enclose any database reserved words in quotes.

When the Data Integration Service generates the default query, it delimits table and field names containing the following characters with double quotes:

```
/ + - = ~ ` ! % ^ & * ( ) [ ] { } ' ; ? , < > \ | <space>
```

Creating a Reserved Words File

Create a reserved words file if any table name or column name in the customized data object contains a database reserved word.

You must have administrator privileges to configure the Data Integration Service to use the reserved words file.

1. Create a file called "reswords.txt."
2. Create a section for each database by entering the database name within square brackets, for example, [Oracle].
3. Add the reserved words to the file below the database name.

For example:

```
[Oracle]
OPTION
START
where
number
[SQL Server]
CURRENT
where
number
```

Entries are not case sensitive.

4. Save the reswords.txt file.
5. In Informatica Administrator, select the Data Integration Service.
6. Edit the custom properties.
7. Add the following custom property:

Name	Value
Reserved Words File	<path>\reswords.txt

8. Restart the Data Integration Service.

Hints

You can add hints to the source SQL query to pass instructions to a database optimizer. The optimizer uses the hints to choose a query run plan to access the source.

The Hints field appears in the **Query** view of a relational data object instance or a customized data object. The source database must be Oracle, Sybase, IBM DB2, or Microsoft SQL Server. The Hints field does not appear for other database types.

When the Data Integration Service generates the source query, it adds the SQL hints to the query exactly as you enter them in the Developer tool. The Data Integration Service does not parse the hints. When you run the mapping that contains the source, the mapping log shows the query with the hints in the query.

The Data Integration Service inserts the SQL hints in a position in the query depending on the database type. Refer to your database documentation for information about the syntax for hints.

Oracle

The Data Integration Service add hints directly after the SELECT/UPDATE/INSERT/DELETE keyword.

```
SELECT /*+ <hints> */ FROM …
```

The '+' indicates the start of hints.

The hints are contained in a comment (/* ... */ or --... until end of line)

Sybase

The Data Integration Service adds hints after the query. Configure a plan name in the hint.

```
SELECT … PLAN <plan>
```

```
select avg(price) from titles plan "(scalar_agg (i_scan type_price_ix titles )"
```

IBM DB2

You can enter the optimize-for clause as a hint. The Data Integration Service adds the clause at the end of the query.

```
SELECT … OPTIMIZE FOR <n> ROWS
```

The optimize-for clause tells the database optimizer how many rows the query might process. The clause does not limit the number of rows. If the database processes more than <n> rows, then performance might decrease.

Microsoft SQL Server

The Data Integration Service adds hints at the end of the query as part of an OPTION clause.

```
SELECT … OPTION ( <query_hints> )
```

Hints Rules and Guidelines

Use the following rules and guidelines when you configure hints for SQL queries:

- If you enable pushdown optimization or if you use a semi-join in a relational data object, then the original source query changes. The Data Integration Service does not apply hints to the modified query.
- You can combine hints with join and filter overrides, but if you configure a SQL override, the SQL override takes precedence and the Data Integration Service does not apply the other overrides.
- The **Query** view shows a simple view or an advanced view. If you enter a hint with a filter, sort, or join override on the simple view, and you the Developer tool shows the full query override on the advanced view.

Creating Hints

Create hints to send instructions to the database optimizer to determine a query plan.

1. Open the customized data object or the relational data object instance.
2. Select the **Read** view.
3. Select the Output transformation.
4. Select the **Query** properties.
5. Select the simple query.
6. Click **Edit** next to the **Hints** field.

   The **Hints** dialog box appears.
7. Enter the hint in the **SQL Query** field.

   The Developer tool does not validate the hint.
8. Click **OK**.
9. Save the data object.

Select Distinct

You can select unique values from sources in a customized data object or a relational data object instance with the select distinct option. When you enable select distinct, the Data Integration Service adds a SELECT DISTINCT statement to the default SQL query.

Use the select distinct option to filter source data. For example, you might use the select distinct option to extract unique customer IDs from a table that lists total sales. When you use the relational data object in a mapping, the Data Integration Service filters data earlier in the data flow, which can increase performance.

Using Select Distinct

Select unique values from a relational source with the **Select Distinct** property.

1. Open the customized data object or relational data object instance.
2. Select the **Read** view.
3. Select the Output transformation.
4. Select the **Query** properties.
5. Select the simple query.
6. Enable the **Select Distinct** option.
7. Save the customized data object.

Filters

You can enter a filter value in a custom query. The filter becomes the WHERE clause in the query SELECT statement. Use a filter to reduce the number of rows that the Data Integration Service reads from the source table.

Entering a Source Filter

Enter a source filter to reduce the number of rows the Data Integration Service reads from the relational source.

1. Open the customized data object or the relational data object instance.
2. Select the **Read** view.
3. Select the Output transformation.
4. Select the **Query** properties.
5. Select the simple query.
6. Click **Edit** next to the **Filter** field.

   The **SQL Query** dialog box appears.
7. Enter the filter condition in the **SQL Query** field.

   You can select column names from the **Columns** list.
8. Click **OK**.
9. Click **Validate** to validate the filter condition.
10. Save the data object.

Sorted Ports

You can sort rows in the default query for a customized data object or a relational data object instance. Select the ports to sort by. The Data Integration Service adds the ports to the ORDER BY clause in the default query.

You might sort the source rows to increase performance when you include the following transformations in a mapping:

- Aggregator. When you configure an Aggregator transformation for sorted input, you can send sorted data by using sorted ports. The group by ports in the Aggregator transformation must match the order of the sorted ports in the customized data object.
- Joiner. When you configure a Joiner transformation for sorted input, you can send sorted data by using sorted ports. Configure the order of the sorted ports the same in each customized data object.

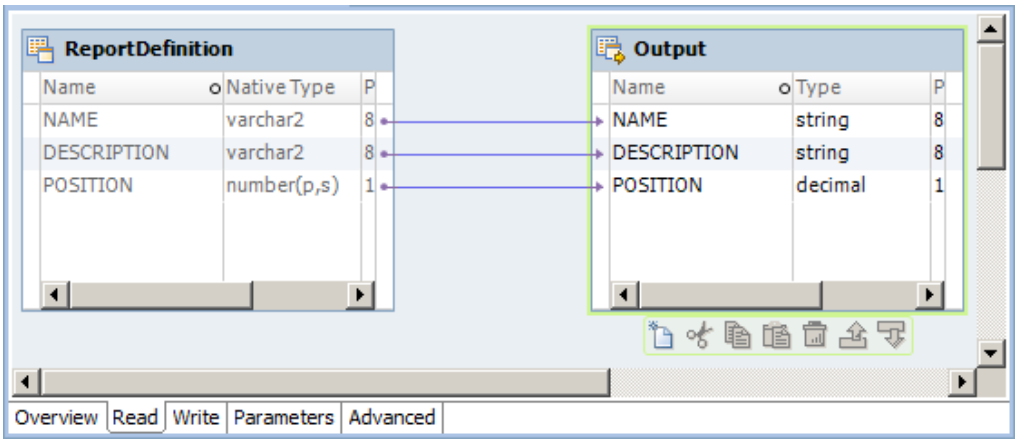
Note: You can also use the Sorter transformation to sort relational and flat file data before Aggregator and Joiner transformations.

Sorting Column Data

Use sorted ports to sort column data in a customized data object or relational data object instance. When you use the data object as a read transformation in a mapping or mapplet, you can pass the sorted data to transformations downstream from the read transformation.

1. Open the customized data object or relational data object instance.
2. Select the **Read** view.

   The following figure shows the **Read** view of a customized data object that is open in the editor:



3. Select the Output transformation.
4. Select the **Query** properties.
5. Select the simple query.
6. Click **Edit** next to the **Sort** field.

   The **Sort** dialog box appears.
7. To specify a column as a sorted port, click the **New** button.
8. Select the column and sort type, either ascending or descending.
9. Repeat steps 7 and 8 to select other columns to sort.

   The Developer tool sorts the columns in the order in which they appear in the **Sort** dialog box.

10. Click **OK**.

    In the **Query** properties, the Developer tool displays the sort columns in the **Sort** field.
11. Click **Validate** to validate the sort syntax.
12. Save the data object.

User-Defined Joins

You can configure a user-defined join in a customized data object or relational data object instance. A user-defined join defines the condition to join data from multiple sources in the same data object.

When you add a user-defined join to a customized data object or a relational data object instance, you can use the data object as a read transformation in a mapping. The source database performs the join before it passes data to the Data Integration Service. Mapping performance increases when the source tables are indexed.

Creat a user-defined join to join data from related sources. The user-defined join overrides the default inner join that the Data Integration creates based on the related keys in each source. When you enter a user-defined join, enter the contents of the WHERE clause that specifies the join condition. If the user-defined join performs an outer join, the Data Integration Service might insert the join syntax in the WHERE clause or the FROM clause, based on the database syntax.

You might need to enter a user-defined join in the following circumstances:

- Columns do not have a primary key-foreign key relationship.
- The datatypes of columns used for the join do not match.
- You want to specify a different type of join, such as an outer join.

Use the following guidelines when you enter a user-defined join in a customized data object or relational data object instance:

- Do not include the WHERE keyword in the user-defined join.
- Enclose all database reserved words in quotes.
- If you use Informatica join syntax, and **Enable quotes in SQL** is enabled for the connection, you must enter quotes around the table names and the column names if you enter them manually. If you select tables and columns when you enter the user-defined join, the Developer tool places quotes around the table names and the column names.

User-defined joins join data from related resources in a database. To join heterogeneous sources, use a Joiner transformation in a mapping that reads data from the sources. To perform a self-join, you must enter a custom SQL query that includes the self-join.

Entering a User-Defined Join

Configure a user-defined join in a customized data object or relational data object to define the join condition for the data object sources.

1. Open the customized data object or relational data object instance.
2. Select the **Read** view.
3. Select the Output transformation.
4. Select the **Query** properties.
5. Select the simple query.
6. Click **Edit** next to the **Join** field.

The **SQL Query** dialog box appears.

7. Enter the user-defined join in the **SQL Query** field.

   You can select column names from the **Columns** list.

8. Click **OK**.
9. Click **Validate** to validate the user-defined join.
10. Save the data object.

Outer Join Support

You can use a customized data object to perform an outer join of two sources in the same database. When the Data Integration Service performs an outer join, it returns all rows from one source resource and rows from the second source resource that match the join condition.

Use an outer join when you want to join two resources and return all rows from one of the resources. For example, you might perform an outer join when you want to join a table of registered customers with a monthly purchases table to determine registered customer activity. You can join the registered customer table with the monthly purchases table and return all rows in the registered customer table, including customers who did not make purchases in the last month. If you perform a normal join, the Data Integration Service returns only registered customers who made purchases during the month, and only purchases made by registered customers.

With an outer join, you can generate the same results as a master outer or detail outer join in the Joiner transformation. However, when you use an outer join, you reduce the number of rows in the data flow which can increase performance.

You can enter two kinds of outer joins:

- Left. The Data Integration Service returns all rows for the resource to the left of the join syntax and the rows from both resources that meet the join condition.
- Right. The Data Integration Service returns all rows for the resource to the right of the join syntax and the rows from both resources that meet the join condition.

Note: Use outer joins in nested query statements when you override the default query.

You can enter an outer join in a user-defined join or in a custom SQL query.

Informatica Join Syntax

When you enter join syntax, use the Informatica or database-specific join syntax. When you use the Informatica join syntax, the Data Integration Service translates the syntax and passes it to the source database during a mapping run.

Note: Always use database-specific syntax for join conditions.

When you use Informatica join syntax, enclose the entire join statement in braces ({Informatica syntax}). When you use database syntax, enter syntax supported by the source database without braces.

When you use Informatica join syntax, use table names to prefix column names. For example, if you have a column named FIRST_NAME in the REG_CUSTOMER table, enter “REG_CUSTOMER.FIRST_NAME” in the join syntax. Also, when you use an alias for a table name, use the alias within the Informatica join syntax to ensure the Data Integration Service recognizes the alias.

You can combine left outer or right outer joins with normal joins in a single data object. You cannot combine left and right outer joins. Use multiple normal joins and multiple left outer joins. Some databases limit you to using one right outer join.

When you combine joins, enter the normal joins first.

Normal Join Syntax

You can create a normal join using the join condition in a customized data object or relational data object instance.

When you create an outer join, you must override the default join. As a result, you must include the normal join in the join override. When you include a normal join in the join override, list the normal join before outer joins. You can enter multiple normal joins in the join override.

To create a normal join, use the following syntax:

```
{ source1 INNER JOIN source2 on join_condition }
```

The following table displays the syntax for normal joins in a join override:

Syntax	Description
source1	Source resource name. The Data Integration Service returns rows from this resource that match the join condition.
source2	Source resource name. The Data Integration Service returns rows from this resource that match the join condition.
join_condition	Condition for the join. Use syntax supported by the source database. You can combine multiple join conditions with the AND operator.

For example, you have a REG_CUSTOMER table with data for registered customers:

CUST_ID	FIRST_NAME	LAST_NAME
00001	Marvin	Chi
00002	Dinah	Jones
00003	John	Bowden
00004	J.	Marks

The PURCHASES table, refreshed monthly, contains the following data:

TRANSACTION_NO	CUST_ID	DATE	AMOUNT
06-2000-0001	00002	6/3/2000	55.79
06-2000-0002	00002	6/10/2000	104.45
06-2000-0003	00001	6/10/2000	255.56
06-2000-0004	00004	6/15/2000	534.95
06-2000-0005	00002	6/21/2000	98.65
06-2000-0006	NULL	6/23/2000	155.65
06-2000-0007	NULL	6/24/2000	325.45

To return rows displaying customer names for each transaction in the month of June, use the following syntax:

```
{ REG_CUSTOMER INNER JOIN PURCHASES on REG_CUSTOMER.CUST_ID = PURCHASES.CUST_ID }
```

The Data Integration Service returns the following data:

CUST_ID	DATE	AMOUNT	FIRST_NAME	LAST_NAME
00002	6/3/2000	55.79	Dinah	Jones
00002	6/10/2000	104.45	Dinah	Jones
00001	6/10/2000	255.56	Marvin	Chi
00004	6/15/2000	534.95	J.	Marks
00002	6/21/2000	98.65	Dinah	Jones

The Data Integration Service returns rows with matching customer IDs. It does not include customers who made no purchases in June. It also does not include purchases made by non-registered customers.

Left Outer Join Syntax

You can create a left outer join with a join override. You can enter multiple left outer joins in a single join override. When using left outer joins with other joins, list all left outer joins together, after any normal joins in the statement.

To create a left outer join, use the following syntax:

```
{ source1 LEFT OUTER JOIN source2 on join_condition }
```

The following tables displays syntax for left outer joins in a join override:

Syntax	Description
source1	Source resource name. With a left outer join, the Data Integration Service returns all rows in this resource.
source2	Source resource name. The Data Integration Service returns rows from this resource that match the join condition.
join_condition	Condition for the join. Use syntax supported by the source database. You can combine multiple join conditions with the AND operator.

For example, using the same REG_CUSTOMER and PURCHASES tables described in "Normal Join Syntax" on page 70, you can determine how many customers bought something in June with the following join override:

```
{ REG_CUSTOMER LEFT OUTER JOIN PURCHASES on REG_CUSTOMER.CUST_ID = PURCHASES.CUST_ID }
```

The Data Integration Service returns the following data:

CUST_ID	FIRST_NAME	LAST_NAME	DATE	AMOUNT
00001	Marvin	Chi	6/10/2000	255.56
00002	Dinah	Jones	6/3/2000	55.79
00003	John	Bowden	NULL	NULL

CUST_ID	FIRST_NAME	LAST_NAME	DATE	AMOUNT
00004	J.	Marks	6/15/2000	534.95
00002	Dinah	Jones	6/10/2000	104.45
00002	Dinah	Jones	6/21/2000	98.65

The Data Integration Service returns all registered customers in the REG_CUSTOMERS table, using null values for the customer who made no purchases in June. It does not include purchases made by non-registered customers.

Use multiple join conditions to determine how many registered customers spent more than $100.00 in a single purchase in June:

```
{REG_CUSTOMER LEFT OUTER JOIN PURCHASES on (REG_CUSTOMER.CUST_ID = PURCHASES.CUST_ID AND
PURCHASES.AMOUNT > 100.00) }
```

The Data Integration Service returns the following data:

CUST_ID	FIRST_NAME	LAST_NAME	DATE	AMOUNT
00001	Marvin	Chi	6/10/2000	255.56
00002	Dinah	Jones	6/10/2000	104.45
00003	John	Bowden	NULL	NULL
00004	J.	Marks	6/15/2000	534.95

You might use multiple left outer joins if you want to incorporate information about returns during the same time period. For example, the RETURNS table contains the following data:

CUST_ID	CUST_ID	RETURN
00002	6/10/2000	55.79
00002	6/21/2000	104.45

To determine how many customers made purchases and returns for the month of June, use two left outer joins:

```
{ REG_CUSTOMER LEFT OUTER JOIN PURCHASES on REG_CUSTOMER.CUST_ID = PURCHASES.CUST_ID
LEFT OUTER JOIN RETURNS on REG_CUSTOMER.CUST_ID = PURCHASES.CUST_ID }
```

The Data Integration Service returns the following data:

CUST_ID	FIRST_NAME	LAST_NAME	DATE	AMOUNT	RET_DATE	RETURN
00001	Marvin	Chi	6/10/2000	255.56	NULL	NULL
00002	Dinah	Jones	6/3/2000	55.79	NULL	NULL
00003	John	Bowden	NULL	NULL	NULL	NULL
00004	J.	Marks	6/15/2000	534.95	NULL	NULL
00002	Dinah	Jones	6/10/2000	104.45	NULL	NULL
00002	Dinah	Jones	6/21/2000	98.65	NULL	NULL

CUST_ID	FIRST_NAME	LAST_NAME	DATE	AMOUNT	RET_DATE	RETURN
00002	Dinah	Jones	NULL	NULL	6/10/2000	55.79
00002	Dinah	Jones	NULL	NULL	6/21/2000	104.45

The Data Integration Service uses NULLs for missing values.

Right Outer Join Syntax

You can create a right outer join with a join override. The right outer join returns the same results as a left outer join if you reverse the order of the resources in the join syntax. Use only one right outer join in a join override. If you want to create more than one right outer join, try reversing the order of the source resources and changing the join types to left outer joins.

When you use a right outer join with other joins, enter the right outer join at the end of the join override.

To create a right outer join, use the following syntax:

```
{ source1 RIGHT OUTER JOIN source2 on join_condition }
```

The following table displays syntax for a right outer join in a join override:

Syntax	Description
source1	Source resource name. The Data Integration Service returns rows from this resource that match the join condition.
source2	Source resource name. With a right outer join, the Data Integration Service returns all rows in this resource.
join_condition	Condition for the join. Use syntax supported by the source database. You can combine multiple join conditions with the AND operator.

PreSQL and PostSQL Commands

You can create SQL commands in a customized data object or relational data object instance. You can run the SQL commands to execute SQL statements such as insert, update, and delete. The Data Integration Service runs the SQL commands against the source relational resource.

When you run the mapping, the Data Integration Service runs PreSQL commands against the source database before it reads the source. You might want to use a PreSQL command if you have to update the source before you use the source in the mapping. For example, you can configure a PreSQL command to delete records in the source and update the source to load the latest records in the source database. When you run the mapping, the source that the mapping uses contains the latest records.

The Data Integration Service can also run PostSQL commands against the source database after it writes to the target. For example, you might want to use a PostSQL command to delete the records in the source after the records are loaded to the mapping target. You can configure a PostSQL command to delete the records after the mapping writes to the target.

Use the following guidelines when you configure PreSQL and PostSQL commands:

- Use any command that is valid for the database type. The Data Integration Service does not allow nested comments, even though the database might allow them.

- Use a semicolon (;) to separate multiple statements. The Data Integration Service issues a commit after each statement.
- The Data Integration Service ignores semicolons within /* ... */.
- If you need to use a semicolon outside comments, you can escape it with a backslash (\). When you escape the semicolon, the Data Integration Service ignores the backslash, and it does not use the semicolon as a statement separator.
- The Developer tool does not validate the SQL in a PreSQL and PostSQL commands.

Adding Pre- and Post-Mapping SQL Commands

You can add pre- and post-mapping SQL commands to a customized data object or relational data object instance. The Data Integration Service runs the SQL commands when you use the data object in a mapping.

1. Open the customized data object.
2. Select the **Read** view.
3. Select the Output transformation
4. Select the **Advanced** properties.
5. Enter a pre-mapping SQL command in the **PreSQL** field.
6. Enter a post-mapping SQL command in the **PostSQL** field.
7. Save the customized data object.

Creating a Custom Query

Create a custom query to issue a special SELECT statement for reading data from the sources. The custom query overrides the default query that the Data Integration Service issues to read source data.

1. Open the customized data object or the relational data object instance.
2. Select the **Read** view.
3. Select the Output transformation.
4. Select the **Query** properties.
5. Select the advanced query.
6. Select **Use custom query**.

   The Data Integration Service displays the query it issues to read source data.
7. Change the query or replace it with a custom query.
8. If you want to push the custom query to the relational data source, select **Push custom query to database**.

   The Data Integration Service does not the push custom query to the database by default. Choose to push the custom query if the query forms a valid subquery for the database.
9. Save the data object.

Nonrelational Data Objects

Import a nonrelational data object to use in a mapping or mapplet. A nonrelational data object is a physical data object that uses a nonrelational data source.

You can import nonrelational data objects for the following connection types:

- Adabas
- IMS
- Sequential
- VSAM

When you import a nonrelational data object, the Developer tool reads the metadata for the object from its PowerExchange data map. A data map associates nonrelational records with relational tables so that the product can use the SQL language to access the data. To create a data map, use the PowerExchange Navigator.

After you import the object, you can include its nonrelational operations as read, write, or lookup transformations in mappings and mapplets. Each nonrelational operation corresponds to a relational table that the data map defines. To view the mapping of fields in one or more nonrelational records to columns in the relational table, double-click the nonrelational operation in the **Object Explorer** view.

For more information about data maps, see the *PowerExchange Navigator Guide*.

Notes:

- When you run a mapping that includes an Update Strategy transformation and writes data to a nonrelational target, the mapping might fail with an error message. The message indicates that the target table has no keys, even though the data map for the target has keys defined.
- For nonrelational data objects with the Sequential connection type, you can parameterize the file name and default connection.

Importing Nonrelational Data Objects

Import one or more nonrelational data objects to use in a mapping or mapplet.

Before you import a nonrelational data object, you need to configure a connection to the database or data set. You also need to create a data map for the object.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Data Object**.
3. Select **Non-relational Data Object** and click **Next**.

   The **New Non-relational Data Object** dialog box appears.
4. Optionally, enter a name for the physical data object.
5. Click **Browse** next to the Connection option and select a connection.
6. Click **Browse** next to the Data Map option.

   The **Select a Data Map** dialog box appears.
7. Optionally, enter filter criteria.
8. In the **Resources** area, expand the connection and schemas as needed and select the desired data maps.

9. Click **OK** to close the **Select a Data Map** dialog box, and then click **Finish**.

   The nonrelational data objects and corresponding nonrelational operations appear under the connection name under **Physical Data Objects** in the project or folder in the **Object Explorer** view.

Note: You can also import nonrelational data objects by using the **Connection Explorer** view.

Creating a Read, Write, or Lookup Transformation from Nonrelational Data Operations

You can add a nonrelational data operation to a mapping or mapplet as a read, write, or lookup transformation.

1. Open the mapping or mapplet in which you want to create a read, write, or lookup transformation.
2. In the **Object Explorer** view, select one or more nonrelational data operations.
3. Drag the nonrelational data operations into the mapping editor.

   The **Add to Mapping** dialog box appears.
4. Select the **Read**, **Write**, or **Lookup** option.

   As independent data object(s) is automatically selected.
5. Click **OK**.

   The Developer tool creates a read, write, or lookup transformation for each nonrelational data operation.

WSDL Data Object

A WSDL data object is a physical data object that uses a WSDL file as a source. You can use a WSDL data object to create a web service or a Web Service Consumer transformation. Import a WSDL file to create a WSDL data object.

After you import a WSDL data object, you can edit general and advanced properties in the **Overview** and **Advanced** views. The **WSDL** view displays the WSDL file content.

Consider the following guidelines when you import a WSDL:

- The WSDL file must be WSDL 1.1 compliant.
- The WSDL file must be valid.
- Operations that you want to include in a web service or a Web Service Consumer transformation must use Document/Literal encoding. The WSDL import fails if all operations in the WSDL file use an encoding type other than Document/Literal.
- The Developer tool must be able to access any schema that the WSDL file references.
- If a WSDL file contains a schema or has an external schema, the Developer tool creates an embedded schema within the WSDL data object.
- If a WSDL file imports another WSDL file, the Developer tool combines both WSDLs to create the WSDL data object.
- If a WSDL file defines multiple operations, the Developer tool includes all operations in the WSDL data object. When you create a web service from a WSDL data object, you can choose to include one or more operations.

WSDL Data Object Overview View

The WSDL data object **Overview** view displays general information about the WSDL and operations in the WSDL.

The following table describes the general properties that you configure for a WSDL data object:

Property	Description
Name	Name of the WSDL data object.
Description	Description of the WSDL data object.

The following table describes the columns for operations defined in the WSDL data object:

Property	Description
Operation	The location where the WSDL defines the message format and protocol for the operation.
Input	The WSDL message name associated with the operation input.
Output	The WSDL message name associated with the operation output.
Fault	The WSDL message name associated with the operation fault.

WSDL Data Object Advanced View

The WSDL data object **Advanced** view displays advanced properties for a WSDL data object.

The following table describes the advanced properties for a WSDL data object:

Property	Description
Connection	Default web service connection for a Web Service Consumer transformation.
File Location	Location where the WSDL file exists.

Importing a WSDL Data Object

To create a web service from a WSDL or to create a Web Service Consumer transformation, import a WSDL data object. You can import a WSDL data object from a WSDL file or a URI that points to the WSDL location. You can import a WSDL data object from a WSDL file that contains either a SOAP 1.1 or SOAP 1.2 binding operation or both.

1. Click **File** > **New** > **Data Object.**
2. Select **WSDL data object** and click **Next**.

   The **New WSDL Data Object** dialog box appears.
3. Click **Browse** next to the **WSDL** option and enter the location of the WSDL. Then, click **OK**.

   When you enter the location of the WSDL, you can browse to the WSDL file or you can enter the URI to the WSDL.

Note: If the URI contains non-English characters, the import might fail. Copy the URI to the address bar in any browser. Copy the location back from the browser. The Developer tool accepts the encoded URI from the browser.

4. Enter a name for the WSDL.
5. Click **Browse** next to the **Location** option to select the project or folder location where you want to import the WSDL data object.
6. Click **Next** to view the operations in the WSDL.
7. Click **Finish**.

   The data object appears under **Physical Data Object** in the project or folder in the **Object Explorer** view.

WSDL Synchronization

You can synchronize a WSDL data object when the WSDL files change. When you synchronize a WSDL data object, the Developer tool reimports the object metadata from the WSDL files.

You can use a WSDL data object to create a web service or a Web Service Consumer transformation. When you update a WSDL data object, the Developer tool updates the objects that reference the WSDL and marks them as changed when you open them. When the Developer tool compares the new WSDL with the old WSDL, it identifies WSDL components through the name attributes.

If no name attribute changes, the Developer tool updates the objects that reference the WSDL components. For example, you edit a WSDL file and change the type for simple element "CustID" from xs:string to xs:integer. When you synchronize the WSDL data object, the Developer tool updates the element type in all web services and Web Service Consumer transformations that reference the CustID element.

If a name attribute changes, the Developer tool marks the objects that reference the WSDL component as changed when you open them. For example, you edit a WSDL and change the name of an element from "Resp" to "RespMsg." You then synchronize the WSDL. When you open a web service that references the element, the Developer tool marks the web service name in the editor with an asterisk to indicate that the web service contains changes. The Developer tool updates the element name in the web service, but it cannot determine how the new element maps to a port. If the Resp element was mapped to a port in the Input transformation or the Output transformation, you must map the RespMsg element to the appropriate port.

The Developer tool validates the WSDL files before it updates the WSDL data object. If the WSDL files contain errors, the Developer tool does not import the files.

Synchronizing a WSDL Data Object

Synchronize a WSDL data object when the WSDL files change.

1. Right-click the WSDL data object in the **Object Explorer** view, and select **Synchronize**.

   The **Synchronize WSDL Data Object** dialog box appears.
2. Click **Browse** next to the **WSDL** field, and enter the location of the WSDL. Then, click **OK**.

   When you enter the location of the WSDL, you can browse to the WSDL file or you can enter the URI to the WSDL.

   Note: If the URI contains non-English characters, the import might fail. Copy the URI to the address bar in any browser. Copy the location back from the browser. The Developer tool accepts the encoded URI from the browser.
3. Verify the WSDL name and location.
4. Click **Next** to view the operations in the WSDL.

5. Click **Finish**.

   The Developer tool updates the objects that reference the WSDL and marks them as changed when you open them.

Certificate Management

The Developer tool must use a certificate to import WSDL data objects and schema objects from a URL that requires client authentication.

By default, the Developer tool imports objects from URLs that require client authentication when the server that hosts the URL uses a trusted certificate. When the server that hosts the URL uses an untrusted certificate, add the untrusted certificate to the Developer tool. If you do not add the untrusted certificate to the Developer tool, the Developer tool cannot import the object. Request the certificate file and password from the server administrator for the URL that you want import objects from.

The certificates that you add to the Developer tool apply to imports that you perform on the Developer tool machine. The Developer tool does not store certificates in the Model repository.

Informatica Developer Certificate Properties

Add certificates to the Developer tool when you want to import objects from a URL that requires client authentication with an untrusted certificate.

The following table describes the certificate properties:

Property	Description
Host Name	Name of the server that hosts the URL.
Port Number	Port number of the URL.
Certificate File Path	Location of the client certificate file.
Password	Password for the client certificate file.

Adding Certificates to Informatica Developer

When you add a certificate, you configure the certificate properties that the Developer tool uses when you import objects from a URL that requires client authentication with an untrusted certificate.

1. Click **Window** > **Preferences**.
2. Select **Informatica** > **Web Services** > **Certificates**.
3. Click **Add**.
4. Configure the certificate properties.
5. Click **OK**.

Synchronization

You can synchronize physical data objects when their sources change. When you synchronize a physical data object, the Developer tool reimports the object metadata from the source you select.

You can synchronize all physical data objects. When you synchronize relational data objects or customized data objects, you can retain or overwrite the key relationships you define in the Developer tool.

You can configure a customized data object to be synchronized when its sources change. For example, a customized data object uses a relational data object as a source, and you add a column to the relational data object. The Developer tool adds the column to the customized data object. To synchronize a customized data object when its sources change, select the **Synchronize input and output** option in the **Overview** properties of the customized data object.

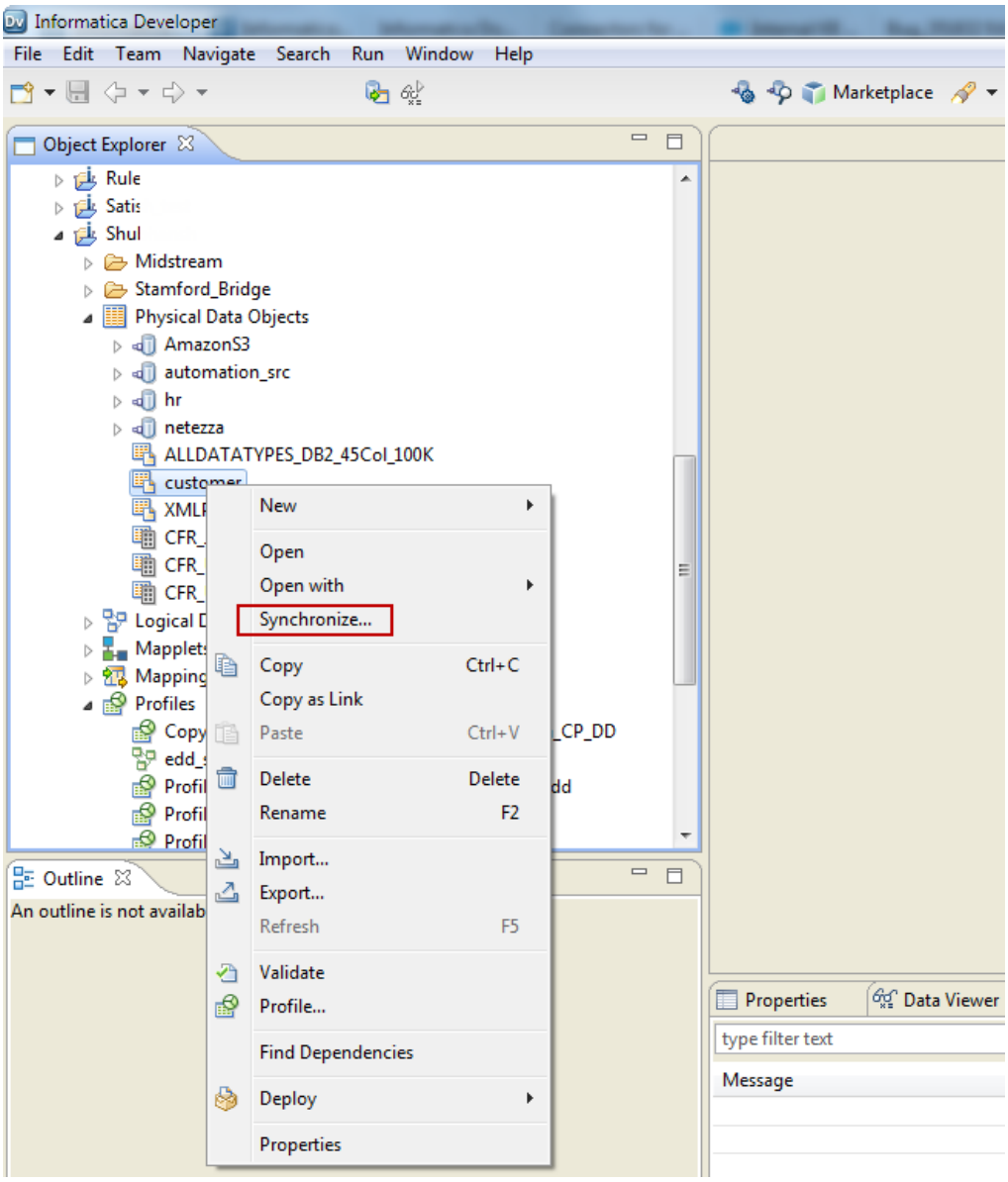
To synchronize any physical data object, right-click the object in the **Object Explorer** view, and select **Synchronize**.

Synchronizing a Flat File Data Object in Informatica Developer

You can synchronize the changes to an external flat file data source with its data object in Informatica Developer. Use the **Synchronize Flat File** wizard to synchronize the data objects.

1. In the **Object Explorer** view, select a flat file data object.
2. Right-click and select **Synchronize**.

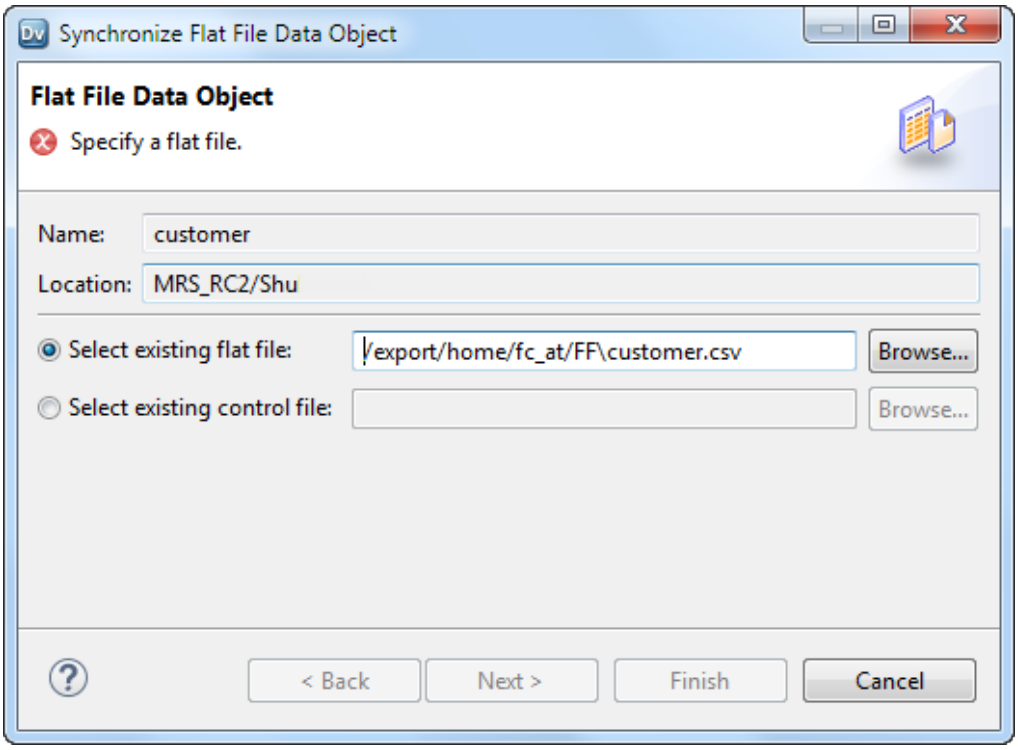
   The following image shows the Synchronize option for a data object:



The **Synchronize Flat File Data Object** wizard appears.

3. In the **Synchronize Flat File Data Object** wizard, verify the flat file path in the **Select existing flat file** field.

   The following image shows the Synchronize Flat File Data Object wizard:



4. Click **Next**.
5. Optionally, select the code page, format, delimited format properties, and column properties.
6. Click **Finish**, and then click **OK**.

Synchronizing a Relational Data Object in Informatica Developer

You can synchronize external data source changes of a relational data source with its data object in Informatica Developer. External data source changes include adding, changing, and removing columns, and changes to rules.

1. In the **Object Explorer** view, select a relational data object.
2. Right-click and select **Synchronize**.

   A message prompts you to confirm the action.
3. To complete the synchronization process, click **OK**.

   A synchronization process status message appears.
4. When you see a **Synchronization complete** message, click **OK**.

   The message displays a summary of the metadata changes made to the data object.

Troubleshooting Physical Data Objects

I am trying to preview a relational data object or a customized data object source transformation and the preview fails.

Verify that the resource owner name is correct.

When you import a relational resource, the Developer tool imports the owner name when the user name and schema from which the table is imported do not match. If the user name and schema from which the table is imported match, but the database default schema has a different name, preview fails because the Data Integration Service executes the preview query against the database default schema, where the table does not exist.

Update the relational data object or the source transformation and enter the correct resource owner name. The owner name appears in the relational data object or the source transformation **Advanced** properties.

CHAPTER 6

Flat File Data Objects

This chapter includes the following topics:

- Flat File Data Objects Overview, 84
- Generate the Source File Name, 85
- Flat File Data Object Overview Properties, 85
- Flat File Data Object Advanced Properties, 86
- Control File, 93
- Update Columns at Run Time, 94
- Generate Column Metadata from Control Files, 95
- Copying from Excel to a Flat File Data Object, 98
- Create a Flat File Data Object, 99

Flat File Data Objects Overview

Create a flat file physical data object to include in a mapping or mapplet. You can add flat file data objects to mappings and mapplets as Read, Write, and Lookup transformations.

A flat file data object can be delimited or fixed-width. You can create flat file data objects from fixed-width and delimited flat files that do not contain binary data.

You can configure a flat file data object to handle the changes in the flat file data source at run time. You can also generate column names for the flat file data object by using the information from the flat file or from the control file. The control file contains information on the column name, precision, scale, and the number of bytes to process.

After you create a flat file data object, use the following views to configure flat file properties:

Overview view

Configure the flat file data object name and description and update column properties.

Parameters view

Create parameters for the flat file data object.

Advanced view

Configure the format and run-time properties that the Data Integration Service uses when it reads data from and writes data to the flat file.

When you add flat file data objects to mappings as Read or Write transformations, you can view the format and run-time properties for the flat file data object in the **Properties** view. You cannot edit the flat file properties within a mapping.

Generate the Source File Name

You can add a file name column to the flat file data object. The file name column helps you identify the source file that contains a particular record of data. This is useful when the data comes from multiple sources.

You can configure the file name column in the Overview view of a flat file data object. The file name column is an optional column in the flat file data object. You can configure the mapping to write the source file name to each source row with the File Name Column port in the flat file data object. The file name column contains the fully qualified path and the file name.

The File Name Column port appears as the last column of the source data object. You can add only one File Name Column port in the source data object. When the port has the name as FileName and if you try to create a File Name Column port, the File Name Column port is named as FileName1.

For example, a mapping uses a source file that contains a list of files. The file names in the file list refers to the department names across the organization. For instance, the SYSA_Finance.txt file contains data from the Finance department. In the mapping, you can use string functions to extract the department name from the output of the file name column. You can use the extracted department name to process data differently for each department.

Flat File Data Object Overview Properties

The Data Integration Service uses overview properties when it reads data from or writes data to a flat file. Overview properties include general properties, which apply to the flat file data object. They also include column properties, which apply to the columns in the flat file data object. The Developer tool displays overview properties for flat files in the **Overview** view.

The following table describes the general properties that you configure for flat files:

Property	Description
Name	Name of the flat file data object.
Description	Description of the flat file data object.

The following table describes the column properties that you configure for flat files:

Property	Description
Name	Name of the column or the file name column port.
Native type	Native datatype of the column.

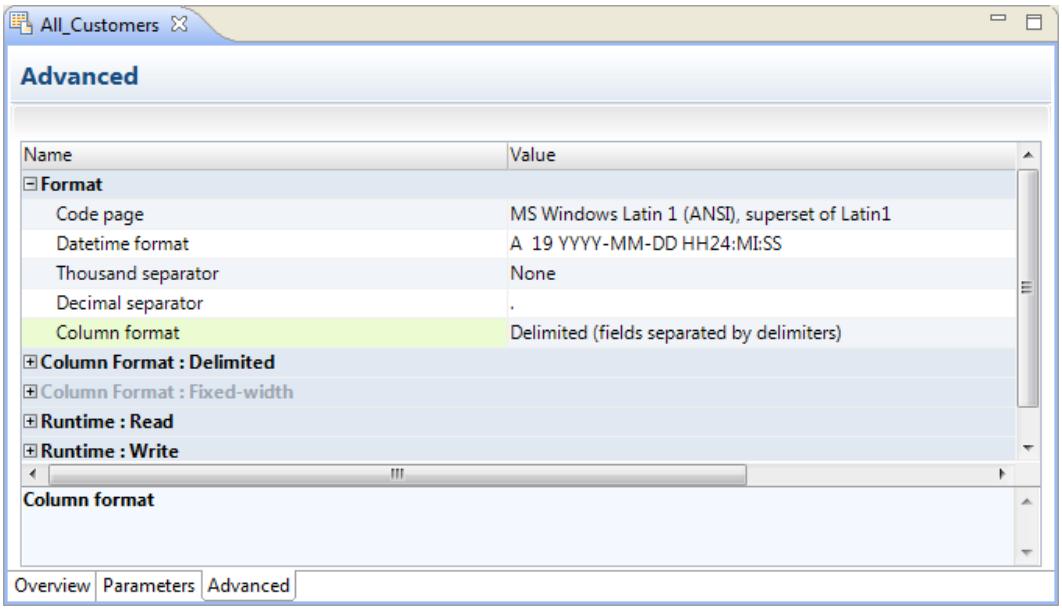
Property	Description
Bytes to process (fixed-width flat files)	Number of bytes that the Data Integration Service reads or writes for the column.
Precision	Maximum number of significant digits for numeric datatypes, or maximum number of characters for string datatypes. For numeric datatypes, precision includes scale.
Scale	Maximum number of digits after the decimal point for numeric values.
Format	Column format for numeric and datetime datatypes. For numeric datatypes, the format defines the thousand separator and decimal separator. Default is no thousand separator and a period (.) for the decimal separator. For datetime datatypes, the format defines the display format for year, month, day, and time. It also defines the field width. Default is "A 19 YYYY-MM-DD HH24:MI:SS."
Visibility	Determines whether the Data Integration Service can read data from or write data to the column. For example, when the visibility is Read, the Data Integration Service can read data from the column. It cannot write data to the column. For flat file data objects, this property is read-only. The visibility is always Read and Write.
Description	Description of the column.

Flat File Data Object Advanced Properties

The Data Integration Service uses advanced properties when it reads data from or writes data to a flat file. Advanced properties include format and run-time properties. The Developer tool displays advanced properties for flat files in the **Advanced** view.

The **Advanced** view contains property sections that you can collapse and expand. The column format sections that display depend on whether you configure a delimited or fixed-width column format.

The following image shows the property sections in the **Advanced** view:



Format Properties

The Developer tool displays format properties for flat file data objects in the **Format** section in the **Advanced** view.

The following table describes the format properties that you configure for flat file data objects:

Property	Description
Code page	Code page of the flat file data object. For source files, use a source code page that is a subset of the target code page. For lookup files, use a code page that is a superset of the source code page and a subset of the target code page. For target files, use a code page that is a superset of the source code page Default is "MS Windows Latin 1 (ANSI), superset of Latin 1."
Datetime format	Defines the display format and the field width for datetime values. Default is "A 19 YYYY-MM-DD HH24:MI:SS."
Thousand separator	Thousand separator for numeric values. Default is None.
Decimal separator	Decimal separator for numeric values. Default is a period (.).
Column Format	Format for the flat file, either delimited or fixed-width.

Column Format: Delimited Properties

When the flat file is delimited, the Developer tool displays delimited properties in the **Column Format: Delimited** section in the **Advanced** view.

The following table describes the delimited properties that you configure for flat file data objects:

Property	Description
Delimiters	Character used to separate columns of data. Click the Delimiters field to select a character or to assign a parameter to the property. Delimiters must be printable characters and must be different from the text qualifier and the escape character if selected. Default is Comma.
Text qualifier	Quote character that defines the boundaries of text strings. If you select a quote character, the Developer tool ignores delimiters within a pair of quotes. Default is No Quotes.
Start import at line	Row at which the Data Integration Service starts importing data. Use this option to skip header rows. Default is 1.
Row delimiter	Octal code for the character that separates rows of data. Default is line feed, \012 LF (\n). **Note:** The row delimiter applies to reading source data. When the Data Integration Service writes to a target file, it always uses the default delimiter, \n.
Escape character	Character used to escape a delimiter character in an unquoted string if the delimiter is the next character after the escape character. If you specify an escape character, the Data Integration Service reads the delimiter character as a regular character embedded in the string. **Note:** You can improve mapping performance slightly if the source file does not contain quotes or escape characters.
Retain escape character in data	Includes the escape character in the output string. Default is disabled.
Treat consecutive delimiters as one	Causes the Data Integration Service to treat one or more consecutive column delimiters as one. Otherwise, the Data Integration Service reads two consecutive delimiters as a null value. Default is disabled.

Column Format: Fixed-width Properties

When the flat file is fixed-width, the Developer tool displays fixed-width properties in the **Column Format: Fixed-width** section in the **Advanced** view.

The following table describes the fixed-width properties that you configure for flat file data objects:

Property	Description
Null character type	Null character type, either text or binary.
Null character value	Character used to represent a null value. The null character can be any valid character in the file code page or any binary value from 0 to 255.
Repeat null character	For source files, causes the Data Integration Service to read repeat null characters in a single field as one null value. For target files, causes the Data Integration Service to write as many null characters as possible into the target field. If you do not enable this option, the Data Integration Service enters one null character at the beginning of the field to represent a null value. Default is disabled.
Start import at line	Row at which the Data Integration Service starts importing data. Use this option to skip header rows. Default is 1.
Number of bytes to skip between records	Number of bytes between the last column of one row and the first column of the next. The Data Integration Service skips the entered number of bytes at the end of each row to avoid reading carriage return characters or line feed characters. Enter 1 for UNIX files and 2 for DOS files. Default is 2.
Line sequential	Causes the Data Integration Service to read a line feed character or carriage return character in the last column as the end of the column. Select this option if the file uses line feeds or carriage returns to shorten the last column of each row. Default is disabled.
Strip trailing blanks	Strips trailing blanks from string values. Default is disabled.
User defined shift state	Allows you to select the shift state for source columns in the Columns properties. Select this option when the source file contains both multibyte and single-byte data, but does not contain shift-in and shift-out keys. If a multibyte file source does not contain shift keys, you must select a shift key for each column in the flat file data object. Select the shift key for each column to enable the Data Integration Service to read each character correctly. Default is disabled.

Run-time: Read Properties

The Developer tool displays run-time properties for flat file sources in the **Run-time: Read section** in the **Advanced** view. The Data Integration Service uses this information when it reads data from a flat file.

The following table describes the read properties that you configure for flat file data objects:

Property	Description
Input type	Type of source input. You can choose the following types of source input: - File. For flat file sources. - Command. For source data or a file list generated by a shell command.
Source type	Indicates source type of files with the same file properties. You can choose one of the following source types: - Direct. A source file that contains the source data. - Indirect. A source file that contains a list of files. The Data Integration Service reads the file list and reads the files in sequential order. - Directory. Source files that are in a directory. You must specify the directory location in the source file directory property. The Data Integration Service reads the files in ascending alphabetic order. The Data Integration Service does not read files in the subdirectories.
Source file name	File name of the flat file source.
Source file directory	Directory where the flat file sources exist. The machine that hosts Informatica services must be able to access this directory. Default is the SourceDir system parameter.
Concurrent Read Partitioning	Order in which multiple partitions read input rows from a source file. If the Data Integration Service does not create partitions for the mapping, it ignores this value. Select one of the following options: - Optimize throughput. The Data Integration Service does not preserve input row order. - Keep relative order. The Data Integration Service preserves the input row order for the rows read by each partition. - Keep absolute order. The Data Integration Service preserves the input row order for all rows read by all partitions.
Connection Type	The type of connection. Select from the following options: - None. The source file does not require a connection. - Hadoop File System. The source file resides in HDFS. Default is None.
Command	Command used to generate the source file data. Use a command to generate or transform flat file data and send the standard output of the command to the flat file reader when the mapping runs. The flat file reader reads the standard output as the flat file source data. Generating source data with a command eliminates the need to stage a flat file source. Use a command or script to send source data directly to the Data Integration Service instead of using a pre-mapping command to generate a flat file source. You can also use a command to generate a file list. For example, to use a directory listing as a file list, use the following command: `cd MySourceFiles; ls sales-records-Sep-*-2005.dat`

Property	Description
Truncate string null	Strips the first null character and all characters after the first null character from string values. Enable this option for delimited flat files that contain null characters in strings. If you do not enable this option, the Data Integration Service generates a row error for any row that contains null characters in a string. Default is disabled.
Line sequential buffer length	Number of bytes that the Data Integration Service reads for each line. This property, together with the total row size, determines whether the Data Integration Service drops a row. If the row exceeds the larger of the line sequential buffer length or the total row size, the Data Integration Service drops the row and writes it to the mapping log file. To determine the total row size, add the column precision and the delimiters, and then multiply the total by the maximum bytes for each character. Default is 1024.
Generate Run-time Column Names	Determines how to generate the column metadata at run time. Select one of the following options: - Automatically (Field1, Field 2…). The Data Integration Service includes column metadata based on the position of the column in the data. The column metadata for the flat file is constructed at run time from the row that the user specifies in the **start from import line** attribute in the format properties of the flat file data object. - From data file header (first line). The first row of the flat file contains a header row that the Data Integration Service uses to determine the column metadata. - From control file. The Data Integration Service constructs column metadata based on the data in a control file, such as column name, precision, data type, scale, and number of bytes to process.
Control file name	Name of the control file. Required if you generate run-time column names from control file.
Control file directory	Directory where the control file exist. Required if you generate run-time column names from control file.
Default Field Type	Data type of the additional ports generated at run time.
Default Precision	Precision of the additional ports generated at run time.
Default Scale	Scale of the additional ports generated at run time.
Constraints	Conditional expression that the values on a data row must satisfy. Use the Expression editor to enter an expression that evaluates to TRUE. When the Data Integration Service reads constraints, it drops the rows that do not evaluate to TRUE. For example, a source flat file has an AGE column. You can set a constraint with AGE < 70 on the flat file data object. The Data Integration Service reads rows from the source flat file with the constraint AGE < 70. If the Data Integration Service reads rows with AGE >= 70, it drops those rows.

Run-time: Write Properties

The Developer tool displays run-time properties for flat file targets in the **Run-time: Write section** in the **Advanced** view. The Data Integration Service uses this information when it writes data to a flat file.

The following table describes the write properties that you configure for flat file data objects:

Property	Description
Append if exists	Appends the output data to the target files and reject files. If you do not select this option, the Data Integration Service truncates the target file and reject file before writing data to them. If the files do not exist, the Data Integration Service creates them. Default is disabled.
Create directory if not exists	Creates the target directory if it does not exist. Default is disabled.
Header options	Creates a header row in the file target. You can choose the following options: - No header. Does not create a header row in the flat file target. - Output field names. Creates a header row in the file target with the output port names . - Use header command output. Uses the command in the Header Command field to generate a header row. For example, you can use a command to add the date to a header row for the file target. Default is no header.
Header command	Command used to generate the header row in the file target.
Footer command	Command used to generate the footer row in the file target.
Output type	Type of target for the mapping. Select File to write the target data to a flat file. Select Command to output data to a command.
Command	Command used to process the target data. On UNIX, use any valid UNIX command or shell script. For example, use the following command to generate a compressed file from the target data on UNIX: `compress -c - > MyTargetFiles/MyCompressedFile.Z` On Windows, use any valid DOS command or batch file. The flat file writer sends the data to the command instead of a flat file target. For example, use cmd as the target command on Windows to avoid staging data in the file system and to avoid any security breaches. You can improve mapping performance by pushing transformation tasks to the command instead of the Data Integration Service. You can also use a command to sort or to compress target data.
Merge command	Merge command used to process merge data for all target partitions. The Data Integration Service must use a concurrent merge type for a command to process merge data. The command might not maintain the order of the target data.
Output file directory	Output directory for the flat file target. The machine that hosts Informatica services must be able to access this directory. Enter multiple directories separated by semicolons to increase performance when multiple partitions write to the flat file target. Default is the TargetDir system parameter.

Property	Description
Output file name	File name of the flat file target. If multiple partitions write to the flat file target and you choose not to merge target data, each partition writes to a separate output file named `<output_file_name><partition_number>.out`.
Merge type	Type of merge that the Data Integration Service performs on the data for partitioned targets. If the Data Integration Service does not create partitions for the target, it ignores this value. Select one of the following options: - No merge. The Data Integration Service concurrently writes the target output to a separate file for each partition. - Sequential. The Data Integration Service creates an output file for each partition and then merges them into a single merge file at the end of the mapping. - File list. The Data Integration Service creates a target file for each partition and creates a file list that contains the paths of the individual files. - Concurrent. The Data Integration Service concurrently writes the data for all target partitions to the merge file. Because the Data Integration Service writes to the merge file concurrently for all partitions, the sort order of the data in the merge file might not be sequential.
Merge file directory	Directory for the merge file for all target partitions. The machine that hosts Informatica services must be able to access this directory. Default is the TargetDir system parameter.
Merge file name	Name of the merge file for all target partitions. Default is the output file name.
Connection type	The type of connection. Select from the following options: - None. The target file does not require a connection. The target file location is specified by the output file directory. - Hadoop File System. The target file is in HDFS. Default is None.

Control File

The Data Integration Service can update the column metadata for the flat file based on a control file.

A control file is a simple text file with a field name, data type, precision, and scale. Each line in the control file contains one column of data.

To generate column names from a control file at run-time, select **From control file** for the **Generate Run-time Column Names** property in the Advanced properties of the flat file data object. You must also configure the Read transformation to get the column metadata at run time.

You can specify any of the following data types in the control file:

- Bigint
- Date/Time
- Decimal
- Double
- Integer

- String
- Text
- TimestampwithTZ

You can use a control file to generate run-time column names for a Read transformation based on a flat file data object or to create a flat file data object. The Data Integration Service constructs the column metadata for the flat file by using the information present in the control file. The column name, precision, data type, and scale attributes are comma delimited. The newline character delimits information across columns.

Update Columns at Run Time

You can configure a flat file data object to accommodate changes to source metadata at run time. The Data Integration Service can read data from a flat file where the number or order of the columns is different from that of the columns in the flat file physical data object.

For example, you have a mapping whose source is provided by another department. The department that provides the source cannot guarantee the order of columns in the source file. Sometimes, the department might change the columns in the file or contain additional columns. You can configure the flat file data object to accept changes in the source metadata at run time.

You can generate the run-time column names automatically from any row in the source, from the flat file header, or from the control file.

You can use input rules to project the run-time columns from the Read transformation based on the flat file data object to the next transformation in the mapping.

You can configure the Read transformation data object property to select **At runtime, get data object columns from the data source** on the Data Object tab. After you configure the property, the options in the flat file data object determine how the Data Integration Service processes the data.

Generate Run-time Column Names Automatically

The Data Integration Service can update the column metadata for the flat file automatically based on the column position.

The Data Integration Service dynamically updates the column metadata for the flat file based on the row that you specify to start the import in the format properties of the flat file. The default data type, precision, and scale is used for the column metadata for the run-time columns without changing the column position.

To generate column names automatically at run time, select **Automatically (Field1, Field 2...)** for the **Generate Run-time Column Names** property in the Advanced properties of the flat file data object. You must also configure the Read transformation to get the column metadata at run time.

For example, you want to configure a flat file data object to accept changes at run time. The data in the flat file determines the column metadata based on the column position. You have defined a flat file data object with the columns `Dept, Name, Place`. You want the Data Integration Service to dynamically construct the column metadata for the flat file at run time.

The input file contains the following information:

```
HR,Bob,Chicago,US,87675
Finance,Mary,California,US,65437,t567,4200
```

In the Format properties, you configured the flat file data object to start the import of data from line 1. Since there are five columns in the first row, the Data Integration Service processes the flat file data object with five

columns. The Data Integration Service creates additional run-time columns in the flat file data object with unique names based on position, Field4 and Field5. The Data Integration Service does not process the additional columns in the second row.

When you add a flat file data object to a mapping, configure the Read transformation to get column metadata at run time. At run time, the Data Integration Service constructs the flat file data object and process the rows in the flat file with the following values:

```
Dept     Name   Place       Field4  Field5
HR       Bob    Chicago     US      87675
Finance  Mary   California  US      65437
```

Generate Run-time Column Names From Data File Header

The Data Integration Service can use a header row to determine the column metadata. The first row of the flat file is the header.

The default data type, precision, and scale is used for the column metadata for the run-time columns and you can change the column position at run time.

To generate column names from a data file header at run-time, select **From data file header (first line)** for the **Generate Run-time Column Names** property in the Advanced properties of the flat file data object. You must also configure the Read transformation to get the column metadata at run time.

For example, you want to configure a flat file data object to accept changes at run time. The data in the flat file determines the column metadata based on the data file header.

You have defined a flat file data object with the columns `Name,Age,Dept`. You can configure the flat file data object advanced property to generate column names at run time from a data file header.

The data flat file contains the following information:

```
Dept    Name  Place       Country  Phone_No.
HR      Bob   Chicago     US       87675
Finance Mary  California  US       65437     t567  4200
```

When you add a flat file data object to a mapping, configure the Read transformation to get column metadata at run time. During run time, the Data Integration Service generates the flat file data object with the following columns: `Dept, Name, Place, Country` and `Phone_No`. Since `Age` column is not present in the flat file header row, the flat file data object constructed at run time does not have the `Age` column.

The Data Integration Service propagates the run-time columns, such as `Country` and `Phone Number` to the downstream transformation based on the mapping rules.

Generate Column Metadata from Control Files

Control files are simple text files based on which you can create flat file data objects. You can also use control files to update columns at run time for a Read transformation based on a flat file data object.

When the Data Integration Service generates columns at run time, it uses the decimal and thousands separators specified in the flat file data object properties. You must specify only the name of the column in the control file if the column exists in the flat file data object.

When you specify the source type as indirect in the run-time properties in the Advanced view of the flat file source, you can generate column names at run time with the control file.

You can create a control file for a delimited or for a fixed-width flat file.

You can parameterize the control file name and the control file directory in the run-time properties of the flat file data object.

The Data Integration Service processes a control file based on the column metadata that you specify in the control file and the input data.

When you generate column names from a control file at run time and the source data object contains the file name column, the Data Integration Service logs a validation error with the duplicate file names.

Control File Formats

Delimited and fixed-width flat files have different control file formats. You can use control files to generate column metadata at run time or to create a flat file data object.

Use one of the following formats to create a control file:

Delimited control file

Each line in a delimited control file has the following format:

```
[column name],<data type>,<precision>,<scale>
```

The following example shows a delimited flat file control file:

```
Dept,String,10
Name
Place,String,20
Country
Phone number,string,30
```

Fixed-width control file

Each line in a fixed-width control file has the following format:

```
[column name],<data type>,<precision>,<scale>,<number of bytes to process>
```

The following example shows a delimited flat file control file:

```
Dept,String,10
Name
Place,String,20
Country
Phone number,string,30
```

For a fixed-width source, the control file contains an additional column for the number of bytes to process. If you do not specify the number of bytes to process, the Data Integration Service uses the value specified for the precision as the number of bytes to process.

Parameterization of Run-time Properties

You can parameterize or specify values for the default scale, precision, and data type of the additional columns from the control file in the run-time properties of the flat file data object. You can parameterize the control file name and the control file directory in the run-time properties of the flat file data object. Configure the parameters on the Advanced tab of the physical data object properties. When you create the transformation from the physical data object, you can use mapping parameters to override the parameter default values.

Run-time Processing of Control Files

When you develop a mapping, you define data object read properties that determine how data is read from a flat file. The Data Integration Service can process the columns of the flat file data object that is based on a control file.

When the Data Integration Service constructs the column metadata based on the control file, it applies the following criteria for processing data:

The Data Integration Service applies default values for the column properties that you do not specify for the flat file data object

The Data Integration Service applies the default run-time properties of the flat file data object when you do not specify the run-time column properties in the control file. When you have additional columns in the control file without a data type, precision, or scale, the Data Integration Service uses the default data type, precision, and scale.

The Data Integration Service processes data as NULL for columns that do not appear in the control file but are present in the flat file data object

When the column in the flat file data object does not exist in the control file, the Data Integration Service processes the data as NULL during data preview. During run time, the Data Integration Service cannot process the column as that column does not exist in the control file.

The Data Integration Service fails mapping when the source metadata mismatches with a control file

The Data Integration Service fails to process the data when the source metadata based on the control file does not match the input data. For example, the Data Integration Service fails the mapping when the source contains a data type in the control file that is not compatible with the data type in the source.

Rules and Guidelines for Control Files

Consider the following guidelines when you use a control file:

- The control file must be accessible to the machine where the Data Integration Service runs.
- After you import a control file as a flat file data object, you cannot change the data type, precision, and scale of the columns in the data object. You can change the column position and column metadata for the additional columns from the control file.
- When you generate column names at run time based on the control file, the lookup cache reflects the metadata in the control file. If you update the same control file with additional columns after you run the mapping once, you must select the **Recache from Lookup source** property from the Runtime properties of the Lookup transformation.
- You cannot specify date format columns in the control file. The Data Integration Service uses the flat file date format from the flat file data object advanced properties in the Advanced view. If the flat file source contains multiple date formats, the Data Integration Service will use only the one date format specified in the Advanced view for all the dates.
- You cannot specify a binary data type in the control file. If the control file contains a TimestampwithTZ data type, ensure that the data type has the precision set to 36 and the scale set to 9 in the control file.

Copying from Excel to a Flat File Data Object

You can configure flat file properties in Excel and copy them into a flat file data object in the Developer tool. Flat file properties include the column name, native type, precision, and scale. You might want to do this when you need to develop or edit a flat file data object with many columns.

Note: You must confirm that the values in each cell are valid before copying the values to a flat file data object. For example, a string type cannot have a scale value other than "0." Precision values cannot be words and type values cannot be numbers. If your information is incorrect, you will receive an error message.

Editing Flat File Data Objects in Excel

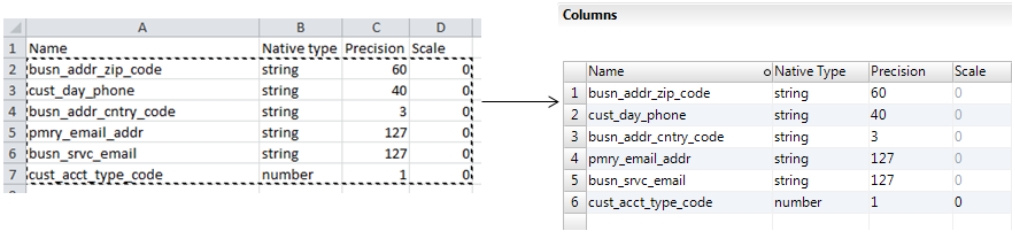
To edit a large portion of a flat file data object, you do not need to change every value in the Developer tool. Instead, you can copy the flat file columns to Excel, change all the values simultaneously using Auto Fill, and then **Paste (Replace)** the flat file back into the Developer tool.

1. To copy the metadata from the Developer tool, right-click within the flat file data object and click **Select All.**
2. Copy the metadata to an Excel spreadsheet.
3. Make changes within the Excel spreadsheet.
4. Copy the metadata from Excel.
5. To update the flat file data object with the changes, right-click within the flat file columns and click **Paste (Replace).**

Copying Metadata to a Flat File Data Object

You can create metadata in Excel and then copy it to a flat file data object in the Developer tool.

1. Create a mapping that includes a flat file data object.
2. Define metadata for a flat file data object in Excel.
3. Copy the metadata from Excel.
4. To move the metadata to the flat file data object, right-click within columns and click **Paste (Replace).** The following image shows a sample Excel table and the resulting flat file data object after you copy the metadata to the Developer tool:



	A	B	C	D
1	Name	Native type	Precision	Scale
2	busn_addr_zip_code	string	60	0
3	cust_day_phone	string	40	0
4	busn_addr_cntry_code	string	3	0
5	pmry_email_addr	string	127	0
6	busn_srvc_email	string	127	0
7	cust_acct_type_code	number	1	0

Columns

	Name	Native Type	Precision	Scale
1	busn_addr_zip_code	string	60	0
2	cust_day_phone	string	40	0
3	busn_addr_cntry_code	string	3	0
4	pmry_email_addr	string	127	0
5	busn_srvc_email	string	127	0
6	cust_acct_type_code	number	1	0

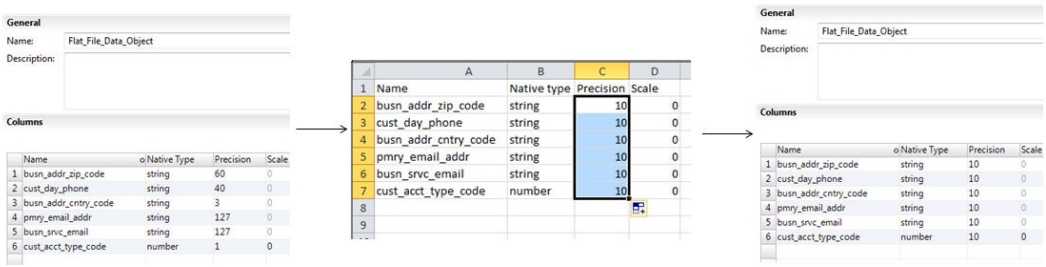
You can still make changes to the flat file data object after you copy the metadata to the Developer tool.

Example, Editing Data Object in Excel

You are developing a flat file data object and you need to change all the precision values to 10. Instead of changing each field individually, you make global changes through Excel.

You copy the metadata from the Developer tool into Excel, make changes to it, and copy it back to the flat file columns in the Developer tool. By using Excel, you avoid having to change each field individually.

The following image shows the process of moving a flat file to Excel, using Auto Fill to change certain values, and then copying the metadata back to the flat file data object in the Developer tool:



Create a Flat File Data Object

You can create a flat file data object as an empty data object, from an existing flat file, or from a control file. You can create a delimited or fixed-width flat file data object using any method.

When you create a flat file data object, you can create the data object in the following ways:

As an empty data object

Create an empty flat file data object when you want to define the data object columns and rows in the Developer tool.

From an existing flat file

Create a flat file data object from an existing flat file when you have a flat file that defines the metadata you want to include in the data object.

From a control file

Create a flat file data object from a control file when you want to define the data object columns and rows based on a control file.

Creating an Empty Flat File Data Object

Create an empty flat file data object when you want to define the data object columns and rows in the Developer tool.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Data Object**.
3. Select **Physical Data Objects** > **Flat File Data Object** and click **Next**.

   The **New Flat File Data Object** dialog box appears.
4. Select **Create as empty**.
5. Enter a name for the data object.
6. Optionally, click **Browse** to select a project or folder for the data object.
7. Click **Next**.
8. Select a code page that matches the code page of the data that you want to process.

9. Select **Delimited** or **Fixed-width**.
10. If you selected **Fixed-width**, click **Finish**. If you selected **Delimited**, click **Next**.
11. Configure the following delimited format properties:

Property	Description
Delimiters	Character used to separate columns of data. If you enter a delimiter that is the same as the escape character or the text qualifier, you might receive unexpected results.
Text Qualifier	Quote character that defines the boundaries of text strings. If you select a quote character, the Developer tool ignores delimiters within a pair of quotes.

12. Click **Finish**.

    The empty data object opens in the editor. Define the columns for the data object in the **Overview** view.

Creating a Flat File Data Object from an Existing Flat File

Create a flat file data object from an existing flat file when you have a flat file that defines the metadata you want to include in the data object.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Data Object**.
3. Select **Physical Data Objects** > **Flat File Data Object** and click **Next**.

   The **New Flat File Data Object** dialog box appears.
4. Select **Create from an existing flat file**.
5. Click **Browse** and navigate to the directory that contains the file.
6. Click **Open**.

   The wizard names the data object the same name as the file you selected.
7. Optionally, edit the data object name.
8. Optionally, click **Browse** to select a project or folder for the data object.
9. Click **Next**.
10. Select a code page that matches the code page of the data that you want to process.
11. Select **Delimited** or **Fixed-width**.
12. Optionally, edit the maximum number of rows to preview.
13. Click **Next**.
14. Configure the format properties, based on whether the flat file is delimited or fixed-width.

- For a delimited flat file, configure the following properties:

Property	Description
Delimiters	Character used to separate columns of data. If you enter a delimiter that is the same as the escape character or the text qualifier, you might receive unexpected results.
Text Qualifier	Quote character that defines the boundaries of text strings. If you select a quote character, the Developer tool ignores delimiters within pairs of quotes.
Import Column Names From First Line	If selected, the Developer tool uses data in the first row for column names. Select this option if column names appear in the first row. The Developer tool prefixes "FIELD_" to field names that are not valid.
Row Delimiter	Specify a line break character. Select from the list or enter a character. Preface an octal code with a backslash (\). To use a single character, enter the character. The Data Integration Service uses only the first character when the entry is not preceded by a backslash. The character must be a single-byte character, and no other character in the code page can contain that byte. Default is line-feed, \012 LF (\n).
Escape Character	Character immediately preceding a column delimiter character embedded in an unquoted string, or immediately preceding the quote character in a quoted string. When you specify an escape character, the Data Integration Service reads the delimiter character as a regular character.
Start Import at Line	Row number at which the Data Integration Service starts reading when it imports the file. For example, if you specify to start at the second row, the Developer tool skips the first row before reading.
Treat Consecutive Delimiters as One	If selected, the Data Integration Service reads one or more consecutive column delimiters as one. Otherwise, the Data Integration Service reads two consecutive delimiters as a null value.
Retain Escape Character in Data	Includes the escape character in the output string.

- For a fixed-width flat file, configure the following properties:

Property	Description
Import Column Names From First Line	If selected, the Developer tool uses data in the first row for column names. Select this option if column names appear in the first row.
Start Import at Line	Row number at which the Data Integration Service starts reading when it imports the file. For example, if you specify to start at the second row, the Developer tool skips the first row before reading.
Column breaks	Configures the column breaks in the fixed-width file. Click **Edit Breaks** to edit column breaks. Or, follow the directions in the wizard to manipulate the column breaks in the file preview window. You can move column breaks by dragging them. Or, double-click a column break to delete it.

15. Click **Next** to preview the flat file data object.
16. Click **Finish**.

    The data object opens in the editor.

Creating a Flat File Data Object from a Control File

Create a flat file data object from a control file when you want to define the data object columns and rows based on a control file.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Data Object**.
3. Select **Physical Data Objects** > **Flat File Data Object** and click **Next**.

   The **New Flat File Data Object** dialog box appears.
4. Select **Create from a control file**.
5. Click **Browse** and navigate to the directory that contains the control file.
6. Click **Open**.

   The wizard names the data object the same name as the control file you selected.
7. Optionally, edit the data object name.
8. Optionally, click **Browse** to select a project or folder for the data object.
9. Click **Next**.
10. Select a code page that matches the code page of the data that you want to process.
11. Select **Delimited** or **Fixed-width**.
12. If you selected **Fixed-width**, click **Finish**. If you selected **Delimited**, click **Next**.
13. Configure the following delimited format properties:

Property	Description
Delimiters	Character used to separate columns of data. If you enter a delimiter that is the same as the escape character or the text qualifier, you might receive unexpected results.
Text Qualifier	Quote character that defines the boundaries of text strings. If you select a quote character, the Developer tool ignores delimiters within a pair of quotes.

14. Click **Finish**.

    The data object opens in the editor.

CHAPTER 7

Logical View of Data

This chapter includes the following topics:

- Logical View of Data Overview, 103
- Developing a Logical View of Data, 104
- Logical Data Object Models, 105
- Logical Data Object Model Properties, 106
- Logical Data Objects, 113
- Logical Data Object Mappings, 117

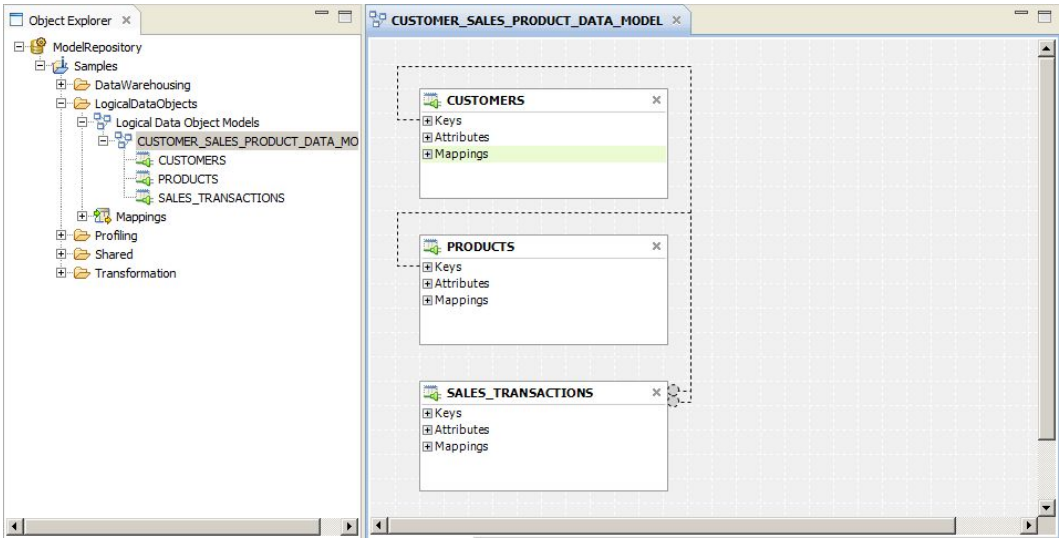
Logical View of Data Overview

A logical view of data is a representation of data that resides in an enterprise. A logical view of data includes a logical data model, logical data objects, and logical data object mappings.

With a logical view of data, you can achieve the following goals:

- Use common data models across an enterprise so that you do not have to redefine data to meet different business needs. It also means if there is a change in data attributes, you can apply this change one time and use one mapping to make this change to all databases that use this data.
- Find relevant sources of data and present the data in a single view. Data resides in various places in an enterprise, such as relational databases and flat files. You can access all data sources and present the data in one view.
- Expose logical data as relational tables to promote reuse.

The following figure shows a sample of related logical data objects:



Logical Data Object Model Example

Create a logical data object model to describe the representation of logical entities in an enterprise. For example, create a logical data object model to present account data from disparate sources in a single view.

American Bank acquires California Bank. After the acquisition, American Bank has the following goals:

- Present data from both banks in a business intelligence report, such as a report on the top 10 customers.
- Consolidate data from both banks into a central data warehouse.

Traditionally, American Bank would consolidate the data into a central data warehouse in a development environment, verify the data, and move the data warehouse to a production environment. This process might take several months or longer. The bank could then run business intelligence reports on the data warehouse in the production environment.

A developer at American Bank can use the Developer tool to create a model of customer, account, branch, and other data in the enterprise. The developer can link the relational sources of American Bank and California bank to a single view of the customer. The developer can then make the data available for business intelligence reports before creating a central data warehouse.

Developing a Logical View of Data

Develop a logical view of data to represent how an enterprise accesses data and uses data.

After you develop a logical view of data, you can add it to a data service to make virtual data available for end users.

Before you develop a logical view of data, you can define the physical data objects that you want to use in a logical data object mapping.

1. Create or import a logical data model.
2. Optionally, add logical data objects to the logical data object model and define relationships between objects.

3. Create a logical data object mapping to read data from a logical data object or write data to a logical data object. A logical data object mapping can contain transformation logic to transform the data.
4. View the output of the logical data object mapping.

Logical Data Object Models

A logical data object model describes the structure and use of data in an enterprise. The model contains logical data objects and defines relationships between them.

Define a logical data object model to create a unified model of data in an enterprise. The data in an enterprise might reside in multiple disparate source systems such as relational databases and flat files. A logical data object model represents the data from the perspective of the business regardless of the source systems. Create a logical data object model to study data, describe data attributes, and define the relationships among attributes.

For example, customer account data from American Bank resides in an Oracle database, and customer account data from California Banks resides in an IBM DB2 database. You want to create a unified model of customer accounts that defines the relationship between customers and accounts. Create a logical data object model to define the relationship.

You can import a logical data object model from a modeling tool. You can also import a logical data object model from an XSD file that you created in a modeling tool. Or, you can manually create a logical data object model in the Developer tool.

You add a logical data object model to a project or folder and store it in the Model repository.

To allow end users to run SQL queries against a logical data object, include it in an SQL data service. Make the logical data object the source for a virtual table. To allow end users to access a logical data object over the Web, include it in a web service. Make the logical data object the source for an operation.

Creating a Logical Data Object Model

Create a logical data object model to define the structure and use of data in an enterprise. When you create a logical data object model, you can add logical data objects. You associate a physical data object with each logical data object. The Developer tool creates a logical data object read mapping for each logical data object in the model.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Logical Data Object Model**.

   The **New** dialog box appears.
3. Select **Logical Data Object Model** and click **Next**.

   The **New Logical Data Object Model** dialog box appears.
4. Enter a name for the logical data object model.
5. To create logical data objects, click **Next**. To create an empty logical data object model, click **Finish**.

   If you click **Next**, the Developer tool prompts you to add logical data objects to the model.
6. To create a logical data object, click the **New** button.

   The Developer tool adds a logical data object to the list.
7. Enter a name in the **Name** column.

8. Optionally, click the **Open** button in the Data Object column to associate a physical data object with the logical data object.

   The **Select a Data Object** dialog box appears.
9. Select a physical data object and click **OK**.
10. Repeat steps 6 through 9 to add logical data objects.
11. Click **Finish**.

    The logical data object model opens in the editor.

Importing a Logical Data Object Model from a Modeling Tool

You can import a logical data object model from a modeling tool or an XSD file. Import a logical data object model to use an existing model of the structure and data in an enterprise.

1. Select the project or folder to which you want to import the logical data object model.
2. Click **File** > **New** > **Logical Data Object Model**.

   The **New Logical Data Object Model** dialog box appears.
3. Select **Logical Data Object Model from Data Model.**
4. Click **Next**.
5. In the Model Type field, select the modeling tool from which you want to import the logical data object model.
6. Enter a name for the logical data object model.
7. Click **Browse** to select the location of the logical data object model.
8. Click **Next**.
9. Browse to the file that you want to import, select the file, and click **Open**.
10. Configure the import properties.
11. Click **Next**.
12. Add logical data objects to the logical data object model.
13. Click **Finish**.

    The logical data objects appear in the editor.

Logical Data Object Model Properties

When you import a logical data object model from a modeling tool, provide the properties associated with the tool.

CA ERwin Data Modeler Import Properties

Configure the import properties when you import a logical data object model from CA ERwin Data Modeler.

The following table describes the properties to configure when you import a model from CA ERwin Data Modeler:

Property	Description
Import UDPs	Specifies how to import user-defined properties. Select one of the following options: - As metadata. Import an explicit value as the property value object. Explicit values are not exported. - As metadata, migrate default values. Import explicit and implicit values as property value objects. - In description, migrate default values. Append the property name and value, even if implicit, to the object description property. - Both, migrate default values. Import the UDP value, even if implicit, both as metadata and in the object's description. Default is As metadata.
Import relationship name	Specifies how to import the relationship names from ERwin. Select one of the following options: - From relationship name - From relationship description Default is From relationship name.
Import IDs	Specifies whether to set the unique ID of the object as the NativeId property.
Import subject areas	Specifies how to import the subject areas from ERwin. Select one of the following options: - As diagrams - As packages and diagrams - As packages and diagrams, assuming one subject area for each entity - Do not import subject areas Default is As diagrams.
Import column order form	Specifies how to import the position of columns in tables. Select one of the following options: - Column order. Order of the columns displayed in the ERwin physical view. - Physical order. Order of the columns in the database, as generated in the SQL DDL. Default is Physical order.
Import owner schemas	Specifies whether to import owner schemas.

IBM Cognos Business Intelligence Reporting - Framework Manager Import Properties

Configure the import properties when you import a logical data object model from IBM Cognos Business Intelligence Reporting - Framework Manager.

The following table describes the properties to configure when you import a model from IBM Cognos Business Intelligence Reporting - Framework Manager:

Property	Description
Folder Representation	Specifies how to represent folders from the Framework Manager. Select one of the following options: - Ignore. Ignore folders. - Flat. Represent folders as diagrams but do not preserve hierarchy. - Hierarchial. Represent folders as diagrams and preserve hierarchy. Default is Ignore.
Package Representation	Specifies how to represent packages from Cognos Framework Manager. Select one of the following options: - Ignore. Ignore subject areas. - Subject Areas. Represent packages as subject areas. - Model. Represent the package as the model. Default is Ignore.
Reverse engineer relationships	Specifes whether the Developer tool computes the relationship between two dbQueries as referential integrity constraints.
Tables design level	Specifies how to control the design level of the imported tables: Select one of the following options: - Logical and physical. The tables appear in both the logical view and in the physical view of the model. - Physical. The tables appear only in the physical view of the model. Default is Physical.
Ignore usage property	Specify whether the usage property of a queryItem should be used.

SAP BusinessObjects Designer Import Properties

Configure the import properties when you import a logical data object model from SAP BusinessObjects Designer.

The following table describes the properties to configure when you import a model from SAP BusinessObjects Designer:

Property	Description
System	Name of the BusinessObjects repository. For BusinessObjects versions 11.x and 12.x (XI), enter the name of the Central Management Server. For BusinessObjects version 5.x and 6.x, enter name of the repository defined by the Supervisor application
Authentication mode	Login authentication mode. This parameter is applicable to SAP BusinessObjects Designer 11.0 and later. Select one of the following authentication modes: - Enterprise. Business Objects Enterprise login - LDAP. LDAP server authentication - Windows AD. Windows Active Directory server authentication - Windows NT. Windows NT domain server authentication - Standalone. Standalone authentication Default is Enterprise.
User name	User name in the BusinessObjects server. For version 11.x and 12.x (XI), you need to be a member of BusinessObjects groups.
Password	Password for the BusinessObjects server.
Silent execution	Specifies whether to execute in interactive or silent mode. Default is Silent.
Close after execution	Specify whether to close BusinessObjects after the Developer Tool completes the model import.
Table design level	Specifies the design level of the imported tables. Select one of the following options: - Logical and physical. The tables appear both in the logical view and in the physical view of the model. - Physical. The tables appear both in the physical view of the model. Default is Physical.
Transform Joins to Foreign Keys	Transforms simple SQL joins in the model into foreign key relationships. Select the parameter if you want to export the model to a tool that only supports structural relational metadata, such as a database design tool.

Property	Description
Class representation	Specifies how to import the tree structure of classes and sub-classes. The Developer Tool imports each class as a dimension as defined by the CWM OLAP standard. The Developer Tool also imports classes and sub-classes as a tree of packages as defined by the CWM and UML standards. Select one of the following options: - As a flat structure. The Developer tool does not create package. - As a simplified tree structure. The Developer tool creates package for each class with sub-classes. - As a full tree structure. The Developer tool creates a package for each class. Default is As a flat structure.
Include List of Values	Controls how the Developer tool imports the list of values associated with objects.
Dimensional properties transformation	Specifies how to transfer the dimension name, description, and role to the underlying table and the attribute name, description, and datatype to the underlying column. Select one of the following options: - Disabled. No property transfer occurs. - Enabled. Property transfer occurs where there are direct matches between the dimensional objects and the relational objects. The Developer tool migrates the dimension names to the relational names. - Enabled (preserve names). Property transfer occurs where there are direct matches between the dimensional objects and the relational objects. The Developer tool preserves the relational names. Default is Disabled.

SAP PowerDesigner CDM Import Properties

Configure the import properties when you import a logical data object model from SAP PowerDesigner CDM.

The following table describes the properties to configure when you import a model from SAP PowerDesigner CDM:

Property	Description
Import UDPs	Specifies how to import user-defined properties. Select one of the following options: - As metadata. Import an explicit value as the property value object. Explicit values are not exported. - As metadata, migrate default values. Import explicit and implicit values as property value objects. - In description, migrate default values. Append the property name and value, even if implicit, to the object description property. - Both, migrate default values. Import the UDP value, even if implicit, both as metadata and in the object's description. Default is As metadata.
Import Association Classes	Specifies whether the Developer tool should import association classes.
Import IDs	Specifies whether to set the unique ID of the object as the NativeId property.

Property	Description
Append volumetric information to the description field	Import and append the number of occurrences information to the description property.
Remove text formatting	Specifies whether to remove or keep rich text formatting. Select this option if the model was generated by PowerDesigner 7.0 or 7.5 Clear this option if the model was generated by PowerDesigner 8.0 or greater.

SAP PowerDesigner PDM Import Properties

Configure the import properties when you import a logical data object model from SAP PowerDesigner PDM.

The following table describes the properties to configure when you import a model from SAP PowerDesigner PDM:

Property	Description
Import UDPs	Specifies how to import user-defined properties. Select one of the following options: - As metadata. Import an explicit value as the property value object. Explicit values are not exported. - As metadata, migrate default values. Import explicit and implicit values as property value objects. - In description, migrate default values. Append the property name and value, even if implicit, to the object description property. - Both, migrate default values. Import the UDP value, even if implicit, both as metadata and in the object's description. Default is As metadata.
Import IDs	Specifies whether to set the unique id of the object as the NativeId property.
Append volumetric information to the description field	Import and append the number of occurrences information to the description property.
Remove text formatting	Specifies whether to remove or keep rich text formatting. Select this option if the model was generated by PowerDesigner 7.0 or 7.5 Clear this option if the model was generated by PowerDesigner 8.0 or greater.

XSD Import Properties

You can import logical data object models from an XSD file exported by a modeling tool.

The following table describes the properties to configure when you import a model from an XSD file:

Property	Description
Elements content name	Attribute to hold the textual content like #PCDATA in the XSD file. Default is As metadata.
Collapse Level	Specifies when to collapse a class. The value you select determines whether the Developer tool imports all or some of the elements and attributes in the XSD file. Select one of the following options: - None. Every XSD element becomes a class and every XSD attribute becomes an attribute. - Empty. Only empty classes collapse into the parent classes. - Single Attribute. Only XSD elements with a single attribute and no children collapse into the parent class. - No Children. Any XSD element that has no child element collapse into the parent class. - All. All collapsible XSD elements collapse into the parent class. Default is All.
Collapse Star	Specifies whether the Developer tool should collapase XML elements with an incoming xlink into the parent class.
Class Type	Specifies whether the Developer tool should create a class type an element collapses into the parent element.
Any	Specifies whether to create a class or entity for the 'xs:any' pseudo-element.
Generate IDs	Specifies whether to generate additional attributes to create primary and foreign keys. By default, the Developer tool does not generate additional attributes.
Import substitutionGroup as	Specifies how to represent inheritance. Select one of the following options: - Generalization. Represents inheritance as generalization. - Roll down. Duplicate inherited attributes in the subclass. Default is Roll down.
Include Path	Path to the directory that contains the included schema files, if any.
UDP namespace	Namespace that contains attributes to be imported as user-defined properties.

Logical Data Objects

A logical data object is an object in a logical data object model that describes a logical entity in an enterprise. It has attributes, keys, and it describes relationships between attributes.

You include logical data objects that relate to each other in a data object model. For example, the logical data objects Customer and Account appear in a logical data object model for a national bank. The logical data object model describes the relationship between customers and accounts.

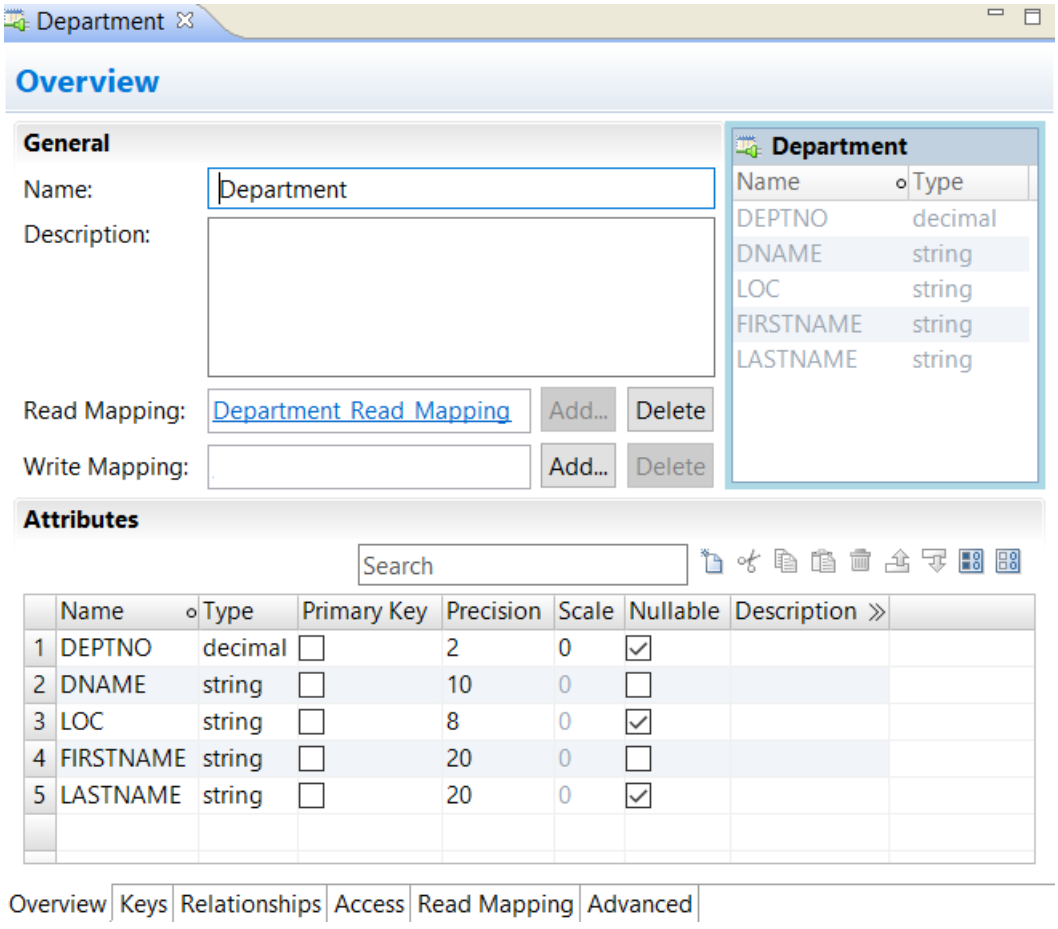
In the model, the logical data object Account includes the attribute Account_Number. Account_Number is a primary key, because it uniquely identifies an account. Account has a relationship with the logical data object Customer, because the Customer data object needs to reference the account for each customer.

You can drag a physical data object into the logical data object model editor to create a logical data object. Or, you can create a logical data object and define the attributes and keys.

Logical Data Object Properties

A logical data object contains properties that define the data object and its relationship to other logical data objects in a logical data object model.

The logical data object properties are in the editor and on editor tabs. The following image shows the logical data object editor:



The following table describes information that appears in the logical data object editor:

Tab Name	Description
Overview	The General area includes the object name, description, and read and write mapping if applicable, of the logical data object. The Attributes area displays the structure of data in a logical data object.
Keys	One or more attributes in a logical data object can be primary keys or unique keys.
Relationships	Associations between logical data objects.
Access	Type of access for a logical data object and each attribute of the data object.
Read Mapping	Logical data object read mapping associated with a logical data object. If the logical data object contains no read mapping, the Read Mapping tab is not visible.
Write Mapping	Logical data object write mapping associated with a logical data object. If the logical data object contains no write mapping, the Write Mapping tab is not visible.

Attribute Relationships

A relationship is an association between primary or foreign key attributes of one or more logical data objects.

You can define the following types of relationship between attributes:

Identifying

A relationship between two attributes where an attribute is identified through its association with another attribute.

For example, the relationship between the Branch_ID attribute of the logical data object Branch and the Branch_Location attribute of the logical data object Customer is identifying. This is because a branch ID is unique to a branch location.

Non-Identifying

A relationship between two attributes that identifies an attribute independently of the other attribute.

For example, the relationship between the Account_Type attribute of the Account logical data object and the Account_Number attribute of the Customer logical data object is non-identifying. This is because you can identify an account type without having to associate it with an account number.

When you define relationships, the logical data object model indicates an identifying relationship as a solid line between attributes. It indicates a non-identifying relationship as a dotted line between attributes.

Creating a Logical Data Object

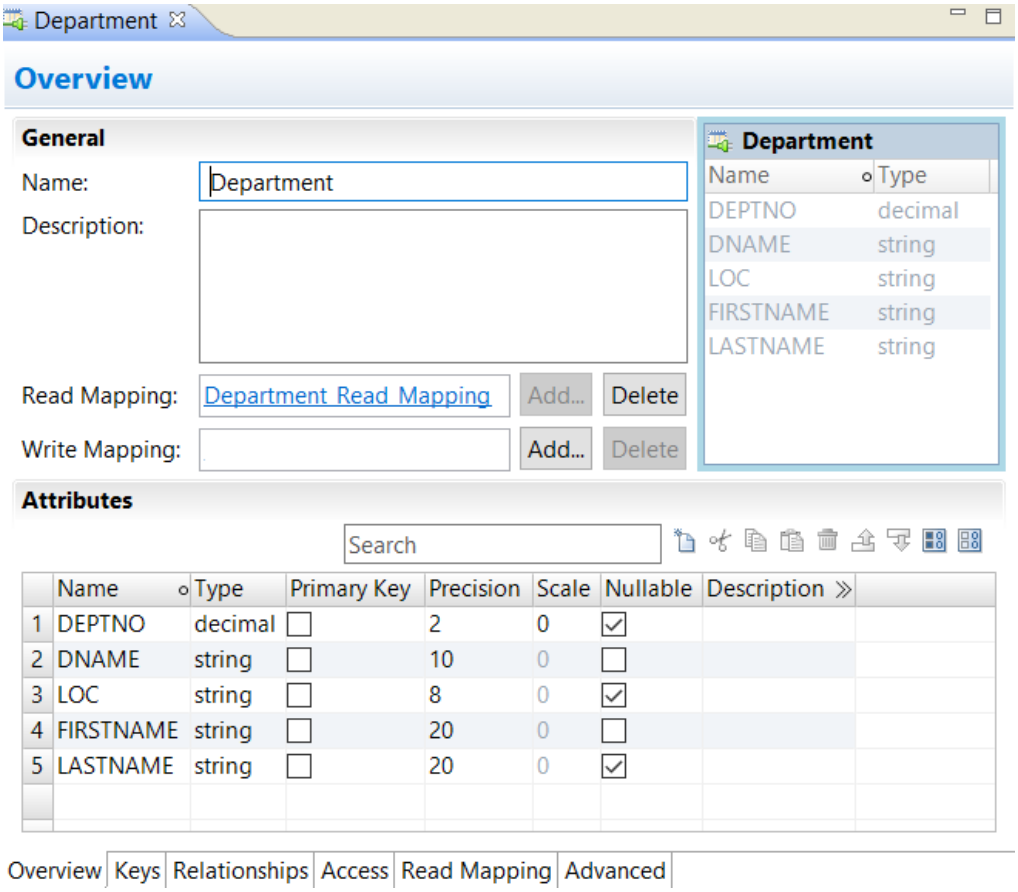
You can create a logical data object in a logical data object model to define a logical entity in an enterprise.

1. Click **File** > **New** > **Logical Data Object**.
2. Enter a logical data object name.
3. Select the logical data object model for the logical data object and click **Finish**.

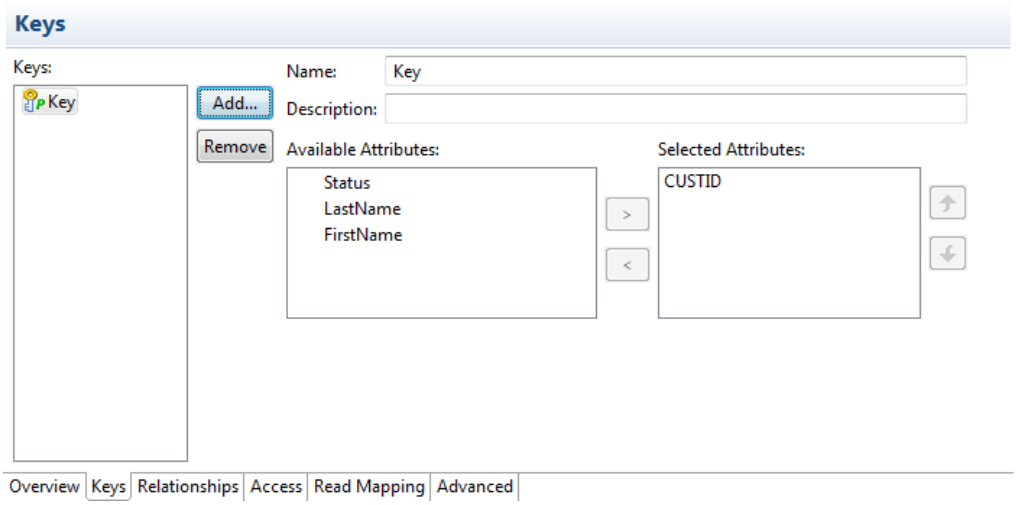
   The logical data object appears in the logical data object model editor.
4. Select the logical data object and click the **Properties** view.

5. On the **Overview** tab in the General area, you can optionally edit the logical data object name and description.
6. On the **Overview** tab in the Attributes area, you can create attributes and specify their datatype and precision.

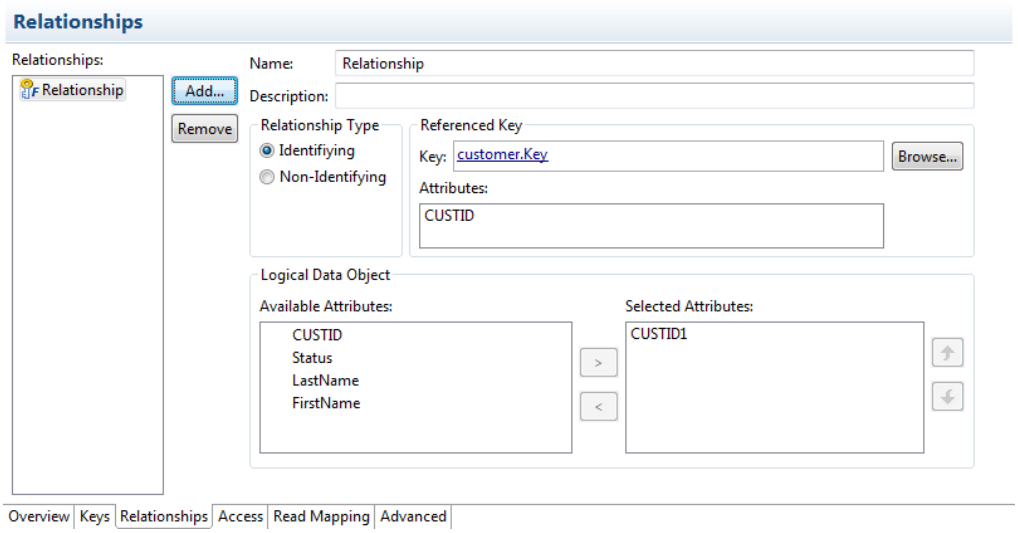
   You can use the options in the toolbar to update the attribute values. The available options are New, Cut, Copy, Paste, Delete, Move Up, Move Down, Enable All Nullable, and Disable All Nullable. You can select or clear all nullable values of the attributes using Enable All Nullable or Disable All Nullable icons.



7. On the **Keys** tab, optionally specify primary and unique keys for the data object.

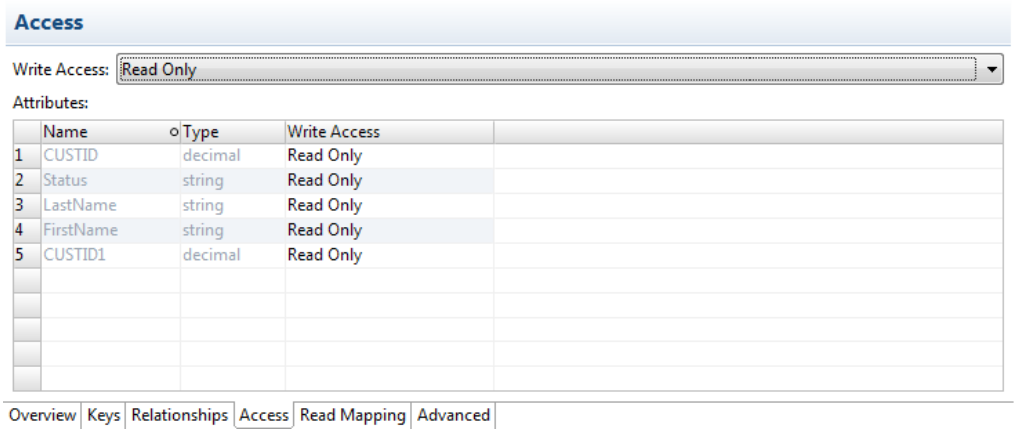


8. On the **Relationships** tab, optionally create relationships between logical data objects.

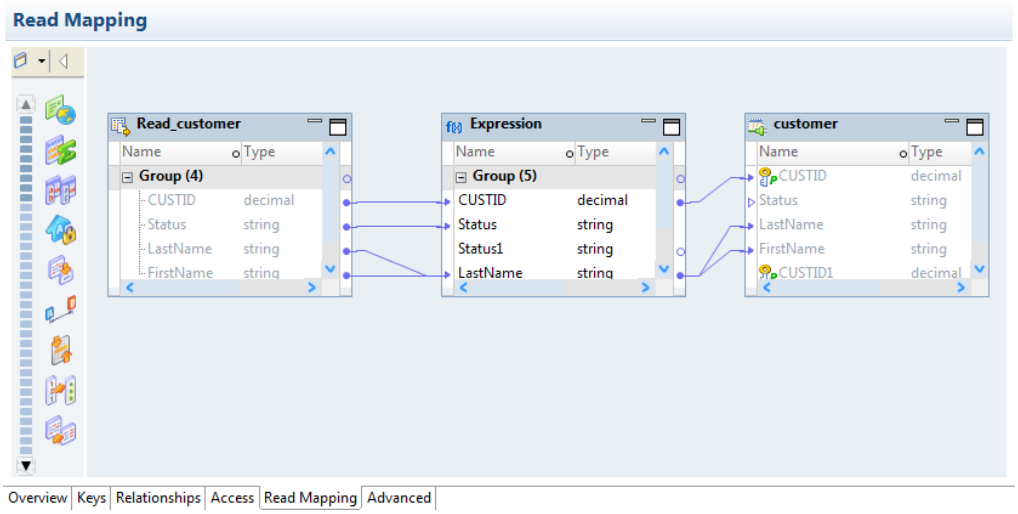


9. On the **Access** tab, optionally edit the type of access for the logical data object and each attribute in the data object.

Default is read only.



10. On the **Read Mapping** tab, optionally create a logical data object read mapping.



11. On the **Write Mapping** tab, optionally create a logical data object write mapping.
12. Save the logical data object.

Logical Data Object Mappings

A logical data object mapping is a mapping that links a logical data object to one or more physical data objects. It can include transformation logic.

A logical data object mapping can be of the following types:

- Read
- Write

You can associate each logical data object with one logical data object read mapping or one logical data object write mapping.

Logical Data Object Read Mappings

A logical data object read mapping contains one or more physical data objects as input and one logical data object as output. The mapping can contain transformation logic to transform the data.

It provides a way to access data without accessing the underlying data source. It also provides a way to have a single view of data coming from more than one source.

For example, American Bank has a logical data object model for customer accounts. The logical data object model contains a Customers logical data object.

American Bank wants to view customer data from two relational databases in the Customers logical data object. You can use a logical data object read mapping to perform this task and view the output in the **Data Viewer** view.

Logical Data Object Write Mappings

A logical data object write mapping contains a logical data object as input. It provides a way to write to targets from a logical data object.

The mapping can contain transformation logic to transform the data. The mapping runs without accessing the underlying data target. It provides a single view of the transformed data without writing to the target.

Creating a Logical Data Object Mapping

You can create a logical data object mapping to link data from a physical data object to a logical data object and transform the data.

1. In the **Object Explorer** view, double-click the logical data object that you want to add the mapping to physical data objects.

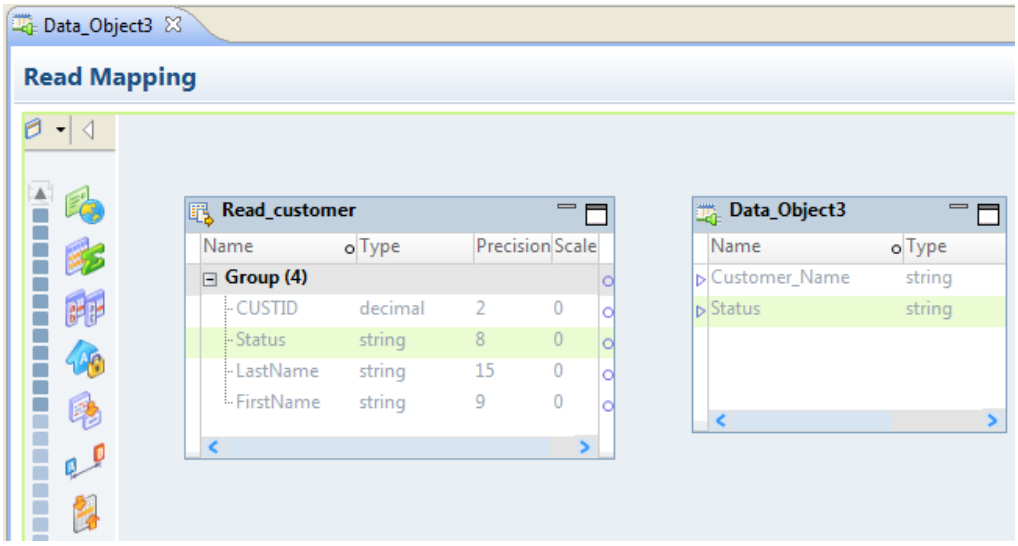
   The logical data object editor opens.
2. On the **Overview** tab in the General area, click **Add** to add a read mapping or a write mapping.
3. Enter a name for the mapping, and then click **Finish**.

   The editor displays the logical data object as the mapping input or output, based on whether the mapping is a read or write mapping.
4. Link data from a physical data object to the logical data object.
   a. Click the **Read Mapping** or **Write Mapping** tab to edit the mapping.

      The mapping editor palette opens.
   b. In the **Object Explorer** view, browse to the physical data object you want to link, and drag the physical data object into the mapping editor palette.

      The **Add to Mapping** dialog box opens.
   c. Choose to create a read mapping, write mapping, or lookup mapping.
   d. If you chose to create a read mapping, designate the object access of the read mapping as a related data object or as an independent data object.
   e. Click **OK**.

The following image shows the result after a user drags the **customer** flat file data object to the mapping editor palette and designates it a read mapping.



5. Optionally, add a reusable object to the mapping.

   You can add a logical data object or other Model repository object.

   a. Right-click in the mapping editor and select **Add Reusable Object**.

   b. Select one of the objects from the Model repository, and click **OK**.

   c. Choose to designate the reusable object a Read mapping, Write mapping, or Lookup mapping.

   d. If you chose to designate the reusable object a Read mapping, designate the object access of the Read mapping as a related data object or as an independent data object.

   e. Click **OK**.

6. Optionally, add additional objects and transformations to the mapping, and create links between mapping objects, then click the **Data Viewer** view and run the mapping.

   Results appear in the **Output** section.

CHAPTER 8

Viewing Data

This chapter includes the following topics:

- Viewing Data Overview, 120
- Configurations, 121
- Exporting Data, 128
- Object Dependencies, 129
- Logs, 130
- Validation Preferences, 131
- Monitoring Jobs from the Developer Tool, 132

Viewing Data Overview

You can run a mapping, view source data, preview data for a transformation, run an SQL query, preview web service messages, or view dependencies on an object.

Use the following views to manipulate and view data and mappings:

Mapping Editor view

Use the mapping editor to run mappings. Run a mapping to move output from sources to targets and transform data. You can run a mapping from the command line or from the **Run** dialog box.

Data Viewer view

Use the **Data Viewer**view to view source data, preview data for a transformation, run an SQL query, or preview web service messages.

Before you can view data, you must select a default Data Integration Service if the domain includes more than one service. You can also add other Data Integration Services to use when you view data. You can create configurations to control settings that the Developer tool applies when you view data.

Note: The maximum number of rows you can preview in the Data Viewer is 100,000.

When you view data in the **Data Viewer** view, you can export the data to a file. You can also access logs that show log events.

Object Dependencies view

Use the **Object Dependencies** view to view object dependencies when you view, modify, or delete Model repository objects.

Configurations

A configuration is a group of settings that the Developer tool applies when you run a mapping, preview data, run an SQL query, or preview web service messages.

A configuration controls settings such as the default Data Integration Service, number of rows to read from a source, default date/time format, and optimizer level. The configurations that you create apply to your installation of the Developer tool.

You can create the following configurations:

- Data viewer configurations. Control the settings the Developer tool applies when you preview output in the **Data Viewer** view.
- Mapping configurations. Control the settings the Developer tool applies when you run mappings through the **Run Configurations** dialog box or from the command line.
- Web service configurations. Controls the settings that the Developer tool applies when you preview the output of a web service in the **Data Viewer** view.

Configuration Properties

The Developer tool applies configuration properties when you preview output or you run mappings. Set configuration properties for the **Data Viewer** view or mappings in the **Run** dialog box.

Data Integration Service Properties

The Developer tool displays the Data Integration Service tab for data viewer, mapping, and web service configurations.

The following table displays the properties that you configure for the Data Integration Service:

Property	Description
Use the default Data Integration Service	Uses the default Data Integration Service to run the mapping. Default is enabled.
Data Integration Service	Specifies the Data Integration Service that runs the mapping if you do not use the default Data Integration Service.

Source Properties

The Developer tool displays the **Source** tab for data viewer, mapping, and web service configurations.

The following table displays the properties that you configure for sources:

Property	Description
Read all rows	Reads all rows from the source. Default is enabled.
Read up to how many rows	Specifies the maximum number of rows to read from the source if you do not read all rows. **Note:** If you enable the this option for a mapping that writes to a customized data object, the Data Integration Service does not truncate the target table before it writes to the target. Default is 1000.
Read all characters	Reads all characters in a column. Default is disabled.
Read up to how many characters	Specifies the maximum number of characters to read in each column if you do not read all characters. The Data Integration Service ignores this property for SAP sources. Default is 4000.

Results Properties

The Developer tool displays the **Results** tab for data viewer and web service configurations.

The following table displays the properties that you configure for results in the **Data Viewer** view:

Property	Description
Show all rows	Displays all rows in the **Data Viewer** view. Default is disabled.
Show up to how many rows	Specifies the maximum number of rows to display if you do not display all rows. Default is 1000.
Show all characters	Displays all characters in a column. Default is disabled.
Show up to how many characters	Specifies the maximum number of characters to display in each column if you do not display all characters. Default is 4000.

Messages Properties

The Developer tool displays the **Messages** tab for web service configurations.

The following table displays the properties that you configure for messages:

Property	Description
Read up to how many characters for request message	Specifies the maximum number of characters to process in the input message.
Show up to how many characters for response message	Specifies the maximum number of characters to display in the output message.

Advanced Properties

The Developer tool displays the **Advanced** tab for data viewer, mapping, and web service configurations.

The following table displays the advanced properties:

Property	Description
Default date time format	Date/time format the Data Integration Services uses when the mapping converts strings to dates. Default is MM/DD/YYYY HH24:MI:SS.
Override tracing level	Overrides the tracing level for each transformation in the mapping. The tracing level determines the amount of information that the Data Integration Service sends to the mapping log files. Choose one of the following tracing levels: - None. The Data Integration Service uses the tracing levels set in the mapping. - Terse. The Data Integration Service logs initialization information, error messages, and notification of rejected data. - Normal. The Data Integration Service logs initialization and status information, errors encountered, and skipped rows due to transformation row errors. Summarizes mapping results, but not at the level of individual rows. - Verbose initialization. In addition to normal tracing, the Data Integration Service logs additional initialization details, names of index and data files used, and detailed transformation statistics. - Verbose data. In addition to verbose initialization tracing, the Data Integration Service logs each row that passes into the mapping. Also notes where the Data Integration Service truncates string data to fit the precision of a column and provides detailed transformation statistics. Default is None.
Sort order	Order in which the Data Integration Service sorts character data in the mapping. Default is Binary.

Property	Description
Optimizer level	Controls the optimization methods that the Data Integration Service applies to a mapping as follows: **Auto** The Data Integration Service applies optimizations based on the execution mode and mapping contents. **None** The Data Integration Service does not apply any optimization. **Minimal** The Data Integration Service applies the early projection optimization method. **Normal** The Data Integration Service applies the early projection, early selection, branch pruning, push-into, global predicate optimization, and predicate optimization methods. **Full** The Data Integration Service applies the cost-based, early projection, early selection, branch pruning, predicate, push-into, semi-join, and dataship-join optimization methods. Default is Auto.
High precision	Runs the mapping with high precision. High precision data values have greater accuracy. Enable high precision if the mapping produces large numeric values, for example, values with precision of more than 15 digits, and you require accurate values. Enabling high precision prevents precision loss in large numeric values. Default is enabled.
Send log to client	Allows you to view log files in the Developer tool. If you disable this option, you must view log files through the Administrator tool. Default is enabled.

Data Viewer Configurations

Data viewer configurations control the settings that the Developer tool applies when you preview output in the **Data Viewer** view.

You can select a data viewer configuration when you preview output for the following objects:

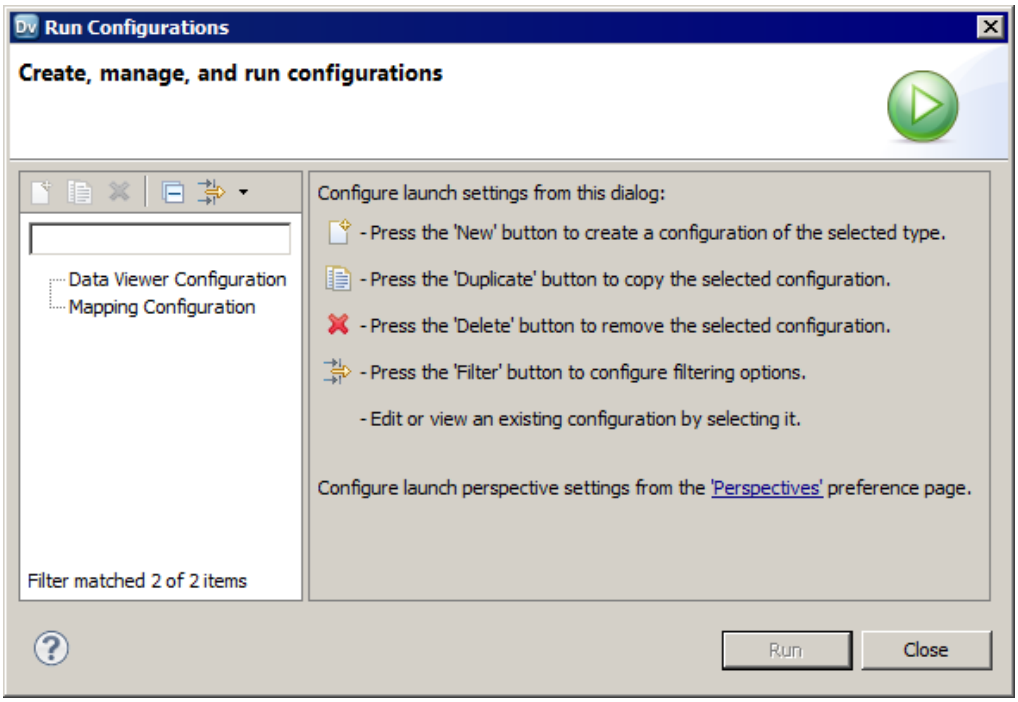
- Custom data objects
- Logical data objects
- Logical data object read mappings
- Physical data objects
- Sources and transformations within mappings
- Virtual stored procedures
- Virtual tables
- Virtual table mappings

Creating a Data Viewer Configuration

Create a data viewer configuration to control the settings the Developer tool applies when you preview output in the **Data Viewer** view.

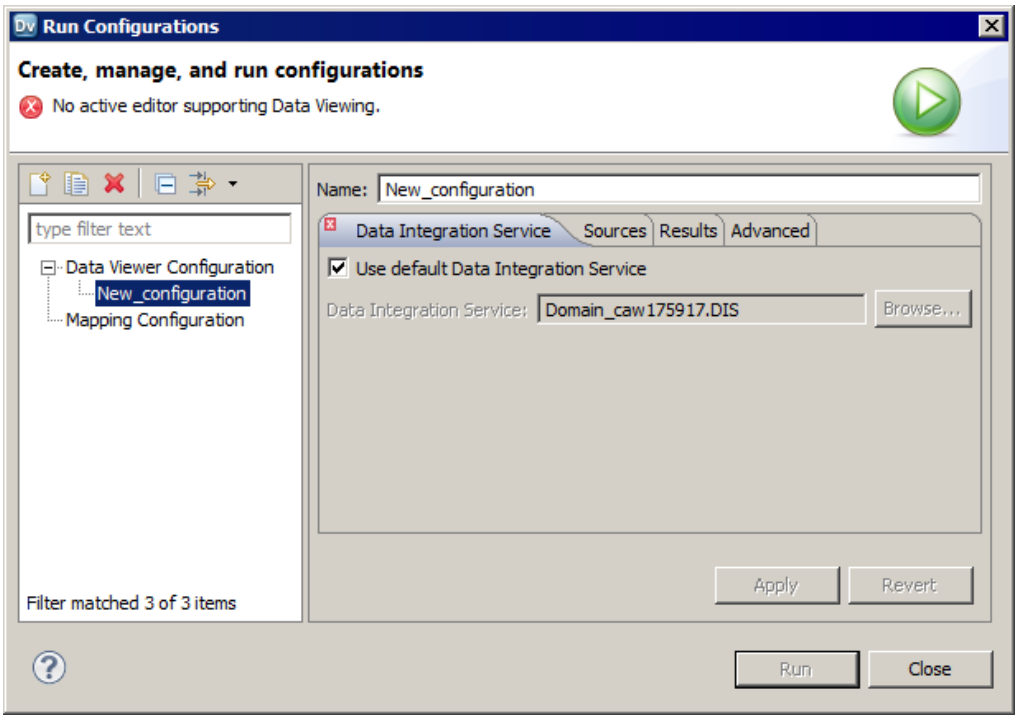
1. Click **Run** > **Open Run Dialog**.

   The **Run Configurations** dialog box appears.



2. Click **Data Viewer Configuration**.
3. Click the **New** button( ).

The right panel of the **Run Configurations** dialog box displays the data viewer configuration properties.



4. Enter a name for the data viewer configuration.
5. Configure the data viewer configuration properties.
6. Click **Apply**.
7. Click **Close**.

   The Developer tool creates the data viewer configuration.

Mapping Configurations

Mapping configurations control the mapping deployment properties that the Developer tool uses when you run a mapping through the **Run Configurations** dialog box or from the command line.

To apply a mapping configuration to a mapping that you run through the Developer tool, run the mapping through the **Run Configurations** dialog box or run the mapping using advanced options. If you run the mapping through the **Run** menu or the mapping editor, the Developer tool runs the mapping with the default mapping deployment properties.

To apply mapping deployment properties to a mapping that you run from the command line, select the mapping configuration when you add the mapping to an application. The mapping configuration that you select applies to all mappings in the application.

You can change the mapping deployment properties when you edit the application. An administrator can also change the mapping deployment properties through the Administrator tool. You must redeploy the application for the changes to take effect.

For more information about using parameter sets with infacmd, see the *Developer Mapping Guide*.

Creating a Mapping Configuration

Create a mapping configuration to control the mapping deployment properties that the Developer tool uses when you run mappings.

1. Click **Run** > **Open Run Dialog**.

   The **Run Configurations** dialog box appears.
2. Click **Mapping Configuration**.
3. Click the **New** button.

   The right panel of the **Run Configurations** dialog box displays the mapping configuration properties.
4. Enter a name for the mapping configuration.
5. Configure the mapping configuration properties.
6. Click **Apply**.
7. Click **Close**.

   The Developer tool creates the mapping configuration.

Web Service Configurations

Web Service configurations control the settings that the Developer tool applies when you preview the output of a web service in the **Data Viewer** view.

Create a web service configuration to control the setting that you want to use for specific web services. You can select a web service configuration when you preview the output of an operation mapping or transformations in an operation mapping.

Note: To create a web service configuration that applies to all web services that you preview, use the **Preferences** dialog box to update the default web service configuration.

Creating a Web Service Configuration

Create a web service configuration to control the settings the Developer tool applies when you preview the output of a web service in the **Data Viewer** view.

1. Click **Run** > **Open Run Dialog**.

   The **Run** dialog box appears.
2. Click **Web Service Configuration**.
3. Click **New**.
4. Enter a name for the web service configuration.
5. Configure the web service configuration properties.
6. Click **Apply**.
7. Click **Close**.

Updating the Default Configuration Properties

You can update the default data viewer, mapping, and web service configuration properties.

1. Click **Window** > **Preferences**.

   The **Preferences** dialog box appears.

2. Click **Informatica** > **Run Configurations**.
3. Select the **Data Viewer**, **Mapping**, or **Web Service** configuration.
4. Configure the default data viewer, mapping, or web service configuration properties.
5. Click **OK**.

   The Developer tool updates the default configuration properties.

Troubleshooting Configurations

I created two configurations with the same name but with different cases. When I close and reopen the Developer tool, one configuration is missing.

Data viewer and mapping configuration names are not case sensitive. If you create multiple configurations with the same name but different cases, the Developer tool deletes one of the configurations when you exit. The Developer tool does not consider the configuration names unique.

I tried to create a configuration with a long name, but the Developer tool displays an error message that says it cannot not write the file.

The Developer tool stores data viewer and mapping configurations in files on the machine that runs the Developer tool. If you create a configuration with a long name, for example, more than 100 characters, the Developer tool might not be able to save the file to the hard drive.

To work around this issue, shorten the configuration name.

My preview data failed for a customized data object.

The Developer tool cannot display data in the Data Viewer view if the customized data object performs a data type conversion and pushdown optimization is not set to source.

To work around this issue, configure pushdown optimization to push processing to the source.

Exporting Data

You can export the data that appears in the **Data Viewer** view to a tab-delimited flat file, such as a TXT or CSV file. You can export the data of a hierarchical data type to a JSON file. Export data when you want to create a local copy of the data.

1. In the **Data Viewer** view, right-click the results and select **Export Data**.
2. Enter a file name and extension.
3. Select the location where you want to save the file.
4. Click **OK**.

Object Dependencies

Before you change or delete an object, you can view object dependencies for all objects in the Model repository that the object has an impact on. You can view object dependencies for all objects that are dependent on the object and objects that this object depends on.

For example, you want to delete a data object that is deployed in multiple applications. However, you do not know if deleting this data object has an impact on the deployed applications. You can view object dependencies to determine if this data object has an impact on the deployed applications. After you delete the data object, you can redeploy the applications.

You can see object dependencies in the **Object Dependencies** view. If a dependent object is in a project that you do not have permission to read, the Developer tool does not display the object in the **Object Dependencies** view. The Developer tool displays a message that states the Developer tool cannot access the object.

View Object Dependencies

You can view object dependencies for an object in the **Object Dependencies** view. You can view dependencies for objects that you select from the **Object Explorer** view or for connections that you select from the **Connection Explorer** view.

You can perform the following tasks to view object dependencies:

View circular dependencies.

A circular dependency occurs when two objects depend on each other. For example object A depends on object B, which depends on object A. When the Developer tool encounters the second instance of the object in the object dependency tree, it does not display further instances of the object. Instead, the Developer tool adds a circle icon against the object to indicate that a circular dependency occurred.

View downstream or upstream dependencies.

View downstream dependencies to view the objects that depend on the selected object. The Developer tool displays downstream dependencies by default.

View upstream dependencies to view objects that the selected object is dependent on.

Filter object dependencies.

Filter object dependencies to narrow the list of dependent objects. You can choose to filter by types of objects or by projects. For example, you might want to see workflows that a particular object has an impact on. You can filter by object type and select workflows.

View object dependencies history.

View history for the last 10 object dependencies.

Viewing Object Dependencies

When you view dependencies, you can apply filters, clear history, or choose more information about particular dependencies.

1. In the **Object Explorer** view or **Connection Explorer** view, right-click an object or connection and click **Find Dependencies**.

   The **Object Dependencies** view displays a list of object dependencies for the selected object or connection in an object dependency tree.
2. Select an object or connection in the object dependency tree to view the object or connection properties.

3. Optionally, to filter dependencies, click the **Filters** icon and choose to filter by types of objects or projects.
4. You can view the following types of dependencies:
   - Circular. The Developer tool adds a circle icon against the object to indicate that a circular dependency occurred.
   - Upstream. Click the **Upstream Dependencies** icon to view the upstream dependencies.
   - Downstream. Click the **Downstream Dependencies** icon to view the downstream dependencies.
5. Optionally, to view dependency history, click the **History** icon to view the history of object dependencies. Optionally, click **Clear History** to clear the history.

Filtering Object Dependencies

You can filter the object dependency tree by **Types** and **Projects**. In the filtered results, some objects may not be direct dependents of the parent object. These objects appear with a plus (+) icon that hides the full path to the parent object in the object dependency tree. You can choose to expand the path to insert all object levels till the parent object.

1. In the **Object Dependencies** view, click the **Filters** icon.

   The **Filters** dialog box opens.
2. Accept the default selections on the **Types** tab to filter by object type.
3. Optionally, click the **Projects** tab and choose to filter by all projects in the Model repository or by each project.
   - To filter by all projects in the Model repository, select the option to include all projects.
   - To filter by each project, select the option to include opened projects.
4. Choose to select an object type or project, select all object types or projects, or clear the selected object types and projects.
   - To select an object type or project, select each object type or project.
   - To select all object types or projects, click **Select All**.
   - To remove all selected object types and projects, click **Select None**.
5. Click **OK**.

   The filtered object dependency tree appears in the **Object Dependencies** view.
6. Optionally, if a plus (+) icon appears against an object name, right-click the object and click **Expand Path** to insert all object levels till the parent object.

Logs

The Data Integration Service generates log events when you run a mapping, preview data, or run an SQL query. Log events include information about the tasks performed by the Data Integration Service, errors, and load summary and transformation statistics.

You can view the logs generated from the Developer tool and save them to a local directory.

You can view log events from the **Show Log** button in the **Data Viewer** view.

When you run a mapping from **Run** > **Run Mapping**, you can view the log events from the **Progress** view. To open the log events in the Developer tool, click the link for the mapping run and select **Go to Log**.

To save the log to a file, click **File** > **Save a Copy As** and choose a directory. By default the log files are stored in the following directory: `c:\[TEMP]\AppData\Local\Temp`.

Log File Format

The information in the log file depends on the sequence of events during the run. The amount of information that is sent to the logs depends on the tracing level.

The Data Integration Service updates log files with the following information when you run a mapping, preview data, or run an SQL query:

Logical DTM messages

Contain information about preparing to compile, to optimize, and to translate the mapping. The log events and the amount of information depends on the configuration properties set.

Data Transformation Manager (DTM) messages

Contain information about establishing a connection to the source, reading the data, transforming the data, and loading the data to the target.

Load summary and transformation statistics messages

Contain information about the number of rows read from the source, number of rows output to the target, number of rows rejected, and the time to execute.

Validation Preferences

You can limit the number of error messages that appear in the **Validation Log** view. You can also group error messages by object or object type in the **Validation Log** view.

Grouping Error Messages

Group error messages in the **Validation Log** view to organize messages by object or object type. Otherwise, messages appear alphabetically.

To group error messages in the **Validation Log** view, select **Menu** > **Group By** and then select **Object** or **Object Type**.

To remove error message groups, select **Menu** > **Group By** > **None**. Error messages appear ungrouped, listed alphabetically in the **Validation Log** view.

Limiting Error Messages

You can limit the number of error messages that appear in the **Validation Log** view. The limit determines how many messages appear in a group or the total number of messages that appear in the **Validation Log** view. Error messages are listed alphabetically and get deleted from bottom to top when a limit is applied.

1. Click **Window** > **Preferences**.

   The **Preferences** dialog box appears.

2. Select **Informatica** > **Validation**.
3. Optionally, set the error limit and configure the number of items that appear.

   Default is 100.
4. To restore the default values, click **Restore Defaults**.
5. Click **Apply**.
6. Click **OK**.

Monitoring Jobs from the Developer Tool

You can access the Monitoring tool from the Developer tool to monitor the status of applications and jobs. As an administrator, you can also monitor applications and jobs in the Administrator tool.

Monitor applications and jobs to view properties, run-time statistics, and run-time reports about the objects.

When you monitor a job, you can view summary statistics or execution statistics for the job. The **Summary Statistics** view displays a graphical overview of the status of jobs in the domain.

The **Execution Statistics** view displays general properties and status information for the jobs. For example, you can see who initiated the job and how long it took the job to complete. If you monitor a mapping job, you can also view throughput and resource usage statistics for the job run.

To monitor applications and jobs from the Developer tool, open the **Progress** view and click **View Menu** > **Monitor Jobs**. Select the Data Integration Service that runs the applications and jobs and click **OK**. The Monitoring tool opens.

CHAPTER 9

Application Deployment

This chapter includes the following topics:

- Application Deployment Overview, 133
- Application Creation, 134
- Application Properties, 135
- Deployment to an Application, 137
- Deployment to an Archive File, 139
- Deployment with Resource Parameters, 139
- Viewing Run-time Objects, 141
- Application Redeployment, 141
- Application State Information, 142
- How to Create, Deploy, and Update an Application, 143

Application Deployment Overview

An application is a deployable object that can contain physical data objects, logical data objects, data services, mappings, mapplets, transformations, web services, workflows.

To make application objects accessible outside of the Developer tool, deploy an application that contains the objects. You can create a new application, or create the application as you deploy an object. When you deploy an application, you isolate the application objects from changes in data structures.

You can deploy application objects in the following ways:

Deploy to a Data Integration Service.

When you deploy an application to a Data Integration Service, the application runs and end users can connect to the application.

Depending on the types of objects in the application, end users with appropriate permissions can query the objects, access web services, or run mappings or workflows from the command line. You can also deploy objects to allow users to query the objects through a third-party client tool.

Deploy to a network file system.

When you deploy an object to a network file system, the Developer tool creates an application archive file. Deploy an object to an archive file if you want to archive the object in a version control system.

If your organization requires that administrators deploy objects to a Data Integration Service, an administrator can deploy the application archive file to a Data Integration Service using the Administrator tool.

You can also import objects from an application archive file into projects or folders in the Model repository.

After you deploy application objects or an application archive file to a Data Integration Service, you can use the Developer tool to connect to the run-time application and view read-only copies of the run-time objects.

Application Creation

You create an application and deploy it to run the mappings and other objects that the application contains.

You can create an application when you deploy an object to a Data Integration Service, or you can create a new application. When you create a new application, you select objects to include in the application.

You can add objects such as mappings, workflows, data objects, and parameter sets. You cannot add a logical data object directly to an application. However, you can create applications for web services and SQL data services that contain logical data objects.

When you create an application, you can create a full application or an incremental application. A full application is an application that you must redeploy each time that you update the application. An incremental application is an application that you can update by deploying an application patch. A patch can update application objects while other application objects continue running, but the patch might require more time to execute compared to redeploying a full application.

After you create an application, the Developer tool validates the application. The validity of an application depends on the validity of the objects it contains. For example, the configuration of links between objects in a mapping must be valid for the application that contains the mapping to be valid. When an application is not valid, errors appear in the Validation Log view or in an error dialog box.

Full Application

A full application is an application that you can deploy to a Data Integration Service. Each time that you edit the application or the application objects, you must redeploy the application to apply the changes.

Note: During application redeployment, all of the objects that the application contains are inaccessible.

Example

You create one mapping that runs a search function, and another mapping that puts selected search results in a shopping cart. You can create an application to contain the two mappings, and then deploy the application to a Data Integration Service. After testing the output of the application objects, you make changes to the objects. To update the application, you redeploy the application. You also deploy the application to an application archive file, and an administrator checks the archive file into a version control system.

Incremental Application

An incremental application is an application that you can deploy to the Data Integration Service and update by deploying an application patch.

The patch includes the objects that you want to update or add to the deployed incremental application. When you deploy the patch, the Data Integration Service updates the deployed application based on the patch contents.

You can use a patch to increase application up-time. For example, you might have an application that contains a streaming mapping. You can update other objects in the application while the streaming mapping continues processing data.

For more information about application patches, see Chapter 10, “Application Patch Deployment” on page 151.

Note: An incremental application cannot contain web services, SQL data services, or REST web services.

Example

You create one hundred mappings and an incremental application that contains the mappings. Then, you deploy the application to a Data Integration Service. After testing the output of the application objects, you make changes to a few of the objects.

To update the objects in the run-time application, you create an application patch that contains the objects that you changed. You deploy the application patch to the Data Integration Service and the service applies the patch to the incremental application to update the objects without stopping other application objects that other users might be running. You also deploy the application and the application patch to archive files, and an administrator checks the archive files into a version control system.

Application Properties

After you create an application, you can edit properties in the **Application** editor.

General Application Properties

The following table describes general application properties that you can set on the **Overview** tab:

Property	Description
Name	Name of the application.
Description	Optional. Description of the application.
Application Type	Type of application. Full or incremental.

Mapping Deployment Properties

The following table describes the mapping deployment properties that you can set on the **Advanced** tab when the application contains a mapping:

Property	Description
Default date time format	Date/time format that the Data Integration Service uses when the mapping converts strings to dates. Default is MM/DD/YYYY HH24:MI:SS.
Override tracing level	Overrides the tracing level for each transformation in the mapping. The tracing level determines the amount of information that the Data Integration Service sends to the mapping log files. Choose one of the following tracing levels: - None. The Data Integration Service does not override the tracing level that you set for each transformation. - Terse. The Data Integration Service logs information about initialization, error messages, and notification of rejected data. - Normal. The Data Integration Service logs information about initialization and status, errors encountered, and skipped rows due to transformation row errors. The Data Integration Service summarizes mapping results, but not at the level of individual rows. - Verbose Initialization. In addition to normal tracing, the Data Integration Service logs additional initialization details, names of index and data files used, and detailed transformation statistics. - Verbose Data. In addition to verbose initialization tracing, the Data Integration Service logs each row that passes into the mapping. The log notes where the Data Integration Service truncates string data to fit the precision of a column. The log contains detailed transformation statistics. The Data Integration Service writes row data for all rows in a block when it processes a transformation. Default is None.
Sort order	Order in which the Data Integration Service sorts character data in the mapping. Default is Binary.

Property	Description
Optimizer level	Controls the optimization methods that the Data Integration Service applies to a mapping as follows: **Auto** The Data Integration Service applies optimizations based on the execution mode and mapping contents. **None** The Data Integration Service does not apply any optimization. **Minimal** The Data Integration Service applies the early projection optimization method. **Normal** The Data Integration Service applies the early projection, early selection, branch pruning, push-into, global predicate optimization, and predicate optimization methods. **Full** The Data Integration Service applies the cost-based, early projection, early selection, branch pruning, predicate, push-into, semi-join, and dataship-join optimization methods. Default is Auto.
High precision	Runs the mapping with high precision. High precision data values have greater accuracy. Enable high precision if the mapping produces large numeric values, for example, values with precision of more than 15 digits, and you require accurate values. Enabling high precision prevents precision loss in large numeric values. Default is enabled.

Deployment to an Application

You can deploy an object as an application or as a data service that is part of an application.

You can deploy some objects as a web service or as an SQL data service. First, you create the application and add the objects. Then, when you deploy the application, the Developer tool prompts you to create a service based on the object. The Developer tool adds the service to the application.

You can also deploy objects to allow users to query the objects through a third-party client tool.

You can deploy an application to a Data Integration Service to allow users to access the mappings, workflows, and other objects that the application contains.

When you deploy an application, the Data Integration Service runs application objects.

The Developer tool prompts you to create an application when you deploy mappings or workflows.

You can add parameter sets to applications that you deploy. A parameter set is an object in the Model repository that contains parameters and parameter values to use with mappings and workflows. Deploy a parameter set with the workflow or mapping to use the parameter values when the Data Integration Service

runs the workflow or mapping. You can add multiple parameter sets to an application and then use different parameter sets for mapping or workflow runs.

When you deploy an application of the same name to the same Data Integration Service, the Data Integration Service overwrites the deployed application and all of the objects in the application.

Deploy Objects as a Web Service

You can deploy the following objects as a web service:

- Mapplets
- Transformations except the Web Service Consumer transformation
- Flat file data objects
- Relational data objects
- Logical data objects

When you deploy an object as a web service, the Developer tool prompts you to create a web service based on the object, and then prompts you to create an application to contain the web service.

When you deploy an object as a web service, you enter the following information:

Property	Description
Name	Name of the web service.
Location	Model repository project folder where you want to put the application.
Namespace	URL where you want users to access the web service. The Data Integration Service overrides the URL defined in the WSDL file.
Prefix	The prefix of the target namespace.

Deploy Objects as an SQL Data Service

You can deploy the following data objects as an SQL data service:

- Physical data objects
- Logical data objects

When you deploy a data object as an SQL data service, the Developer tool prompts you to create an SQL data service based on the object, and then prompts you to create an application to contain the service.

When you deploy an object as an SQL data service, you enter the following information:

Property	Description
Name	Name of the web service.
Location	Model repository project folder where you want to put the application.

Deployment to an Archive File

An archive file stores application objects and metadata in an XML format.

You can create the following types of archive files:

Application archive file

An application archive file is an XML file that stores all of the information about application objects and metadata for an application. The file has an .iar extension.

Application patch archive file

An application patch archive file is an XML file that stores information about an application patch. The file stores information about the application objects in the patch, the patch description, the name of the corresponding incremental application, and whether the patch retains or discards state information. The file has a .piar extension.

You might want to create an archive file for any of the following reasons:

- Deploy the application or application patch. If your organization restricts the ability to deploy applications or application patches to a Data Integration Service to administrators, an administrator can deploy an archive file to the Data Integration Service.

  To deploy an application, the administrator can use the Administrator tool, or infacmd dis deployApplication. To deploy an application patch, the administrator can use infacmd tools patchApplication.
- Import the application or application patch to a Model repository. An administrator can import an archive file to a Model repository using infacmd tools importObjects.
  If you import an archive file that contains objects that are already in the Model repository, the Developer tool creates a new project folder.

  Note: The release version of the Model repository where you import the application must be compatible with the version from which you exported the application to an archive file.
- Archive the file in another system. For example, if the Model repository is not integrated with a version control system, an administrator can check the archive file into a version control system for project auditing or object recovery.

Deployment with Resource Parameters

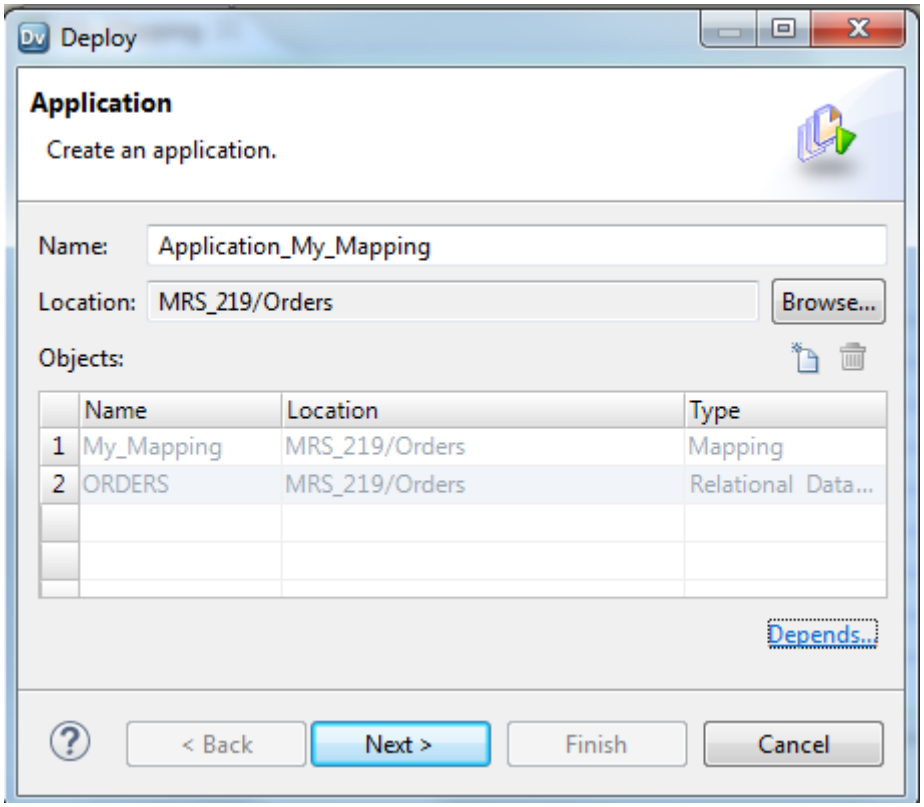
When you deploy a mapping that has resource parameters in the source, the lookup, or the target transformation, you can view a list of the data objects that the application requires to run the mapping.

You can check the data objects in the resource parameter values when you create an application or when you deploy a mapping. To view the data objects, click the **Depends** link in the **Create Application** dialog box or in the **Deploy** dialog box. When you view the data objects, you can choose to exclude a data object from the application. You change the value of the resource parameter to a different data object at run time.

To exclude a data object from the application, clear the Include/Exclude checkbox.

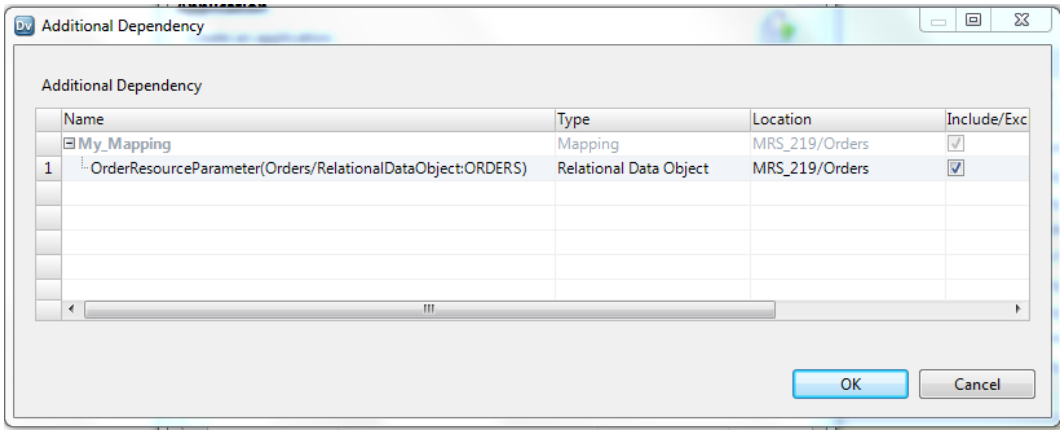
A mapping can include multiple mapplets and a mapplet can include other mapplets. The dependencies list does not show a resource parameter value if another parameter overrides it in the parameter hierarchy.

The following image shows the **Depends** link in the **Deploy** dialog box:



When you click the **Depends** link, the **Additional Dependency** dialog box appears. The Developer tool shows a list of the data objects that the application requires to run the mapping.

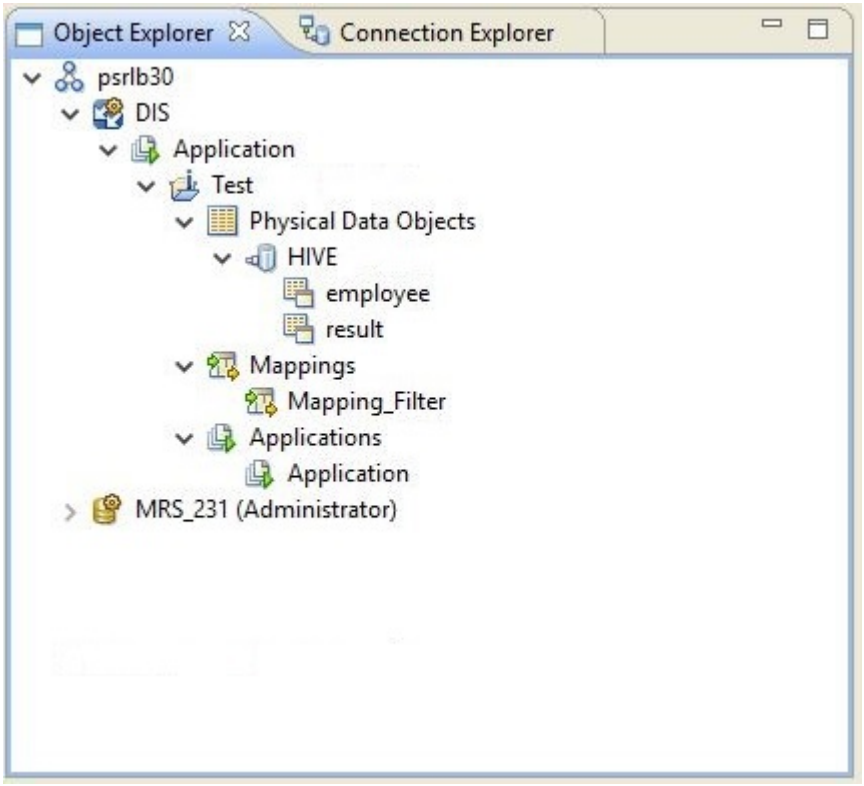
The following image shows the **Additional Dependency** dialog box:



Viewing Run-time Objects

Connect to a run-time application to view read-only copies of run-time objects in a deployed application.

The following image shows a run-time application in the **Object Explorer** view:



You can open the run-time objects in the **Editor** to verify that the application or application patch was deployed correctly. After you redeploy an application or deploy an application patch, you can refresh the run-time application to view the updated objects.

You can also reference the run-time objects while you are editing the corresponding design-time objects based on the results of testing or new release requirements. Because a run-time object is isolated from changes in data structures in the corresponding design-time object, you can continue editing the design-time object without altering the run-time object.

Application Redeployment

After you change the contents of an application, you might want to redeploy the application to apply the changes.

You might redeploy the application for the following reasons:

- You edit, add, or delete application objects in a full application and you want to apply the changes to the deployed full application.
- You delete application objects in an incremental application and you want to apply the changes to the deployed incremental application.

- An application patch for an incremental application contains a large number of objects and it is inefficient for the user to deploy the patch. Deploying a patch might be inefficient if the patch contains a large number of objects due to patch inheritance or there is a large number of affected objects in the run-time application.

When you deploy an edited application, you choose to retain or discard application state information. State information refers to mapping properties, mapping outputs, and the properties of run-time objects.

If you want to redeploy an application, you must choose to stop the application. If you do not want to abort running objects, you can rename the application or deploy the application to a different service.

When you update an application and export it to a network file system, you can replace the application archive file or cancel the deployment. If you replace the application archive file, the Developer tool replaces the objects in the application and resets the object properties.

Application State Information

When you update a deployed application by redeploying the application or by deploying an application patch, you can choose to retain the state information for the deployed application or to discard it.

State information refers to mapping properties and the properties of run-time objects such as mapping outputs or the Sequence Generator transformation. When you retain state information, you retain these settings and properties in the deployed application. When you discard state information, you discard the state of these settings and properties in the deployed application.

Example: Retaining or discarding configurable properties

An application includes a mapping with configurable run-time properties. You set the High Precision property to True. After you deploy the application, you edit the mapping and change the High Precision property to False. When you redeploy the application and check **Retain state information**, the Data Integration Service retains state information in the deployed application and does not recognize changes to the run-time properties. You must uncheck **Retain state information** for the change to the property to take effect.

Example: Retaining or discarding sequences

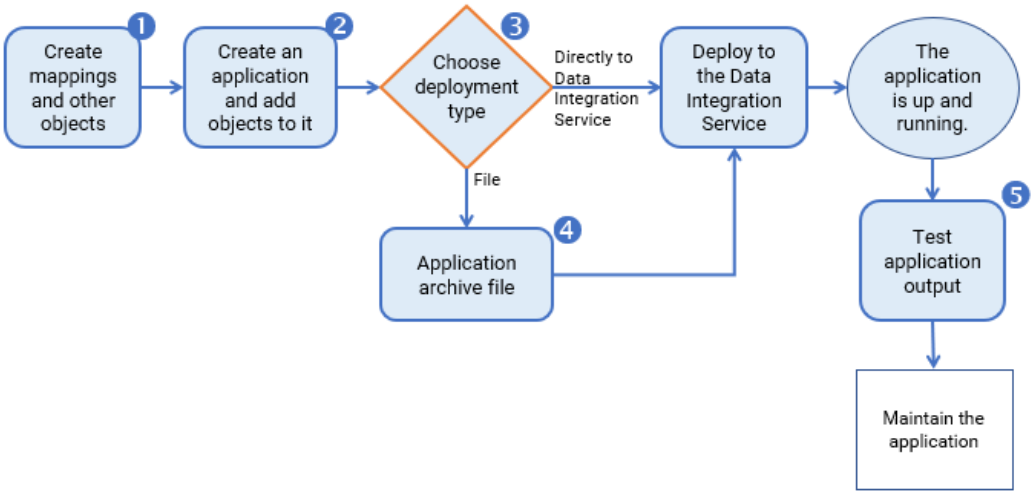
A mapping includes a Sequence Generator transformation that generates unique keys for rows in a target table. After you deploy the application that contains the mapping, the mapping runs and the Sequence Generator transformation generates keys for rows 1-2000. The next time the mapping runs, the Sequence Generator transformation would generate keys beginning with 2001, but you decide you want the sequence to start at 10000. Edit the Sequence Generator transformation to specify the new start value. When you redeploy the application and check **Retain state information**, the Data Integration Service retains state information in the deployed application and does not recognize changes to the setting. When the mapping runs again, the sequence resumes at 2001. You must uncheck **Retain state information** for the change to the setting to take effect.

Note: Your choice to retain or discard application state has no effect on the Reset property of a Sequence Generator transformation.

How to Create, Deploy, and Update an Application

Create and edit mappings and other objects and deploy them in an application to make them accessible to end users.

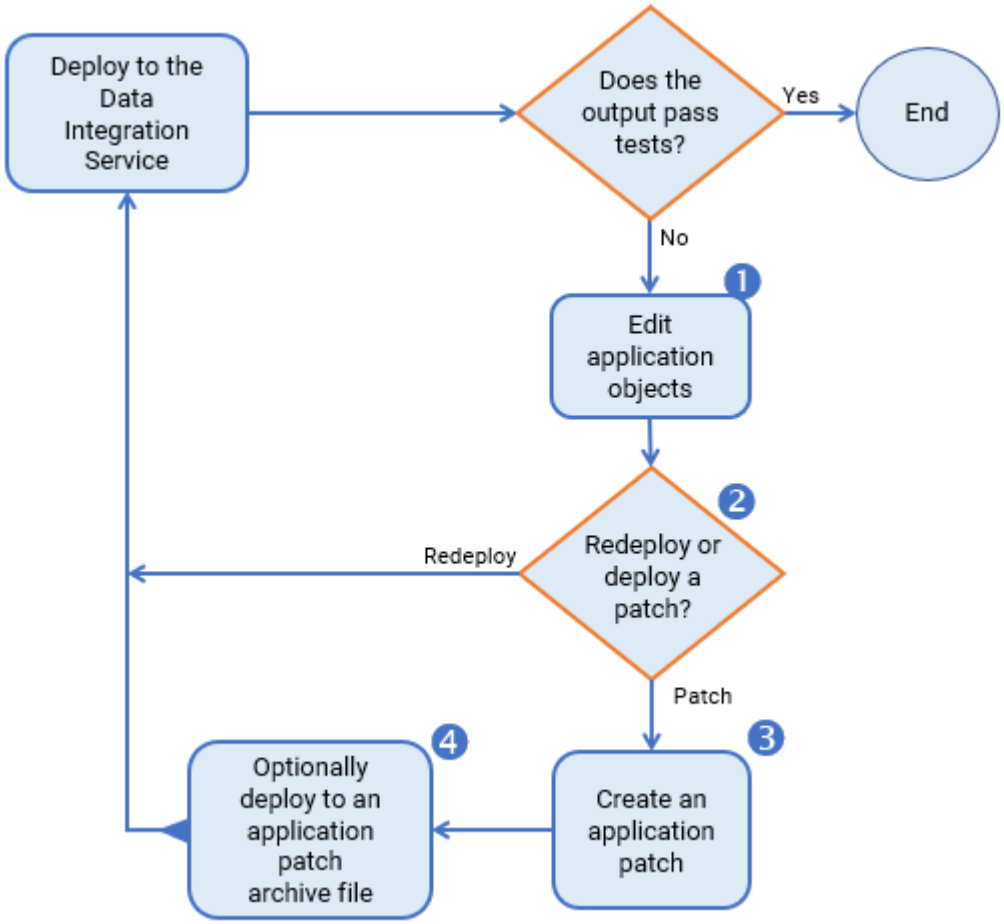
The following image shows the process of developing and deploying an application:



1. Create mappings, workflows, transformations, and other objects to access and transform data.
2. Create an application and add objects to the application.
3. Choose to deploy the application directly to a Data Integration Service or to an application archive file:
   - Deploy the application directly to a Data Integration Service to allow the Data Integration Service to run objects.
   - Deploy the application to an application archive file to allow an administrator to archive the file.
4. Deploy the archive file to a Data Integration Service.
5. Test the application to ensure that it meets organizational requirements.

After you test the application, you can maintain the application by editing application objects and updating the deployed application. To help you edit the application objects, you can connect to the run-time application to review how the run-time objects are designed.

The following image shows how an application is maintained:



1. If the output does not meet requirements, edit the objects in the application.
2. Choose to update the application by redeploying the application or by deploying an application patch. You can deploy an application patch only if the application is an incremental application.
3. If you choose to deploy a patch, create the patch using the Incremental Deployment Wizard.
4. Deploy the patch to the Data Integration Service, optionally deploying the application patch to an archive file.

For information on how to create and deploy an application patch, see the "Application Patch Deployment" chapter.

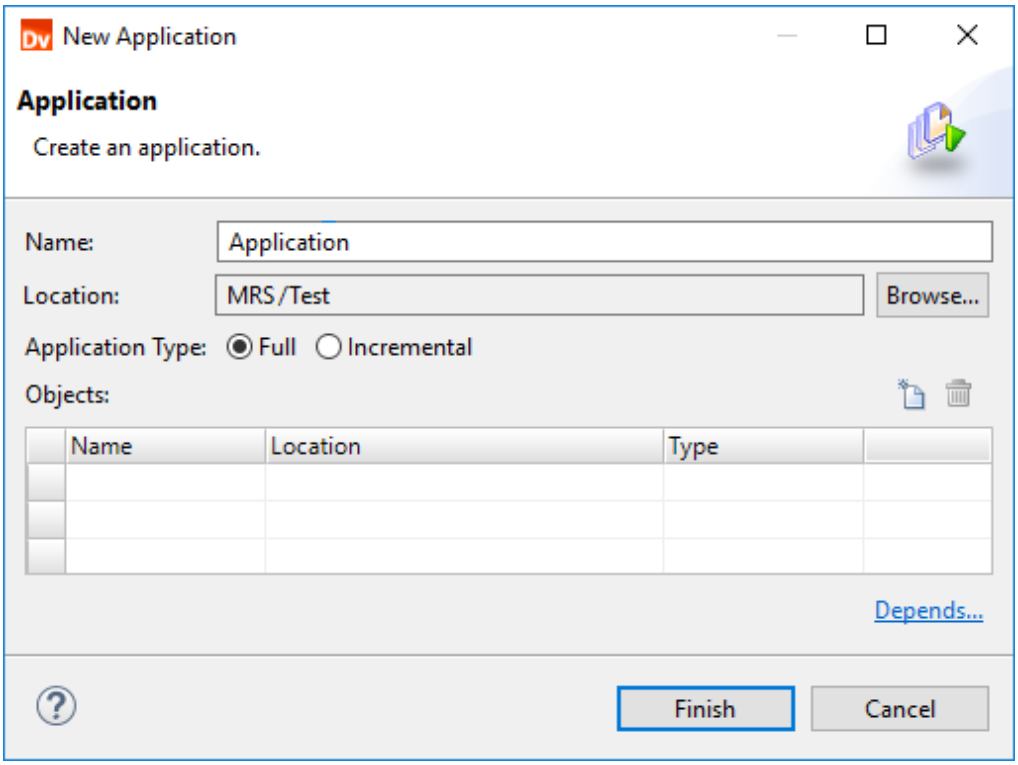
Note: An administrator can also deploy an application and perform other administrative tasks from the command line or from the Administrator tool. For information about these tasks, see the *Informatica Application Service Guide*.

Creating an Application

When you create an application, you select the objects to include in the application. Create an application when you want to deploy one or more objects so end users can access the data through third-party tools.

1. Select a project or folder in the **Object Explorer** view.
2. Click **File** > **New** > **Application**.

The **New Application** dialog box appears.



3. Enter a name for the application.

   The application name is part of the session log file name when you deploy the application. Depending on the application, the session log file name might include the application name, feature name, connection ID, and the date and time.

   The session log file name is included with the log directory path in the log file path. On Windows operating systems, the log file path has a limit of 259 characters. Consider the length limitation when you name the application.

4. Click **Browse** to select the application location.

   You must create the application in a project or a folder.

5. Select the application type. You can select Full or Incremental.

   After you create the application, you cannot edit the application type.

6. Click the Add button to add objects to the application.

   The **Add Objects** dialog box appears.

7. Select one or more objects and click **OK**.

   The Developer tool lists the objects you select in the **New Application** dialog box.

8. Click **Finish** to create the application.

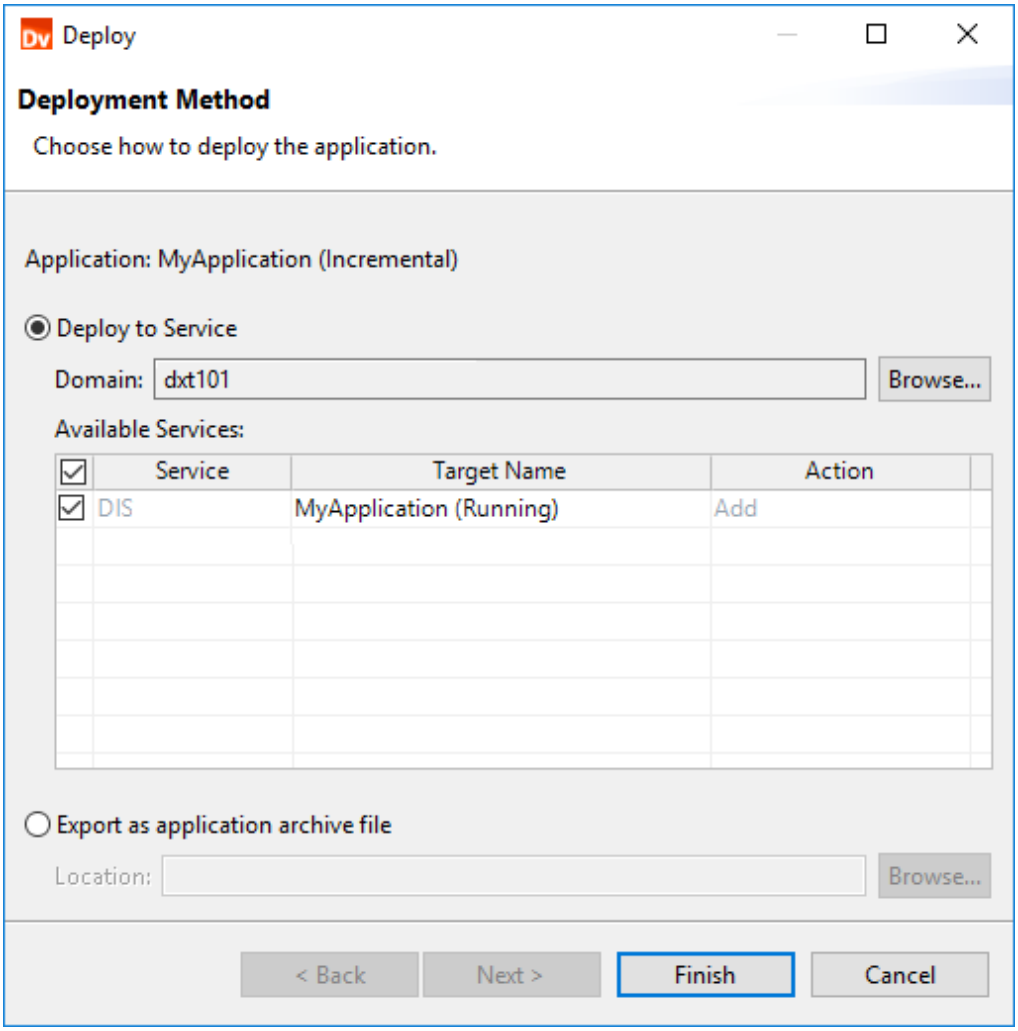
After you create an application, you can deploy it to a Data Integration Service or to an application archive file.

Deploying an Application

After you create an application, deploy the application to a Data Integration Service to run application objects, or deploy it to an application archive file.

1. Right-click an application in the **Object Explorer** view and select **Deploy**.

   The **Deploy** dialog box appears.



2. Choose one of the following options to deploy the application:
   - To deploy the application to a Data Integration Service, select **Deploy to Service**.
     - Optionally, click **Browse** to select a domain.
     - Select the Data Integration Services to which you want to deploy the application.
   - To deploy the application to an application archive file, select **Export as application archive file**. Then click the **Browse** button to select the directory where you want to save the file.

If you deploy the application to a Data Integration Service, the Data Integration Service runs the application and the objects that it contains. You can test the output of application objects and validate the output against requirements. If necessary, edit the application objects and update the deployed application.

If you deploy the application to an application archive file, you can store the file, make it available for import to another Model repository, or deploy the application from the file to a Data Integration Service at a later time.

Deploying an Object

Deploy an object to a Data Integration Service as an application, or deploy the object to an application archive file to export the object as an application.

1. Choose to deploy an executable object or a flat file data object.
   - To deploy a mapping, workflow, or other executable object, right-click the object and select **Deploy**.
   - To deploy a flat file data object, right-click the object and choose to deploy the object as a web service, SQL data service, or REST web service.

   The Developer tool prompts you to create an application.
2. Enter an application name.
3. To choose a location for the application, accept the default location, or click **Browse** and select another location.
4. Select the application type. You can select Full or Incremental.

   If you deploy a flat file data object as a web service, SQL data service, or REST web service, the application cannot be an incremental application.

   After you create the application, you cannot edit the application type.
5. To add objects to the application, click the Add button and select objects.
6. Click **Next**.
7. Choose one of the following options to deploy the application:
   - To deploy the application to a Data Integration Service, select **Deploy to Service**.
     - If the Developer tool contains the connection information for multiple domains, click **Browse** to select a domain.
     - Select the Data Integration Services to which you want to deploy the application.
   - To deploy the application to an application archive file, select **Export as application archive file**. Then click the **Browse** button to select the directory where you want to save the file.
8. To deploy the object to a web service, complete the following steps:
   a. Configure properties for the web service.
   b. To add operations to the web service, click **Next**.

      By default, the Developer tool creates an operation for each object that you deploy as a web service.
   c. Select each operation, operation input, and operation output to display and configure the properties.
   d. Click **Finish**.
9. To deploy the object to an SQL data service, complete the following steps:
   a. Enter a name for the SQL data service.
   b. Accept the default location, or click **Browse** to select a Model repository and a project folder location for the SQL data service.
   c. Click **Next**.

      The **Add Virtual Tables to SQL Data Service** dialog box appears.
   d. Click the Add button.
   e. Enter a name for the virtual table.
   f. Click the Open button in the **Data Object** column.

      The **Select a Data Object** dialog box appears.

g. Select a physical data object and click **OK**.

h. Enter the virtual schema name in the **Virtual Schema** column.

i. Select **Read** in the Data Access column to link the virtual table with the data object. Select **None** if you do not want to link the virtual table with the data object.

10. To deploy an object to a REST web service, complete the following steps:

a. Configure properties for the REST web service.

b. Select the columns to include in the definition.

c. Select the HTTP method.

11. Click **Finish**.

The Developer tool deploys the application to the Data Integration Service. The Data Integration Service runs the application and the objects that it contains.

You can test the output of application objects and validate the output against requirements. If necessary, edit the application objects and update the deployed application.

Importing Application Archives

Import an application or an application patch that was deployed to an archive file to make the file contents available in the Model repository.

If you import a file that was archived from a different Model repository or was retrieved from a version control system, you can import the application and the application objects that were used to create the archive file.

1. Click **File** > **Import**.

   The **Import** wizard appears.

2. Select **Informatica** > **Application Archive**.

3. Click **Next**.

4. Click **Browse** to select the archive file. To import an application archive file, use the .iar extension. To import an application patch archive file, use the .piar extension.

   The Developer tool lists the archive file contents.

5. Select the Model repository into which you want to import the application.

6. Click **Finish**.

The Developer tool imports the application into the repository.

Connecting to a Run-time Application

Connect to a run-time application so that you can use the **Editor** to view the run-time objects in the application.

1. Click **File** > **Connect to Run-time Application**.

2. Select a domain.

3. Select a Data Integration Service.

4. Select a deployed application on the Data Integration Service.

5. Click **OK**.

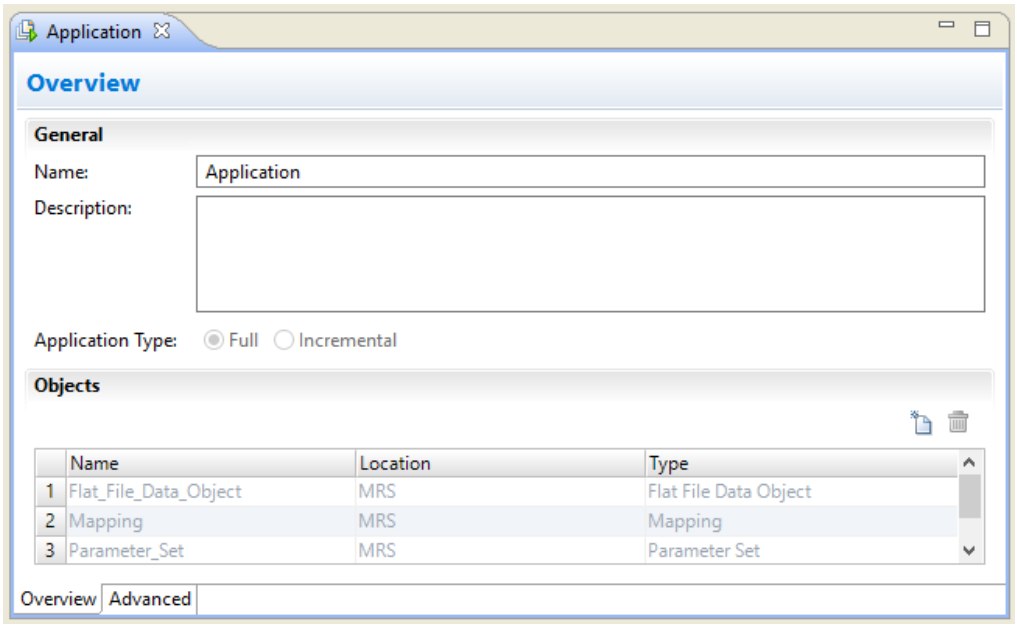
   The run-time application appears in the **Object Explorer** view. You can expand the application to view the run-time objects in the application.

Editing an Application

Edit application objects, or edit the application to add and delete objects, and edit application properties.

1. To edit an application object, open the object from the **Object Explorer** view. Make the required changes.
2. To edit the application, right-click the application in the **Object Explorer** and select **Open**.

   The **Application Editor** appears.



3. Click the Add button to add objects to the application.

   The **Add Objects** dialog box appears.
4. Select one or more objects and click **OK**.
5. Select one or more objects to delete and click the Delete button.
6. Save the application.

After you edit the application, update the application to propagate the changes to the deployed application.

Redeploying an Application to a Data Integration Service

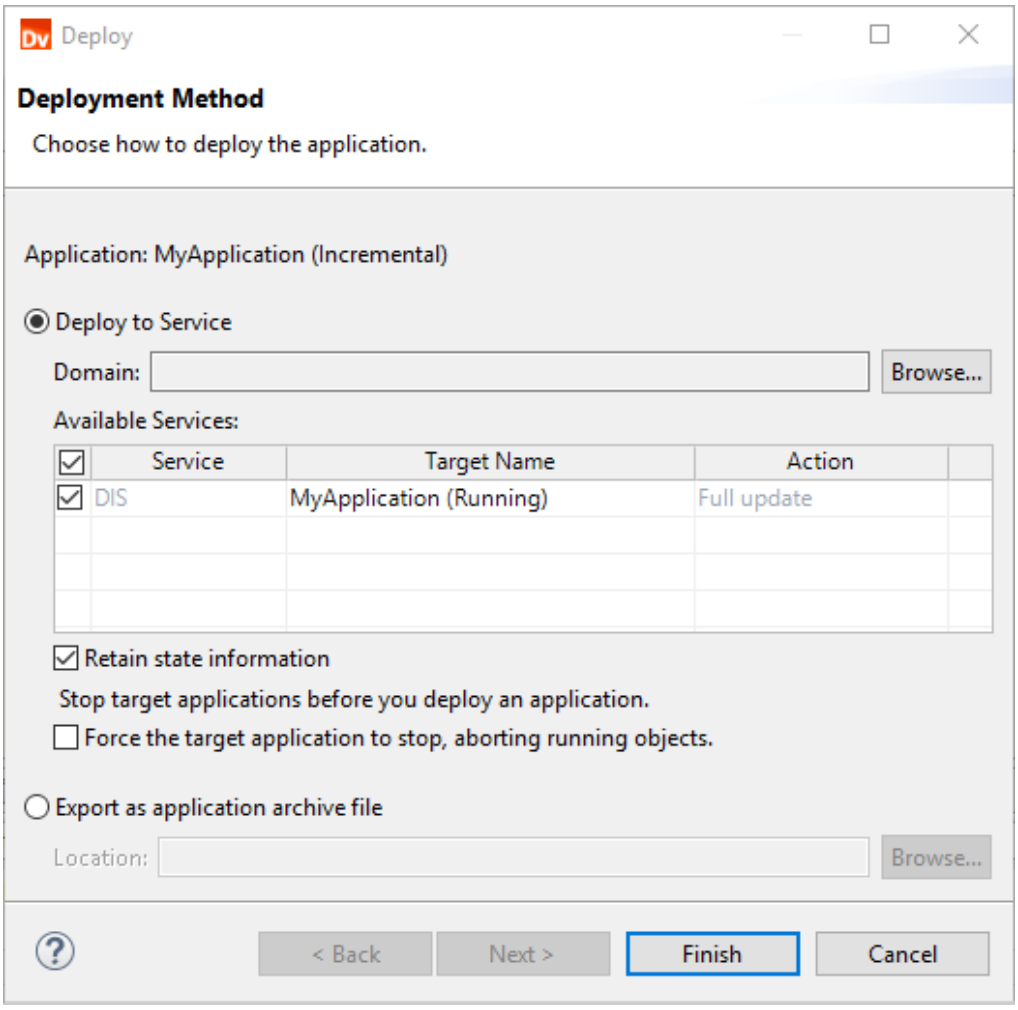
Redeploy an application to a Data Integration Service to replace the run-time application on the Data Integration Service with the design-time application.

1. Right-click an application in the **Object Explorer** view and click **Deploy**.

   The **Deploy** dialog box appears.
2. Select **Deploy to Service**.
3. If no default Data Integration Service is set, click **Browse** to select the domain.

   The **Choose Domain** dialog box appears.
4. Select a domain and click **OK**, and then select a Data Integration Service.

   The Target Name column displays the application with the same name by default. For example, the following image shows the deployment of the application MyApplication:



5. Click the Target Name column to choose a different target application on the Data Integration Service.
6. To retain the state of run-time objects that are part of the application, select **Retain state information**.
7. Optionally, select **Force target application to stop, aborting running objects**.

   You can redeploy an application without stopping it only if the state of the application is not Running. If the administrator has not stopped the application, verify with the administrator whether you can choose to force the application to stop.
8. Click **Finish**.

After you redeploy the application, you can validate it against organizational or project requirements.

CHAPTER 10

Application Patch Deployment

This chapter includes the following topics:

- Application Patch Deployment Overview, 151
- Application Object Instances, 152
- Application Object Dependencies, 152
- Incremental Deployment Wizard, 155
- Choose the Application Patch Type, 159
- Determine Application Object Impact, 165
- Specify a Patch Deployment Strategy, 169
- Deploying an Application Patch, 171
- Rules and Guidelines for Application Patch Deployment, 173

Application Patch Deployment Overview

An application patch is a deployable object that you can use to update the objects in a deployed incremental application.

To design an application patch, you must consider the design-time and run-time instances of the application objects and the dependencies between the application objects in order to decide the type of application patch that you want to create. Some application objects might be related to the other objects that you want to update, and other application objects might be affected by the objects that you want to update.

You can use a patch type to choose how to inherit the related objects and determine how much impact you want the patch to have on the deployed application. To manage patch inheritance and object impact, you can use the tools in the Incremental Deployment Wizard to understand the contents of a patch and how the Data Integration Service will apply the contents to the deployed application.

Note: You cannot use an application patch to perform the following tasks:

- Update application and mapping deployment properties, such as the application description, the default date time format, or the sort order.
- Add web services, SQL data services, or REST web services to the deployed application.
- Delete application objects from the deployed application.

Application Object Instances

The application objects in an application have design-time instances and run-time instances.

A design-time instance of an application object refers to the application object that developers create and edit in the Developer tool. When you deploy an application that contains the object to the Data Integration Service, you generate a run-time instance of the application object. The run-time instance of the object is isolated from changes in data structures in the design-time instance, so you can continue editing the design-time object in the Developer tool without altering the run-time object.

The Incremental Deployment Wizard uses both the design-time and run-time instances of application objects to decide how to apply patch contents to a deployed application. For example, the wizard decides how the patch inherits other objects based on the set of dependencies of the design-time and run-time instances of a selected object, and it fetches run-time instances of application objects to determine how they are impacted.

Application Object Dependencies

An application object that you select to include in an application patch might be related to other application objects based on the selected object's set of dependencies.

Objects within an application can contain the following dependencies:

Direct

A direct dependency is an application object that the selected object uses and any direct dependencies of the direct dependency.

The following table lists the direct dependencies for each object type:

Object	Possible Direct Dependencies
Workflow	- Mapping - Mapplet - Reusable transformation - Logical data object - Physical data object
Mapping	- Mapplet - Reusable transformation - Logical data object - Physical data object
Mapplet	- Reusable transformation - Logical data object - Physical data object
Reusable Transformation	- Logical data object - Physical data object
Logical Data Object	- Physical data object

Object	Possible Direct Dependencies
Physical Data Object	- A physical data object cannot have a direct dependency.
Parameter Set	- A parameter set cannot have a direct dependency or be a direct dependency.

Indirect

An indirect dependency is an application object that uses the selected object, or an application object that uses a direct dependency of the selected object. Indirect dependencies include any direct dependencies of the indirect dependency.

Remote

A remote dependency is an application object that is an indirect dependency of an indirect dependency for the selected object. Remote dependencies include any direct dependencies of the remote dependency.

An application patch inherits other application objects based on a selected object's dependencies. For example, you can create an application patch that inherits a selected object's direct and indirect dependencies.

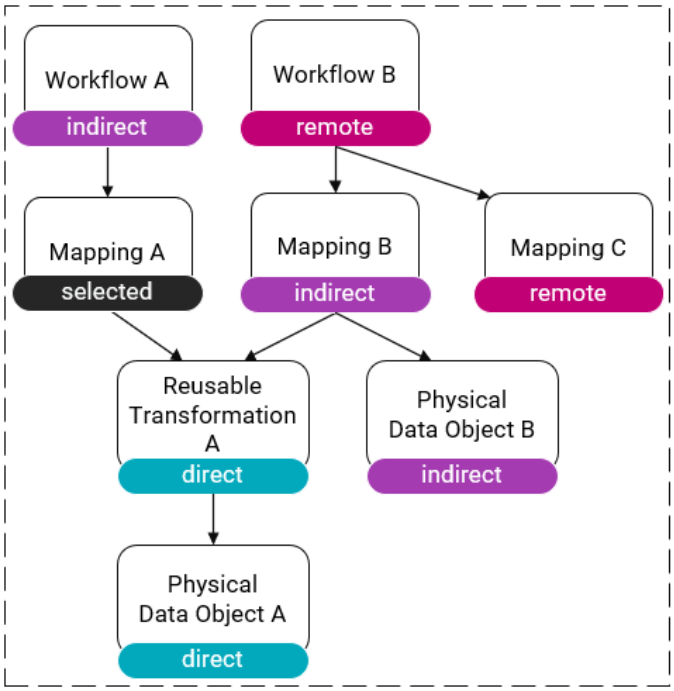
You cannot select mapplets, reusable transformations, and logical data objects in an application patch, but the patch can inherit these objects.

For more information about identifying indirect and remote dependencies, see the following Informatica How-To Library article:

Identifying Indirect and Remote Dependencies for an Application Patch in the Developer Tool

Example. Dependency Sets

The following image shows the set of dependencies for an application object `Mapping A`:



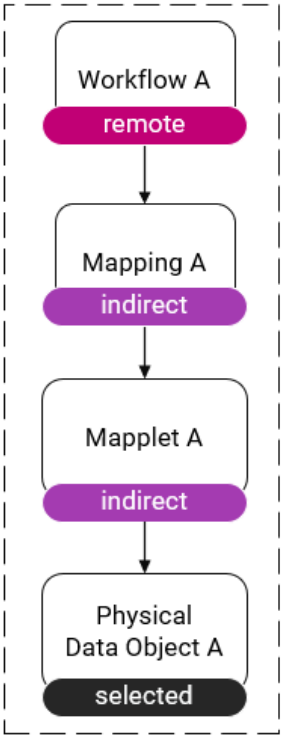
The following table describes the set of dependencies for the mapping `Mapping A`:

Dependency Type	Object	Description
Direct	Reusable Transformation A	The reusable transformation is a direct dependency because the mapping `Mapping A` uses the reusable transformation.
Direct	Physical Data Object A	The physical data object is a direct dependency because the mapping `Mapping A` ultimately uses the data object through the reusable transformation `Reusable Transformation A`.
Indirect	Workflow A	The workflow is an indirect dependency because the workflow uses the mapping `Mapping A`.
Indirect	Mapping B	The mapping `Mapping B` is an indirect dependency because the mapping `Mapping B` uses a direct dependency of the mapping `Mapping A`: the reusable transformation `Reusable Transformation A`.
Indirect	Physical Data Object B	The physical data object is an indirect dependency because it is a direct dependency for the mapping `Mapping B` which is an indirect dependency for the mapping `Mapping A`.
Remote	Workflow B	The workflow is a remote dependency because it is an indirect dependency for the mapping `Mapping B` which is an indirect dependency for the mapping `Mapping A`.
Remote	Mapping C	The mapping `Mapping C` is a remote dependency because it is a direct dependency for the workflow `Workflow B` which is a remote dependency for the mapping `Mapping A`.

Relationship Priority

Relationships between physical data objects, mappings, and workflows take priority over relationships between other types of objects.

For example, the following image shows a set of dependencies for a data object `Physical Data Object A`:



The mapping `Mapping A` is an indirect dependency because the relationship between the mapping `Mapping A` and the data object `Physical Data Object A` takes priority over the following relationships:

- The relationship between the data object `Physical Data Object A` and the mapplet `Mapplet A`.
- The relationship between the mapplet `Mapplet A` and the mapping `Mapping A`.

Since the mapping `Mapping A` is an indirect dependency, its direct dependency `Mapplet A` is an indirect dependency for the data object `Physical Data Object A`.

Incremental Deployment Wizard

The Incremental Deployment Wizard is the wizard that you use to create an application patch.

To run the wizard, right-click an incremental application and select **Update Application Objects**.

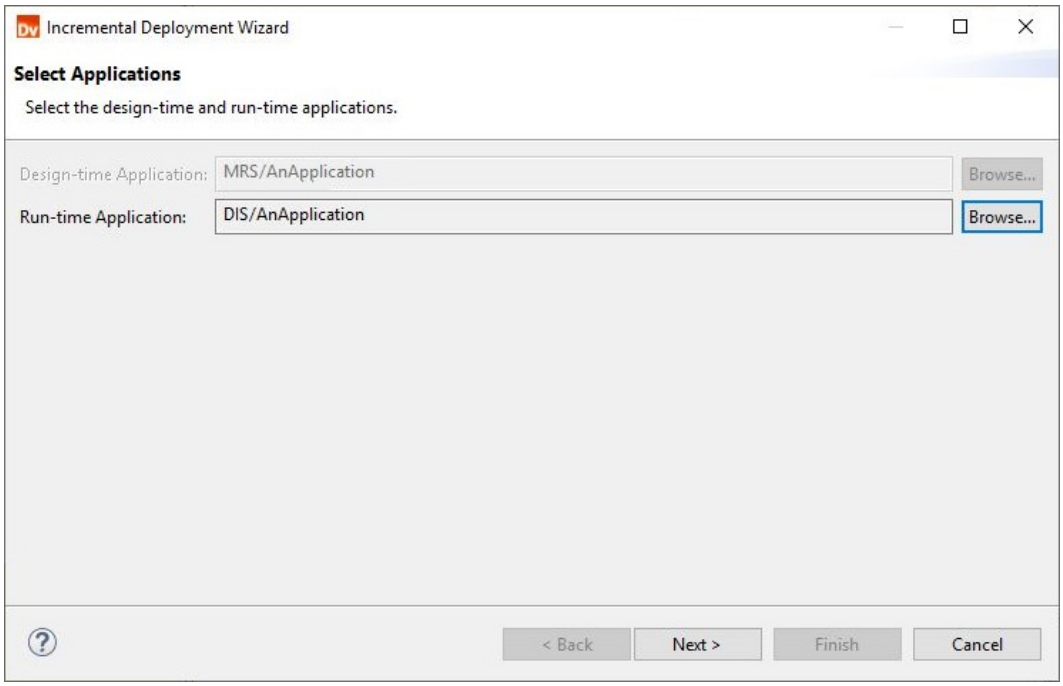
You use the following pages in the wizard to create an application patch:

1. On the **Select Applications** page, you select the run-time application that you want to update.
2. On the **Source Object Selection** page, you choose the objects that you want to include in the patch. You can also preview the type and amount of impact that the patch will have on the run-time application.
3. On the **Patch Deployment Strategy** page, you define additional details about the patch and decide how to deploy the patch.

Select Applications

On the **Select Applications** page, select a run-time application that you want to update.

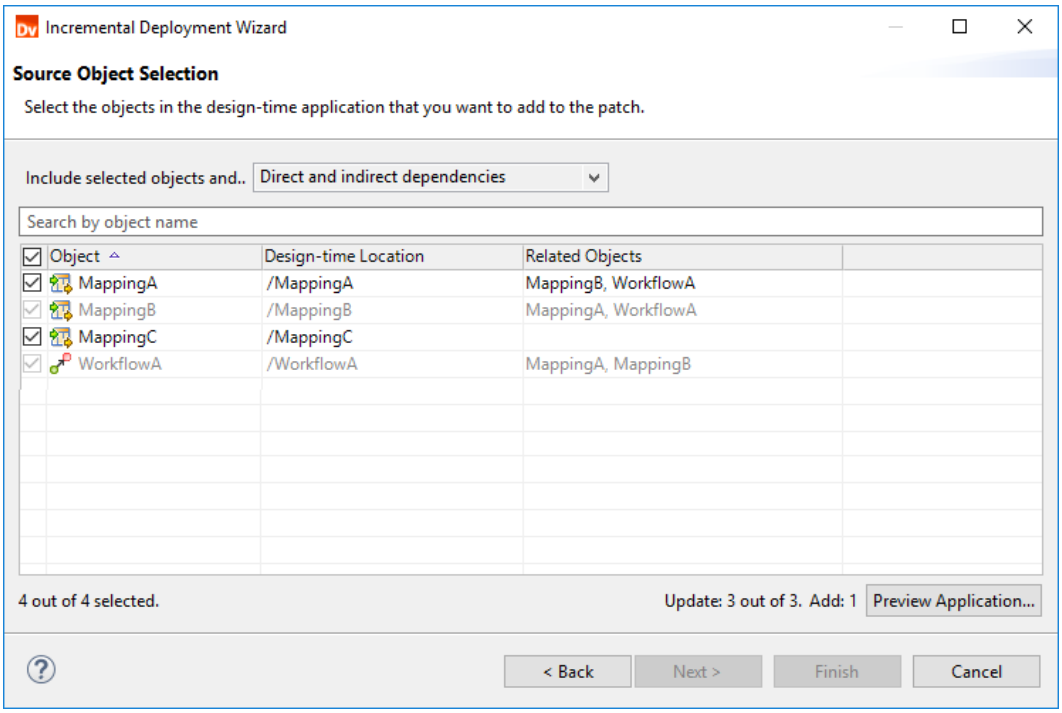
The following image shows the **Select Applications** page:



Source Object Selection

On the **Source Object Selection** page, choose the type of patch that you want to create and select the objects that you want to include in the patch.

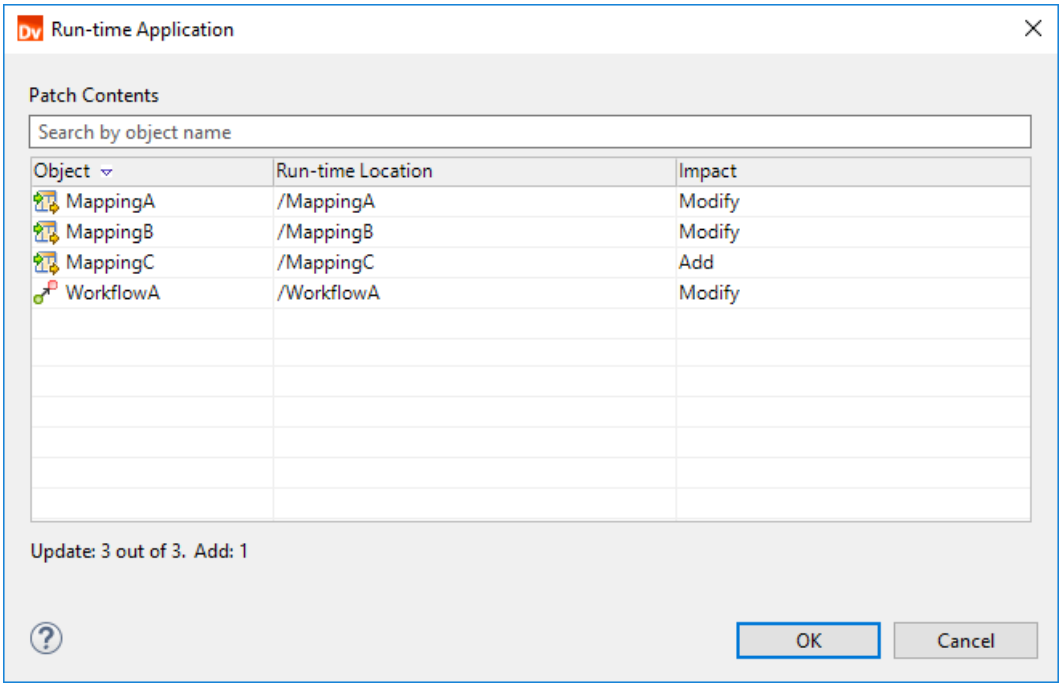
The following image shows the **Source Object Selection** page:



For each object, you can use the Design-time Location column to view the path to the object in the design-time application. The wizard also lists the objects' dependencies in the Related Objects column based on the patch type. For example, for a patch type that inherits direct and indirect dependencies, the wizard lists each object's direct and indirect dependencies.

In the bottom-right corner of the page, you can preview a summary of the impact on the run-time objects. The Update statistic shows the total number of objects that will be modified or affected. The Add statistic shows the total number of objects that will be added. To view the impact on a particular run-time object, you can preview the application.

The following image shows the application preview:

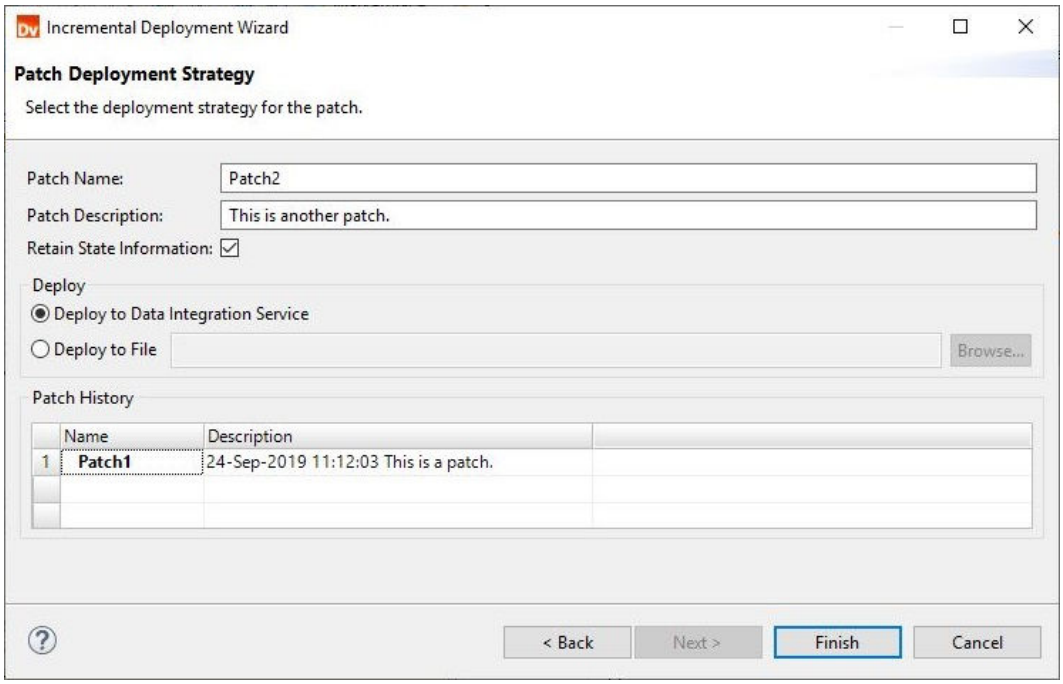


You can match the list of run-time objects in the preview to the design-time objects by matching the object names and matching the run-time location to the design-time location. For each object, the impact lists whether the object will be modified, affected, added, or unchanged.

Patch Deployment Strategy

On the **Patch Deployment Strategy** page, describe the patch, define how you want to deploy it, and choose whether the patch retains state information.

The following image shows the **Patch Deployment Strategy** page:



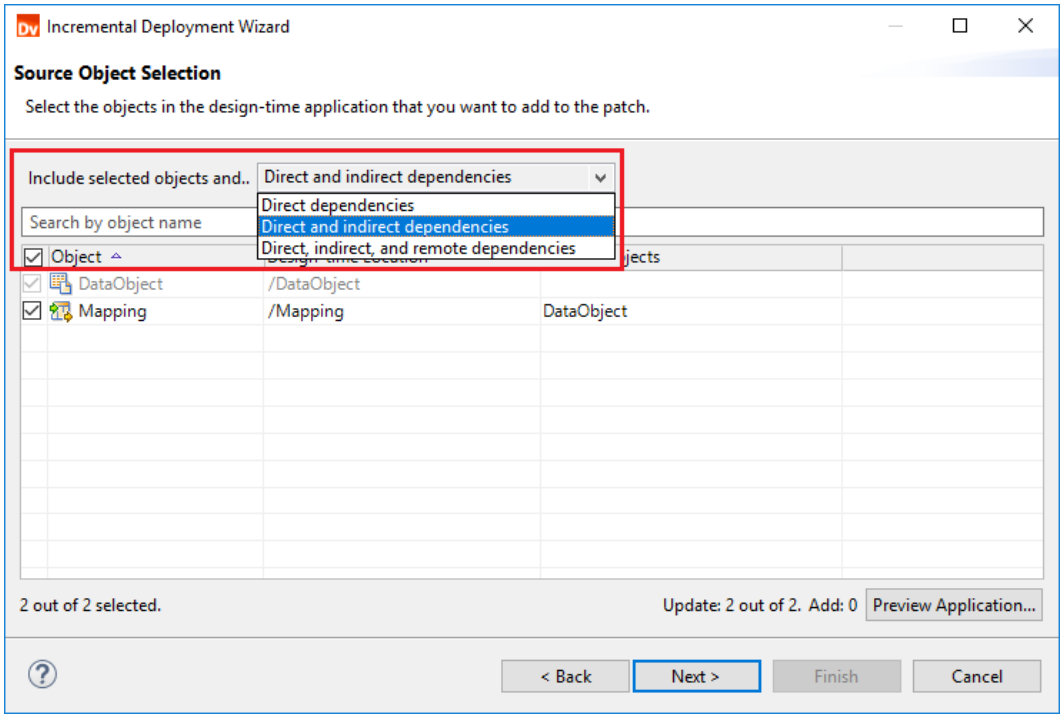
When you choose a deployment strategy, you can also use the patch history to review the patches that have been deployed to update the run-time application.

Choose the Application Patch Type

When you use the Incremental Deployment Wizard, you can choose the type of application patch that you want to create. The application patch type determines the level of dependencies that the patch inherits.

A patch can inherit direct, indirect, and remote dependencies. With each level of dependencies, the patch inherits more objects. The inherited objects are included in the patch, and the overall impact that the patch has on the run-time application increases. You can choose the dependencies to inherit depending on the application design and the application objects that you want to update.

The following image shows the list of patch types in the wizard:

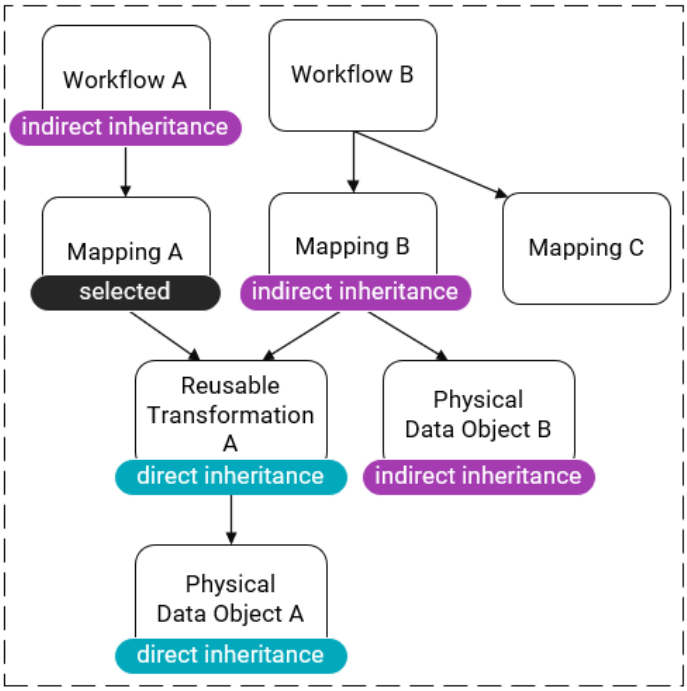


When you deploy the patch, the Data Integration Service replaces the run-time instances of the selected objects and the inherited dependencies in the run-time application, and it might affect other run-time objects. The Data Integration Service updates the affected objects to use the replaced run-time objects, but the affected objects are not replaced with their design-time instances.

If the application contains highly-dependent objects, the dependencies that an application patch inherits can be inversely related to the number of affected objects in the run-time application. As an application patch inherits fewer dependencies, the run-time application might contain more affected objects.

Example. Dependency Inheritance

The following image shows the direct and indirect dependencies that a patch inherits where the selected object is a mapping `Mapping A`:



The patch inherits the following direct and indirect dependencies:

- Direct dependencies. The patch inherits the reusable transformation `Reusable Transformation A` and the data object `Physical Data Object A` because the objects are direct dependencies for the selected mapping `Mapping A`.
- Indirect dependencies. The patch inherits the mapping `Mapping B` and the data object `Physical Data Object B` because the objects are indirect dependencies for the selected mapping `Mapping A`.

Note that the patch does not inherit the workflow `Workflow B` and the mapping `Mapping C` because the objects are remote dependencies for the selected mapping `Mapping A`.

Inheriting Only Direct Dependencies

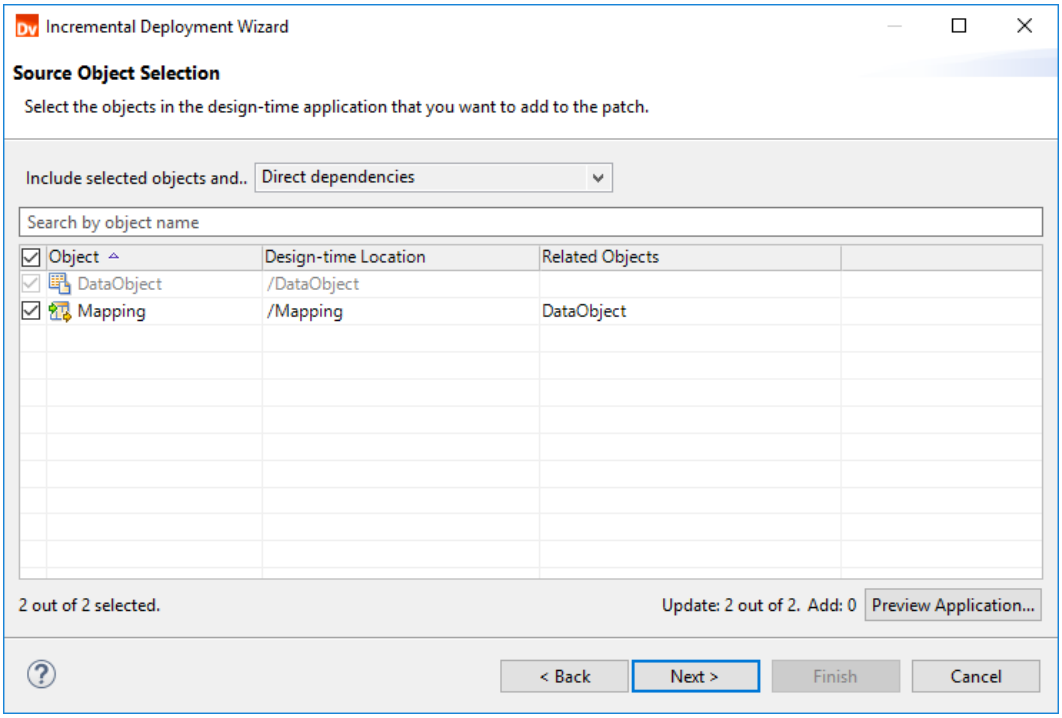
Create a patch that inherits only direct dependencies when you do not have complete information about the design-time and run-time applications in order to minimize the impact that you have on the objects in the run-time application.

Example

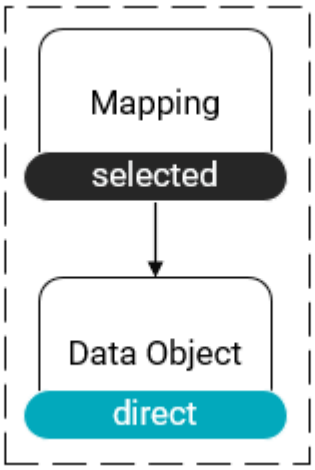
You share an application with thirty other users. The application object that you need to update is a single streaming mapping that reads data from a Kafka data object. After testing the mapping in the run-time application, you edit the mapping in the Developer tool and deploy an application patch to update the mapping in the run-time application.

When you create the application patch, you select the mapping and choose to inherit only direct dependencies. When you select the mapping, the patch inherits the data object that the mapping uses.

The following image shows how the inheritance appears in the Incremental Deployment Wizard:



The following image indicates the selected object and the inherited direct dependency:



Inheriting Direct and Indirect Dependencies

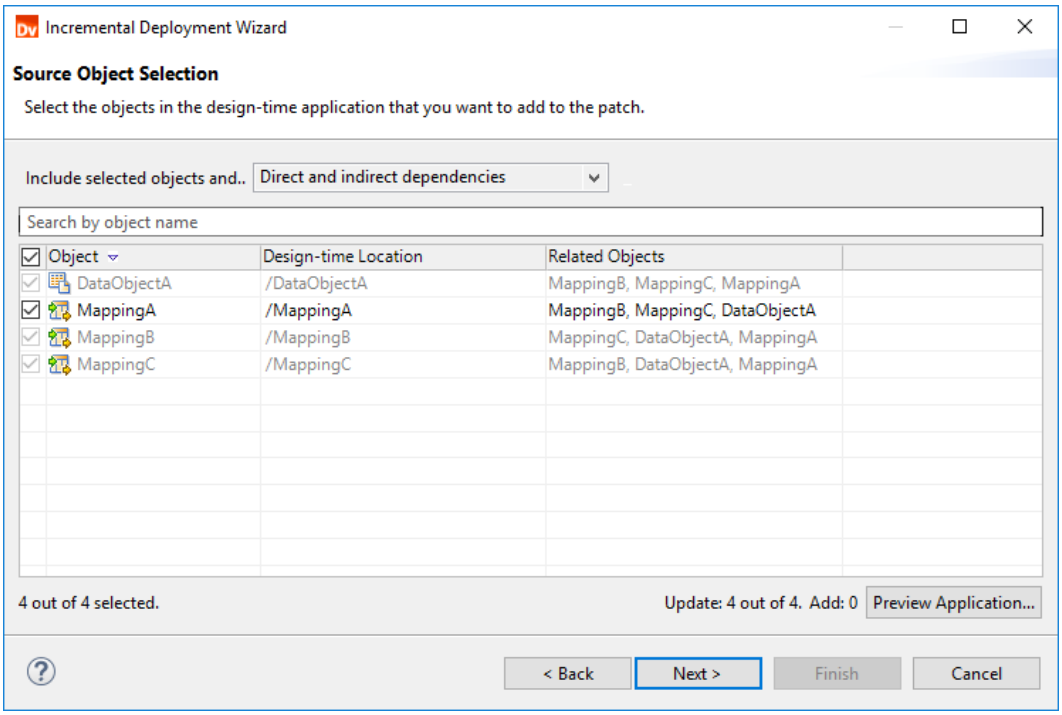
Choose to inherit direct and indirect dependencies when you edit an application object that is used by many other application objects in order to propagate the changes to the parent objects.

Example

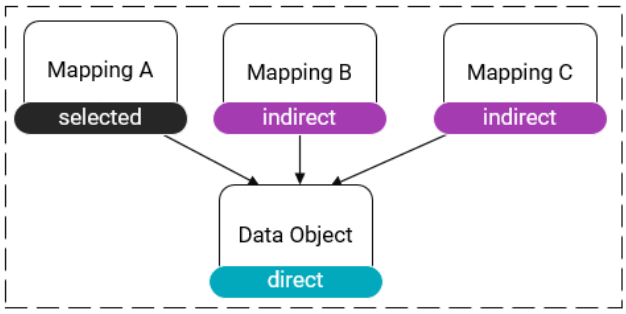
You have an application that contains multiple mappings that use the same relational data object. Each mapping ultimately transforms the data in the data object for a different purpose. To schedule mapping runs, you deploy the application to the Data Integration Service.

Later, you use the Developer tool to synchronize the relational data object with the relational database. To propagate the synchronized data to the mappings in the run-time application, you create a patch that inherits direct and indirect dependencies:

When you create the patch, you select one of the mappings that uses the data object and the patch inherits every other mapping that uses the data object. The following image shows how the inheritance appears in the Incremental Deployment Wizard:



The following image indicates the selected object and the inherited indirect dependencies:



Inheriting Direct, Indirect, and Remote Dependencies

Inherit direct, indirect, and remote dependencies when you have complete information about the design-time and run-time applications.

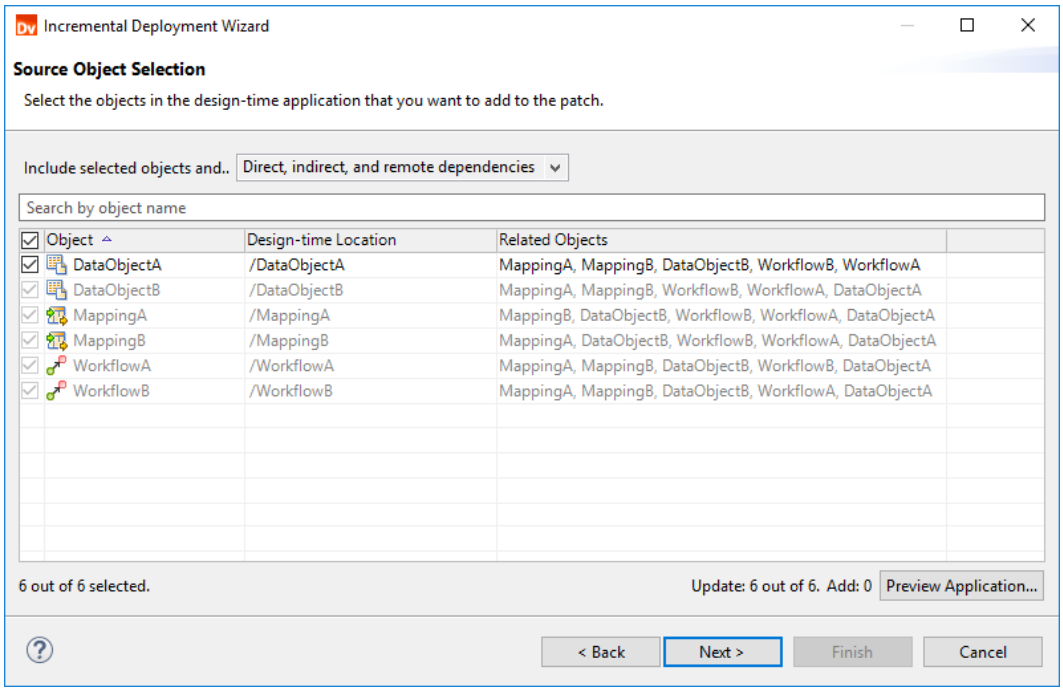
You might have complete information about an application when you are the sole developer or the functional administrator for an application. When you make changes to the design-time application, you might expect all of the changes to be propagated to the run-time application to ensure that the run-time application mirrors the design-time application.

Example

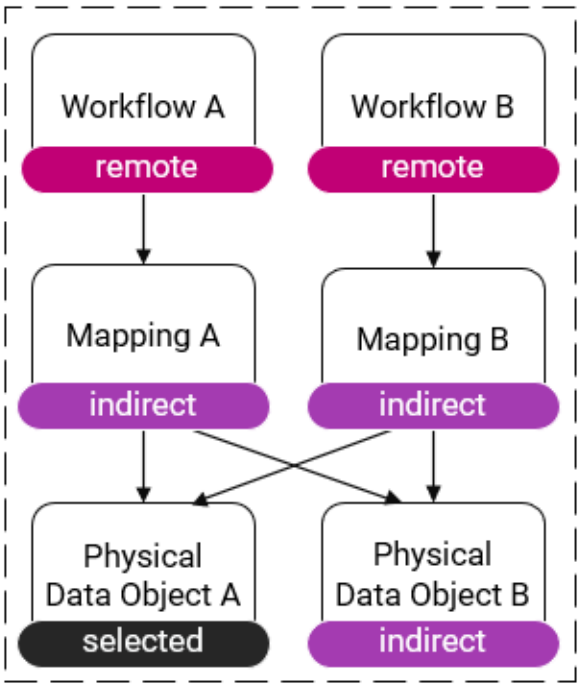
You manage an application that contains workflows that run mappings which share different data objects. After you deploy the application and test the outputs, you edit one of the data objects.

To propagate the changes in the data object to every other application object that uses the data object, you create a patch that inherits direct, indirect, and remote dependencies. When you select the data object, the patch inherits any workflows and mappings that use the data object.

The following image shows how the inheritance appears in the Incremental Deployment wizard:



The following image indicates the selected object and the inherited direct, indirect, and remote dependencies:



Note: To propagate the changes in the data object `Physical Data Object A` to the mappings and workflows that use the data object in the run-time application, you can also use a patch that inherits only direct dependencies. If you select the data object, the wizard identifies the mappings and workflows to be affected objects and the Data Integration Service updates the mappings and workflows during patch deployment.

A patch that inherits direct, indirect, and remote dependencies, however, provides more transparency about how the objects will be updated. It provides a higher guarantee that updated objects in the run-time application will transform data in the same way as the objects in the design-time application.

Determine Application Object Impact

When you create and deploy an application patch, the impact that the patch has on the run-time application extends to the objects that you modify as well as any affected objects. If multiple users share the application, the patch might impact objects that other users maintain.

To understand how much impact a patch will have on the run-time application, you can perform the following tasks:

- Preview the object impact. The object impact is a summary that describes how each application object will be impacted.
- Identify the affected objects. Affected objects might be objects that belong to other users. You can identify affected objects to understand how other users' objects might change.
- Review the scope of the patch. The scope is a numerical summary of how much many objects a patch will impact.

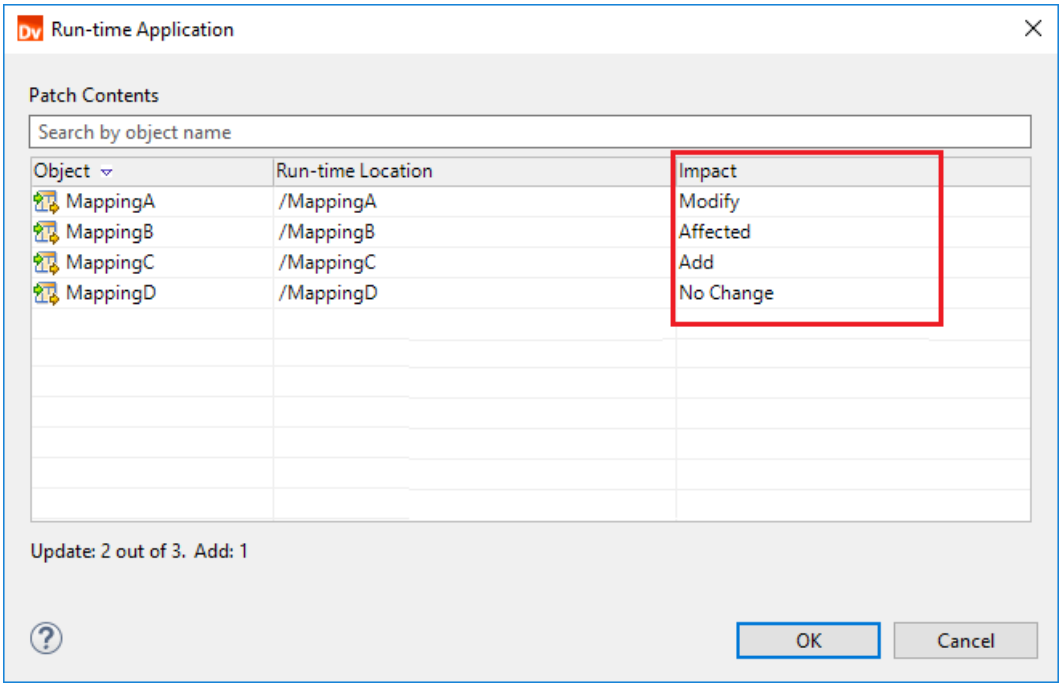
Preview Object Impact

The Incremental Deployment Wizard analyzes the impact to the run-time application while you add objects to the application patch and the patch inherits the objects' dependencies. You can preview the impact on each application object to understand how the run-time instance of an object will change after you deploy the patch.

The preview lists the following types of impact for each application object:

- Modify. A run-time instance of the object exists in the run-time application, and the Data Integration Service will replace the run-time instance with the design-time instance.
- Affected. The wizard will fetch the object from the run-time application because its run-time dependencies will be modified. The Data Integration Service updates the affected object in the run-time application to use the modified dependencies, but the service does not replace the it.
- Add. A run-time instance of the object does not exist in the run-time application, so the Data Integration Service will create a run-time instance of the object in the run-time application.
- No Change. The object will not be modified, affected, or added to the run-time application.

The following image shows where you can preview the object impact in the Developer tool:



Identify Affected Objects

An affected object is an object that the Incremental Deployment Wizard fetches from the run-time application in order to maintain the deployed application. Users who own the affected objects will not be able to run the objects while the Data Integration Service applies a patch.

The wizard fetches affected objects while you design the application patch. You can view the objects that are affected when you preview object impact.

The wizard fetches affected objects for the following reasons:

Change propagation

When you select objects in an application patch and the patch inherits other application objects, the wizard must decide how the changes in the objects will be propagated to their parent objects in the run-time application. The wizard identifies the parent objects to be affected due to the changes.

When you deploy the patch, the Data Integration Service does not replace the run-time instances of affected objects, but it updates the affected objects to consume any changes that are made to their run-time direct dependencies.

Note: If a workflow is an affected object, the workflow is updated the next time that it runs. For example, if you modify a mapping and a workflow uses the mapping in a Mapping task, the workflow uses the modified mapping in subsequent executions.

Validity

The wizard must ensure that application objects in the run-time application will be valid after the Data Integration Service applies a patch. After the wizard decides how changes will be propagated to affected objects, the wizard validates the affected objects. An affected object might not be valid if it is not compatible with a modified object.

Any application patch type can result in affected objects. For example, when you create a patch that inherits direct, indirect, and remote dependencies, a remote dependency might have a direct dependency that the patch inherits. If the direct dependency is used by a different object in the run-time application, the parent object becomes an affected object.

After the Data Integration Service updates an affected object, the data in the affected object might not be consistent with the application design. Consider a run-time application that contains a mapping which uses a mapplet. If the modified mapplet uses a different transformation, the affected mapping might be valid but it might also transform the data in an unpredictable way.

If possible, try to select the objects that you want to update, or design an application patch to inherit the objects to minimize the number of affected objects and guarantee that the objects in the run-time application transform data in the same way as the objects in the design-time application.

Example. Change Propagation and Validity

You deploy a patch that modifies a data object. The wizard propagates the modifications in the data object to any other run-time objects that use the data object. As a result, an affected run-time mapping that uses the data object will use the modified data object after the Data Integration Service applies the patch.

If a mapping uses the modified data object and the data object contains fewer rows than before the patch was applied, the mapping might no longer be valid. The data object might not be compatible with the transformations in the mapping due to a fewer number of links between upstream and downstream transformations. If the affected mapping is not valid, you cannot deploy the patch.

Deleted Objects

When you delete an application object from the design-time application and deploy a patch, the Data Integration Service does not delete the object from the run-time application. In some cases, the deleted object might be an affected object.

If the wizard identifies a deleted object as an affected object, the wizard will attempt to propagate changes in the deleted object's child objects to the deleted object. The wizard will also attempt to validate the deleted object. If the deleted object is not valid, you cannot deploy the patch.

If users are not running deleted objects, redeploy the application to remove the objects from the run-time application.

Example. Identifying Affected Objects

The following example shows how you can identify the affected objects that the Incremental Deployment Wizard fetches from the run-time application.

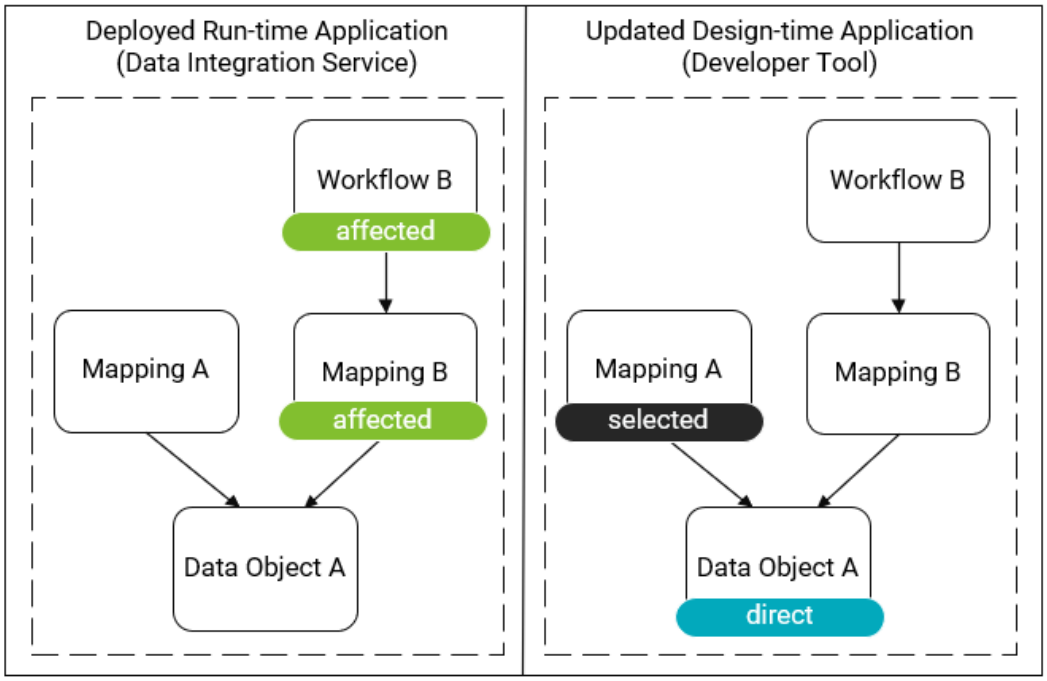
You might have an application that contains two mappings `Mapping A` and `Mapping B` that share a physical data object `Data Object A`, and a workflow `Workflow B` that uses the mapping `Mapping B`. After you deploy the application, you edit the mapping `Mapping A` and decide to update the mapping in the run-time application.

You create a patch that inherits only direct dependencies and select the mapping `Mapping A`. Because you select the mapping, the patch inherits the data object that the mapping uses, `Data Object A`.

Since the wizard expects the Data Integration Service to replace the run-time instances of the mapping and the data object, the wizard locates any parent objects that use the mapping or the data object in the run-time application. The mapping `Mapping B` uses the data object, so the wizard marks the mapping as an affected object.

Because the mapping `Mapping B` will be updated in the run-time application and the workflow `Workflow B` uses the mapping `Mapping B`, the workflow `Workflow B` is also an affected object.

The following image shows the design-time and run-time conditions, the selected mapping, the dependencies that the patch inherits, and the affected objects:



Note: The run-time instances of the objects `Mapping B` and `Workflow B` will not be replaced with their design-time instances. If you edited the mapping `Mapping B` in the design-time application, the run-time instance of `Mapping B` will be different from the design-time instance.

Identify the Scope of Object Impact

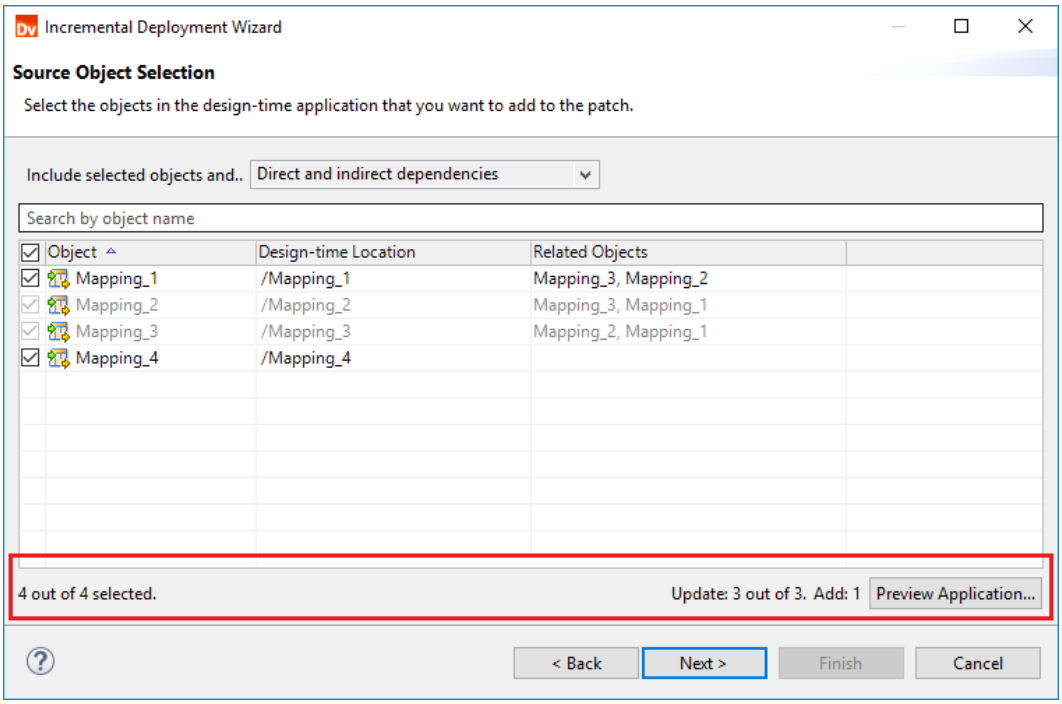
Identify the scope of the impact that an application patch will have on the deployed application to understand how many objects will experience down-time during patch deployment. When an application has multiple users, a larger scope lowers the number of objects that are available to those users.

To identify the scope, use values from the impact summary based on the following object types:

- Selected objects. The sum of the objects that you select to appear in the patch and the objects that the patch inherits. The maximum value is the number of application objects in the design-time application.
- Updated objects. The sum of the objects that will be modified or affected. The maximum value is the number of application objects in the run-time application.
- Added objects. The number of objects that will be added to the run-time application.

Note: The number of selected objects might be different from the number of updated objects if the design-time application contains a different set of dependencies from the run-time application.

The following image shows where the Incremental Deployment wizard displays the summary:



Specify a Patch Deployment Strategy

Specify a patch deployment strategy to determine how you want to make the contents of an application patch available to end users depending on organizational policies.

You can deploy an application patch in the following ways:

- Deploy to a Data Integration Service. When you deploy the patch to a Data Integration Service, the service automatically updates the corresponding incremental application based on the patch contents. Objects that are not impacted will continue running, and impacted objects will be updated with the patch contents.

- Deploy to an archive file. When you deploy the patch to an archive file, you can save the patch as a file on your local machine, store the file in a version control system, and deploy the patch to a Data Integration Service at a later time.

Deploying a patch does not change the state of the application. For example, a stopped application does not begin running.

If you cancel the deployment, the Data Integration Service might partially apply the patch contents to the deployed application. Canceling deployment does not roll back the incremental application.

Patch Deployment Strategy Properties

On the **Patch Deployment Strategy** page, you can configure general properties and deployment options. You can also review the patch history of the run-time application.

The following table describes the general properties that you can configure:

General Property	Description
Patch Name	Name of the patch.
Patch Description	Optional. Description of the patch. The patch description cannot exceed 270 characters.
Retain State Information	Optional. Indicates whether state information is retained or discarded. State information refers to mapping properties and the properties of run-time objects such as mapping outputs or the Sequence Generator transformation. By default, state information is retained. **Note:** Your choice to retain or discard application state has no effect on the Reset property of a Sequence Generator transformation. For more information about state information, see "Application State Information" on page 142.

The following table describes the patch deployment options that you choose from:

Deployment Option	Description
Deploy to Data Integration Service	Select this option to deploy the patch to a Data Integration Service.
Deploy to File	Select this option to deploy the patch to an application patch archive file.

The following table describes the properties that you can review for each patch in the patch history:

Patch Property	Description
Name	Name of the deployed patch.
Description	Description of the deployed patch. The time that the patch was created is appended to the beginning of the patch description.

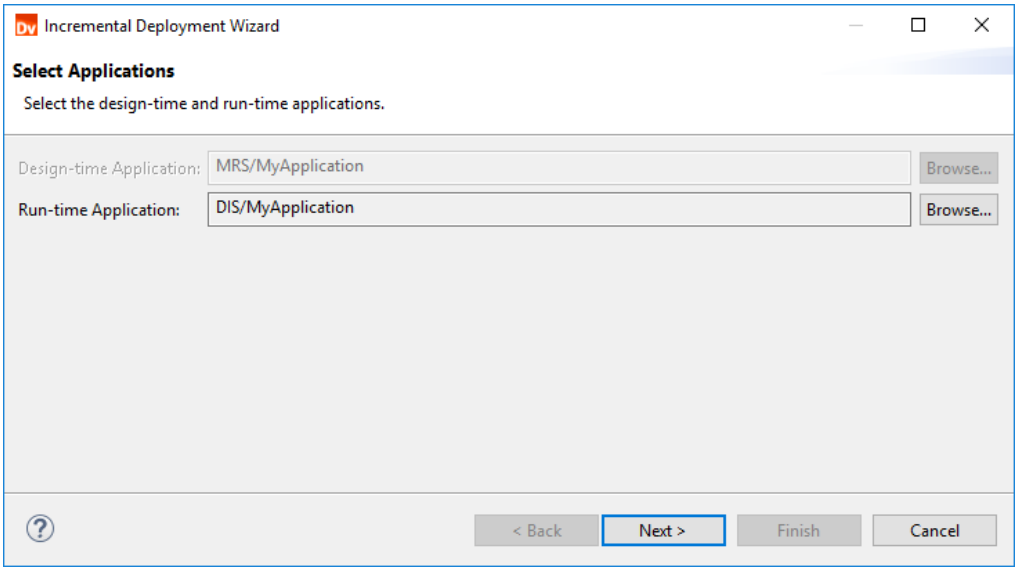
Note: By default, the patch history lists patches by the time that they were created.

Deploying an Application Patch

If you edit an object in an incremental application, you can update the object in the deployed application by creating and deploying an application patch. Deploy the application patch to a Data Integration Service or to an application patch archive file.

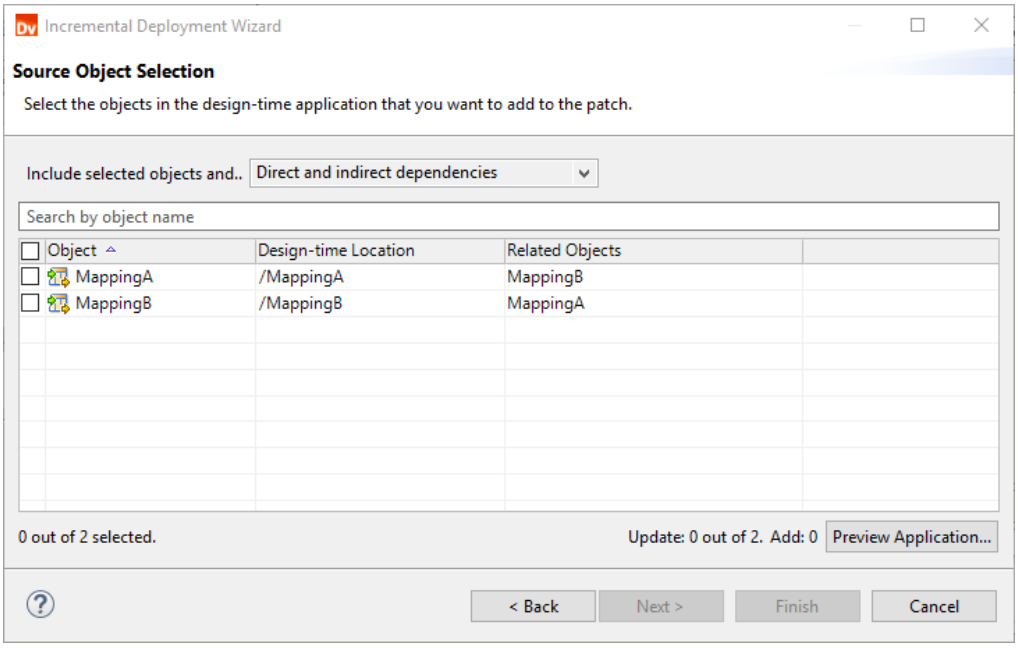
1. In the **Object Explorer** view, right-click an incremental application and select **Update Application Objects**.

   The **Select Applications** page of the Incremental Deployment Wizard appears.



2. On the **Select Applications** page, click **Browse** and select the run-time application that you want to update on the Data Integration Service.
3. Click **Next**.

   The wizard fetches the run-time application. Then, the **Source Object Selection** page appears.

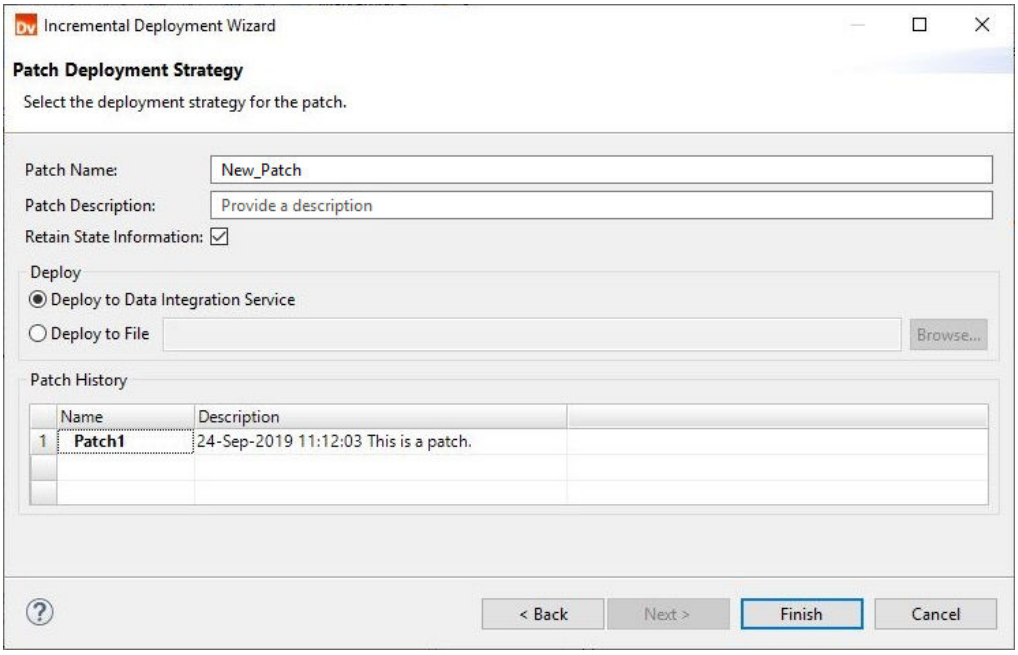


4. On the **Source Object Selection** page, select the objects in the design-time application that you want the Data Integration Service to replace in the run-time application.
5. Select one of the following application patch types:
   - Direct dependencies. The patch inherits the direct dependencies of the selected objects.
   - Direct and indirect dependencies. The patch inherits the direct and indirect dependencies of the selected objects.
   - Direct, indirect, and remote dependencies. The patch inherits the direct, indirect, and remote dependencies of the selected objects.

   Default is Direct and indirect dependencies.

   The dependencies appear in the **Related Objects** column. For example, for an application patch that inherits direct and indirect dependencies, the column lists the selected objects' direct and indirect dependencies.
6. To review the impact on each object in the run-time application, click **Preview Application**.
7. Click **Next**.

   The **Patch Deployment Strategy** page appears.



8. Enter the name of the patch.
9. Optionally, enter a description of the patch.
10. Optionally, choose to retain state information.

    If you choose to not retain state information, state information will be discarded.
11. Select one of the following deployment options:
    - Deploy to Data Integration Service. Deploys the application patch to the Data Integration Service. The Data Integration Service applies the application patch to the run-time application.
    - Deploy to File. Deploys the application patch to an archive file. If you deploy the patch to an archive file, select a directory on your local machine where you want to save the file.
12. Review the patch history.

    The patch history lists the application patches that have been deployed to update the incremental application.

Rules and Guidelines for Application Patch Deployment

Refer to the following rules and guidelines when you update an incremental application by deploying an application patch:

- If you select to update application objects from the Object Explorer view, the Incremental Deployment Wizard begins constructing the application dependencies. You can view the progress in the Progress view, but you cannot cancel the operation.
- When you create a patch, the Incremental Deployment wizard does not list any objects that are not objects in the application.

  For example, an application can contain a mapping and exclude the data objects that the mapping uses to transform data. In this case, the data objects are not application objects. When you create a patch, you will not be able to see the data objects, but the wizard will process the data objects when it identifies the dependencies between application objects.
- If you change the name or the project of an application object in the design-time application and update the object using an application patch, the object is duplicated in the run-time application under different paths.
- If the application description of the run-time application is empty the first time that you deploy a patch, the Incremental Deployment Wizard updates the description using the application's patch history. When you deploy additional patches, the wizard appends the patch names to the description. The wizard continues appending patch names until you edit the application description using any client.

CHAPTER 11

Application Patch Deployment Example

This chapter includes the following topics:

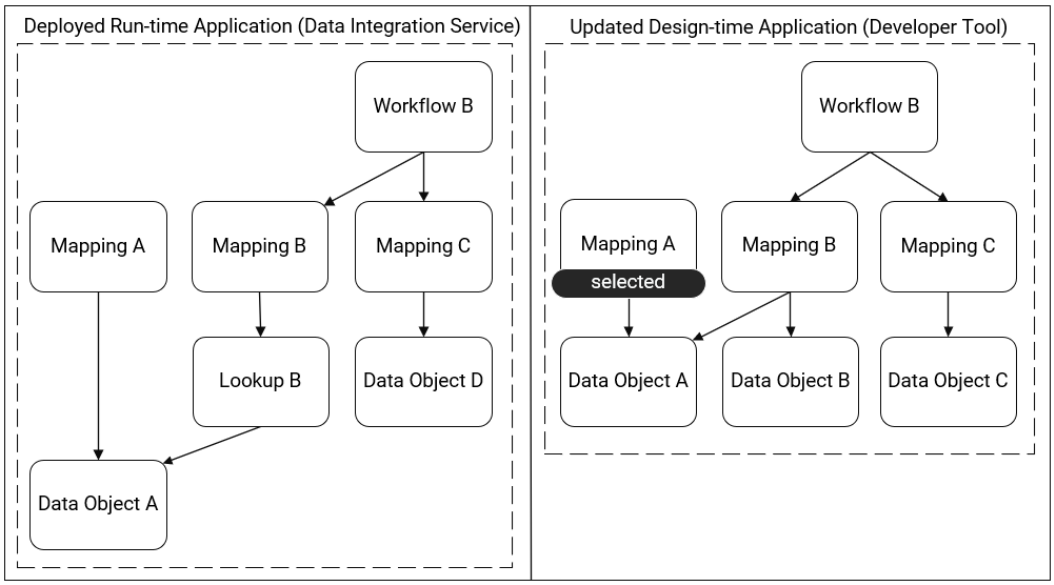
- Application Patch Deployment Example Overview, 174
- Patch Type: Direct, 176
- Patch Type: Direct and Indirect, 178
- Patch Type: Direct, Indirect, and Remote, 180
- Conclusion, 182

Application Patch Deployment Example Overview

You are a DevOps engineer who observes changes in design-time objects and merges the changes to the corresponding run-time objects in a deployed application on the Data Integration Service.

Recently, a developer edited a mapping `Mapping A` in an incremental application. You want to create a patch to update the mapping in the run-time application.

The following image shows the run-time application on the Data Integration Service and the design-time application in the Developer tool where the mapping `Mapping A` is the selected object:



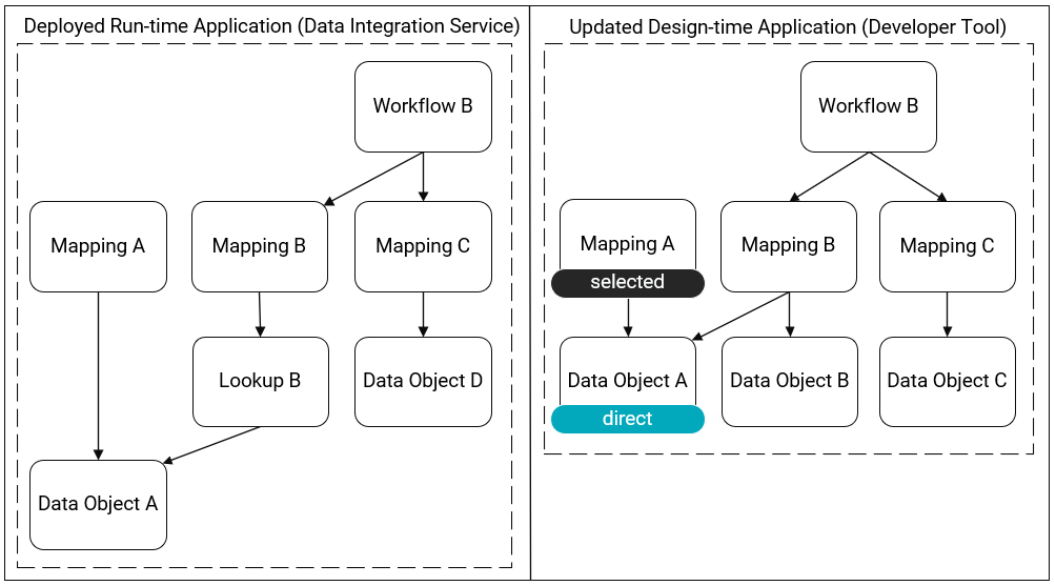
To create an application patch, you need to choose the dependencies that you want the patch to inherit. You identify the inherited objects, assess the object impact, and evaluate the scope of the changes to the deployed application.

Patch Type: Direct

This section describes an application patch that inherits only direct dependencies.

Identify the Inherited Objects

The following image shows the objects that the patch includes when you inherit only direct dependencies:

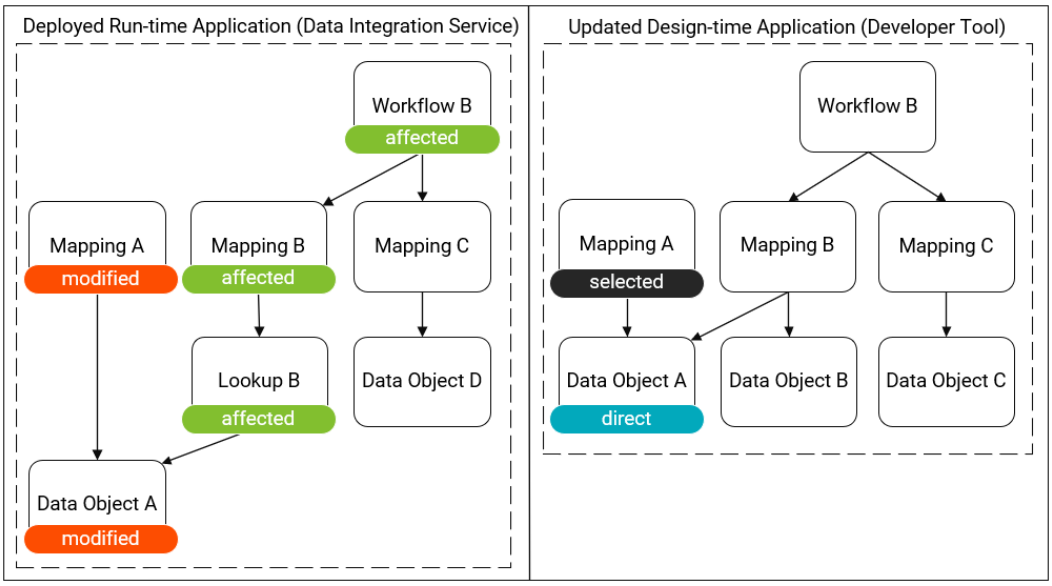


The following table lists the dependencies that the patch inherits when the mapping `Mapping A` is the selected object:

Type of Dependency	Dependencies
Direct	- Data Object A

Assess the Object Impact

The following image shows the impact to the run-time objects after the patch is deployed:



Evaluate the Scope of the Changes

The following table shows the number of objects that are impacted during patch deployment:

Scope Item	Number of Objects
Selected	2
Updated	Modified: 2 Affected: 3* Total: 5*
Added	0
* *The reusable transformation* `Lookup B` *behaves as an affected object at run time, but the Incremental Deployment wizard does not list reusable transformations as affected objects. In the Incremental Deployment wizard, the number of affected objects is 2.*	

Findings

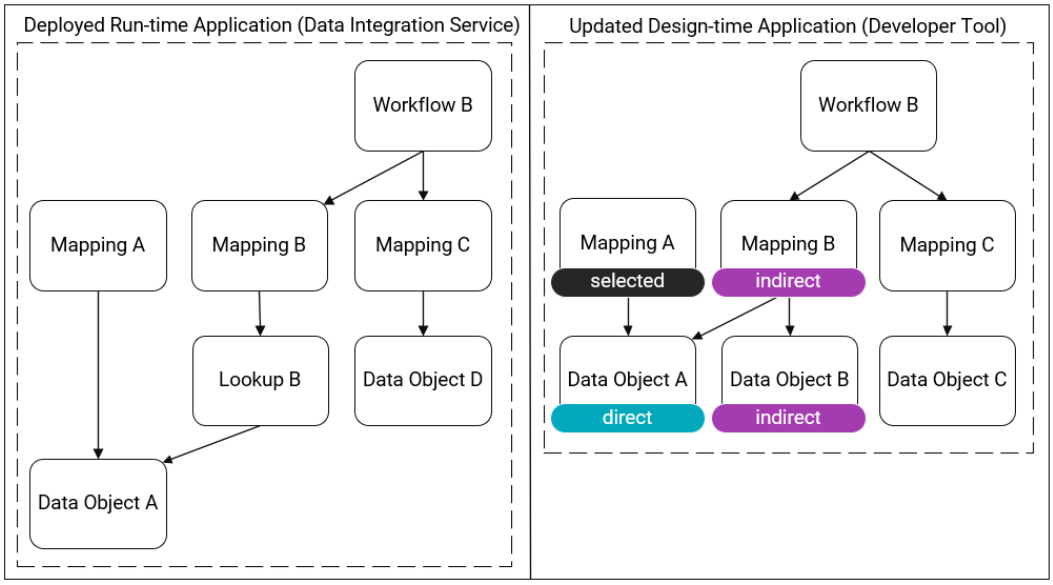
A patch that inherits only direct dependencies has a low impact on the run-time application, but the run-time application contains affected objects. Affected objects can be unpredictable compared to modified objects, so there is less transparency about how the application will transform data after you apply the patch.

Patch Type: Direct and Indirect

This section describes an application patch that inherits direct and indirect dependencies.

Identify the Inherited Objects

The following image shows the objects that the patch includes when you inherit direct and indirect dependencies:

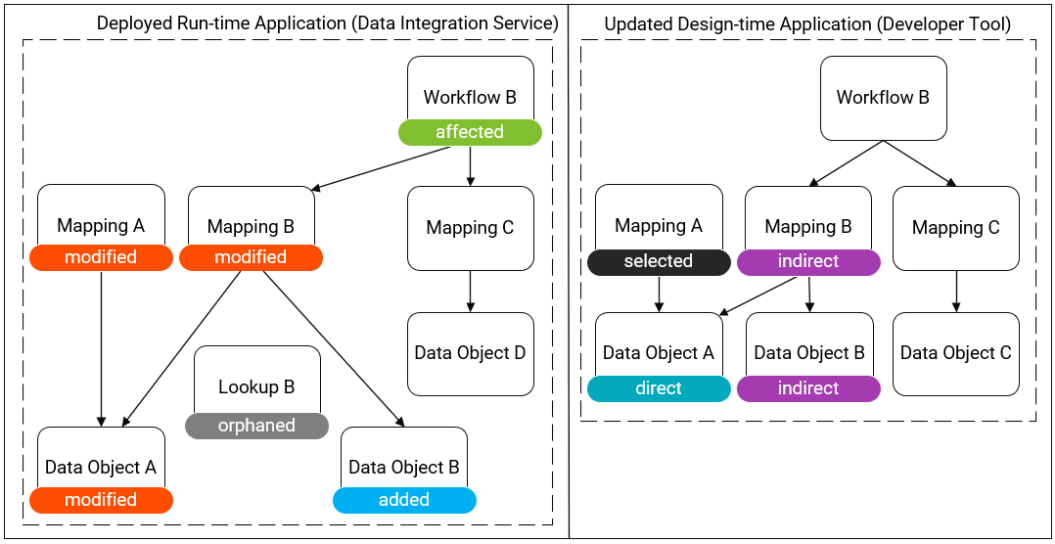


The following table lists the dependencies that the patch inherits when the mapping `Mapping A` is the selected object:

Type of Dependency	Dependencies
Direct	- Data Object A
Indirect	- Mapping B - Data Object B

Assess the Object Impact

The following image shows the impact to the run-time objects after the patch is deployed:



Because the patch inherits the mapping `Mapping B` as an indirect dependency, the mapping will use its design-time dependencies and discard its previous run-time dependencies. It no longer uses the reusable transformation `Lookup B` in the run-time application. Instead, it uses the data objects `Data Object A` and `Data Object B`. The transformation `Lookup B` becomes an orphan in the run-time application.

Evaluate the Scope of the Changes

The following table shows the number of objects that are impacted during patch deployment:

Scope Item	Number of Objects
Selected	4
Updated	Modified: 3 Affected: 1 Total: 4
Added	1

Findings

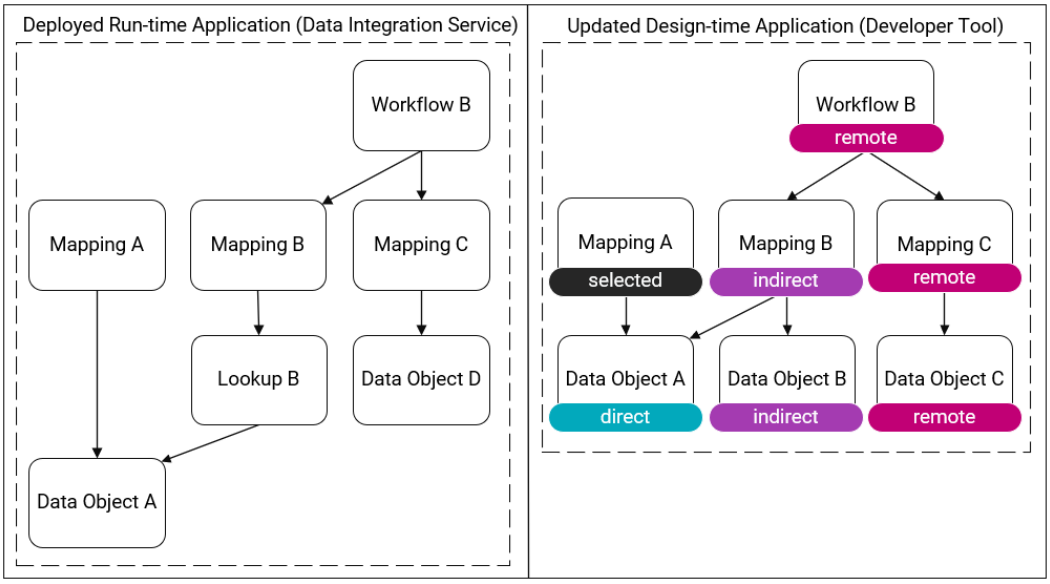
Inheriting direct and indirect dependencies has a similar impact to inheriting only direct dependencies, but fewer objects are affected. The mapping `Mapping B` is modified instead of affected, so it is updated in a more predictable way.

Patch Type: Direct, Indirect, and Remote

This section describes an application patch that inherits direct, indirect, and remote dependencies.

Identify the Inherited Objects

The following image shows the objects that the patch includes when you inherit direct, indirect, and remote dependencies:

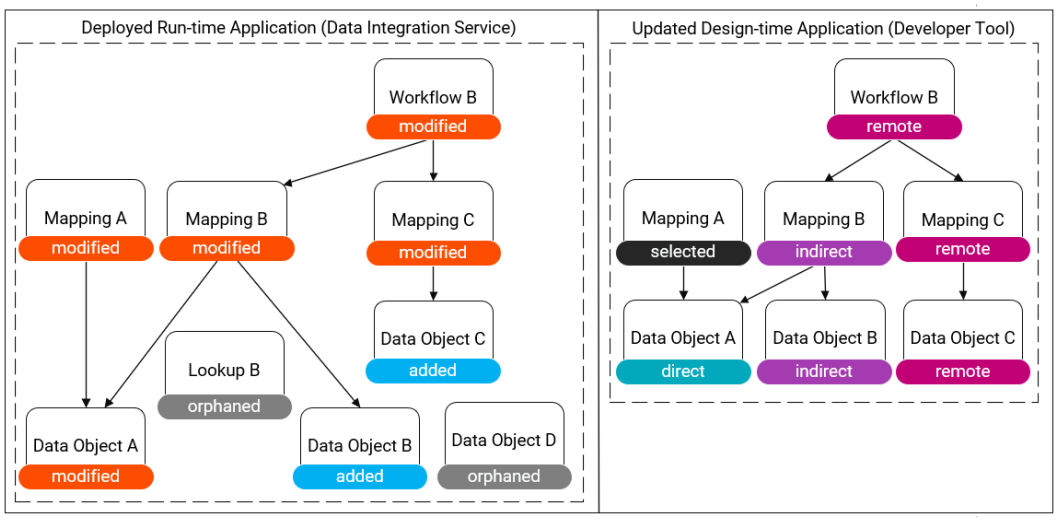


The following table lists the dependencies that the patch inherits when the mapping `Mapping A` is the selected object:

Type of Dependency	Dependencies
Direct	- Data Object A
Indirect	- Mapping B - Data Object B
Remote	- Workflow B - Mapping C - Data Object C

Assess the Object Impact

The following image shows the impact to the run-time objects after the patch is deployed:



Because the patch inherits mappings `Mapping B` and `Mapping C`, the mappings use their design-time dependencies in the run-time application. The reusable transformation `Lookup B` and the data object `Data Object B` become orphans in the run-time application.

Evaluate the Scope of the Changes

The following table shows the number of objects that are impacted during patch deployment:

Scope Item	Number of Objects
Selected	7
Updated	Modified: 5 Affected: 0 Total: 5
Added	2

Findings

If you create a patch that inherits direct, indirect, and remote dependencies, the changes to the run-time application are transparent. The updated and added objects in the run-time application match the selected objects in the design-time application and you can expect the run-time application to transform data in the same way as the objects in the design-time application.

The trade-off is that all objects experience down-time, and you might be better off by redeploying the application. When you redeploy the application, you update the same objects with the benefit of removing the orphaned objects from the run-time application: the reusable transformation `Lookup B` and the data object `Data Object D`.

Conclusion

In this example, creating a patch that inherits direct and indirect dependencies has the least impact on the run-time application, so it maximizes object up-time. It allows you to take full advantage of the abilities of an incremental application.

You might choose to create a patch that inherits direct and indirect dependencies if the application is deployed in a development or QA environment. For example, you might want to test that the selected mapping `Mapping A` runs successfully using sample data before you begin processing enterprise data.

If the application is deployed in a user acceptance testing (UAT) or production environment and you have the capacity to stop the application, you might choose to redeploy the application instead. Redeploying the application creates the most transparent changes in the run-time application to ensure that the data is transformed according to your requirements.

CHAPTER 12

Continuous Integration and Continuous Delivery (CI/CD)

This chapter includes the following topics:

- CI/CD Overview, 183
- REST API and infacmd Quick Reference, 184
- Continuous Integration, 187
- Continuous Delivery, 188
- Sample Automated Integration Process, 189
- Sample Automated Delivery Process, 191

CI/CD Overview

CI/CD, or continuous integration and continuous delivery, is a practice that automates the integration and delivery operations in a CI/CD pipeline. You can automate each integration and delivery operation using the Data Integration Service REST API or the infacmd command line programs.

A CI/CD pipeline includes the integration operations that developers use to design objects and the delivery operations that deliver the objects to the production environment. You can use the REST API or infacmd to automate integration and delivery in the following ways:

- Deploy and test every change that a developer makes to an object. Developers receive instant feedback about whether objects pass or fail testing and the types of changes that objects require.
- Deliver objects that pass testing to the production environment. Based on organizational requirements, you can deliver objects to additional requirements, such as QA and UAT, before delivering the objects to the production environment.

Continuous integration and continuous delivery are fully logged and visible to the entire team so that team members can allocate time away from manual tasks.

REST API and infacmd Quick Reference

Refer to the common integration and delivery operations that you can use to set up continuous integration and continuous delivery.

For more information about the REST API requests, see one of the following locations:

- REST API documentation that you access through the Data Integration Service process properties or the REST Operations Hub Service process properties.
- *Informatica Application Service Guide*.

For more information about the infacmd commands, see the *Informatica Command Reference*.

Get Information about Objects

The following table lists the operations that you can automate to get information about objects:

Operation	REST API	infacmd
Query design-time objects	Get design-time objects	dis queryDesignTimeObjects
Query run-time objects	Get run-time objects	dis queryRunTimeObjects
List applications	Get run-time objects	dis listApplications
List application objects	Get run-time objects	dis listApplicationObjects
List parameter sets	Get run-time objects	dis listParameterSets
List application patches	Not applicable	dis listPatchNames
Compare mappings	Compare two queried mappings	dis compareMapping

Tag Objects

The following table lists the operations that you can automate to tag objects:

Operation	REST API	infacmd
Tag an object	Assign tags to design-time objects	dis tag
Replace object tags	Replace repository object tags	dis replaceAllTag
Untag an object	Remove tags from design-time objects	dis untag

Detect and Resolve Conflicts

The following table lists the operations that you can automate to detect and resolve conflicts:

Operation	REST API	infacmd
Unlock an object	Not applicable	mrs unlockObject
List locked objects	Not applicable	mrs listLockedObjects

Perform Version Control Operations

The following table lists the operations that you can automate to perform version control operations:

Operation	REST API	infacmd
Check in an object	Perform version control operations	mrs checkInObject
Check out an object	Perform version control operations	Not applicable
Undo a check-out	Perform version control operations	mrs undoCheckout
Reassign a checked-out object	Perform version control operations	mrs reassignCheckedOutObject
List checked-out objects	Get design-time objects	mrs listCheckedOutObjects

Note: Version control operations are available only if the Model repository is integrated with a version control system.

Deploy Objects

The following table lists the operations that you can automate to deploy objects:

Operation	REST API	infacmd
Deploy objects to an application archive file	Not applicable*	tools deployApplication
Deploy objects to an application patch archive file	Deploy design-time objects to an archive file	dis deployObjectsToFile
Deploy an application archive file to a Data Integration Service	Deploy an application	dis deployApplication
Deploy an application patch archive file to a Data Integration Service	Update an application	tools patchApplication
Redeploy an application archive file to a Data Integration Service	Update an application	dis updateApplication
* You cannot use the REST API to deploy objects to an application archive file. Use an application patch archive file to deploy an application to a Data Integration Service for the first time or to redeploy an application.		

Manage Deployed Applications

The following table lists the operations that you can automate to manage deployed applications:

Operation	REST API	infacmd
Start an application	Start or stop an application	dis startApplication
Stop an application	Start or stop an application	dis stopApplication
Backup an application	Not applicable	dis backupApplication

Operation	REST API	infacmd
Restore an application	Not applicable	dis restoreApplication
Rename an application	Not applicable	dis renameApplication
Undeploy an application	Undeploy an application	dis undeployApplication
Run a deployed mapping	Run a deployed mapping	ms runMapping

Using the REST API or infacmd

To decide whether to use the Data Integration Service REST API or the infacmd command line programs to automate integration and delivery operations, assess the requirements of your organization.

The REST API and infacmd have the following differences:

- You can achieve more use cases with infacmd compared to REST API. For example, you can use an infacmd command to backup applications.
- To use infacmd, you have to install Informatica services or the infacmd utilities. In contrast, the REST API does not have any setup requirements and you can run the REST API in environments that do not have Informatica services installed on the client machine.

For information about the Data Integration Service REST API, see the *Informatica Application Service Guide*.

For information about the infacmd command line programs, see the *Informatica Command Reference*.

Using the REST API to Deploy and Redeploy an Application

The Data Integration Service REST API uses application patch archive files to deploy objects. Instead of using an application archive file, you use an application patch archive file to deploy objects to a Data Integration Service for the first time and to redeploy an application.

Use an application patch archive file to perform the following tasks:

Deploy an application for the first time

To deploy objects as an application to a Data Integration Service for the first time, complete the following tasks:

1. Use the REST API `Deploy design-time objects to an archive file` to create an application patch archive file.
2. Use the REST API `Deploy an application` to deploy the patch to a Data Integration Service.

   The Data Integration Service uses the objects in the patch to create an incremental application.

Redeploy an application

To redeploy an application, use the REST API `Update an application` and configure the following components:

1. Specify the application patch archive file.
2. Set the request payload parameter `"redeployApplication":true`.

   The Data Integration Service replaces the objects in the deployed incremental application with the objects in the patch.

Continuous Integration

Continuous integration ensures that every change that a developer makes to an object is deployed and tested in the development environment before the object is approved for delivery to additional environments.

To set up continuous integration, you can automate the following tasks:

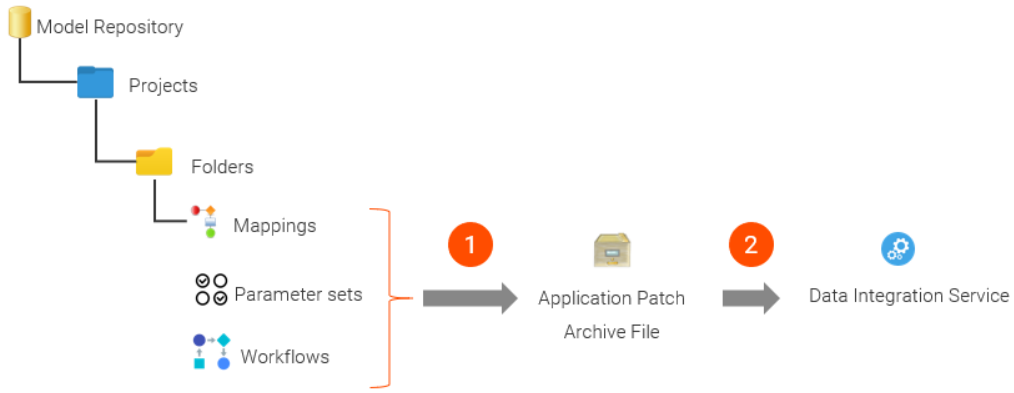
- Deploy objects to run sanity tests.
- Run version control operations to assign objects to developers.
- Detect and resolve conflicts due to object locks.

Deploy Objects

Set up continuous integration to ensure that every change that a developer makes to an object is deployed and tested.

To isolate objects for testing, you deploy the objects to an application archive file and then to a Data Integration Service. When the object is deployed to a Data Integration Service, developers can continue to access and edit the object in the Developer tool without interfering with test results.

The following image shows how objects are deployed:



1. The automated process deploys objects to an application patch archive file.
2. The automated process deploys the archive file to a Data Integration Service.

After an object is deployed, you can runs sanity tests to verify that mappings run successfully. To perform further testing, you can use a third-party tool to verify that the mapping output is correct. Based on test results, you can approve the object for delivery to additional environments or assign the object to a developer for editing.

Assign Objects

In a team-based development environment, you can set up continuous integration to assign objects to developers.

For example, if a mapping fails a sanity test, you can assign the mapping for editing to the developer that last modified the mapping. To assign the mapping, you can use the REST API `Perform version control operations` or the command `infacmd mrs reassignCheckedOutObject`.

To assign an object to a developer or to reassign an object from one developer to another developer, you must be able to perform version control operations. Ensure that the Model repository is integrated with a version control system, such as GIT, SVN, or Perforce.

For information about how to integrate a Model repository with a version control system, see the *Informatica Application Service Guide*.

Resolve Object Conflicts

If an object requires changes, you can set up continuous integration to use intent-based locks to detect and resolve conflicts between objects and facilitate developer collaboration.

An intent-based lock prevents other developers from saving changes to the object. Occasionally, locks create conflicts between developers that require access to the same object.

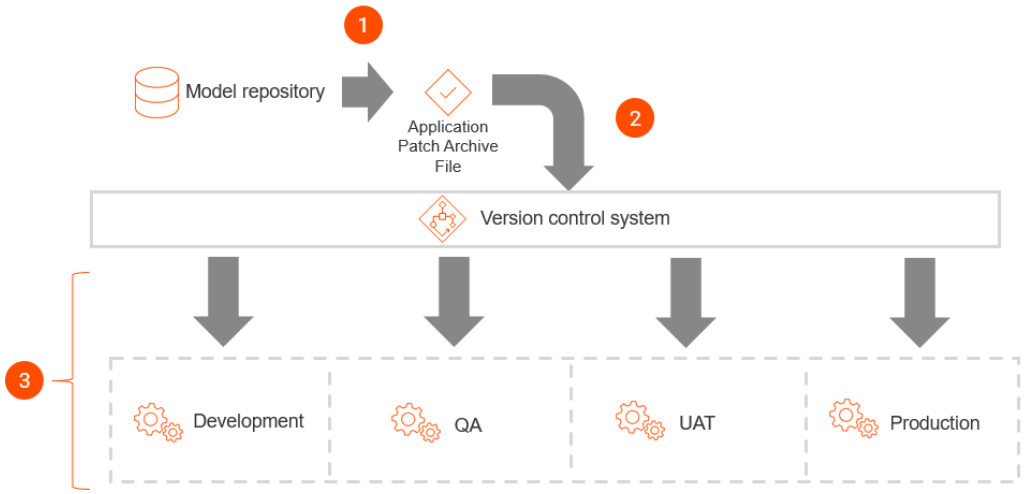
For example, a developer locks an object before going on vacation. The lock creates a conflict for other developers that require access to the object. To resolve such conflicts, you can use a third-party tool to read employee work schedules, detect potential conflicts, and pass logic to infacmd commands that list the locked objects and unlock the objects if necessary.

Continuous Delivery

As objects pass testing in the development environment, the objects are continuously delivered to the production environment. Based on organizational requirements, you can set up continuous delivery to additional environments, such as QA and UAT, before delivering the objects to the production environment.

To simplify continuous delivery, you can use archive files to create consistency between the objects that were tested and approved in the development environment and the objects that are delivered to additional environments. To create consistency, set up continuous delivery to deploy objects to one archive file, store the archive file in a version control system, and deploy the archive file to each environment.

The following image shows how one application patch archive file can be deployed to development, QA, UAT, and production environments:



1. Objects are deployed to an application patch archive file.
2. The archive file is stored in a version control system.
3. The same archive file is deployed to development, QA, UAT, and production environments.

Sample Automated Integration Process

This section shows how you can set up continuous integration to deploy and test every change that a developer makes to mappings in the development environment.

Set up continuous integration to perform the following tasks:

1. Prepare mappings for deployment.
2. Deploy mappings to a Data Integration Service.
3. Run sanity tests.
4. Prepare mappings for editing.

Step 1. Prepare Mappings for Deployment

Before you deploy mappings for testing, identify the mappings that have been changed and mark the mappings for deployment.

Set up continuous integration to perform the following integration operations:

1. Identify mappings that have been changed.

To identify mappings that have been changed, continuously compare each design-time mapping to the corresponding run-time mapping that is deployed to the Data Integration Service.

To compare mappings, pass a query to the REST API `Compare two queried mappings` or to the command `infacmd dis compareMapping`.

2. Assign a tag to mark the mappings for deployment.

If the comparison report identifies differences between a design-time mapping and the corresponding run-time mapping, assign a tag to the design-time mapping that marks the mapping for deployment.

For example, assign the tag `DeploymentInProgress` to the mapping. You can pass the tag to the REST API `Assign tags to design-time objects` or to the command `infacmd dis tag`.

Step 2. Deploy Mappings to a Data Integration Service

After mappings are prepared for deployment, deploy the changed mappings to an application patch archive file and then to a Data Integration Service.

Set up continuous integration to perform the following integration operations:

1. Run a query to identify mappings that are marked for deployment.

To identify the mappings that require deployment, query the design-time objects that have the tag `DeploymentInProgress`. Pass the query to the REST API `Get design-time objects` or to the command `infacmd dis queryDesignTimeObjects`.

2. Deploy the mappings to an application patch archive file.

If the query returns at least one mapping, deploy the mappings to an application patch archive file.

Pass the query to the REST API `Deploy design-time objects to an archive file` or to the command `infacmd dis deployObjectsToFile`. The REST API request or the infacmd command saves the archive file on the Data Integration Service machine.

3. Deploy the archive file to a Data Integration Service.

Deploy the archive file to a Data Integration Service in the development environment. Pass the location of the archive file to the REST API `Update an application` or to the command `infacmd tools patchApplication`.

Step 3. Run Sanity Tests

When the mappings are updated on the Data Integration Service, run sanity tests to verify that mappings run successfully.

Set up continuous integration to perform the following integration operations:

1. Verify that deployed mappings run successfully.

To verify that deployed mappings run successfully, perform the following tasks:

1. To run deployed mappings, use the REST API `Run a deployed mapping` or the command `infacmd ms runMapping`.
2. Run sanity tests based on organizational requirements. For example, if you automate integration tasks using a script in a third-party tool, you can write test cases to read the data in the mapping target and verify that the mapping output is correct.

2. Assign tags to mark mappings that require additional changes.

To mark mappings that require additional changes, pass the tag `RequiresChanges` to the REST API `Assign tags to design-time objects` or to the command `infacmd dis tag`.

You can also remove the tag `DeploymentInProgress` from the mappings that require changes. Pass the tag `DeploymentInProgress` to the REST API `Remove tags from design-time objects` or to the command `infacmd dis untag`.

3. Assign tags to approve mappings that ran successfully.

To approve mappings that ran successfully, pass the tag `ReadyForDelivery` to the REST API `Replace repository object tags` or to the command `infacmd dis replaceAllTag`. The REST API request or the infacmd command replaces all tags that are assigned to the mapping with the tag `ReadyForDelivery`.

Step 4. Prepare Mappings for Editing

If updated mappings fail testing, prepare the mappings for editing.

Set up continuous integration to perform the following integration operations:

1. Assign the mappings to the appropriate developer for editing.

To assign mappings to the appropriate developer, perform the following tasks:

1. Query the design-time mappings that have the tag `RequiresChanges` using the REST API `Get design-time objects` or the command `infacmd dis queryDesignTimeObjects`.
2. Based on the third-party tool that you use, you can parse the query result to get the user name of the developer that last modified each mapping.
3. To assign the mapping to the developer for editing, use the REST API `Perform version control operations` or the command `infacmd mrs reassignCheckedOutObject`.

   Note: Version control operations are available only if the Model repository is integrated with a version control system.

2. Notify the developer about the test results.

Optionally, you can use a third-party tool to notify developers about the mappings that require changes.

For example, you can query the design-time mappings that have the tag `RequiresChanges` using the REST API `Get design-time objects` or the command `infacmd dis queryDesignTimeObjects`.

Then, you can run a script in a third-party tool to perform the following tasks:

1. Read the query results to parse the response and get the user name of the developer that last modified the mapping.
2. Look up the e-mail address that is associated with the user name in the employee database.
3. Generate an e-mail message to convey information about the mappings that failed testing and the reason that the mappings failed.

Sample Automated Delivery Process

This section shows how you can set up continuous delivery to deploy mappings to the production environment as the mappings are approved in the development environment.

Set up continuous delivery to perform the following delivery operations:

1. Identify mappings that are approved for delivery.

To identify mappings that are approved for delivery, perform the following tasks:

1. Continuously check whether any mappings in the development environment have the tag `ReadyForDelivery`.
2. Query the design-time mappings for the tag `ReadyForDelivery`.

3. Pass the query to the REST API `Get design-time objects` or to the command `infacmd dis queryDesignTimeObjects`.

2. Deploy the mappings to an application patch archive file.

If the query returns at least one mapping, deploy the mappings to an application patch archive file.

Pass the query to the REST API `Deploy design-time objects to an archive file` or to the command `infacmd dis deployObjectsToFile`. The REST API request or the infacmd command save the archive file on the Data Integration Service machine.

3. Deploy the archive file to a Data Integration Service.

Deploy the archive file to a Data Integration Service in the production environment. Pass the location of the archive file to the REST API `Update an application` or to the command `infacmd tools patchApplication`.

CHAPTER 13

Object Import and Export

This chapter includes the following topics:

- Object Import and Export Overview, 193
- Import and Export Objects, 194
- Object Export, 195
- Object Import, 196

Object Import and Export Overview

You can export multiple objects from a project to an ..xml file. When you import objects, you can choose individual objects in the ..xml file or all the objects in the ..xml file.

You can export objects to an .xml file and then import objects from the .xml file. When you export objects, the Developer tool creates an .xml file that contains the metadata of the exported objects. Use the .xml file to import the objects into a project or folder. You can also import and export objects through infacmd command.

Export and import objects to accomplish the following tasks:

Deploy metadata into production

After you test a mapping in a development repository, you can export it to an .xml file and then import it from the .xml file into a production repository.

Archive metadata

You can export objects to an .xml file that you no longer need before you remove them from the repository.

Share metadata

You can share metadata with a third party. For example, you can send a mapping to someone else for testing or analysis.

Copy metadata between repositories

You can copy objects between repositories that you cannot connect to from the same client. Export the object and transfer the .xml file to the target machine. Then import the object from the .xml file into the target repository. You can export and import objects between repositories with the same version. If the objects contain tags, the Developer tool automatically imports into the repository.

You can use infacmd to generate a readable .xml file from an export file. You can also edit the object names in the readable .xml file, and then update the export .xml file before you import the objects into a repository.

You can upgrade objects exported to an .xml file from a previous Informatica release to the current metadata format, and then import the updated objects into the current release. Use the mrs UpgradeExportedObjects command to upgrade the exported objects to the current metadata format before you import the objects.

Import and Export Objects

You can import and export projects and objects in a project. You can also import and export application archive files in a repository.

When you export an object, the Developer tool also exports the dependent objects. A dependent object is an object that is used by another object. For example, a physical data object used as a mapping input is a dependent object of that mapping. When you import an object, the Developer tool imports all the dependent objects.

When you export or import objects in a project or folder, the Model Repository Service preserves the object hierarchy.

The following table lists objects and dependent objects that you can export:

Object	Dependency
Application	- SQL data services, mappings, or workflows and their dependent objects
Project	- Projects contain other objects, but they do not have dependent objects
Folder	- Folders contain other objects, but they do not have dependent objects
Reference table	- Reference tables do not have dependent objects
Content set	- Content sets do not have dependent objects
Physical data object (except for customized data object)	- Physical data objects do not have dependent objects
Customized data object	- Physical data objects
Logical data object model	- Logical data objects - Physical data objects - Reusable transformations and their dependent objects - Mapplets and their dependent objects
Transformation	- Physical data objects - Reference tables - Content sets
Mapplet	- Logical data objects - Physical data objects - Reusable transformations and their dependent objects - Mapplets and their dependent objects

Object	Dependency
Mapping	- Logical data objects - Physical data objects - Reusable transformations and their dependent objects - Mapplets and their dependent objects
SQL data service	- Logical data objects - Physical data objects - Reusable transformations and their dependent objects - Mapplets and their dependent objects
Web service	- Operation mappings
Workflow	- Mappings and their dependent objects

Object Export

When you export an object, the Developer tool creates an XML file that contains the metadata of the objects.

You can choose the objects to export. You must also choose to export all dependent objects. The Developer tool exports the objects and the dependent objects. The Developer tool exports the last saved version of the object. The Developer tool includes Cyclic Redundancy Checking Value (CRCVALUE) codes in the elements in the XML file. If you modify attributes in an element that contains a CRCVALUE code, you cannot import the object. If you want to modify the attributes use the infacmd xrf command.

You can also export objects with the infacmd tools ExportObjects command.

Exporting Objects

To use Model repository objects in another repository, you can export the objects as an XML metadata file.

1. Click **File** > **Export**.

   The **Export** wizard opens.
2. Select **Informatica** > **Export Object Metadata File**.

   Click **Next**.
3. Click **Browse**. Select the repository project that contains the objects to export.

   Click **Next**.
4. Select one or more objects to export. If you highlighted a repository object before you started the export process, the wizard selects the object for you.
5. Enter a file name and a location for the XML metadata file. The Developer tool exports all the objects that you select to a single file.

   Click **Next**.
6. The wizard displays any dependent object that the metadata objects use.

   Click **Next** to accept the dependent objects.

7. If the objects that you select include reference data objects, select the **Export content** option and verify the export settings:
   - Verify the name and location for the reference data files that you export. The Developer tool Service exports the reference data files to a single ZIP file. By default, the wizard exports the ZIP file and the XML metadata file to the same directory.
   - Verify the code page that the reference data uses. The default code page is UTF-8. If you export reference table data, accept the default code page.
   - Verify the probabilistic model data to export. By default, the wizard exports all model data. If the objects that you select do not include a probabilistic model, the export process ignores the option.
8. Click **Finish** to export the selected objects.

   The Developer tool exports the object metadata to an XML file and exports any dependent reference data files to a ZIP file.

Object Import

You can import a project or objects within a project from an export file. You can import the objects and any dependent objects into a project or folder.

You can import a project or individual objects. Import a project when you want to reuse all objects in the project. Import individual objects when you want to reuse objects across projects.

When you import an object, the Developer tool lists all the dependent objects. You must add each dependent object to the target before you can import the object.

When you import objects, an object in the export file might have the same name as an object in the target project or folder. You can choose how you want to resolve naming conflicts.

You can also import objects with the `infacmd tools ImportObjects` command.

You cannot import objects from an export file that you created in a previous version.

Importing Projects

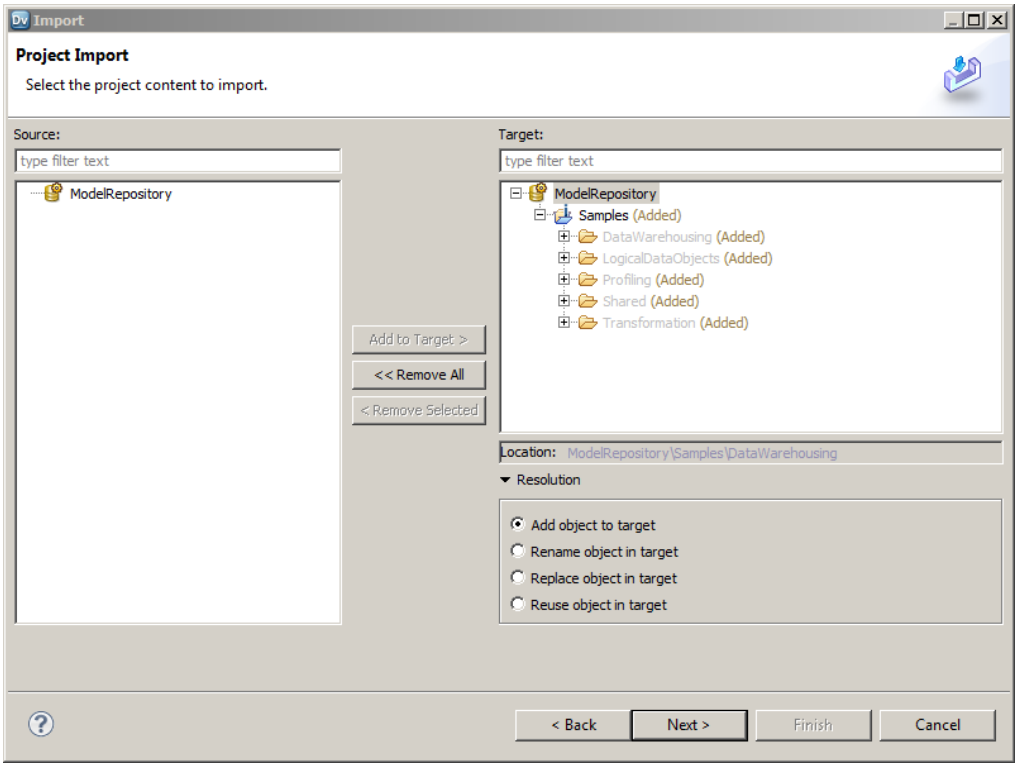
You can import a project from an XML file into the target repository. You can also import the contents of the project into a project in the target repository.

1. Click **File** > **Import**.
2. Select **Informatica** > **Import Object Metadata File (Basic)**.
3. Click **Next**.
4. Click **Browse** and select the export file that you want to import.
5. Click **Next**.
6. Select the project or select "<project name> Project Content" in the Source pane.
   - If you select the project in the Source pane, select the Model Repository Service in the Target pane where you want to import the project.
   - If you select the project content in the Source pane, select the project to which you want to import the project contents in the Target pane.
7. Click **Add to Target** to add the project to the target.

Tip: You can also drag the project from the Source pane into the repository in the Target pane. Or, you can drag the project content in the Source pane into a project in the Target pane.

8. Click **Resolution** to specify how to handle duplicate objects.

   You can rename the imported object, replace the existing object with the imported object, or reuse the existing object. The Developer tool renames all the duplicate objects by default.



9. Click **Next**.

   The Developer tool lists any reference table data that you are importing. Specify the additional reference table settings.

10. Click **Next**.

    The Developer tool summarizes the objects to be imported. Click **Link Source and Target Objects** to link the objects in the Source and Target display panes when you select one of the objects. For example, if you select this option and then select an object in the Source pane, the Developer tool selects the same object in the Target pane.

11. Map the connections from the import file to the target domain connections in the Additional Import Settings pane. You can also select whether to overwrite existing tags on the objects.

12. Click **Finish**.

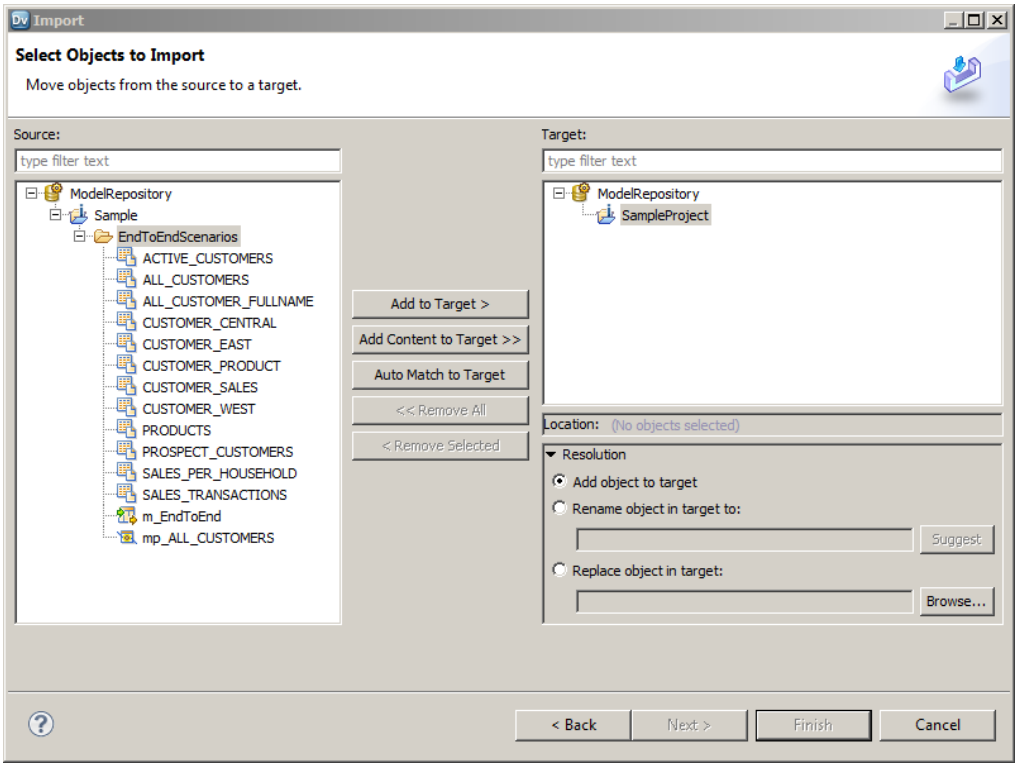
If you chose to rename the duplicate project, the Model Repository Service appends a number to the object name. You can rename the project after you import it.

Importing Objects

You can import objects from an XML file or application archive file. You import the objects and any dependent objects into a project.

1. Click **File** > **Import**.

2. Select **Informatica** > **Import Object Metadata File (Advanced)**.
3. Click **Next**.
4. Click **Browse** to select the export file that you want to import.
5. Click **Next**.
6. Select the object in the Source pane that you want to import.
7. Select the project in the Target pane to which you want to import the object.
8. Click **Add to Target** to add the object to the target.



If you click **Auto Match to Target**, the Developer tool tries to match the descendants of the current source selection individually by name, type, and parent hierarchy in the target selection and adds the objects that match.

If you want to import all the objects under a folder or a project, select the target folder or project and click **Add Content to Target**.

Tip: You can also drag the object from the Source pane into the required project in the Target pane. Press the control key while you drag to maintain the object hierarchy in source and target.

9. Click to specify how to handle duplicate objects.

   You can rename the imported object, replace the existing object with the imported object, or reuse the existing object. The Developer tool renames all the duplicate objects by default.

10. Click **Next**.

    The Developer tool lists any dependent objects in the import file.

11. Add dependent objects to a target folder or project.
12. Click **Next**.

    The Developer tool lists any reference table data that you are importing. Specify the additional reference table settings.

13. Click **Next**.

    The Developer tool summarizes the objects to be imported. Click **Link Source and Target Objects** to link the objects in the Source and Target display panes when you select one of the objects. For example, if you select this option and then select an object in the Source pane, the Developer tool selects the same object in the Target pane.

14. Map the connections from the import file to the target domain connections in the Additional Import Settings pane. You can also select whether to overwrite existing tags on the objects.

15. Click **Finish**.

If you choose to rename the duplicate project, the **Import** wizard names the imported project as "<Original Name>_<number of the copy>." You can rename the project after you import it.

Importing Objects from a Previous Informatica Release

You can upgrade objects exported from a previous Informatica release to the current metadata format, and then import the upgraded objects into the current Informatica release. You must upgrade the exported objects to the current metadata format before you import the objects into the current release.

You might import objects from a previous Informatica release into the current Informatica release in an enterprise with multiple Informatica releases installed. For example, you might export objects from a previous Informatica release development domain, and then import the objects into a current Informatica release test domain.

You export objects from an Informatica 10.1 or 10.1.1 Model repository to an .xml file that contains the object metadata. You use the mrs UpgradeExportedObjects command to upgrade the exported objects to the current metadata format. You must specify the path and file name of the .xml file that contains the exported objects when you run the command.

Do not modify the .xml file before you run the mrs UpgradeExportedObjects command. If you modify the file before you run the command, the objects in the file might not upgrade successfully.

The mrs UpgradeExportedObjects command upgrades the objects, and then generates an .xml file containing the upgraded objects. You must specify the file name and path for the .xml file. You can import the .xml file into a current Informatica release repository.

The mrs UpgradeExportedObjects command requires access to an Informatica 10.2 Model Repository Service. You must supply the service name of an Informatica 10.2 Model Repository Service running within the domain when you run the command.

For more information about the mrs UpgradeExportedObjects command, see the *Informatica Command Reference*.

APPENDIX A

Data Type Reference

This appendix includes the following topics:

- Data Type Reference Overview, 200
- Transformation Data Types, 201
- Flat File and Transformation Data Types, 210
- DB2 for LUW and Transformation Data Types, 211
- DB2 for i5/OS, DB2 for z/OS, and Transformation Datatypes, 213
- JDBC and Transformation Datatypes, 214
- Microsoft SQL Server and Transformation Data Types, 215
- Nonrelational and Transformation Datatypes, 218
- ODBC and Transformation Data Types, 220
- Oracle and Transformation Data Types, 221
- SAP HANA and Transformation Datatypes, 224
- XML and Transformation Datatypes, 226
- Converting Data, 228

Data Type Reference Overview

When you create a mapping, you create a set of instructions for the Data Integration Service to read data from a source, transform it, and write it to a target. The Data Integration Service transforms data based on dataflow in the mapping, starting at the first transformation in the mapping, and the data type assigned to each port in a mapping.

The Developer tool displays two types of data types:

Native data types

Native data types are specific to the relational table or flat file used as a physical data object. Native data types appear in the physical data object column properties.

Transformation data types

Transformation data types are set of data types that appear in the transformations. They are internal data types based on ANSI SQL-92 generic data types, which the Data Integration Service uses to move data across platforms. Transformation data types appear in all transformations in a mapping.

Transformation data types include the following data types:

- Primitive data type. Represents a single data value in a single column position.
- Complex data type. Represents multiple data values in a single column position.

When the Data Integration Service reads source data, it converts the native data types to the comparable transformation data types before transforming the data. When the Data Integration Service writes to a target, it converts the transformation data types to the comparable native data types.

When you specify a multibyte character set, the data types allocate additional space in the database to store characters of up to three bytes.

Transformation Data Types

The following table describes the transformation data types:

Data Type	Size in Bytes	Description
Array	Unlimited number of characters.	Complex data type. You can use arrays with complex sources and targets.
Bigint	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision of 19, scale of 0 Integer value.
Binary	Precision	1 to 104,857,600 bytes You cannot use binary data for COBOL or flat file sources. You cannot use binary data for flat file sources.
Date/Time	16 bytes	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. Precision of 29, scale of 9 (precision to the nanosecond) Combined date/time value.
Decimal	8 bytes (if high precision is off or precision is greater than 28) 8 bytes (if high precision is off or precision is greater than 38) 16 bytes (if precision <= 18 and high precision is on) 20 bytes (if precision > 18 and <= 28) 24 bytes (if precision > 28 and <= 38)	Decimal value with declared precision and scale. Scale must be less than or equal to precision. Precision 1 to 28 digits, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Double	8 bytes	Double-precision floating-point numeric value. You can edit the precision and scale. The scale must be less than or equal to the precision.

Data Type	Size in Bytes	Description
Integer	4 bytes	-2,147,483,648 to 2,147,483,647 Precision of 10, scale of 0 Integer value.
Map	Unlimited number of characters.	Complex data type. You can use maps with complex sources and targets.
Nstring	Unicode mode: (precision + 1) * 2 ASCII mode: precision + 1	1 to 104,857,600 characters Fixed-length or varying-length string.
Ntext	Unicode mode: (precision + 1) * 2 ASCII mode: precision + 1	1 to 104,857,600 characters Fixed-length or varying-length string.
Real	8 bytes	Precision of 7, scale of 0 Double-precision floating-point numeric value.
Small Integer	4 bytes	-32,768 and 32,767 Precision of 5, scale of 0 Integer value.
String	Unicode mode: (precision + 1) * 2 ASCII mode: precision + 1	1 to 104,857,600 characters Fixed-length or varying-length string.
Struct	Unlimited number of characters.	Complex data type You can use structs with complex sources and targets.
Text	Unicode mode: (precision + 1) * 2 ASCII mode: precision + 1	1 to 104,857,600 characters Fixed-length or varying-length string.
timestampWithTZ	40 bytes	Aug. 1, 1947 A.D to Dec. 31, 2040 A.D. -12:00 to +14:00 Precision of 36 and scale of 9. (precision to the nanosecond) Timestamp with Time Zone data type does not support the following time zone regions: - AFRICA_CAIRO - AFRICA_MONROVIA - EGYPT - AMERICA_MONTREAL

Effective in October 2024 release, Informatica Developer (the Developer Tool) and the following application services are available for preview:

- Data Integration Service
- Model Repository Service

Preview functionality is supported for evaluation purposes but is unwarranted and is not supported in production environments or any environment that you plan to push to production. Informatica intends to include the preview functionality in an upcoming release for production use, but might choose not to in accordance with changing market or technical circumstances. For more information, contact Informatica Global Customer Support.

Integer Data Types

You can pass integer data from sources to targets and perform transformations on integer data. The transformation language supports Bigint, Integer, and Small Integer data types.

You can pass integer data from sources to targets and perform transformations on integer data. The transformation language supports Bigint and Integer data types.

The transformation integer data types represent exact values.

Integer Values in Calculations

When you use integer values in calculations, the CDI-PC Integration Service sometimes converts integer values to floating-point numbers before it performs the calculation. For example, to evaluate MOD(12.00, 5), the CDI-PC Integration Service converts the integer value “5” to a floating-point number before it performs the division operation. The CDI-PC Integration Service converts integer values to double or decimal values depending on whether you enable high precision.

When you use integer values in calculations, the Data Integration Service sometimes converts integer values to floating-point numbers before it performs the calculation. For example, to evaluate MOD(12.00, 5), the Data Integration Service converts the integer value “5” to a floating-point number before it performs the division operation. The Data Integration Service converts integer values to double or decimal values depending on whether you enable high precision.

The CDI-PC Integration Service converts integer values in the following arithmetic operations:

The Data Integration Service converts integer values in the following arithmetic operations:

Arithmetic Operation	High Precision Disabled	High Precision Enabled
Functions and calculations that cannot introduce decimal points. For example, integer addition, subtraction, and multiplication, and functions such as CUME, MOVINGSUM, and SUM.	No conversion[1]	Decimal
Non-scientific functions and calculations that can introduce decimal points. For example, integer division, and functions such as AVG, MEDIAN, and PERCENTILE.	Double	Decimal
All scientific functions and the EXP, LN, LOG, POWER, and SQRT functions.	Double	Double

1. If the calculation produces a result that is out of range, the Integration Service writes a row error.

The transformation Double data type supports precision of up to 15 digits, while the Bigint data type supports precision of up to 19 digits. Therefore, precision loss can occur in calculations that produce Bigint values with precision of more than 15 digits.

For example, an expression transformation contains the following calculation:

```
POWER( BIGINTVAL, EXPVAL )
```

Before it performs the calculation, the CDI-PC Integration Service converts the inputs to the POWER function to double values. If the BIGINTVAL port contains the Bigint value 9223372036854775807, the CDI-PC Integration Service converts this value to 9.22337203685478e+18, losing the last 4 digits of precision. If the EXPVAL port contains the value 1.0 and the result port is a Bigint, this calculation produces a row error since the result, 9223372036854780000, exceeds the maximum bigint value.

When you use an Integer data type in a calculation that can produce decimal values and you enable high precision, the CDI-PC Integration Service converts the integer values to decimal values.

Before it performs the calculation, the Data Integration Service converts the inputs to the POWER function to double values. If the BIGINTVAL port contains the Bigint value 9223372036854775807, the Data Integration Service converts this value to 9.22337203685478e+18, losing the last 4 digits of precision. If the EXPVAL port contains the value 1.0 and the result port is a Bigint, this calculation produces a row error since the result, 9223372036854780000, exceeds the maximum bigint value.

When you use an Integer data type in a calculation that can produce decimal values and you enable high precision, the Data Integration Service converts the integer values to decimal values.

For transformations that support the Decimal data type with precision up to 28 digits, precision loss does not occur in a calculation unless the result produces a value with precision greater than 28 digits in high precision mode. In this case, the CDI-PC Integration Service stores the result as a double. If the port precision is less than or equal to 28 digits and the result produces a value greater than 28 digits in high precision mode, the CDI-PC Integration Service rejects the row.

For transformations that support the Decimal data type with precision up to 28 digits, precision loss does not occur in a calculation unless the result produces a value with precision greater than 28 digits in high precision mode. In this case, the Data Integration Service stores the result as a double. If the port precision is less than or equal to 28 digits and the result produces a value greater than 28 digits in high precision mode, the Data Integration Service rejects the row.

For transformations that support the Decimal data type with precision up to 38 digits, precision loss does not occur in a calculation unless the result produces a value with precision greater than 38 digits in high precision mode. In this case, the Data Integration Service stores the result as a double. If the port precision is less than or equal to 38 digits and the result produces a value greater than 38 digits in high precision mode, the Data Integration Service rejects the row.

Integer Constants in Expressions

The Integration Service interprets constants in an expression as floating-point values, even if the calculation produces an integer result. For example, in the expression INTVALUE + 1000, the Integration Service converts the integer value “1000” to a double value if high precision is not enabled. It converts the value 1000 to a decimal value if high precision is enabled. To process the value 1000 as an integer value, create a variable port with an Integer data type to hold the constant and modify the expression to add the two ports.

NaN Values

NaN (Not a Number) is a value that is usually returned as the result of an operation on invalid input operands, especially in floating-point calculations. For example, when an operation attempts to divide zero by zero, it returns a NaN result.

Operating systems and programming languages may represent NaN differently. For example the following list shows valid string representations of NaN:

```
nan
NaN
```

```
NaN%
NAN
NaNQ
NaNS
qNaN
sNaN
1.#SNAN
1.#QNAN
```

The Integration Service converts QNAN values to 1.#QNAN on Win64EMT platforms. 1.#QNAN is a valid representation of NaN.

Write Integer Values to Flat Files

When writing integer values to a fixed-width flat file, the file writer does not verify that the data is within range. For example, the file writer writes the result 3,000,000,000 to a target Integer column if the field width of the target column is at least 13. The file writer does not reject the row because the result is outside the valid range for Integer values.

Binary Data Type

If a mapping includes binary data, set the precision for the transformation binary data type so that the Integration Service can allocate enough memory to move the data from source to target.

You cannot use binary data types for COBOL or flat file sources.

You cannot use binary data types for flat file sources.

Date/Time Data Type

The Date/Time data type handles years from 1 A.D. to 9999 A.D. in the Gregorian calendar system. Years beyond 9999 A.D. cause an error.

The Date/Time data type supports dates with precision to the nanosecond. The data type has a precision of 29 and a scale of 9. Some native data types have a smaller precision. When you import a source that contains datetime values, the import process imports the correct precision from the source column. For example, the Microsoft SQL Server Datetime data type has a precision of 23 and a scale of 3. When you import a Microsoft SQL Server source that contains Datetime values, the Datetime columns in the mapping source have a precision of 23 and a scale of 3.

The Integration Service reads datetime values from the source to the precision specified in the mapping source. When the Integration Service transforms the datetime values, it supports precision up to 29 digits. For example, if you import a datetime value with precision to the millisecond, you can use the ADD_TO_DATE function in an Expression transformation to add nanoseconds to the date.

If you write a Date/Time value to a target column that supports a smaller precision, the Integration Service truncates the value to the precision of the target column. If you write a Date/Time value to a target column that supports a larger precision, the Integration Service inserts zeroes in the unsupported portion of the datetime value.

Timestamp with Time Zone

Timestamp with Time Zone is a variant of the Timestamp data type that includes a time zone offset, TIME ZONE_HOUR: TIME ZONE_MINUTE, with or without daylight savings or time zone region name. The time zone offset is the difference, in hours and minutes, between the local time zone and UTC (Coordinated Universal Time).

For example, '16-JUN-08 07.01.25.376000 PM -06:00'. In the example, -06:00 is the time zone offset.

Another example is, '05-JUN-2008 07:01:25.376000 PM America/Los_Angeles'. In the example, America/Los_Angeles is the time zone region name.

When you import the Timestamp with Time Zone data type into the Developer tool, the associated transformation data type is timestampWithTZ.

timestampWithTZ has a precision of 36 and a scale of 9. Timestamp with Time Zone displacement value range is from -12:00 < UTC < +14:00.

The Data Integration Service can process Timestamp with Time Zone data type in Oracle and flat file data objects. Timestamp with Time Zone data type is applicable only for the data within the range of Aug 1947 to Dec 2040.

Timestamp with Time Zone data type does not support the following time zone regions in the Developer tool:

- AFRICA_CAIRO
- AFRICA_MONROVIA
- EGYPT
- AMERICA_MONTREAL

Timestamp with Time Zone data type uses the IANA standard for the Time Zone Database version 2015b. To avoid data corruption when using the Timestamp with Time Zone data type, ensure that the Oracle DST patch 24 is present on the Oracle server and client. When you enable data object caching, ensure that the Oracle DST patch 24 is present on the database server used for data object caching.

If the patch is not present, download the Oracle DST patch 24 from the Oracle website. To verify that the Oracle server uses the Oracle DST patch 24, run the following command in the command prompt at the Oracle server:

```
SELECT VERSION FROM v$timezone_file;
```

You can verify that the version appears as 24. You can also verify the version when you run the following command at the Oracle server:

```
SELECT TZ_VERSION FROM registry$database;
```

You can verify that the TZ_VERSION appears as 24.

Timestamp with Local Time Zone

Timestamp with Local Time Zone is a variant of the Timestamp data type where the timestamp data is normalized to the database time zone. The time zone displacement is not part of the column data. When the Data Integration Service reads the data, Oracle returns the data in the time zone of the Data Integration Service.

When you import the Timestamp with Local Time Zone data type into the Developer tool, the associated transformation data type is date/time.

For example, '04-APR-10 10.27.451 AM'

Timestamp (9) with Local Time Zone has a precision of 29 and a scale of 9. It is mapped to the date/time (29,9) transformation data type.

To set the default session time zone when the Data Integration Service reads or writes the Timestamp with Local Time Zone data, specify the ORA_SDTZ environment variable. You can set the ORA_SDTZ environment variable to any of the following values:

- Operating system local time zone ('OS_TZ')
- Database time zone ('DB_TZ')
- Absolute offset from UTC (for example, '-05:00')

- Time zone region name (for example, 'America/Los_Angeles')

Supported Time Zones for Oracle

In the following install location, you can see that the time zones file contains the list of all supported time zones for Oracle except the four time zones of AFRICA_CAIRO, AFRICA_MONROVIA, EGYPT, and AMERICA_MONTREAL:

```
<Informatica installation directory>/services/shared/timezones/timezones.txt
```

If you want to add time zones later based on Oracle support for additional time zones, you can place a new file in the same install location that includes the new Time Zones.

Decimal and Double Data Types

You can pass decimal and double data from sources to targets and perform transformations on decimal and double data.

The transformation language supports the following data types:

Decimal

Precision 1 to 28 digits, scale 0 to 28 . You cannot use decimal values with scale greater than precision or a negative precision. Transformations display any range you assign to a Decimal data type, but the CDI-PC Integration Service supports precision only up to 28 digits.

For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. You cannot use decimal values with a scale greater than the precision or with a negative precision. Transformations display any range that you assign to the Decimal data type, but the Data Integration Service supports precision only up to 38 digits or 28 digits depending on the transformation.

When you enable high precision and the port precision is greater than 38 digits or 28 digits, depending on the transformation, the Data Integration Service stores the result as a double.

When you enable high precision and the port precision is greater than 28 digits, the CDI-PC Integration Service stores the result as a double.

Double

Double-precision floating-point numeric value.

You can edit the precision and scale. The scale must be less than or equal to the precision.

Decimal and Double Values in Calculations

The following table lists how the CDI-PC Integration Service handles decimal values based on high precision configuration:

The following table lists how the Data Integration Service handles decimal values based on high precision configuration:

Port Data Type	Precision	High Precision Off	High Precision On
Decimal	0 to 15	Decimal	Decimal
Decimal	15 to 28 15 to 38 for transformations that support the Decimal data type with precision up to 38 digits. 15 to 28 for transformations that support the Decimal data type with precision up to 28 digits.	Double	Decimal
Decimal	Over 28 Over 38 for transformations that support the Decimal data type with precision up to 38 digits. Over 28 for transformations that support the Decimal data type with precision up to 28 digits.	Double	Double

When you enable high precision, the CDI-PC Integration Service converts numeric constants in any expression function to Decimal. If you do not enable high precision, the CDI-PC Integration Service converts numeric constants to Double.

When you enable high precision, the Data Integration Service converts numeric constants in any expression function to Decimal. If you do not enable high precision, the Data Integration Service converts numeric constants to Double.

You can ensure the maximum precision for numeric values greater than 28 or 38 digits depending on the transformation. Before you perform any calculations or transformations with the transformation functions, truncate or round any large numbers.

For transformations that support Decimal data type of precision up to 38 digits, use the Decimal data type and enable high precision to ensure precision of up to 38 digits.

For transformations that support Decimal data type of precision up to 28 digits, use the Decimal data type and enable high precision to ensure precision of up to 28 digits.

Do not use the Double data type for data that you use in an equality condition, such as a lookup or join condition.

Do not use the Double data type for data that you use in an equality condition, such as a lookup or join condition.

Precision Loss for Decimal and Double Values

Precision loss can occur with decimal and double data types in a calculation when the result produces a value with a precision greater than the maximum allowed digits.

If the precision is greater than the maximum allowed digits, the Data Integration Service stores the result as a double.

If the precision is greater than the maximum allowed digits, the PowerCenter Integration Service stores the result as a double.

Precision loss can also occur during intermediate calculations if there is a change in the precision or scale of intermediate results. The precision and scale of intermediate results can affect the precision or scale of the final result.

Precision Loss when High Precision is Disabled

If you disable high precision, the PowerCenter Integration Service converts decimal values to double. Precision loss occurs if the decimal value has a precision greater than 15 digits. For example, you have a mapping with Decimal (20,0) that passes the number 40012030304957666903. If you disable high precision, the PowerCenter Integration Service converts the decimal value to double and passes 4.00120303049577 x 10^{19}.

If you disable high precision, the Data Integration Service converts decimal values to double. Precision loss occurs if the decimal value has a precision greater than 15 digits. For example, you have a mapping with Decimal (20,0) that passes the number 40012030304957666903. If you disable high precision, the Data Integration Service converts the decimal value to double and passes 4.00120303049577 x 10^{19}.

Additional precision loss can occur if you use expressions in transformations that have intermediate calculations. If you disable high precision, the Data Integration Service converts the intermediate result to double and uses the intermediate result with lower precision to perform the following calculation in the expression. As a result, the Data Integration Service might produce a different output for expressions based on the order of the arguments. For example, when you disable high precision, the output for tan(90*$pi /180) is different from tan(90/180*$pi). The Data Integration Service performs the intermediate calculations in these expressions as follows:

```
tan(90*$pi/180) = tan(1.5707963267950003) = -9.649380295141232E12
tan(90/180*$pi) = tan(1.570796326795) = -9.670099380792184E12
```

Rounding Methods for Double Values

Due to differences in system run-time libraries and the computer system where the database processes double datatype calculations, the results may not be as expected. The double datatype conforms to the IEEE 794 standard. Changes to database client library, different versions of a database or changes to a system run-time library affect the binary representation of mathematically equivalent values. Also, many system run-time libraries implement the round-to-even or the symmetric arithmetic method. The round-to-even method states that if a number falls midway between the next higher or lower number it round to the nearest value with an even least significant bit. For example, with the round-to-even method, 0.125 is rounded to 0.12. The symmetric arithmetic method rounds the number to next higher digit when the last digit is 5 or greater. For example, with the symmetric arithmetic method 0.125 is rounded to 0.13 and 0.124 is rounded to 0.12.

To provide calculation results that are less susceptible to platform differences, the Integration Service stores the 15 significant digits of double datatype values. For example, if a calculation on Windows returns the number 1234567890.1234567890, and the same calculation on UNIX returns 1234567890.1234569999, the Integration Service converts this number to 1234567890.1234600000.

String Data Types

The transformation data types include the following string data types:

- Nstring
- Ntext
- String
- Text

Although the Nstring, Ntext, String, and Text data types support the same precision up to 104,857,600 characters, the Integration Service uses String to move string data from source to target and Text to move

text data from source to target. Because some databases store text data differently than string data, the Integration Service needs to distinguish between the two types of character data. If the source qualifier displays String, set the target column to String. Likewise, if the source qualifier displays Text, set the target column to Text, Long, or Long Varchar, depending on the source database.

Although the String and Text data types support the same precision up to 104,857,600 characters, the Integration Service uses String to move string data from source to target and Text to move text data from source to target. Because some databases store text data differently than string data, the Integration Service needs to distinguish between the two types of character data. In general, the smaller string data types, such as Char and Varchar, display as String in transformations, while the larger text data types, such as Text, Long, and Long Varchar, display as Text.

In general, the smaller string data types, such as Char and Varchar, display as String in the Source Qualifier, Lookup, and Stored Procedure transformations, while the larger text data types, such as Text, Long, and Long Varchar, display as Text in the Source Qualifier.

Use Nstring, Ntext, String, and Text interchangeably within transformations. However, in the Source Qualifier, Lookup, and Stored Procedure transformations, the target data types must match. The database drivers need to match the string data types with the transformation data types, so that the data passes accurately. For example, Nchar in a lookup table must match Nstring in the Lookup transformation.

Use String and Text interchangeably within transformations. However, in Lookup transformations, the target data types must match. The database drivers need to match the string data types with the transformation data types, so that the data passes accurately. For example, Varchar in a lookup table must match String in the Lookup transformation.

Flat File and Transformation Data Types

Flat file data types map to transformation data types that the DataPowerCenter Integration Service uses to move data across platforms.

The following table compares flat file data types to transformation data types:

Flat File Data type	Transformation Data type	Range
Bigint	Bigint	Precision of 19 digits, scale of 0
Datetime	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Double	Double	Precision of 15 digits
Int	Integer	-2,147,483,648 to 2,147,483,647
Nstring	String	1 to 104,857,600 characters

Flat File Data type	Transformation Data type	Range
Number	Decimal	Precision 1 to 28, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode. If the precision is greater than 15, the Data Integration Service converts decimal values to double in low-precision mode.
String	String	1 to 104,857,600 characters
TimestampWithTZ	TimestampWithTZ	Aug. 1, 1947 A.D to Dec. 31, 2040 A.D. -12:00 to +14:00 Precision of 36 and scale of 9. (precision to the nanosecond) Timestamp with Time Zone data type does not support the following time zone regions: - AFRICA_CAIRO - AFRICA_MONROVIA - EGYPT - AMERICA_MONTREAL **Note:** TimestampWithTZ is applicable only to the native environment.

When the DataPowerCenter Integration Service reads non-numeric data in a numeric column from a flat file, it drops the row and writes a message in the log. Also, when the DataPowerCenter Integration Service reads non-datetime data in a datetime column from a flat file, it drops the row and writes a message in the log.

DB2 for LUW and Transformation Data Types

DB2 for LUW data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares DB2 for LUW data types and transformation data types:

Data Type	Range	Transformation	Range
Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision 19, scale 0
Blob	1 to 2,147,483,647 bytes	Binary	1 to 104,857,600 bytes
Char	1 to 254 characters	String	1 to 104,857,600 characters
Char for bit data	1 to 254 bytes	Binary	1 to 104,857,600 bytes

Data Type	Range	Transformation	Range
Clob	1 to 2,447,483,647 bytes	Text	1 to 104,857,600 characters
Date	0001 to 9999 A.D. Precision 19; scale 0 (precision to the day)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Dbclob	1 to 1073741823 bytes	Dbclob	1 to 104,857,600 bytes
Decimal	Precision 1 to 31, scale 0 to 31	Decimal	Precision 1 to 31, scale 0 to 31 Precision 1 to 28, scale 0 to 28
Float	Precision 1 to 15	Double	Precision 15
Graphic	1 to 127 bytes	Graphic	1 to 127 bytes
Integer	-2,147,483,648 to 2,147,483,647	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Long Varchar	32700 bytes	Long Varchar	32700 bytes
Long Vargraphic	163350 bytes	Long Vargraphic	163350 bytes
Numeric	Precision 1 to 31, scale 0 to 31	Decimal	Precision 1 to 28, scale 0 to 28
Smallint	-32,768 to 32,767	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Time[1]	24-hour time period Precision 19, scale 0 (precision to the second)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Timestamp	26 bytes Precision 26, scale 6 (precision to the microsecond)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Varchar	Up to 4,000 characters	String	1 to 104,857,600 characters
Varchar for bit data	Up to 4,000 bytes	Binary	1 to 104,857,600 bytes
Vargraphic	1 to 16336 bytes	Vargraphic	1 to 16336 bytes

1. When the Data Integration Service converts the DB2 Time data type to the transformation Date/Time data type, the date value is added to the time value.

DB2 for i5/OS, DB2 for z/OS, and Transformation Datatypes

DB2 for i5/OS and DB2 for z/OS datatypes map to transformation datatypes in the same way that IBM DB2 datatypes map to transformation datatypes. The Data Integration Service uses transformation datatypes to move data across platforms.

The following table compares DB2 for i5/OS and DB2 for z/OS datatypes with transformation datatypes:

Datatype	Range	Transformation	Range
Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision 19, scale 0
Char	1 to 254 characters	String	1 to 104,857,600 characters
Char for bit data	1 to 254 bytes	Binary	1 to 104,857,600 bytes
Date	0001 to 9999 A.D. Precision 19; scale 0 (precision to the day)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Decimal	Precision 1 to 31, scale 0 to 31	Decimal	Precision 1 to 28, scale 0 to 28. Precision 1 to 31, scale 0 to 31.
Float	Precision 1 to 15	Double	Precision 15
Integer	-2,147,483,648 to 2,147,483,647	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Smallint	-32,768 to 32,767	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Time[1]	24-hour time period Precision 19, scale 0 (precision to the second)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Timestamp[2]	26 bytes Precision 26, scale 6 (precision to the microsecond)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Varchar	Up to 4,000 characters	String	1 to 104,857,600 characters
Varchar for bit data	Up to 4,000 bytes	Binary	1 to 104,857,600 bytes

1. When the Data Integration Service converts the DB2 Time data type to the transformation Date/Time data type, the date value is added to the time value.

2. DB2 for z/OS Version 10 extended-precision timestamps map to transformation datatypes as follows:
- If scale=6, then precision=26 and transformation datatype=date/time
- If scale=0, then precision=19 and transformation datatype=string
- If scale=1-5 or 7-12, then precision=20+scale and transformation datatype=string

Unsupported DB2 for i5/OS and DB2 for z/OS Datatypes

The Developer tool does not support certain DB2 for i5/OS and DB2 for z/OS datatypes.

The Developer tool does not support DB2 for i5/OS and DB2 for z/OS large object (LOB) datatypes. LOB columns appear as unsupported in the relational table object, with a native type of varchar and a precision and scale of 0. The columns are not projected to customized data objects or outputs in a mapping.

JDBC and Transformation Datatypes

When the Data Integration Service reads data from a JDBC source, it converts the native datatypes into the corresponding JDBC datatypes and then to the transformation datatypes. It uses the transformation datatypes to move data across platforms.

The following table compares the JDBC datatypes to the transformation datatypes:

JDBC Datatype	Transformation	Range
Bigint	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision 19, scale 0
Binary*	Binary	1 to 104,857,600 bytes
Bit	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Blob*	Binary	1 to 104,857,600 bytes
Boolean	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Char*	String	1 to 104,857,600 characters
Clob*	Text	1 to 104,857,600 characters
Date	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Decimal	Decimal	Precision 1 to 28, scale 0 to 28
Double	Double	Precision 15
Float	Double	Precision 15
Integer	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Long VarBinary*	Binary	1 to 104,857,600 bytes
Long Varchar*	Text	1 to 104,857,600 characters
Numeric	Decimal	Precision 1 to 28, scale 0 to 28

JDBC Datatype	Transformation	Range
Real	Double	Precision 15
Smallint	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Time	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Timestamp	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Tinyint	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Varchar*	String	1 to 104,857,600 characters
Varbinary*	Binary	1 to 104,857,600 bytes
*If you specify the port precision greater than 100 MB, the Developer Tool defaults the port precision to 4000.		

Microsoft SQL Server and Transformation Data Types

Microsoft SQL Server data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares Microsoft SQL Server data types and transformation data types:

Microsoft SQL Server	Range	Transformation	Range
Binary	1 to 8,000 bytes	Binary	1 to 104,857,600 bytes
bigint	- 9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision 19, scale 0
Bit	1 bit	String	1 to 104,857,600 characters
Char	1 to 8,000 characters	String	1 to 104,857,600 characters
Date	Jan 1, 0001 A.D. to Dec 31, 9999 A.D.	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D.
Datetime	Jan 1, 1753 A.D. to Dec 31, 9999 A.D. Precision 23, scale 3 (precision to 3.33 milliseconds)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)

Microsoft SQL Server	Range	Transformation	Range
Datetime2	Jan 1, 0001 A.D. 00:00:00 to Dec 31, 9999 A.D. 23:59:59.9999999	Timestamp	Precision 22 to 27
Decimal	Precision 1 to 38, scale 0 to 38	Decimal	Precision 1 to 28, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Float	-1.79E+308 to 1.79E+308	Double	Precision 15
Image	1 to 2,147,483,647 bytes	Binary	1 to 104,857,600 bytes
Int	-2,147,483,648 to 2,147,483,647	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Money	-922,337,203,685,477.5807 to 922,337,203,685,477.5807	Decimal	Precision 1 to 28, scale 0 to 28
nchar	1 to 4000 characters	String	1 to 104,857,600 characters
ntext	1 to 1,073,741,823 bytes	Text	1 to 104,857,600 characters
Numeric	Precision 1 to 38, scale 0 to 38	Decimal	Precision 1 to 28, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
nvarchar	1 to 4000 characters	String	1 to 104,857,600 characters
Real	-3.40E+38 to 3.40E+38	Double	Precision 15
Smalldatetime	Jan 1, 1900, to June 6, 2079 Precision 19, scale 0 (precision to the minute)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)

Microsoft SQL Server	Range	Transformation	Range
Smallint	-32,768 to 32,768	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Smallmoney	-214,748.3648 to 214,748.3647	Decimal	Precision 1 to 28, scale 0 to 28
Sysname	1 to 128 characters	String	1 to 104,857,600 characters
Text	1 to 2,147,483,647 characters	Text	1 to 104,857,600 characters
Time	00:00:00.0000000 to 23:59:59.9999999	Time	00:00:00 to 23:59:59.9999999 Precision 29, Scale 7
Timestamp	8 bytes	Binary	1 to 104,857,600 bytes
Tinyint	0 to 255	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Uniqueidentifier	Precision 38, scale 0	String	To successfully move or change Uniqueidentifier data, ensure that the data is in the following format: `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx` where `x` is a hexadecimal digit in the range 0-9 or a-f.
Varbinary	1 to 8,000 bytes	Binary	1 to 104,857,600 bytes
Varchar	1 to 8,000 characters	String	1 to 104,857,600 characters

Uniqueidentifier Data Type

Uniqueidentifier is a Microsoft SQL Server data type that is used to store Globally Unique Identifiers (GUIDs). It can store 16 bytes of data.

The Developer tool treats the Uniqueidentifier data type as String. To move or change Uniqueidentifier data, connect the Uniqueidentifier column to a String column. To successfully move or change Uniqueidentifier data, ensure that the data is in the following format:

`xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`

where `x` is a hexadecimal digit in the range 0-9 or a-f.

For example, `6F9619FF-8B86-D011-B42D-00C04FC964FF` is a valid Uniqueidentifier value.

The Developer tool can store 16 bytes of Uniqueidentifier data in 36 characters. However, since Uniqueidentifier data can be represented within two curly brackets, the Developer tool assigns two additional characters to the precision to accommodate the curly brackets. When you connect a Uniqueidentifier column to a String column, set the precision of the String column to 38 to successfully move or change Uniqueidentifier data.

Nonrelational and Transformation Datatypes

Nonrelational datatypes map to transformation datatypes that the Data Integration Service uses to move data across platforms.

Nonrelational datatypes apply to the following types of connections:

- Adabas
- IMS
- Sequential
- VSAM

The following table compares nonrelational datatypes and transformation datatypes:

Nonrelational	Precision	Transformation	Range
BIN	10	Binary	1 to 104,857,600 bytes
CHAR	10	String	1 to 104,857,600 characters Fixed-length or varying-length string.
DATE	10	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. Combined date/time value, with precision to the nanosecond.
DOUBLE	18	Double	Precision of 15 digits Double-precision floating-point numeric value.
FLOAT	7	Double	Precision of 15 digits Double-precision floating-point numeric value.
NUM8	3	Integer	Precision of 10 and scale of 0 Integer value.
NUM8U	3	Integer	Precision of 10 and scale of 0 Integer value.
NUM16	5	Integer	Precision of 10 and scale of 0 Integer value.
NUM16U	5	Integer	Precision of 10 and scale of 0 Integer value.
NUM32	10	Integer	Precision of 10 and scale of 0 Integer value.
NUM32U	10	Double	Precision of 15 digits Double-precision floating-point numeric value.
NUM64	19	Decimal	Precision 1 to 28 digits, scale 0 to 28 Decimal value with declared precision and scale. Scale must be less than or equal to precision. If you pass a value with negative scale or declared precision greater than 28, the Data Integration Service converts it to a double.

Nonrelational	Precision	Transformation	Range
NUM64U	19	Decimal	Precision 1 to 28 digits, scale 0 to 28 Decimal value with declared precision and scale. Scale must be less than or equal to precision. If you pass a value with negative scale or declared precision greater than 28, the Data Integration Service converts it to a double.
NUMCHAR	100	String	1 to 104,857,600 characters Fixed-length or varying-length string.
PACKED	15	Decimal	Precision 1 to 28 digits, scale 0 to 28 Decimal value with declared precision and scale. Scale must be less than or equal to precision. If you pass a value with negative scale or declared precision greater than 28, the Data Integration Service converts it to a double.
TIME	5	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. Combined date/time value, with precision to the nanosecond.
TIMESTAMP	5	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. Combined date/time value, with precision to the nanosecond.
UPACKED	15	Decimal	Precision 1 to 28 digits, scale 0 to 28 Decimal value with declared precision and scale. Scale must be less than or equal to precision. If you pass a value with negative scale or declared precision greater than 28, the Data Integration Service converts it to a double.
UZONED	15	Decimal	Precision 1 to 28 digits, scale 0 to 28 Decimal value with declared precision and scale. Scale must be less than or equal to precision. If you pass a value with negative scale or declared precision greater than 28, the Data Integration Service converts it to a double.
VARBIN	10	Binary	1 to 104,857,600 bytes
VARCHAR	10	String	1 to 104,857,600 characters Fixed-length or varying-length string.
ZONED	15	Decimal	Precision 1 to 28 digits, scale 0 to 28 Decimal value with declared precision and scale. Scale must be less than or equal to precision. If you pass a value with negative scale or declared precision greater than 28, the Data Integration Service converts it to a double.

ODBC and Transformation Data Types

ODBC data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares ODBC data types, such as Microsoft Access or Excel, to transformation data types:

Data Type	Transformation	Range
Bigint	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision 19, scale 0
Binary	Binary	1 to 104,857,600 bytes
Bit	String	1 to 104,857,600 characters
Char	String	1 to 104,857,600 characters
Date	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Decimal	Decimal	Precision 1 to 28, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Double	Double	Precision 15
Float	Double	Precision 15
Integer	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Long Varbinary	Binary	1 to 104,857,600 bytes
Nchar	String	1 to 104,857,600 characters
Nvarchar	String	1 to 104,857,600 characters
Ntext	Text	1 to 104,857,600 characters
Numeric	Decimal	Precision 1 to 28, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Real	Double	Precision 15
Smallint	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0

Data Type	Transformation	Range
Text	Text	1 to 104,857,600 characters
Time	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Timestamp	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Tinyint	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
Varbinary	Binary	1 to 104,857,600 bytes
Varchar	String	1 to 104,857,600 characters

Oracle and Transformation Data Types

Oracle data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares Oracle data types and transformation data types:

Oracle	Range	Transformation	Range
Blob	Up to 4 GB	Binary	1 to 104,857,600 bytes
Char(L)	1 to 2,000 bytes	String	1 to 104,857,600 characters
Clob	Up to 4 GB	Text	1 to 104,857,600 characters
Date	Jan. 1, 4712 B.C. to Dec. 31, 4712 A.D. Precision 19, scale 0	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Float	Precision of 1 to 15, scale 0	Double	Precision of 15, scale 0

Oracle	Range	Transformation	Range
Long	Up to 2 GB	Text	1 to 104,857,600 characters If you include Long data in a mapping, the Integration Service converts it to the transformation String data type, and truncates it to 104,857,600 characters.
Long Raw	Up to 2 GB	Binary	1 to 104,857,600 bytes
Nchar	1 to 2,000 bytes	String	1 to 104,857,600 characters
Nclob	Up to 4 GB	Text	1 to 104,857,600 characters
Number	Precision of 1 to 38	Double	Precision of 15
Number(P,S)	Precision of 1 to 38, scale of 0 to 38	Decimal	Precision 1 to 28 digits, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Nvarchar2	1 to 4,000 bytes	String	1 to 104,857,600 characters
Raw	1 to 2,000 bytes	Binary	1 to 104,857,600 bytes
Timestamp	Jan. 1, 4712 B.C. to Dec. 31, 9999 A.D. Precision 19 to 29, scale 0 to 9 (precision to the nanosecond)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)

Oracle	Range	Transformation	Range
Timestamp with Time Zone	Jan. 1, 4712 B.C. to Dec. 31, 9999 A.D. -12:00 to +14:00 Precision of 26 to 36, scale 0 to 9. (precision to the nanosecond)	timestampWithTZ	Aug. 1, 1947 A.D to Dec. 31, 2040 A.D. -12:00 to +14:00 Precision of 36 and scale of 9. (precision to the nanosecond) Timestamp with Time Zone data type does not support the following time zone regions: - AFRICA_CAIRO - AFRICA_MONROVIA - EGYPT - AMERICA_MONTREAL
Timestamp with Local Time Zone	Jan. 1, 4712 B.C. to Dec. 31, 9999 A.D. Precision 19 to 29, scale 0 to 9 (precision to the nanosecond)	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Varchar	1 to 4,000 bytes	String	1 to 104,857,600 characters
Varchar2	1 to 4,000 bytes	String	1 to 104,857,600 characters
XMLType	Up to 4 GB	Text	1 to 104,857,600 characters

Number(P,S) Data Type

The Developer tool supports Oracle Number(P,S) values with negative scale.

However, it does not support Number(P,S) values with scale greater than precision 38 or a negative precision.

However, it does not support Number(P,S) values with scale greater than precision 28 or a negative precision.

If you import a table with an Oracle Number with a negative scale, the Developer tool displays it as a Decimal data type. However, the Data Integration Service converts it to a double.

Char, Varchar, Clob Data Types

When the Data Integration Service uses the Unicode data movement mode, it reads the precision of Char, Varchar, and Clob columns based on the length semantics that you set for columns in the Oracle database.

If you use the byte semantics to determine column length, the Data Integration Service reads the precision as the number of bytes. If you use the char semantics, the Data Integration Service reads the precision as the number of characters.

Unsupported Oracle Data Types

The Developer tool does not support certain Oracle data types.

The Developer tool does not support the following Oracle data types:

- Bfile
- Interval Day to Second
- Interval Year to Month
- Mslabel
- Raw Mslabel
- Rowid

SAP HANA and Transformation Datatypes

SAP HANA datatypes map to transformation datatypes that the Data Integration Service uses to move data across platforms.

The following table compares SAP HANA datatypes and transformation datatypes:

SAP HANA Datatype	Range	Transformation Datatype	Range
Alphanum	Precision 1 to 127	Nstring	1 to 104,857,600 characters
Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision 19, scale 0
Binary	Used to store bytes of binary data	Binary	1 to 104,857,600 bytes
Blob	Up to 2 GB	Binary	1 to 104,857,600 bytes
Clob	Up to 2 GB	Text	1 to 104,857,600 characters
Date	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. Precision 10, scale 0	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)

SAP HANA Datatype	Range	Transformation Datatype	Range
Decimal (precision, scale) or Dec (p, s)	Precision 1 to 34	Decimal	Precision 1 to 28, scale 0 to 28 For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Double	Specifies a single-precision 64-bit floating-point number	Double	Precision 15
Float	Precision 1 to 53	Double	Precision 15
Integer	-2,147,483,648 to 2,147,483,647	Integer	-2,147,483,648 to 2,147,483,647 Precision 10, scale 0
NClob	Up to 2 GB	Ntext	1 to 104,857,600 characters
Nvarchar	Precision 1 to 5000	Nstring	1 to 104,857,600 characters
Real	Specifies a single-precision 32-bit floating-point number	Real	Precision 7, scale 0
Seconddate	0001-01-01 00:00:01 to 9999-12-31 24:00:00	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Shorttext	Specifies a variable-length character string, which supports text search and string search features	Nstring	1 to 104,857,600 characters
Smalldecimal	Precision 1 to 16	Decimal	Precision 1 to 28, scale 0 to 28
Smallint	-32,768 to 32,767	Small Integer	Precision 5, scale 0
Text	Specifies a variable-length character string, which supports text search features	Text	1 to 104,857,600 characters
Time	24-hour time period	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Timestamp	0001-01-01 00:00:00.0000000 to 9999-12-31 23:59:59.9999999	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Tinyint	0 to 255	Small Integer	Precision 5, scale 0

SAP HANA Datatype	Range	Transformation Datatype	Range
Varchar	Precision 1 to 5000	String	1 to 104,857,600 characters
Varbinary	1 to 5000 bytes	Binary	1 to 104,857,600 bytes

XML and Transformation Datatypes

XML datatypes map to transformation datatypes that the Data Integration Service uses to move data across platforms.

The Data Integration Service supports all XML datatypes specified in the W3C May 2, 2001 Recommendation. However, the Data Integration Service may not support the entire XML value range. For more information about XML datatypes, see the W3C specifications for XML datatypes at the following location: http://www.w3.org/TR/xmlschema-2.

The following table compares XML datatypes to transformation datatypes:

Datatype	Transformation	Range
anyURI	String	1 to 104,857,600 characters
base64Binary	Binary	1 to 104,857,600 bytes
boolean	String	1 to 104,857,600 characters
byte	Integer	-2,147,483,648 to 2,147,483,647
date	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
dateTime	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
decimal	Decimal	Precision 1 to 28, scale 0 to 28
double	Double	Precision of 15 digits
duration	String	1 to 104,857,600 characters
ENTITIES	String	1 to 104,857,600 characters
ENTITY	String	1 to 104,857,600 characters
float	Double	Precision of 15 digits
gDay	String	1 to 104,857,600 characters
gMonth	String	1 to 104,857,600 characters
gMonthDay	String	1 to 104,857,600 characters

Datatype	Transformation	Range
gYear	String	1 to 104,857,600 characters
gYearMonth	String	1 to 104,857,600 characters
hexBinary	Binary	1 to 104,857,600 bytes
ID	String	1 to 104,857,600 characters
IDREF	String	1 to 104,857,600 characters
IDREFS	String	1 to 104,857,600 characters
int	Integer	-2,147,483,648 to 2,147,483,647
integer	Integer	-2,147,483,648 to 2,147,483,647
language	String	1 to 104,857,600 characters
long	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Name	String	1 to 104,857,600 characters
NCName	String	1 to 104,857,600 characters
negativeInteger	Integer	-2,147,483,648 to 2,147,483,647
NMTOKEN	String	1 to 104,857,600 characters
NMTOKENS	String	1 to 104,857,600 characters
nonNegativeInteger	Integer	-2,147,483,648 to 2,147,483,647
nonPositiveInteger	Integer	-2,147,483,648 to 2,147,483,647
normalizedString	String	1 to 104,857,600 characters
NOTATION	String	1 to 104,857,600 characters
positiveInteger	Integer	-2,147,483,648 to 2,147,483,647
QName	String	1 to 104,857,600 characters
short	Integer	-2,147,483,648 to 2,147,483,647
string	String	1 to 104,857,600 characters
time	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
token	String	1 to 104,857,600 characters
unsignedByte	Integer	-2,147,483,648 to 2,147,483,647
unsignedInt	Integer	-2,147,483,648 to 2,147,483,647

Datatype	Transformation	Range
unsignedLong	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsignedShort	Integer	-2,147,483,648 to 2,147,483,647

Converting Data

You can convert data from one data type to another.

To convert data from one data type to another, use one the following methods:

- Pass data between ports (port-to-port data conversion).
- Use transformation functions to convert data.
- Use transformation arithmetic operators to convert data.

Port-to-Port Data Conversion

The Data Integration Service converts data based on the data type of the port. Each time data passes through a port, the Data Integration Service identifies the data type assigned to the port and converts the data based on the supported data type if required.

Pass Data Between Ports of the Same Data Type

You can create a mapping to pass data between ports of the same data type. When you pass data between ports of the same data type, the Data Integration Service passes data without changing the data or data type.

When you pass data between ports of the same numeric data type and the data is transferred between transformations, the Data Integration Service does not always convert the data to the precision and scale of the port that the data is passed to.

You can transfer Decimal data between ports of different precision, scale, and precision modes in the following ways:

Pass Decimal data in low-precision mode

If you pass data to a Decimal port in low-precision mode, the Data Integration Service converts all Decimal ports to Double with a precision of 15 and a scale of 0. For example, you pass a value from Decimal (14, 3) to Decimal (9, 5) in low-precision mode. The Data Integration Service stores the value internally and does not truncate the data in low-precision mode.

Pass Decimal data with reduced scale in high-precision mode

If you pass data to a Decimal port in high-precision mode and with reduced scale between the Decimal ports, scale truncation occurs. For example, you pass a value from Decimal (18, 5) to Decimal (18, 2). When you pass 18.01234, the Data Integration Service truncates the scale of the data and the output data is 18.01.

Pass Decimal data with reduced precision in high-precision mode

You can pass data to a Decimal port in high-precision mode with reduced precision. For example, you pass a value from Decimal (19, 5) to a Decimal (17, 2) in high-precision mode. When the output field contains a value that exceeds 17 digits, the Data Integration Service rejects the row.

Pass Data Between Ports of Different Data Types

When you pass data between ports of different data types, the Data Integration Service uses the conversion functions in the transformation language to convert the data from one data type to another.

For example, you connect a String port to an Integer port. When the Data Integration Service runs the mapping, it uses the TO_INTEGER function to convert the input data from a string to an integer data type.

When the Data Integration Service performs port-to-port conversions, the data that you pass must be valid for the conversion data type. Any value that the Data Integration Service cannot convert results in a transformation row error. For example, you connect a String port that contains the value "9,000,000,000,000,000,000.777" to a Bigint port. The Data Integration Service cannot convert the String to a Bigint value and returns an error.

The Data Integration Service performs port-to-port conversions between transformations and between the last transformation in a pipeline and a target.

The following table describes the port-to-port conversions that the Data Integration Service performs:

Data Type	Bigint	Integer	Decimal	Double	String, Text	Date/ Time	Binary	Timestamp with Time Zone
Bigint	No	Yes	Yes	Yes	Yes	No	No	No
Integer	Yes	No	Yes	Yes	Yes	No	No	No
Decimal	Yes	Yes	No	Yes	Yes	No	No	No
Double	Yes	Yes	Yes	No	Yes	No	No	No
String, Text	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes
Date/Time	No	No	No	No	Yes	Yes	No	No
Binary	No	No	No	No	No	No	Yes	No
Timestamp with Time Zone	No	No	No	No	Yes	No	No	Yes

APPENDIX B

Keyboard Shortcuts

This appendix includes the following topics:

- Keyboard Shortcuts for Advanced Mapping Options, 230
- Keyboard Shortcuts for Objects, 231
- Keyboard Shortcuts for Ports, 232
- Keyboard Shortcuts for the Transformation Palette, 233
- Keyboard Shortcuts for the Workbench, 233

Keyboard Shortcuts for Advanced Mapping Options

You can use keyboard shortcuts to navigate the Run Mapping Using Advanced Options dialog box.

You can perform the following tasks with keyboard shortcuts:

Select a mapping configuration.

Press **Shift+a**.

Specify a custom mapping configuration.

Press **Shift+c**.

Use the default Data Integration Service.

Press **Shift+u**.

Data Integration Service.

Press **Shift+i**. To browse for more options, press **Shift+b**.

Override tracing level.

Press **Shift+t**.

Optimizer level.

Press **Shift+z**.

Apply the default values in the mapping.

Press **Shift+d**.

Apply a parameter set.

Press **Shift+s**. To browse for more options, press **Shift+o**.

Apply a parameter file.

Press **Shift+f**. To browse for more options, press **Shift+o**.

Keyboard Shortcuts for Objects

You can use keyboard shortcuts to work with objects that are in the editor.

When you select an object in the editor, you can change the appearance of the object and move the object. When an object is selected, a green border appears around the object. A dotted line also appears around a selected object indicating that the object is in focus. You can focus on an object while allowing other objects to be selected. Move the focus from a selected object to another object when you want to select multiple objects in the editor.

Note: The navigation order of objects in the editor is from top to bottom and left to right.

You can perform the following tasks with keyboard shortcuts:

Select an object.

When the editor is selected, press **Tab**. The object is also in focus. Press **Tab** again to select the next object.

Select the previous object.

Press **Shift+Tab**.

Find a particular object.

Press **Ctrl+O**.

Focus on the next object.

When an object is in focus, press **Ctrl+Tab** to move the focus to the next object. The previous object remains selected. Press **Ctrl+Tab** again to focus on the next object.

Select multiple objects individually.

When an object is in focus, press **Ctrl+Tab** to move the focus to the next object that you want to select. Then press **Ctrl+Spacebar** to select the focused object. Repeat these steps to select multiple objects individually.

Iconize selected objects.

Press **Ctrl+I**.

Restore selected objects.

Press **Ctrl+R**.

Resize selected objects.

Press and hold **Ctrl+Shift**, and then use the **Up**, **Down**, **Left**, and **Right** arrow keys.

Move selected objects.

Use the arrow keys to move one or more objects one pixel at a time in any direction.

Go from an object to the editor.

When one or multiple objects are selected or in focus, press **Esc**.

Keyboard Shortcuts for Ports

You can use keyboard shortcuts to work with ports.

When you select a port, you can edit the port and link the port to other ports. When a port is selected, the entire port appears green. A dotted line also appears around one port value of the selected port indicating that the port is in focus. You can focus on a port while allowing other ports to be selected. Move the focus from one port to another port when you want to select multiple ports individually.

You can perform the following tasks with keyboard shortcuts:

Select a port.

When an object is selected and in focus, press **Ctrl+G**. Use the **Up** and **Down** arrow keys to select a different port.

Select multiple ports.

Press **Shift+Up** or **Shift+Down**.

Select multiple ports individually.

Press and hold the **Ctrl** key, and then use the **Up** and **Down** arrow keys to focus on the port that you want to select. Then press **Ctrl+Spacebar** to select the port. Repeat these steps to select multiple ports individually.

Focus on and edit the next port value.

Press **Tab**.

Focus on and edit the previous port value.

Press **Shift+Tab**.

Go from a port to the object.

Press **Esc**.

Link ports in two objects.

Select the ports that you want to link in the first object, and then press **Ctrl+L**. The selected ports become highlighted. Navigate to the second object and select the ports that you want to link. Press **Ctrl +L** to link the ports.

End linking mode.

Press **Ctrl+Alt+L** to clear the ports that you selected in the first object.

Select a link.

Press **Ctrl+G** from the selected port to select the outgoing link from that port.

Select the next link.

Press **Tab**.

Select the previous link.

Press **Shift+Tab**.

Keyboard Shortcuts for the Transformation Palette

You can use keyboard shortcuts to navigate the **Transformation** palette.

You can perform the following tasks with keyboard shortcuts:

Go from the editor to the Transformation palette.

When an editor is selected, press **Ctrl+Shift+P**.

Navigate the Transformation palette.

Press **Tab**, or use the **Up** and **Down** arrow keys.

Go from the Transformation palette to the editor.

Press **Esc**.

Keyboard Shortcuts for the Workbench

You can use keyboard shortcuts to navigate editors and views in the workbench.

When you select an editor, you can navigate within the editor. A green border appears around an editor when it is selected.

You can focus on editors and views. A blue border appears around an editor or view when it is in focus. A dotted line appears around the tab of a view that is in focus. When a view is in focus, you can select the view or you can focus on another view.

You can perform the following tasks with keyboard shortcuts:

Focus on a view.

When an editor is selected, press **Shift+Tab**. Then use the **Left** and **Right** arrow keys to focus on another view within the same editor. Alternatively, press **Shift+Tab** until another view in the workbench is in focus.

Select the first area in a view.

When a view is in focus in an editor, press **Ctrl+Tab**. Press the **Tab** button three times to select the next widget.

Select the previous area in a view.

Press **Ctrl+Shift+Tab**.

In the mapping Properties view, select and read a description.

Press **Ctrl+A**.

Minimize an editor.

When a view is in focus, press **Shift+Tab** to select the **Minimize** control, and then press the spacebar.

Maximize an editor.

When a view is in focus, press **Shift+Tab** to select the **Minimize** control. Use the **Right** arrow key to select the **Maximize** control, and then press the spacebar.

Select an area of the workbench.

When an editor is selected, press **Ctrl+Tab** to select another area of the workbench, such as a view. Press the **Ctrl+Tab** again to select the next area.

Select the previous area of the workbench.

Press **Ctrl+Shift+Tab**.

Jump to the error message in a dialog box.

Press **Ctrl+M**.

APPENDIX C

Connection Properties

This appendix includes the following topics:

- Connection Properties Overview, 236
- Amazon Redshift Connection Properties, 236
- Amazon S3 Connection Properties, 239
- Greenplum Connection Properties, 242
- Google BigQuery Connection Properties, 243
- Google Cloud Storage Connection Properties, 245
- IBM DB2 Connection Properties, 246
- IBM DB2 for i5/OS Connection Properties, 249
- IBM DB2 for z/OS Connection Properties, 252
- JDBC Connection Properties, 255
- JDBC V2 Connection Properties, 257
- JD Edwards EnterpriseOne Connection Properties, 259
- LDAP Connection Properties, 260
- Microsoft Azure Blob Storage Connection Properties, 261
- Microsoft Azure Cosmos DB SQL API Connection Properties, 263
- Microsoft Azure Data Lake Storage Gen2 Connection Properties, 263
- MS SQL Server Connection Properties, 265
- Netezza Connection Properties, 269
- ODBC Connection Properties, 270
- Oracle Connection Properties, 271
- Salesforce Connection Properties, 274
- Salesforce Marketing Cloud Connection Properties, 275
- SAP Connection Properties, 276
- Snowflake Connection Properties, 279
- Teradata Parallel Transporter Connection Properties, 280
- Tableau Connection Properties, 283
- Identifier Properties in Database Connections, 283
- PowerExchange for PostgreSQL Connection Properties, 286
- Microsoft Dynamics 365 for Sales Connection Properties, 287
- PowerExchange for Oracle E-Business Suite Connection Properties, 289

- Siebel Application Connections for Sources, Targets, and EIM Invoker Transformations, 289
- Microsoft Dynamics CRM Connection, 290
- PowerExchange for Essbase Connections, 291
- Vertica Relational Connection Properties, 292
- PowerExchange for Db2 Warehouse Connections, 293
- PowerExchange for HANA Connections, 293

Connection Properties Overview

Connection properties enable the Informatica client to connect to data sources.

This chapter contains connection properties for each of the connections you can create and manage through Informatica clients.

Amazon Redshift Connection Properties

When you set up an Amazon Redshift connection, you must configure the connection properties.

The following table describes the Amazon Redshift connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters:~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 4,000 characters.
Location	The domain where you want to create the connection.
Type	The connection type. Select Amazon Redshift in the Database.

The **Details** tab contains the connection attributes of the Amazon Redshift connection. The following table describes the connection attributes:

Property	Description
Authentication Type	The authentication method to log in to Amazon Redshift. You can select one of the following authentication methods: - Default. Uses the username and password to connect to Amazon Redshift. - Redshift IAM Authentication via AssumeRole. Uses AssumeRole for IAM authentication to connect to Amazon Redshift.
Username	User name of the Amazon Redshift account.
Password	Password for the Amazon Redshift account.
Access Key ID	Amazon S3 bucket access key ID to stage data in the S3 bucket using unload and copy commands.
Secret Access Key	Amazon S3 bucket secret access key ID to stage the data in the S3 bucket.
Master Symmetric Key	Optional. Provide a 256-bit AES encryption key in the Base64 format when you enable client-side encryption. You can generate a key using a third-party tool. If you specify a value, ensure that you specify the encryption type as client-side encryption in the advanced target properties.
JDBC URL	Amazon Redshift connection URL.
Cluster Region	Optional. The AWS cluster region in which the bucket you want to access resides. Select a cluster region if you choose to provide a custom JDBC URL that does not contain a cluster region name in the **JDBC URL** connection property. If you specify a cluster region in both **Cluster Region** and **JDBC URL** connection properties, the Data Integration Service ignores the cluster region that you specify in the **JDBC URL** connection property. To use the cluster region name that you specify in the **JDBC URL** connection property, select **None** as the cluster region in this property. Select one of the following cluster regions: - Asia Pacific (Mumbai) - Asia Pacific (Seoul) - Asia Pacific (Singapore) - Asia Pacific (Sydney) - Asia Pacific (Tokyo) - AWS GovCloud (US) - Canada (Central) - China (Beijing) - China (Ningxia) - EU (Ireland) - EU (Frankfurt) - EU (London) - EU (Paris) - South America (Sao Paulo) - US East (Ohio) - US East (N. Virginia) - US West (N. California) - US West (Oregon) Default is None. You can only read data from or write data to the cluster regions supported by AWS SDK used by PowerExchange for Amazon Redshift.

Property	Description
Customer Master Key ID	Optional. Specify the customer master key ID generated by AWS Key Management Service (AWS KMS) or the Amazon Resource Name (ARN) of your custom key for cross-account access. You must generate the customer master key corresponding to the region where Amazon S3 bucket resides. You can specify any of the following values: **Customer generated customer master key** Enables client-side or server-side encryption. **Default customer master key** Enables client-side or server-side encryption. Only the administrator user of the account can use the default customer master key ID to enable client-side encryption.
IAM Assume Role ARN	The Amazon Resource Number (ARN) of the IAM role assumed by the user to use the dynamically generated temporary security credentials. Set the value of this property if you want to use the temporary security credentials to access Amazon Redshift. For more information about how to get the ARN of the IAM role, see the AWS documentation.
Cluster Identifier	The unique identifier of the cluster that hosts Amazon Redshift for which you are requesting the security credentials. Specify the cluster name.
Database Name	Name of the Amazon Redshift database.
Auto Create DBUser	Select to create a new Amazon Redshift database user at run time. Default is disabled. **Note:** Use this option to create a new Amazon Redshift database user. If you create a new user directly using the AWS command line, the user is not created in Amazon Redshift.
Database Group	Name of the database group. Specify the database user and group details after you log in to the Amazon Redshift database. **Note:** If you do not specify a group, the user is added to a public group.
Expiration Time	The time duration that the password for the Amazon Redshift database user expires. Specify a value between 900 seconds and 3600 seconds. Default is 900.
Use EC2 Role to Assume Role	Select to enable the EC2 role to assume another IAM role specified in the IAM AssumeRole ARN option.
IAM User Access Key	The access key of the IAM user that has permissions to assume the IAM AssumeRole ARN.
IAM User Secret Access Key	The secret access key of the IAM user that has permissions to assume the IAM Assume Role ARN.

Amazon S3 Connection Properties

When you set up an Amazon S3 connection, you must configure the connection properties.

The following table describes the Amazon S3 connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters:~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Optional. The description of the connection. The description cannot exceed 4,000 characters.
Location	The domain where you want to create the connection.
Type	The Amazon S3 connection type.
Access Key	Access key to access the Amazon S3 bucket. Provide the access key value based on the following authentication methods: - Basic authentication: provide the actual access key value. - IAM authentication: do not provide the access key value. - Temporary security credentials via assume role: provide access key of an IAM user with no permissions to access Amazon S3 bucket.
Secret Key	Secret access key to access the Amazon S3 bucket. The secret key is associated with the access key and uniquely identifies the account. Provide the access key value based on the following authentication methods: - Basic authentication: provide the actual access secret value. - IAM authentication: do not provide the access secret value. - Temporary security credentials via assume role: provide access secret of an IAM user with no permissions to access Amazon S3 bucket.
IAM Role ARN	The ARN of the IAM role assumed by the user to use the dynamically generated temporary security credentials. Enter the value of this property if you want to use the temporary security credentials to access the AWS resources. If you want to use the temporary security credentials with IAM authentication, do not provide the Access Key and Secret Key connection properties. If you want to use the temporary security credentials without IAM authentication, you must enter the value of the Access Key and Secret Key connection properties. For more information about how to obtain the ARN of the IAM role, see the AWS documentation.
Folder Path	The complete path to Amazon S3 objects. The path must include the bucket name and any folder name. Do not use a slash at the end of the folder path. For example, `<bucket name>/<my folder name>`.
Master Symmetric Key	Optional. Provide a 256-bit AES encryption key in the Base64 format when you enable client-side encryption. You can generate a master symmetric key using a third-party tool.

Property	Description
S3 Account Type	The type of the Amazon S3 account. Select **Amazon S3 Storage** or **S3 Compatible Storage**. Select the Amazon S3 storage option to use the Amazon S3 services. Select the S3 compatible storage option to specify the endpoint for a third-party storage provider such as Scality RING. By default, Amazon S3 storage is selected.
REST Endpoint	The S3 storage endpoint. Specify the S3 storage endpoint in HTTP/HTTPs format when you select the S3 compatible storage option. For example, `http://s3.isv.scality.com`.
Region Name	Select the AWS region in which the bucket you want to access resides. Select one of the following regions: - Asia Pacific (Mumbai) - Asia Pacific (Seoul) - Asia Pacific (Singapore) - Asia Pacific (Sydney) - Asia Pacific (Tokyo) - AWS GovCloud (US) - AWS GovCloud (US-East) - Canada (Central) - China (Beijing) - China (Hong Kong) - China (Ningxia) - EU (Ireland) - EU (Frankfurt) - EU (London) - EU (Paris) - EU (Stockholm) - Middle East (Bahrain) - South America (Sao Paulo) - US East (Ohio) - US East (N. Virginia) - US West (N. California) - US West (Oregon) Default is US East (N. Virginia). Not applicable for S3 compatible storage.
Customer Master Key ID	Optional. Specify the customer master key ID or alias name generated by AWS Key Management Service (AWS KMS) or the Amazon Resource Name (ARN) of your custom key for cross-account access. You must generate the customer master key for the same region where Amazon S3 bucket reside. You can specify any of the following values: **Customer generated customer master key** Enables client-side or server-side encryption. **Default customer master key** Enables client-side or server-side encryption. Only the administrator user of the account can use the default customer master key ID to enable client-side encryption.
Federated SSO IdP	SAML 2.0-enabled identity provider for the federated user single sign-on to use with the AWS account. PowerExchange for Amazon S3 supports only the `ADFS 3.0` identity provider. Select `None` if you do not want to use federated user single sign-on.

Property	Description
AWS VPC Endpoint Type	Specifies how the PowerExchange for Amazon S3 communicates with AWS services. Choose Interface to route traffic through a private network path using AWS PrivateLink. Select None to use standard public AWS service endpoints. Default is None.
S3 VPC Endpoint DNS Name	Enter the DNS name of the S3 VPC interface endpoint. Required if you select Interface as the endpoint type. For example: `vpce-xxxxx.s3.region.vpce.amazonaws.com`
STS VPC Endpoint DNS Name	Enter the DNS name of the STS VPC interface endpoint. For example, `vpce-xxxxx.sts.region.vpce.amazonaws.com` Required for private IAM role assumption and federated authentication.
KMS VPC Endpoint DNS Name	Enter the DNS name of the KMS VPC interface endpoint. Required when you enable private encryption and decryption operations. For example: `vpce-xxxxx.kms.region.vpce.amazonaws.com`.

Federated user single sign-on connection properties

Configure the following properties when you select `ADFS 3.0` in **Federated SSO IdP**:

Property	Description
Federated User Name	User name of the federated user to access the AWS account through the identity provider.
Federated User Password	Password for the federated user to access the AWS account through the identity provider.
IdP SSO URL	Single sign-on URL of the identity provider for AWS.
SAML Identity Provider ARN	ARN of the SAML identity provider that the AWS administrator created to register the identity provider as a trusted provider.
Role ARN	ARN of the IAM role assumed by the federated user.

Temporary security credentials related properties

Configure the following properties when you enable **Use Temporary Credentials**:

Property	Description
Temporary Access Key	Public part of the temporary credentials or temporary user name.
Temporary Secret Key	Confidential part of the temporary credentials or temporary password.
Session Token	A unique session identifier generated with the temporary credentials.

Greenplum Connection Properties

Use a Greenplum connection to connect to a Greenplum database. The Greenplum connection is a relational type connection. You can create and manage a Greenplum connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

When you create a Greenplum connection, you enter information for metadata and data access.

The following table describes Greenplum connection properties:

Property	Description
Name	Name of the Greenplum relational connection.
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or fewer and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Description of the connection. The description cannot exceed 765 characters.
Location	Domain on which you want to create the connection.
Type	Type of connection.

The user name, password, driver name, and connection string are required to import the metadata. The following table describes the properties for metadata access:

Property	Description
User Name	User name with permissions to access the Greenplum database.
Password	Password to connect to the Greenplum database.
Driver Name	The name of the Greenplum JDBC driver. For example: `com.pivotal.jdbc.GreenplumDriver` For more information about the driver, see the Greenplum documentation.
Connection String	Use the following connection URL: `jdbc:pivotal:greenplum://<hostname>:<port>;DatabaseName=<database_name>` For more information about the connection URL, see the Greenplum documentation.

PowerExchange for Greenplum uses the host name, port number, and database name to create a control file to provide load specifications to the Greenplum gpload bulk loading utility. It uses the Enable SSL option and the certificate path to establish secure communication to the Greenplum server over SSL.

The following table describes the connection properties for data access:

Property	Description
Host Name	Host name or IP address of the Greenplum server.
Port Number	Greenplum server port number. If you enter 0, the gpload utility reads from the environment variable $PGPORT. Default is 5432.
Database Name	Name of the database.
Enable SSL	Select this option to establish secure communication between the gpload utility and the Greenplum server over SSL.
Certificate Path	Path where the SSL certificates for the Greenplum server are stored. For information about the files that need to be present in the certificates path, see the gpload documentation.

Google BigQuery Connection Properties

When you set up a Google BigQuery connection, you must configure the connection properties.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes the Google BigQuery connection properties:

Property	Description
Service Account ID	Specifies the client_email value present in the JSON file that you download after you create a service account in Google BigQuery.
Service Account Key	Specifies the private_key value present in the JSON file that you download after you create a service account in Google BigQuery.
Connection mode	The mode that you want to use to read data from or write data to Google BigQuery. Select one of the following connection modes: - Simple. Flattens each field within the Record data type field as a separate field in the mapping. - Hybrid. Displays all the top-level fields in the Google BigQuery table including Record data type fields. PowerExchange for Google BigQuery displays the top-level Record data type field as a single field of the String data type in the mapping. - Complex. Displays all the columns in the Google BigQuery table as a single field of the String data type in the mapping. Default is Simple.
Schema Definition File Path	Specifies a directory on the client machine where the must create a JSON file with the sample schema of the Google BigQuery table. The JSON file name is the same as the Google BigQuery table name. Alternatively, you can specify a storage path in Google Cloud Storage where the must create a JSON file with the sample schema of the Google BigQuery table. You can download the JSON file from the specified storage path in Google Cloud Storage to a local machine.

Property	Description
Project ID	Specifies the project_id value present in the JSON file that you download after you create a service account in Google BigQuery. If you have created multiple projects with the same service account, enter the ID of the project that contains the dataset that you want to connect to.
Storage Path	This property applies when you read or write large volumes of data. Path in Google Cloud Storage where the creates a local stage file to store the data temporarily. You can either enter the bucket name or the bucket name and folder name. For example, enter `gs://<bucket_name>` or `gs://<bucket_name>/<folder_name>`
Dataset ID	Not applicable for PowerExchange for Google BigQuery.
Use Legacy SQL For Custom Query	Not applicable for PowerExchange for Google BigQuery.
Dataset Name for Custom Query	Not applicable for PowerExchange for Google BigQuery.
Staging Dataset	The Google BigQuery dataset name where you want to create the staging table to stage the data. You can define a Google BigQuery dataset that is different from the source or target dataset.
Region ID	The region name where the Google BigQuery dataset resides. For example, if you want to connect to a Google BigQuery dataset that resides in Las Vegas region, specify **us-west4** as the **Region ID**. **Note:** In the **Storage Path** connection property, ensure that you specify a bucket name or the bucket name and folder name that resides in the same region as the dataset in Google BigQuery. For more information about the regions supported by Google BigQuery, see the following Google BigQuery documentation:https://cloud.google.com/bigquery/docs/locations
Optional Properties	Specifies whether you can configure certain source and target functionalities through custom properties. You can select one of the following options: - None. Select if you do not want to configure any custom properties. - Required. If you want to specify custom properties to configure the source and target functionalities. Default is None.
Provide Optional Properties	Comma-separated key-value pairs of custom properties to enable certain source and target functionalities. Appears only when you select **Required** in the Optional Properties. For more information about the list of custom properties that you can specify, see the Informatica Knowledge Base article: https://kb.informatica.com/faq/7/Pages/26/632722.aspx

Connection Modes

You can configure a Google BigQuery connection to use one of the following connection modes:

Simple mode

If you use simple mode, PowerExchange for Google BigQuery flattens each field within the Record data type field as a separate field in the Google BigQuery data object.

Hybrid mode

If you use hybrid mode, PowerExchange for Google BigQuery displays all the top-level fields in the Google BigQuery table including Record data type fields. PowerExchange for Google BigQuery displays the top-level Record data type field as a single field of the String data type in the Google BigQuery data object.

Complex mode

If you use complex mode, PowerExchange for Google BigQuery displays all the columns in the Google BigQuery table as a single field of the String data type in the Google BigQuery data object.

You can use Google BigQuery views for simple, complex, and hybrid connection modes.

Google Cloud Storage Connection Properties

When you set up a Google Cloud Storage connection, you must configure the connection properties.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes the Google Cloud Storage connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters:~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. The ID must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Optional. The description of the connection. The description cannot exceed 4,000 characters.
Location	The domain where you want to create the connection.
Type	The connection type. Select **Google Cloud Storage**.
Project ID	Specifies the project_id value present in the JSON file that you download after you create a service account. If you have created multiple projects with the same service account, enter the ID of the project that contains the bucket that you want to connect to.
Service Account ID	Specifies the client_email value present in the JSON file that you download after you create a service account.
Service Account Key	Specifies the private_key value present in the JSON file that you download after you create a service account.

IBM DB2 Connection Properties

Use an IBM DB2 connection to access IBM DB2. An IBM DB2 connection is a relational database connection. You can create and manage an IBM DB2 connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes DB2 connection properties:

Property	Description
Database Type	The database type.
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: `~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /`
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 765 characters.
User Name	The database user name.
Password	The password for the database user name.
Pass-through security enabled	Enables pass-through security for the connection. When you enable pass-through security for a connection, the domain uses the client user name and password to log into the corresponding database, instead of the credentials defined in the connection object.
Connection String for data access	The DB2 connection URL used to access metadata from the database. `dbname` Where `dbname` is the alias configured in the DB2 client.

Property	Description

Metadata Access Properties: Connection String	Use the following metadata connection string URL:
	`jdbc:informatica:db2://<host name>:<port>;DatabaseName=<database name>`
	When you import a table, by default, all tables are displayed under the default schema name. To view tables under a specific schema instead of the default schema, you can specify the schema name from which you want to import the table. Include the ischemaname parameter in the URL to specify the schema name. For example, use the following syntax to import a table from a specific schema:
	`jdbc:informatica:db2://<host name>:<port>;DatabaseName=<database name>;ischemaname=<schema_name>`
	To search for a table in multiple schemas and import it, you can specify multiple schema names in the ischemaname parameter. The schema name is case sensitive. You cannot use special characters when you specify multiple schema names. Use the pipe (|) character to separate multiple schema names. For example, use the following syntax to search for a table in three schemas and import it:
	`jdbc:informatica:db2://<host name>:<port>;DatabaseName=<database name>;ischemaname=<schema_name1>|<schema_name2>|<schema_name3>`
	When you specify multiple schema names, you must clear the **Show Default Schema Only** option to view the tables under the specified schema names.

Property	Description

AdvancedJDBCSecurityOptions	Database parameters for metadata access to a secure database. Informatica treats the value of the AdvancedJDBCSecurityOptions field as sensitive data and stores the parameter string encrypted.
	To connect to a secure database, include the following parameters:
	- EncryptionMethod. Required. Indicates whether data is encrypted when transmitted over the network. This parameter must be set to SSL.
	- ValidateServerCertificate. Optional. Indicates whether Informatica validates the certificate that is sent by the database server.
	If this parameter is set to True, Informatica validates the certificate that is sent by the database server. If you specify the HostNameInCertificate parameter, Informatica also validates the host name in the certificate.
	If this parameter is set to false, Informatica does not validate the certificate that is sent by the database server. Informatica ignores any truststore information that you specify.
	- HostNameInCertificate. Optional. Host name of the machine that hosts the secure database. If you specify a host name, Informatica validates the host name included in the connection string against the host name in the SSL certificate.
	- cryptoProtocolVersion. Optional. If you enable TLS for the IBM DB2 instance, set the cryptoProtocolVersion parameter as follows:
	cryptoProtocolVersion=TLSv<version number>.
	For example, cryptoProtocolVersion=TLSv1.2
	Note: The version number must be the same as the TLS version you configured for the server.
	- TrustStore. Required. Path and file name of the truststore file.
	Note: If you configure SSL or TLS and specify only the file name, you must copy the truststore file to the Informatica installation directory. To test the connection and import metadata, copy the truststore file to the following directory:
	`<Informatica client installation directory>/clients/DeveloperClient`
	To run the mapping, copy the truststore file to the following directory:
	`<Informatica server installation directory>/tomcat/bin`
	Note: If you configure SSL or TLS and specify only the file name, you must copy the truststore file to the following directory to test the connection: `<Informatica server installation directory>/tomcat/bin`
	- TrustStorePassword. Required. Password for the truststore file for the secure database.
	Note: Informatica appends the secure JDBC parameters to the connection string. If you include the secure JDBC parameters directly to the connection string, do not enter any parameters in the AdvancedJDBCSecurityOptions field.

Data Access Properties: Connection String	The connection string used to access data from the database. For IBM DB2 this is `<database name>`
Code Page	The code page used to read from a source database or to write to a target database or file.
Environment SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the connection environment SQL each time it connects to the database.

Property	Description
Transaction SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the transaction environment SQL at the beginning of each transaction.
Retry Period	This property is reserved for future use.
Tablespace	The tablespace name of the database.
SQL Identifier Character	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type. Select (None) if the database uses regular identifiers. When the Data Integration Service generates SQL queries, the service does not place delimited characters around any identifiers. Select a character if the database uses delimited identifiers. When the Data Integration Service generates SQL queries, the service encloses delimited identifiers within this character.
Support Mixed-case Identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.
ODBC Provider	ODBC. The type of database to which ODBC connects. For pushdown optimization, specify the database type to enable the Data Integration Service to generate native database SQL. The options are: - Other - Sybase - Microsoft_SQL_Server Default is Other.

IBM DB2 for i5/OS Connection Properties

Use an IBM DB2 for i5/OS connection to access tables in IBM DB2 for i5/OS. An IBM DB2 for i5/OS connection is a relational database connection. You can create and manage an IBM DB2 for i5/OS connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes DB2 for i5/OS connection properties:

Property	Description
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 255 characters.
Connection Type	The connection type (DB2I).
Username	A database user name.
Password	A password for the specified user name or a valid PowerExchange passphrase. A PowerExchange passphrase can be from 9 to 31 characters in length and can contain the following characters: - Uppercase and lowercase letters - The numbers 0 to 9 - Spaces - The following special characters: ' - ; # \ , . / ! % & * ( ) _ + { } : @ | < > ? **Note:** The first character is an apostrophe. Passphrases cannot include single quotation marks (‘), double quotation marks (“), or currency symbols. To use passphrases, ensure that the PowerExchange Listener runs with a security setting of SECURITY=(1,N) or higher in the DBMOVER member. For more information, see "SECURITY Statement" in the *PowerExchange Reference Manual*.
Pass-through security enabled	Enables pass-through security for the connection.
Database name	The database instance name.
Location	Node name for the location of the PowerExchange Listener that connects to DB2. The node name is defined in the first parameter of the NODE statement in the PowerExchange dbmover.cfg configuration file.
Environment SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service executes the connection environment SQL each time it connects to the database.

Property	Description
Database file overrides	Specifies the i5/OS database file override in the following format: `from_file/to_library/to_file/to_member` Where: - *from_file* is the file to be overridden. - *to_library* is the new library to use. - *to_file* is the file in the new library to use. - *to_member* is optional and is the member in the new library and file to use. *FIRST is used if nothing is specified. You can specify up to eight unique file overrides on a single connection. A single override applies to a single source or target. When you specify more than one file override, enclose the string of file overrides in double quotes (") and include a space between each file override. **Note:** If you specify both **Library List** and **Database File Overrides** and a table exists in both, the **Database File Overrides** value takes precedence.
Library list	List of libraries that PowerExchange searches to qualify the table name for Select, Insert, Delete, or Update statements. PowerExchange searches the list if the table name is unqualified. Separate libraries with commas. **Note:** If you specify both **Library List** and **Database File Overrides** and a table exists in both, the **Database File Overrides** value takes precedence.
Code Page	The code page used to read from a source database or write to a target database or file.
SQL Identifier character to use	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type.
Support mixed case identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.
Isolation level	Commit scope of the transaction. Select one of the following options: - **None** - **CS**. Cursor stability. - **RR**. Repeatable Read. - **CHG**. Change. - **ALL** Default is CS.
Encryption type	Optional. The type of encryption that the Data Integration Service uses. Select one of the following options: - **None** - **AES** Default is None. **Note:** Informatica recommends that you use Secure Sockets Layer (SSL) authentication instead of configuring the **Encryption type** and **Encryption level** connection properties. SSL authentication provides stricter security and is used by several Informatica products. For more information about implementing SSL authentication in a PowerExchange network, see the *PowerExchange Reference Manual*.

Property	Description
Encryption level	If you select **AES** for **Encryption Type**, select one of the following options to indicate the encryption level that the Data Integration Service uses: - **1**. Use a128-bit encryption key. - **2**. Use a 192-bit encryption key. - **3**. Use a 256-bit encryption key. If you do not select **AES** for **Encryption Type**, this option is ignored. Default is 1.
Pacing size	Optional. Amount of data that the source system can pass to the PowerExchange Listener. Set the pacing size if an external application, database, or the Data Integration Service node is a bottleneck. Use lower values for faster performance. The minimum value and default value is 0. A value of 0 provides the best performance.
Interpret as rows	Optional. Select this option to express the pacing size as a number of rows. Clear this option to express the pacing size in kilobytes. By default, this option is not selected and the pacing size is in kilobytes.
Compression	Optional. Select this option to enable source data compression. By compressing data, you can decrease the amount of data that Informatica applications send over the network. By default, this option is not selected and compression is disabled.
Array size	Optional. The number of records in the storage array for the worker threads. This option is applicable when you set the **Worker Threads** option to a value greater than 0. Valid values are 25 to 5000. Default is 25.
Write mode	Optional. Mode in which the Data Integration Service sends data to the PowerExchange Listener. Select one of the following write modes: - **CONFIRMWRITEON**. Sends data to the PowerExchange Listener and waits for a response before sending more data. Select this option when error recovery is a priority. However, this option might degrade performance. - **CONFIRMWRITEOFF**. Sends data to the PowerExchange Listener without waiting for a response. Use this option if you can reload the target table when an error occurs. - **ASYNCHRONOUSWITHFAULTTOLERANCE**. Sends data to the PowerExchange Listener without waiting for a response. This option also enables error detection. This option combines the speed of CONFIRMWRITEOFF and the data integrity of CONFIRMWRITEON. Default is CONFIRMWRITEON.
Reject file	Overrides the default prefix of PWXR for the reject file. PowerExchange creates the reject file on the target machine when the write mode is ASYNCHRONOUSWITHFAULTTOLERANCE. Enter PWXDISABLE to prevent the creation of the reject files.

IBM DB2 for z/OS Connection Properties

Use an IBM DB2 for z/OS connection to access tables in IBM DB2 for z/OS. An IBM DB2 for z/OS connection is a relational database connection. You can create and manage an IBM DB2 for z/OS connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes DB2 for z/OS connection properties:

Property	Description
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Description of the connection. The description cannot exceed 255 characters.
Connection Type	Connection type (DB2Z).
Username	Database user name.
Password	Password for the specified user name or a valid PowerExchange passphrase. A PowerExchange passphrase can be from 9 to 128 characters in length and can contain the following characters: - Uppercase and lowercase letters - The numbers 0 to 9 - Spaces - The following special characters: ' - ; # \ , . / ! % & * ( ) _ + { } : @ | < > ? **Note:** The first character is an apostrophe. Passphrases cannot include single quotation marks (‘), double quotation marks (“), or currency symbols. To use passphrases, ensure that the PowerExchange Listener runs with a security setting of SECURITY=(1,N) or higher in the DBMOVER member. For more information, see "SECURITY Statement" in the *PowerExchange Reference Manual*. The allowable characters in the IBM IRRPHREX exit do not affect the allowable characters in PowerExchange passphrases. **Note:** A valid RACF passphrase can be up to 100 characters in length. PowerExchange truncates passphrases longer than 100 characters when passing them to RACF for validation.
Pass-through security enabled	Enables pass-through security for the connection.
DB2 Subsystem ID	Name of the DB2 subsystem.
Location	Node name for the location of the PowerExchange Listener that connects to DB2. The node name is defined in the first parameter of the NODE statement in the PowerExchange dbmover.cfg configuration file.
Environment SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service executes the connection environment SQL each time it connects to the database.
Correlation ID	Value to be concatenated to prefix PWX to form the DB2 correlation ID for DB2 requests.

Property	Description
Code Page	Code page used to read from a source database or write to a target database or file.
SQL identifier character to use	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type. Select (None) if the database uses regular identifiers. When the Data Integration Service generates SQL queries, the service does not place delimited characters around any identifiers. Select a character if the database uses delimited identifiers. When the Data Integration Service generates SQL queries, the service encloses delimited identifiers within this character.
Support mixed case identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.
Encryption type	Optional. The type of encryption that the Data Integration Service uses. Select one of the following options: - **None** - **AES** Default is None. **Note:** Informatica recommends that you use Secure Sockets Layer (SSL) authentication instead of configuring the **Encryption Type** and **Level** connection properties. SSL authentication provides stricter security and is used by several Informatica products. For more information about implementing SSL authentication in a PowerExchange network, see the *PowerExchange Reference Manual*.
Encryption level	If you select **AES** for **Encryption Type**, select one of the following options to indicate the encryption level that the Data Integration Service uses: - **1**. Use a128-bit encryption key. - **2**. Use a 192-bit encryption key. - **3**. Use a 256-bit encryption key. If you do not select **AES** for **Encryption Type**, this option is ignored. Default is 1.
Pacing size	Optional. Amount of data that the source system can pass to the PowerExchange Listener. Set the pacing size if an external application, the Data Integration Service, or the database node is a bottleneck. User lower values for faster performance. The minimum value and default value is 0. A value of 0 provides the best performance.
Interpret as rows	Optional. Select this option to express the pacing size as a number of rows. Clear this option to express the pacing size in kilobytes. By default, this option is not selected and the pacing size is in kilobytes.
Compression	Optional. Select this option to enable source data compression. By compressing data, you can decrease the amount of data that Informatica applications send over the network. By default, this option is not selected and compression is disabled.
Offload processing	Optional. Controls whether to offload some bulk data processing from the source machine to the Data Integration Service machine. Select one of the following options: - **AUTO**. The Data Integration Service determines whether to use offload processing. - **Yes**. Use offload processing. - **No**. Do not use offload processing. Default is No.

Property	Description
Worker threads	Optional. Number of threads that the Data Integration Service uses to process bulk data when offload processing is enabled. For optimal performance, this value should not exceed the number of available processors on the Data Integration Service machine. Valid values are 1 through 64. Default is 0, which disables multithreading.
Array size	Optional. The number of records in the storage array for the worker threads. This option is applicable when you set the **Worker Threads** option to a value greater than 0. Valid values are 1 to 5000. Default is 25.
Write mode	Mode in which the Data Integration Service sends data to the PowerExchange Listener. Configure one of the following write modes: - **CONFIRMWRITEON**. Sends data to the PowerExchange Listener and waits for a response before sending more data. Select if error recovery is a priority. This option might decrease performance. - **CONFIRMWRITEOFF**. Sends data to the PowerExchange Listener without waiting for a response. Use this option when you can reload the target table if an error occurs. - **ASYNCHRONOUSWITHFAULTTOLERANCE**. Sends data to the PowerExchange Listener without waiting for a response. This option also provides the ability to detect errors. This provides the speed of Confirm Write Off with the data integrity of Confirm Write On. Default is CONFIRMWRITEON.
Reject file	Overrides the default prefix of PWXR for the reject file. PowerExchange creates the reject file on the target machine when the write mode is ASYNCHRONOUSWITHFAULTTOLERANCE. Enter PWXDISABLE to prevent the creation of reject files.

JDBC Connection Properties

You can use a JDBC connection to access tables in a database. You can create and manage a JDBC connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes JDBC connection properties:

Property	Description
Database Type	The database type.
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: `~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /`
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 765 characters.
User Name	The database user name.

Property	Description
Password	The password for the database user name.

JDBC Driver Class Name	Name of the JDBC driver class.
	The following list provides the driver class name that you can enter for the applicable database type:
	- DataDirect JDBC driver class name for Oracle:
	`com.informatica.jdbc.oracle.OracleDriver`
	- DataDirect JDBC driver class name for IBM DB2:
	`com.informatica.jdbc.db2.DB2Driver`
	- DataDirect JDBC driver class name for Microsoft SQL Server:
	`com.informatica.jdbc.sqlserver.SQLServerDriver`
	- DataDirect JDBC driver class name for Sybase ASE:
	`com.informatica.jdbc.sybase.SybaseDriver`
	- DataDirect JDBC driver class name for Informix:
	`com.informatica.jdbc.informix.InformixDriver`
	- DataDirect JDBC driver class name for MySQL:
	`com.informatica.jdbc.mysql.MySQLDriver`
	For more information about which driver class to use with specific databases, see the vendor documentation.

Connection String	Connection string to connect to the database. Use the following connection string: `jdbc:<subprotocol>:<subname>` For more information about the connection string to use with specific drivers, see the vendor documentation.
Environment SQL	Optional. Enter SQL commands to set the database environment when you connect to the database. The Data Integration Service executes the connection environment SQL each time it connects to the database. **Note:** If you enable Sqoop, Sqoop ignores this property.
Transaction SQL	Optional. Enter SQL commands to set the database environment when you connect to the database. The Data Integration Service executes the transaction environment SQL at the beginning of each transaction. **Note:** If you enable Sqoop, Sqoop ignores this property.
SQL Identifier Character	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type. **Note:** If you enable Sqoop, Sqoop ignores this property.
Support Mixed-case Identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled. **Note:** If you enable Sqoop, Sqoop honors this property when you generate and execute a DDL script to create or replace a target at run time. In all other scenarios, Sqoop ignores this property.

Property	Description
Use Sqoop Connector	Enables Sqoop connectivity for the data object that uses the JDBC connection. You can configure Sqoop connectivity for relational data objects, customized data objects, and logical data objects that are based on a JDBC-compliant database. Select **Sqoop v1.x** to enable Sqoop connectivity. Default is **None**.
Sqoop Arguments	Enter the arguments that Sqoop must use to connect to the database. Separate multiple arguments with a space. To run the mapping on the Blaze engine with the Teradata Connector for Hadoop (TDCH) specialized connectors for Sqoop, you must define the TDCH connection factory class in the Sqoop arguments. The connection factory class varies based on the TDCH Sqoop Connector that you want to use. **Note:** To run the mapping with a generic JDBC connector instead of the specialized Cloudera or Hortonworks connector, you must define the --driver and --connection-manager Sqoop arguments in the JDBC connection. If you define the --driver and --connection-manager arguments in the Read or Write transformation of the mapping, Sqoop ignores the arguments. If you do not enter Sqoop arguments, the Data Integration Service constructs the Sqoop command based on the JDBC connection properties.

JDBC V2 Connection Properties

When you set up a JDBC V2 connection, you must configure the connection properties.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes the JDBC V2 connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters:~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 4,000 characters.
Location	The domain where you want to create the connection.
Type	The connection type. Select JDBC V2.

The **Details** tab contains the connection attributes of the JDBC V2 connection. The following table describes the connection attributes:

Property	Description
Username	The database user name. User name with permissions to access the database that supports the Type 4 JDBC driver.
Password	The password for the database user name.
Schema Name	Optional. The schema name to connect in the database. If you do not specify the schema name, all the schemas available in the database are listed.

JDBC Driver Class Name	Name of the JDBC driver class.
	The following list provides the driver class name that you can enter for the applicable database type:
	- JDBC driver class name for Azure SQL Database:
	`com.microsoft.sqlserver.jdbc.SQLServerDriver`
	- JDBC driver class name for Aurora PostgreSQL:
	`org.postgresql.Driver`
	- JDBC driver class name for SAP HANA Database:
	`com.sap.db.jdbc.Driver`
	- JDBC driver class name for Phoenix HBase Database:
	`org.apache.phoenix.jdbc.PhoenixDriver`
	For more information about which driver class to use with specific databases, see the third-party vendor documentation.

Connection String	Connection string to connect to the database.
	Use the following connection string:
	`jdbc:<subprotocol>:<subname>`
	The following list provides sample connection strings that you can enter for the applicable database type:
	- Connection string for Azure SQL Database JDBC driver:
	`jdbc:sqlserver://<host>:<port>;database=<database_name>`
	- Connection string for Aurora PostgreSQL JDBC driver:
	`jdbc:postgresql://<host>:<port>[/<database_name>]`
	- Connection string for SAP HANA Database driver:
	`jdbc:sap://<host>:<port>/?databaseName=<Database_Name>`
	- Connection string for Phoenix HBase Database driver:
	  - using the Developer Tool:
	`jdbc:<protocol variant>[:<server list>[:<port>[:<zk root node>[:<principal>[:<keytab file>]]]]]`
	For example,
	`jdbc:phoenix:testhost1,testhost2,testhost3:2182:/hbase:principal@TESTKRB.com:\etc\config\test.keytab`
	  - using the Metadata Access Service:
	`jdbc:<protocol variant>[:<server list>[:<port>[:<zk root node>]]]`
	For example,
	`jdbc:phoenix:testhost1,testhost2,testhost3:2182:/hbase`
	For more information about the connection string to use with specific drivers, see the third-party vendor documentation.

Sub Type	The database type to which you want to connect. You can select from the following database types to connect: - **Azure SQL Database**. Connects to Azure SQL database. - **PostgreSQL**. Connects to Aurora PostgreSQL database. - **SAP HANA Database**. Connects to SAP HANA database. - **Phoenix Hbase**. Connects to Phoenix HBase database. - **Others** . Connects to any database that supports the Type 4 JDBC driver.

Property	Description
Support Mixed-case Identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. For example, Aurora PostgreSQL database supports mixed-cased characters. You must enable this property to connect to the Aurora PostgreSQL database. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.
SQL Identifier Character	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type. **Note:** Select **SQL Identifier Character** as **None** when you specify the SAP HANA Database subtype.

JD Edwards EnterpriseOne Connection Properties

Use a JD Edwards EnterpriseOne connection to connect to a JD Edwards EnterpriseOne object.

The following table describes the JD Edwards EnterpriseOne connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. It cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	The string that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 765 characters.
Location	The Informatica domain where you want to create the connection.
Type	The connection type. Select JD Edwards EnterpriseOne.
Host Name	JD Edwards EnterpriseOne server host name.
Enterprise Port	JD Edwards EnterpriseOne server port number. Default is 6016.
User Name	The JD Edwards EnterpriseOne database user name.
Password	The password for the JD Edwards EnterpriseOne database user.
Environment	Name of the JD Edwards EnterpriseOne environment you want to connect to.
Role	Role of the JD Edwards EnterpriseOne user. Default is *ALL.
User Name	The JD Edwards EnterpriseOne database user name.

Property	Description
Password	Password for the database user.
Driver Class Name	The following list provides the driver class name that you can enter for the applicable database type: - DataDirect JDBC driver class name for Oracle: `com.informatica.jdbc.oracle.OracleDriver` - DataDirect JDBC driver class name for IBM DB2: `com.informatica.jdbc.db2.DB2Driver` - DataDirect JDBC driver class name for Microsoft SQL Server: `com.informatica.jdbc.sqlserver.SQLServerDriver` For more information about which driver class to use with specific databases, see the vendor documentation.
Connection String	The connection string to connect to the database. Use the following connection string: The JDBC connection string uses the following syntax: - For Oracle: `jdbc:informatica:oracle://<host name>:<port>,ServiceName=<db service name>` - For DB2: `jdbc:informatica:db2://<host name>:<port>;databaseName=<db name>` - For Microsoft SQL: `jdbc:informatica:sqlserver://<host name>:<port>;databaseName=<db name>`

LDAP Connection Properties

Use an LDAP connection to connect to an LDAP object.

The following table describes the LDAP connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. It cannot exceed 128 characters, contain spaces, or contain the following special characters: `~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /`
ID	The string that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 765 characters.
Location	The Informatica domain where you want to create the connection.
Type	The connection type. Select LDAP.
Host Name	LDAP directory server host name.

Property	Description
Port	Listening port for the LDAP server. Port number to communicate with the LDAP directory service. You can use port 389 for a non-SSL LDAP connection or port 636 for an LDAP connection over SSL. To connect to an Active Directory Global Catalog, you can use port 3268 for a non-SSL connection or port 3269 for a connection over SSL. The maximum port number is 65535.
Anonymous Connection	Establishes an anonymous connection with the LDAP directory server. Select anonymous connection to access a directory server as an anonymous user without authentication. **Note:** You cannot establish an anonymous connection with Active Directory.
User Name	The LDAP user name to connect to the LDAP directory server.
Password	The password to connect to the LDAP directory server.
Secure Connection	Establishes a secure connection with the LDAP directory server through the TLS protocol.
TrustStore File Name	The file name of the truststore that contains the TLS certificate to establish a secure connection with the LDAP directory server. Default is `infa_truststore.jks`. Required if you select Secure Connection. Contact the LDAP Administrator for the truststore file name and password.
TrustStore Password	The password for the truststore file that contains the SSL certificate.
KeyStore File Name	The file name of the keystore that contains the keys and certificates required to establish a secure communication with the LDAP directory server. Required if you select Secure Connection. Contact the LDAP Administrator for the keystore file name and password.
KeyStore Password	The password for the keystore file required for secure communication.

Microsoft Azure Blob Storage Connection Properties

Use a Microsoft Azure SQL Blob Storage connection to access a Microsoft Azure Blob Storage.

Note: The order of the connection properties might vary depending on the tool where you view them.

You can create and manage a Microsoft Azure Blob Storage connection in the Administrator tool or the Developer tool. The following table describes the Microsoft Azure Blob Storage connection properties:

Property	Description
Name	Name of the Microsoft Azure Blob Storage connection.
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Description of the connection.
Location	The domain where you want to create the connection.
Type	Type of connection. Select Azure Blob Storage.

The **Connection Details** tab contains the connection attributes of the Microsoft Azure Blob Storage connection. The following table describes the connection attributes:

Property	Description
Account Name	Name of the Microsoft Azure Storage account.
Authorization Type	Authorization type. You can select any of the following authorization mechanisms: - Shared Key Authorization - Shared Access Signatures
Account Key	Microsoft Azure Storage access key. Applies when you select shared key authorization.
SAS Token	SAS URI with SAS token that you generate on Microsoft Azure portal for your account. Applies when you select shared access signature authorization type. **Note:** You must provide a valid SAS URI with a valid SAS token.
Container Name	The root container or sub-folders with the absolute path. **Note:** To import complex files, specify only the root container.
Endpoint Suffix	Type of Microsoft Azure end-points. You can select any of the following end-points: - `core.windows.net`: Default - `core.usgovcloudapi.net`: To select the US government Microsoft Azure end-points - `core.chinacloudapi.cn`: Not applicable

Microsoft Azure Cosmos DB SQL API Connection Properties

Use a Microsoft Azure Cosmos DB connection to connect to the Cosmos DB database. When you create a Microsoft Azure Cosmos DB connection, you enter information for metadata and data access.

The following table describes the Microsoft Azure Cosmos DB connection properties:

Property	Description
Name	Name of the Cosmos DB connection.
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Description of the connection. The description cannot exceed 765 characters.
Location	The project or folder in the Model repository where you want to store the Cosmos DB connection.
Type	Select Microsoft Azure Cosmos DB SQL API.
Cosmos DB URI	The URI of Microsoft Azure Cosmos DB account.
Key	The primary and secondary key to which provides you complete administrative access to the resources within Microsoft Azure Cosmos DB account.
Database	Name of the database that contains the collections from which you want to read or write JSON documents.

Note: You can find the Cosmos DB URI and Key values in the **Keys** settings on Azure portal. Contact your Azure administrator for more details.

Microsoft Azure Data Lake Storage Gen2 Connection Properties

Use a Microsoft Azure Data Lake Storage Gen2 connection to access a Microsoft Azure Data Lake Storage Gen2.

Note: The order of the connection properties might vary depending on the tool where you view them.

You can create and manage a Microsoft Azure Data Lake Storage Gen2 connection in the Administrator tool or the Developer tool. The following table describes the Microsoft Azure Data Lake Storage Gen2 connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 4,000 characters.
Location	The domain where you want to create the connection.
Type	The connection type. Select Microsoft Azure Data Lake Storage Gen2.

The following table describes the properties for metadata access:

Property	Description
Account Name	The Microsoft Azure Data Lake Storage Gen2 account name or the service name.
Authentication Type	Authentication type to access the Microsoft Azure Data Lake Storage Gen2 account. Select one of the following options: - **Service Principal Authentication**. Select to use the client ID, client secret, and tenant ID to connect to Microsoft Azure Data Lake Storage Gen2. - **Shared Key Authentication**. Select to use the account key to connect to Microsoft Azure Data Lake Storage Gen2. **Note:** You cannot use shared key authentication in a streaming mapping.
Client ID	Applicable for Service Principal Authentication. The ID of your application to complete the OAuth Authentication in the Azure Active Directory (AD).
Client Secret	Applicable for Service Principal Authentication. The client secret key to complete the OAuth Authentication in the Azure Active Directory.
Tenant ID	Applicable for Service Principal Authentication. The Directory ID of the Azure Active Directory.
Account Key	Applicable for Shared Key Authentication. The account key for the Microsoft Azure Data Lake Storage Gen2 account.
File System Name	The name of an existing file system in the Microsoft Azure Data Lake Storage Gen2.

Property	Description
Directory Path	The path of an existing directory without the file system name. There is no default directory. You can select one of the following syntax: - / for root directory. - /dir1 - dir1/dir2
Adls Gen2 End-point	Type of Microsoft Azure endpoints. You can select any of the following endpoints: - core.windows.net: Default - core.usgovcloudapi.net: To select the Azure Government endpoints

For more information about creating a client ID, client secret, tenant ID, and file system name, contact the Azure administrator or see Microsoft Azure Data Lake Storage Gen2 documentation.

MS SQL Server Connection Properties

Use a Microsoft SQL Server connection to access Microsoft SQL Server. A Microsoft SQL Server connection is a connection to a Microsoft SQL Server relational database. You can create and manage a Microsoft SQL Server connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes MS SQL Server connection properties:

Property	Description
Database Type	The database type.
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 765 characters.
Use trusted connection	Enables the application service to use Windows authentication to access the database. The user name that starts the application service must be a valid Windows user with access to the database. By default, this option is cleared. **Note:** Windows and NTLM authentication is not certified for a Microsoft SQL Server 2017 version hosted on Linux.
User Name	The database user name. Required if Microsoft SQL Server uses NTLMv1 or NTLMv2 authentication.
Password	The password for the database user name. Required if Microsoft SQL Server uses NTLMv1 or NTLMv2 authentication.

Property	Description
Pass-through security enabled	Enables pass-through security for the connection. When you enable pass-through security for a connection, the domain uses the client user name and password to log into the corresponding database, instead of the credentials defined in the connection object.

Metadata Access Properties: Connection String	Connection string used to access metadata from the database.
	Use the following connection string:
	`jdbc:informatica:sqlserver://<host name>:<port>;DatabaseName=<database name>`
	To test the connection with NTLM authentication, include the following parameters in the connection string:
	- AuthenticationMethod. The NTLM authentication version to use.
	Note: UNIX supports NTLMv1 and NTLMv2 but not NTLM.
	- Domain. The domain that the SQL server belongs to.
	The following example shows the connection string for a SQL server that uses NTLMv2 authentication in a NT domain named Informatica.com:
	`jdbc:informatica:sqlserver://host01:1433;DatabaseName=SQL1;AuthenticationMethod=ntlm2java;Domain=Informatica.com`
	If you connect with NTLM authentication, you can enable the **Use trusted connection** option in the MS SQL Server connection properties. If you connect with NTLMv1 or NTLMv2 authentication, you must provide the user name and password in the connection properties.

Property	Description

AdvancedJDBCSecurityOptions	Database parameters for metadata access to a secure database. Informatica treats the value of the AdvancedJDBCSecurityOptions field as sensitive data and stores the parameter string encrypted.
	To connect to a secure database, include the following parameters:
	- EncryptionMethod. Indicates whether data is encrypted when transmitted over the network. This parameter must be set to SSL.
	- ValidateServerCertificate. Optional. Indicates whether Informatica validates the certificate that is sent by the database server.
	If this parameter is set to True, Informatica validates the certificate that is sent by the database server. If you specify the HostNameInCertificate parameter, Informatica also validates the host name in the certificate.
	If this parameter is set to False, Informatica does not validate the certificate that is sent by the database server. Informatica ignores any truststore information that you specify.
	- HostNameInCertificate. Optional. Host name of the machine that hosts the secure database. If you specify a host name, Informatica validates the host name included in the connection string against the host name in the SSL certificate.
	- cryptoProtocolVersion. Optional. If you enable TLS for the Microsoft SQL Server instance, set the cryptoProtocolVersion parameter as follows:
	cryptoProtocolVersion=TLSv<version number>.
	For example, cryptoProtocolVersion=TLSv1.2
	Note: The version number must be the same as the TLS version you configured for the server.
	- TrustStore. Required. Path and file name of the truststore file.
	Note: If you configure SSL or TLS and specify only the file name, you must copy the truststore file to the Informatica installation directory. To test the connection and import metadata, copy the truststore file to the following directory:
	`<Informatica client installation directory>/clients/DeveloperClient`
	To run the mapping, copy the truststore file to the following directory:
	`<Informatica server installation directory>/tomcat/bin`
	Note: If you configure SSL or TLS and specify only the file name, you must copy the truststore file to the following directory to test the connection:
	`<Informatica server installation directory>/tomcat/bin`
	- TrustStorePassword. Required. Password for the truststore file for the secure database.
	Not applicable for ODBC.
	Note: Informatica appends the secure JDBC parameters to the connection string. If you include the secure JDBC parameters directly to the connection string, do not enter any parameters in the AdvancedJDBCSecurityOptions field.

Property	Description
Data Access Properties: Provider Type	The connection provider that you want to use to connect to the Microsoft SQL Server database. You can select the following provider types: - ODBC - Oldeb(Deprecated) Default is ODBC. **Note:** Although the Microsoft SQL Server connection user interface shows the OLEDB provider type as deprecated, Informatica supports the OLEDB provider type. For more information about the OLEDB provider type support statement, see the following Knowledge Base article KB 522895.
Use DSN	Enables the Data Integration Service to use the Data Source Name for the connection. If you select the Use DSN option, the Data Integration Service retrieves the database and server names from the DSN. If you do not select the Use DSN option, you must provide the database and server names.

Connection String	Use the following connection string if you do not enable DSN mode:
	`<server name>@<database name>`
	If you enable DSN mode, use the following connection strings:
	`<DSN Name>`

Code Page	The code page used to read from a source database or to write to a target database or file.
Domain Name	The name of the domain.
Packet Size	The packet size used to transmit data. Used to optimize the native drivers for Microsoft SQL Server.
Owner Name	The name of the owner of the schema. **Note:** When you generate a table DDL through a dynamic mapping or through the **Generate and Execute DDL** option, the DDL metadata does not include schema name and owner name properties.
Schema Name	The name of the schema in the database. You must specify the schema name for the Profiling Warehouse if the schema name is different from the database user name. You must specify the schema name for the data object cache database if the schema name is different from the database user name and if you configure user-managed cache tables. **Note:** When you generate a table DDL through a dynamic mapping or through the **Generate and Execute DDL** option, the DDL metadata does not include schema name and owner name properties.
Environment SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the connection environment SQL each time it connects to the database.
Transaction SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the transaction environment SQL at the beginning of each transaction.
Retry Period	This property is reserved for future use.

Property	Description
SQL Identifier Character	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type. Select (None) if the database uses regular identifiers. When the Data Integration Service generates SQL queries, the service does not place delimited characters around any identifiers. Select a character if the database uses delimited identifiers. When the Data Integration Service generates SQL queries, the service encloses delimited identifiers within this character.
Support Mixed-case Identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.
ODBC Provider	ODBC. The type of database to which ODBC connects. For pushdown optimization, specify the database type to enable the Data Integration Service to generate native database SQL. The options are: - Other - Sybase - Microsoft_SQL_Server Default is Other.

Netezza Connection Properties

Use a Netezza connection to access a Netezza database. The Netezza connection is a database connection. You can create and manage a Netezza connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes the Netezza connection properties:

Property	Description
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Description of the connection. The description cannot exceed 765 characters.
Location	Domain where you want to create the connection.
Type	Connection type. Select **Netezza**.

Property	Description
User name	User name with the appropriate permissions to access the Netezza database.
Password	Password for the database user name.
JDBC Url	JDBC URL that the Developer tool must use when it connects to the Netezza database. Use the following format: `jdbc:netezza://<hostname>:<port>/<database name>`
Connection String	Name of the ODBC data source that you want to use to connect to the Netezza database.
Timeout	Number of seconds that the Developer tool waits for a response from the Netezza database before it closes the connection.

ODBC Connection Properties

Use an ODBC connection to access ODBC data. An ODBC connection is a relational database connection. You can create and manage an ODBC connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes ODBC connection properties:

Property	Description
Database Type	The database type.
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: `~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /`
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 765 characters.
User Name	The database user name.
Password	The password for the database user name.
Pass-through security enabled	Enables pass-through security for the connection. When you enable pass-through security for a connection, the domain uses the client user name and password to log into the corresponding database, instead of the credentials defined in the connection object.

Property	Description
Data Access Properties: Connection String	The ODBC connection URL used to access metadata from the database. `<data source name>`
Code Page	The code page used to read from a source database or to write to a target database or file.
Environment SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the connection environment SQL each time it connects to the database.
Transaction SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the transaction environment SQL at the beginning of each transaction.
Retry Period	This property is reserved for future use.
SQL Identifier Character	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type. Select (None) if the database uses regular identifiers. When the Data Integration Service generates SQL queries, the service does not place delimited characters around any identifiers. Select a character if the database uses delimited identifiers. When the Data Integration Service generates SQL queries, the service encloses delimited identifiers within this character.
Support Mixed-case Identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.
ODBC Provider	The type of database to which ODBC connects. For pushdown optimization, specify the database type to enable the Data Integration Service to generate native database SQL. The options are: - Other - Sybase - Microsoft_SQL_Server - Snowflake Default is Other.

Note: Use an ODBC connection to connect to Microsoft SQL Server when the Data Integration Service runs on UNIX or Linux. Use a native connection to Microsoft SQL Server when the Data Integration Service runs on Windows.

Oracle Connection Properties

Use an Oracle connection to connect to an Oracle database. The Oracle connection is a relational connection type. You can create and manage an Oracle connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes Oracle connection properties:

Property	Description
Database Type	The database type.
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: `~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /`
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	The description of the connection. The description cannot exceed 765 characters.
User Name	The database user name.
Password	The password for the database user name.
Pass-through security enabled	Enables pass-through security for the connection. When you enable pass-through security for a connection, the domain uses the client user name and password to log into the corresponding database, instead of the credentials defined in the connection object.

Metadata Access Properties: Connection String	Connection string used to access metadata from the database.
	Use the following connection string:
	`jdbc:informatica:oracle://<host_name>:<port>;SID=<database name>`
	Use the following connection string to connect to Oracle database through Oracle Connection Manager:
	`jdbc:informatica:oracle:TNSNamesFile=<fully qualified path to the tnsnames.ora file>;TNSServerName=<TNS server name>;`

Property	Description

AdvancedJDBCSecurityOptions	Database parameters for metadata access to a secure database. Informatica treats the value of the AdvancedJDBCSecurityOptions field as sensitive data and stores the parameter string encrypted.
	To connect to a secure database, include the following parameters:
	- EncryptionMethod. Required. Indicates whether data is encrypted when transmitted over the network. This parameter must be set to SSL.
	- ValidateServerCertificate. Optional. Indicates whether Informatica validates the certificate that is sent by the database server.
	If this parameter is set to True, Informatica validates the certificate that is sent by the database server. If you specify the HostNameInCertificate parameter, Informatica also validates the host name in the certificate.
	If this parameter is set to false, Informatica does not validate the certificate that is sent by the database server. Informatica ignores any truststore information that you specify.
	- HostNameInCertificate. Optional. Host name of the machine that hosts the secure database. If you specify a host name, Informatica validates the host name included in the connection string against the host name in the SSL certificate.
	- cryptoProtocolVersion. Optional. If you enable TLS for the Oracle instance, set the cryptoProtocolVersion parameter as follows:
	cryptoProtocolVersion=TLSv<version number>.
	For example, cryptoProtocolVersion=TLSv1.2
	Note: The version number must be the same as the TLS version you configured for the server.
	- TrustStore. Required. Path and file name of the truststore file.
	Note: If you configure SSL or TLS and specify only the file name, you must copy the truststore file to the Informatica installation directory. To test the connection and import metadata, copy the truststore file to the following directory:
	`<Informatica client installation directory>/clients/DeveloperClient`
	To run the mapping, copy the truststore file to the following directory:
	`<Informatica server installation directory>/tomcat/bin`
	Note: If you configure SSL or TLS and specify only the file name, you must copy the truststore file to the following directory to test the connection: `<Informatica server installation directory>/tomcat/bin`
	- TrustStorePassword. Required. Password for the truststore file for the secure database.
	- KeyStore. Required. Path and file name of the keystore file.
	- KeyStorePassword. Required. Password for the keystore file for the secure database.
	Note: Informatica appends the secure JDBC parameters to the connection string. If you include the secure JDBC parameters directly to the connection string, do not enter any parameters in the AdvancedJDBCSecurityOptions field.

Data Access Properties: Connection String	Use the following connection string: `<database name>.world`
Code Page	The code page used to read from a source database or to write to a target database or file.

Property	Description
Environment SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the connection environment SQL each time it connects to the database.
Transaction SQL	SQL commands to set the database environment when you connect to the database. The Data Integration Service runs the transaction environment SQL at the beginning of each transaction.
Retry Period	This property is reserved for future use.
Enable Parallel Mode	Enables parallel processing when loading data into a table in bulk mode. By default, this option is cleared.
SQL Identifier Character	Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type. Select (None) if the database uses regular identifiers. When the Data Integration Service generates SQL queries, the service does not place delimited characters around any identifiers. Select a character if the database uses delimited identifiers. When the Data Integration Service generates SQL queries, the service encloses delimited identifiers within this character.
Support Mixed-case Identifiers	Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property. When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.

Salesforce Connection Properties

Use a Salesforce connection to connect to a Salesforce object. The Salesforce connection is an application connection type. You can create and manage a Salesforce connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes Salesforce connection properties:

Property	Description
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. It cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.

Property	Description
Description	The description of the connection. The description cannot exceed 765 characters.
Location	The Informatica domain where you want to create the connection.
Type	The connection type. You can select the Standard connection type or OAuth connection type.
User Name	Applicable for Standard connection type. Salesforce user name.
User Password	Applicable for Standard connection type. Password for the Salesforce user name. To access Salesforce outside the trusted network of your organization, you must append a security token to your password to log in to the API or a desktop client. To receive or reset your security token, log in to Salesforce and click **Setup** > **My Personal Information** > **Reset My Security Token**. Password is case sensitive.
Service URL	URL of the Salesforce service you want to access. For example, `https://login.salesforce.com/services/Soap/u/<version>` In a test or development environment, you might want to access the Salesforce Sandbox testing environment. For more information about the Salesforce Sandbox, see the Salesforce documentation.
Refresh Token	Applicable for OAuth connection type. Refresh Token of Salesforce.
Consumer Key	Applicable for OAuth connection type. The Consumer Key obtained from Salesforce, required to generate the Refresh Token. For more information about how to generate the Consumer Key, see the Salesforce documentation.
Consumer Secret	Applicable for OAuth connection type. The Consumer Secret obtained from Salesforce, required to generate the Refresh Token. For more information about how to generate the Consumer Secret, see the Salesforce documentation.

Salesforce Marketing Cloud Connection Properties

Use a Salesforce Marketing Cloud connection to connect to a Salesforce Marketing Cloud object. You can create and manage a Salesforce Marketing Cloud connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes the Salesforce Marketing Cloud connection properties:

Connection property	Description
Name	Name of the Salesforce Marketing Cloud connection.
ID	The Data Integration Service uses the ID to identify the connection.

Connection property	Description
Description	Optional. The description of the connection.
Location	Informatica domain where you want to create the connection.
Type	Connection type. Select Salesforce Marketing Cloud.

Salesforce Marketing Cloud Url	The URL that the Data Integration Service uses to connect to the Salesforce Marketing Cloud WSDL.
	The following URL is an example for OAuth 1.0 URL:
	`https://webservice.s7.exacttarget.com/etframework.wsdl`
	The following URL is an example for OAuth 2.0 URL:
	`https://<SUBDOMAIN>.soap.marketingcloudapis.com/etframework.wsdl`

Username	User name of the Salesforce Marketing Cloud account.
Password	Password for the Salesforce Marketing Cloud account.
ClientId	The client ID of Salesforce Marketing Cloud required to generate a valid access token.
ClientSecret	The client secret of Salesforce Marketing Cloud required to generate a valid access token.
Enable Logging	When you enable logging you can see the session log for the tasks.
UTC Offset	The Secure Agent uses the UTC offset connection property to read data from and write data to Salesforce Marketing Cloud in UTC offset time zone.
Batch Size	Number of rows that the Secure Agent writes in a batch to the target. When you insert or update data and specify the contact key, the data associated with the specified contact ID is inserted or updated in a batch to Salesforce Marketing Cloud. When you upsert data to Salesforce Marketing Cloud, do not specify the contact key.
Enable Multiple BU	Select this option if there are multiple business units in your Salesforce Marketing Cloud account. You can use the Salesforce Marketing Cloud connection to access data across all business units.

SAP Connection Properties

Use an SAP connection to access an SAP table or an SAP BW object. The SAP connection is an enterprise application connection. You can create and manage an SAP connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes the SAP connection properties:

Property	Description
Username	Required. User name for the SAP source system that you want to access.
Password	Required. Password for the user name.
Connection type	Required. Type of connection that you want to create. Select one of the following values: - Application. Create an application connection when you want to connect to a specific SAP application server. - Load balancing. Create a load balancing connection when you want to use SAP load balancing. Default is Application. Based on the connection type you select, the corresponding connection property fields become available in the **Connection Details** dialog box. The Developer tool greys out the connection property fields that are not applicable for a particular connection type.
Host name	Required when you create an SAP application connection. Host name or IP address of the SAP server that you want to connect to.
System number	Required when you create an SAP application connection. SAP system number.
Message host name	Required when you create an SAP load balancing connection. Host name of the SAP message server.
R3 name/SysID	Required when you create an SAP load balancing connection. Name of the SAP system.
Group	Required when you create an SAP load balancing connection. Group name of the SAP application server.
Client	Required. SAP client number.
Language	Optional. Language that you want to use for mappings and workflows. Must be compatible with the Developer tool code page. If you leave this option blank, the Developer tool uses the default language of the SAP system.
Trace	Optional. Use this option to track the JCo calls that the SAP system makes. SAP stores the information about the JCo calls in a trace file. Specify one of the following values: - 0. Off - 1. Full Default is 0. You can access the trace files from the following directories: - `<Informatica installation directory>/tomcat/bin` directory on the machine where you installed the Informatica services - `<Informatica installation directory>/clients/DeveloperClient` directory on the machine where you installed the Developer tool
Additional parameters	Optional. Enter any other connection parameter that you want to use. Use the following format: `<parameter name>=<value>`

Property	Description
Staging directory	Path in the SAP system where the stage file will be created.
Source directory	Path that contains the source file. The path must be accessible to the Data Integration Service.
Use FTP	Enables FTP access to SAP.
FTP user	Required when you use FTP. User name to connect to the FTP server.
FTP password	Required when you use FTP. Password for the FTP user.

FTP host	Required when you use FTP.
	Host name or IP address of the FTP server.
	Optionally, you can specify a port number from 1 through 65535, inclusive. Default for FTP is 21.
	Use one of the following syntax to specify the host name:
	`- hostname:port_number`
	`- IP address:port_number`
	When you specify a port number, enable that port number for FTP on the host machine.
	If you enable SFTP, specify a host name or port number for an SFTP server. Default for SFTP is 22.

Retry period	Number of seconds that the Data Integration Service attempts to reconnect to the FTP host if the connection fails. If the Data Integration Service cannot reconnect to the FTP host in the retry period, the mapping or workflow fails. Default is 0. A value of 0 indicates an infinite retry period.
Use SFTP	Enables SFTP access to SAP.
Public key file name	Required when you enable SFTP and the SFTP server uses public key authentication. Public key file path and file name.
Private key file name	Required when you enable SFTP and the SFTP server uses public key authentication. Private key file path and file name.
Private key file name password	Required when you enable SFTP, and the SFTP server uses public key authentication and the private key is encrypted. Password to decrypt the private key file.
Port Range	HTTP port range that the Data Integration Service must use to read data from the SAP server in streaming mode. Enter the minimum and maximum port numbers with a hyphen as the separator. The minimum and maximum port number can range between 10000 and 65535. You can also specify the port range according to your organization. Default is 10000-65535.

Property	Description
Use HTTPS	Select this option to enable HTTPS streaming when you read data from SAP tables. By default, the **Use HTTPS** check box is not selected. For more information about configuring HTTPS for table reader mappings in streaming mode, see the article "HTTPS Configuration for Table Reader Mappings in Streaming Mode for PowerExchange for SAP NetWeaver" on the Informatica Documentation Portal.
Key store file path	Required when you use HTTPS. Path to the keystore file that contains the private or public key pairs and the associated certificates.
Key store password	Required when you use HTTPS. Password for the keystore file.
Private key password	Required when you use HTTPS. Password to decrypt the private key file.

Snowflake Connection Properties

When you set up a Snowflake connection, you must configure the connection properties.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes the Snowflake connection properties:

Property	Description
Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters:~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. The ID must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Optional. The description of the connection. The description cannot exceed 4,000 characters.
Location	The domain where you want to create the connection.
Type	The connection type. Select SnowFlake.
Snowflake Authentication Type	Specify whether to connect to Snowflake using the standard or the key pair authentication. Standard authentication uses the Snowflake account user name and password. Key pair authentication uses the public key and the corresponding private key file (encrypted or unencrypted). If the private key file is encrypted, provide the private key password file. Provide these credentials along with your Snowflake account username. You can enable key pair authentication for both Native and Spark engines.

Property	Description
Username	The user name to connect to the Snowflake account.
Password	Required for standard authentication. Enter the password to connect to the Snowflake account. Or, provide a Programmatic Access Token (PAT) for Multi-Factor Authentication (MFA). For further information on how to generate the PAT, refer the Snowflake documentation for PAT.
Account	The name of the Snowflake account.
Private Key	Required for the key pair authentication. Fully qualified path to the private key file that the Data Integration Service uses to access Snowflake.
Private Key Password	Required for the key pair authentication. Password for the private key file.
Warehouse	The Snowflake warehouse name.
Role	The Snowflake role assigned to the user.

Additional JDBC URL Parameters	Enter one or more JDBC connection parameters in the following format:
	`<param1>=<value>&<param2>=<value>&<param3>=<value>....`
	For example:
	`user=jon&warehouse=mywh&db=mydb&schema=public`
	To access Snowflake through Okta SSO authentication, enter the web-based IdP implementing SAML 2.0 protocol in the following format:
	`authenticator=https://<Your_Okta_Account_Name>.okta.com`
	Note: Microsoft ADFS is not supported.
	For more information about configuring Okta authentication, see the following website: https://docs.snowflake.net/manuals/user-guide/admin-security-fed-auth-configure-snowflake.html#configuring-snowflake-to-use-federated-authentication

Teradata Parallel Transporter Connection Properties

Use a Teradata PT connection to access Teradata tables. The Teradata PT connection is a database type connection. You can create and manage a Teradata PT connection in the Administrator tool or the Developer tool.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes Teradata PT connection properties:

Property	Description
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Description of the connection. The description cannot exceed 765 characters.
Location	Domain where you want to create the connection.
Type	Connection type. Select Teradata PT.
User Name	Teradata database user name with the appropriate read and write permissions to access the database.
Password	Password for the Teradata database user name.
Driver Name	Name of the Teradata JDBC driver.
Connection String	Connection string used to access metadata from the database. Use the following connection string: jdbc:teradata://<hostname>/database=<database name>,tmode=ANSI,charset=UTF8

The following table describes the properties for data access:

Property	Description
TDPID	Name or IP address of the Teradata database machine.
Database Name	Teradata database name. If you do not enter a database name, Teradata PT API uses the default login database name.
Data Code Page	Code page associated with the database. When you run a mapping that writes data to a Teradata target, the code page of the Teradata PT connection must be the same as the code page of the Teradata target. Default is UTF-8.
Tenacity	Number of hours that Teradata PT API continues trying to log on when the maximum number of operations run on the Teradata database. Must be a positive, non-zero integer. Default is 4.
Max Sessions	Maximum number of sessions that Teradata PT API establishes with the Teradata database. Must be a positive, non-zero integer. Default is 4.
Min Sessions	Minimum number of Teradata PT API sessions required for the Teradata PT API job to continue. Must be a positive integer between 1 and the **Max Sessions** value. Default is 1.

Property	Description
Sleep	Number of minutes that Teradata PT API pauses before it retries to log on when the maximum number of operations run on the Teradata database. Must be a positive, non-zero integer. Default is 6.
Use Metadata JDBC URL for TDCH	Indicates that the Teradata Connector for Hadoop (TDCH) must use the JDBC URL that you specified in the connection string under the metadata access properties. Default is selected. Clear this option to enter a different JDBC URL that TDCH must use when it runs the mapping.
TDCH JDBC Url	Enter the JDBC URL that TDCH must use when it runs a Teradata mapping. Use the following format: `jdbc:teradata://<hostname>/database=<database name>,tmode=ANSI,charset=UTF8` This field is available only when you clear the **Use Metadata JDBC URL for TDCH** option.
Data Encryption	Enables full security encryption of SQL requests, responses, and data on Windows. Default is disabled.
Additional Sqoop Arguments	This property is applicable if you use a Hortonworks or Cloudera cluster, and run a Teradata mapping on the Blaze or Spark engine through Sqoop. Enter the arguments that Sqoop must use to process the data. For example, enter `--method split.by.amp`. Separate multiple arguments with a space. See the Hortonworks for Teradata Connector and Cloudera Connector Powered by Teradata documentation for a list of arguments that you can specify. **Note:** If you use Hortonworks for Teradata Connector, the --split-by argument is required if you add two or more source tables in the read operation. If you use Cloudera Connector Powered by Teradata, the --split-by argument is required in the source connection if the source table does not have a primary key defined.
Authentication Type	Method to authenticate the user. Select one of the following authentication types: - Native. Authenticates your user name and password against the Teradata database specified in the connection. - LDAP. Authenticates user credentials against the external LDAP directory service. Default is Native.

Tableau Connection Properties

Use a Tableau connection to connect to Tableau. When you create a Tableau connection, you enter information to access Tableau.

The following table describes the Tableau connection properties:

Property	Description
Name	Name of the Tableau connection.
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.
Description	Description of the connection. The description cannot exceed 765 characters.
Location	The Informatica domain where you want to create the connection.
Type	Type of connection. Select Tableau.

The following table describes the properties to connect to Tableau:

Connection Property	Description
Tableau Product	The name of the Tableau product to which you want to connect. You can choose one of the following Tableau products to publish the TDE or TWBX file: - Tableau Desktop. Creates a TDE file in the Data Integration Service machine. You can then manually import the TDE file to Tableau Desktop. - Tableau Server. Publishes the generated TDE or TWBX file to Tableau Server. - Tableau Online. Publishes the generated TDE or TWBX file to Tableau Online.
Connection URL	URL of Tableau Server or Tableau Online to which you want to publish the TDE or TWBX file. The URL has the following format: `http://<Host name of Tableau Server or Tableau Online>:<port>`
User Name	User name of the Tableau Server or Tableau Online account.
Password	Password for the Tableau Server or Tableau Online account.
Content URL	The name of the site on Tableau Server or Tableau Online where you want to publish the TDE or TWBX file. Contact the Tableau administrator to provide the site name.

Identifier Properties in Database Connections

When you create most relational database connections, you must configure database identifier properties.

A database identifier is a database object name. Tables, views, columns, indexes, triggers, procedures, constraints, and rules can have identifiers. You use the identifier to reference the object in SQL queries. A

database can have regular identifiers or delimited identifiers that must be enclosed within delimited characters.

Regular Identifiers

Regular identifiers comply with the format rules for identifiers. Regular identifiers do not require delimited characters when they are used in SQL queries.

For example, the following SQL statement uses the regular identifiers *MYTABLE* and *MYCOLUMN*:

```
SELECT * FROM MYTABLE
WHERE MYCOLUMN = 10
```

Delimited Identifiers

Delimited identifiers must be enclosed within delimited characters because they do not comply with the format rules for identifiers.

Databases can use the following types of delimited identifiers:

Identifiers that use reserved keywords

If an identifier uses a reserved keyword, you must enclose the identifier within delimited characters in an SQL query. For example, the following SQL statement accesses a table named *ORDER*:

```
SELECT * FROM "ORDER"
WHERE MYCOLUMN = 10
```

Identifiers that use special characters

If an identifier uses special characters, you must enclose the identifier within delimited characters in an SQL query. For example, the following SQL statement accesses a table named *MYTABLE$@*:

```
SELECT * FROM "MYTABLE$@"
WHERE MYCOLUMN = 10
```

Case-sensitive identifiers

By default, identifiers in IBM DB2, Microsoft SQL Server, and Oracle databases are not case sensitive. Database object names are stored in uppercase, but SQL queries can use any case to refer to them. For example, the following SQL statements access the table named *MYTABLE*:

```
SELECT * FROM mytable
SELECT * FROM MyTable
SELECT * FROM MYTABLE
```

To use case-sensitive identifiers, you must enclose the identifier within delimited characters in an SQL query. For example, the following SQL statement accesses a table named *MyTable*:

```
SELECT * FROM "MyTable"
WHERE MYCOLUMN = 10
```

Identifier Properties

When you create most database connections, you must configure database identifier properties. The identifier properties that you configure depend on whether the database uses regular identifiers, uses keywords or special characters in identifiers, or uses case-sensitive identifiers.

Configure the following identifier properties in a database connection:

SQL Identifier Character

Type of character that the database uses to enclose delimited identifiers in SQL queries. The available characters depend on the database type.

Select (None) if the database uses regular identifiers. When the Data Integration Service generates SQL queries, the service does not place delimited characters around any identifiers.

Select a character if the database uses delimited identifiers. When the Data Integration Service generates SQL queries, the service encloses delimited identifiers within this character.

Support Mixed-case Identifiers

Enable if the database uses case-sensitive identifiers. When enabled, the Data Integration Service encloses all identifiers within the character selected for the **SQL Identifier Character** property.

In the Informatica client tools, you must refer to the identifiers with the correct case. For example, when you create the database connection, you must enter the database user name with the correct case.

When the **SQL Identifier Character** property is set to none, the **Support Mixed-case Identifiers** property is disabled.

Example: Database Uses Regular Identifiers

In this example, the database uses regular identifiers. No identifiers contain a reserved keyword or a special character. The database uses identifiers that are not case sensitive.

In the database connection, set the **SQL Identifier Character** property to (None). When **SQL Identifier Character** is set to none, the **Support Mixed-case Identifiers** property is disabled.

When the Data Integration Service generates SQL queries, the service does not place delimited characters around any identifiers.

Example: Database Uses Keywords or Special Characters in Identifiers

In this example, the database uses keywords or special characters in some identifiers. The database uses identifiers that are not case sensitive.

In the database connection, configure the identifier properties as follows:

1. Set the **SQL Identifier Character** property to the character that the database uses for delimited identifiers.

   This example sets the property to *"" (quotes)*.

2. Clear the **Support Mixed-case Identifiers** property.

When the Data Integration Service generates SQL queries, the service places the selected character around identifiers that use a reserved keyword or that use a special character. For example, the Data Integration Service generates the following query:

```
SELECT * FROM "MYTABLE$@"   /* identifier with special characters enclosed within
delimited
          character */
WHERE MYCOLUMN = 10   /* regular identifier not enclosed within delimited character */
```

Example: Database Uses Case-Sensitive Identifiers

In this example, the database uses case-sensitive identifiers. The database might use keywords or special characters in some identifiers, or it might not.

In the database connection, configure the identifier properties as follows:

1. Set the **SQL Identifier Character** property to the character that the database uses for delimited identifiers.

   This example sets the property to *"" (quotes)*.

2. Select the **Support Mixed-case Identifiers** property.

When the Data Integration Service generates SQL queries, the service places the selected character around all identifiers. For example, the Data Integration Service generates the following query:

```
SELECT * FROM "MyTable"    /* case-sensitive identifier enclosed within delimited
character */
WHERE "MYCOLUMN" = 10    /* regular identifier enclosed within delimited character */
```

PowerExchange for PostgreSQL Connection Properties

When you configure a PowerExchange for PostgreSQL connection, you define the connection attributes that the CDI-PC Integration Service uses to connect to the PostgreSQL database.

The following table describes the PostgreSQL connection properties:

Connection Property	Description
Host Name	Host name of the PostgreSQL server to which you want to connect.
Port	Port number for the PostgreSQL server to which you want to connect. Default is 5432.
Database	The PostgresSQL database name.
User Name	User name to access the PostgreSQL database.
Password	Password for the PostgreSQL database user name.
Encryption Method	Determines whether the data exchanged between the CDI-PC Integration Service and the PostgreSQL database server is encrypted: Select one of the following encryption methods: - noEncryption. Establishes a connection without using SSL. Data is not encrypted. - SSL. Establishes a connection using SSL. Data is encrypted using SSL. If the PostgreSQL database server does not support SSL, the connection fails. - requestSSL. Attempts to establish a connection using SSL. If the PostgreSQL database server does not support SSL, the CDI-PC Integration Service establishes an unencrypted connection. Default is noEncryption.
Validate Server Certificate	Applicable if you enable the encryption method to SSL or requestSSL. Select the **Validate Server Certificate** option so that the CDI-PC Integration Service validates the server certificate that is sent by the PostgreSQL database server. If you specify the **Hostname In Certificate** parameter, the CDI-PC Integration Service also validates the host name in the certificate.
TrustStore	Applicable if you select SSL or requestSSL as the encryption method and the Validate Server Certificate option. The path and name of the truststore file, which contains the list of the Certificate Authorities (CAs) that the PostgreSQL client trusts.

Connection Property	Description
TrustStore Password	Applicable if you select SSL or requestSSL as the encryption method and the Validate Server Certificate option. The password to access the truststore file that contains the SSL certificate.
Host Name In Certificate	Optional when you select SSL or requestSSL as the encryption method and the Validate Server Certificate option. Specifying a host name ensures additional security and the CDI-PC Integration Service validates the host name included in the connection with the host name in the SSL certificate.
KeyStore	Applicable if you select SSL as the encryption method and when client authentication is enabled on the PostgreSQL database server. The path and the file name of the key store. The keystore file contains the certificates that the PostgreSQL client sends to the PostgreSQL server in response to the server's certificate request.
KeyStore Password	Applicable if you select SSL as the encryption method and when client authentication is enabled on the PostgreSQL database server. The password for the keystore file required for secure communication.
Key Password	Applicable if you select SSL as the encryption method and when client authentication is enabled on the PostgreSQL database server. Required when individual keys in the keystore file have a different password than the keystore file.
Additional Connection Properties	Additional connection parameters that you want to use. You must provide the connection parameters as semicolon-separated key-value pairs. For example, `ConnectionRetryCount=2; ConnectionRetryDelay=5`
Crypto Protocol Versions	Required if you enable the encryption method to SSL or requestSSL. Specifies a cryptographic protocol or a list of cryptographic protocols when you use an encrypted connection. You can select from the following protocols: - SSLv3 - TLSv1 - TLSv1_1 - TLSv1_2

Microsoft Dynamics 365 for Sales Connection Properties

When you configure a Microsoft Dynamics 365 for Sales connection, you define the connection attributes that the CDI-PC Integration Service uses to connect to the Microsoft Dynamics 365 for Sales database.

to be conreffed

The following table describes the Microsoft Dynamics 365 for Sales connection properties:

Property	Description
Runtime Environment	The name of the runtime environment where you want to run the tasks.
Authentication Type	The authentication method that the connector must use to login to the web application. Select one of the following authentication types: **OAuth 2.0 Password Grant.** It requires you to use the ADFS configuration. Specify the web API URL, username, password, and application ID. You additionally need the security token service URL to access Microsoft Dynamics 365 for Sales on-premises. Applies to Microsoft Dynamics 365 for Sales on-premise. **OAuth 2.0 Client Certificate Grant.** Requires you to specify the web API URL, application ID, tenant ID, keystore file, keystore password, key alias, and key password. Applies to Microsoft Dynamics 365 for Sales online. **OAuth 2.0 Client Secret.**Requires you to specify the application ID and client secret. Applies to Microsoft Dynamics 365 for Sales online. For more information on how to configure OAuth with IFD(ADFS) configuration, see the knowledge base article: Configuration of on-premise endpoint
Web API URL	The URL of the Microsoft Dynamics 365 for Sales endpoint.
Username	The user name to connect to the Microsoft Dynamics 365 for Sales account.
Password	The password to connect to the Microsoft Dynamics 365 for Sales account.
Application ID	The Azure application ID for Microsoft Dynamics 365 for Sales.
Tenant ID	The directory ID for Azure Active Directory.
Keystore File	The location and the file name of the key store. Not applicable when you use the Hosted Agent.
Keystore Password	The password for the keystore file required for secure communication.
Key Alias	The alias name for the individual key.
Key Password	The password for the individual keys in the keystore file required for secure communication. Not applicable when you use the Hosted Agent.
Retry Error Codes	The comma-separated http error codes for which the retries are made.
Retry Count	The number of retries to get the response from an endpoint based on the retry interval. Default is 5.
Retry Interval	The time in seconds to wait before Microsoft Dynamics 365 for Sales Connector retries for a response. Default is 60.
Client Secret	The client secret key to connect to Microsoft Dynamics 365 for Sales account.

Property	Description
Server Type	The Microsoft Dynamics 365 for Sales server that you want to access. You can select the server type from the following list: - **Microsoft Dynamics Online.** Select this option to connect to Microsoft Dynamics 365 for Sales deployed online. Requires you to enter the Web API URL, client secret, and certificate password. - **Microsoft Dynamics On-premise.** Select this option to connect to Microsoft Dynamics 365 for Sales deployed on-premises. Requires you to enter the ADFS client ID as Application ID and certificate password.
Security Token Service URL	The service URL with the security token. For example, `http://<customer_STS_URL>/adfs/oauth2/token`.

PowerExchange for Oracle E-Business Suite Connection Properties

When you configure an Oracle E-Business Suite connection, you define the connection attributes that the CDI-PC Integration Service uses to connect to the Oracle E-Business Suite.

The following table describes the connection properties:

Property	Description
User Name	User name to connect to Oracle E-Business Suite. If you are configuring a connection for an E-Business Suite target, the user name must be apps for the CDI-PC Integration Service to be able to execute the concurrent program.
Password	Password for the user name. You cannot use a parameter to specify the password.
Connect String	ODBC data source name. Use Informatica-certified ODBC drivers for ODBC data source connections.
Apps Schema Name	Name of the application schema that contains metadata for Oracle E-Business Suite. Default is apps.

Siebel Application Connections for Sources, Targets, and EIM Invoker Transformations

The Siebel Sources, Targets, and EIM Invoker transformations use the Siebel connection application connection to connect to the Siebel repository. When you configure an application connection, you must specify the connection attributes for the Siebel repository.

The following table describes the application connection properties:

Connection Parameter	Description
Protocol	Protocol used to connect to Siebel. Specify the following protocol parameters: - Transport. Enter HTTP or TCP/IP. Default is TCP/IP. - Encryption. Enter NONE or RSA. Default is NONE. - Compression. Enter NONE or ZLIB. Default is ZLIB. Specify the parameters in the following format: `siebel[[.transport][.[encryption][.[compression]]]]`
User Name	User name to connect to Siebel.
Password	Password for the user name.
Siebel Server Host	Host name or IP address of the Siebel server. If you configure native load balancing, specify the virtual host name.
SCBroker Port	Siebel Connection Broker port number.
Enterprise Server	Enterprise Server name.
Application Object Manager	Siebel application business object manager.
Encoding	Encoding defined in the code page the CDI-PC Integration Service uses to communicate with the Siebel Server. Default is UTF-8.

Microsoft Dynamics CRM Connection

A Microsoft Dynamics CRM connection extracts data from and loads data to the Microsoft Dynamics CRM. PowerExchange for Microsoft Dynamics CRM uses SOAP to connect to Microsoft Dynamics CRM.

The following table describes the Microsoft Dynamics CRM connection properties:

Property	Description
Service Type	Type of Web service for passport authentication. Select Organizational service. **Note:** You can select Discovery service for active directory and claim-based authentication.
Application Server	Name of the server that hosts Microsoft Dynamics CRM. For active directory and claims-based authentication, provide the fully qualified domain name. The fully-qualified domain name must be the same as in the SSL certificate file. For passport authentication, specify the CRM Organization web service URL.
Port	Port in which Microsoft Dynamics CRM is configured. Configure for active directory, claims-based, and passport authentication.

Property	Description
Use SSL/TLS	Secure method to connect to the application server by using SSL or TLS security protocols. Configure for active directory and claims-based authentication.
Organization Name	Name of the organization that you want to connect to. Use one of the following options: - Name of the organization that appears when you log in to Microsoft Dynamics CRM. - Name that appears in the Deployment Manager. - The unique organization name. To find the unique organization name, log in to Microsoft Dynamics account and click **Settings** > **Customizations** > **Developer Resources**. Configure for active directory and claims-based authentication only.
Domain	Domain to which the user belongs. You must provide the complete domain name. Configure for active directory and claims-based authentication.
Authentication Type	Authentication type for the connection. Select one of the following authentication types based on the Microsoft Dynamics CRM deployment: - Passport. To connect to an online deployment of Microsoft Dynamics CRM using OAuth authentication. - Claims-based. To connect to an on-premise and Internet-facing deployment of Microsoft Dynamics CRM. - Active directory. To connect to on-premise deployment of Microsoft Dynamics CRM.
Security Token Service	Microsoft Dynamics CRM security token service URL. For example, https://sts1.company.com. Configure for claims-based authentication.

PowerExchange for Essbase Connections

When you configure a Essbase connection, you define the connection attributes that the CDI-PC Integration Service uses to connect to Essbase.

The following table describes the Essbase connection properties that you must configure:

Connection Attribute	Description
User Name	User name to connect to Essbase.
Password	Password to connect to Essbase.
ServerHost	Essbase server name.
Application	Name of the application. Default is none. **Note:** For a Unicode application, specify the name of the Unicode application.
Database	Name of the database. Default is none.

Vertica Relational Connection Properties

The following table describes the properties that you must configure for a Vertica relational connection:

Property	Description
Name	Enter a name for the connection.
Type	The connection type is set by default. You cannot edit this value.
User Name	Enter the user name to connect to the Vertica database. The user must have read permissions on the schema where the Vertica source table resides. Similarly, the user must have write permissions on the schema where the Vertica target table resides.
Password	Enter the password to connect to the Vertica database.
Use Parameter in Password	Indicates that the password for the repository user name is a session parameter, $Param*Name*. Define the password in the workflow or session parameter file, and encrypt it by using the *pmpasswd* CRYPT_DATA option. Default is disabled.
Connect String	Enter the name of the ODBC data source that you created for the Vertica database.
Code Page	Select the code page that the CDI-PC Integration Service Service must use to read or write data.
Connection Environment SQL	Runs an SQL command with each database connection. Default is disabled.
Transaction Environment SQL	Runs an SQL command before the initiation of each transaction. Default is disabled.
Connection Retry Period	The number of seconds that the CDI-PC Integration Service attempts to reconnect to the database if the connection fails. If the CDI-PC Integration Service cannot connect to the database in the retry period, the session fails. Default value is 0.
Load Balance	Select this option to enable load balancing on the Vertica database. When you enable load balancing, the CDI-PC Integration Service requests a load balanced connection with the Vertica database. Default is enabled. For more information on load balancing, see the Vertica database documentation.

PowerExchange for Db2 Warehouse Connections

The following table describes the Db2 Warehouse connection properties that you must configure:

Property	Description
User Name	Database user name with the appropriate read and write database permissions to access Db2 Warehouse.
Use Parameter in Password	Indicates the password for the database user name is a session parameter, $Param*Name*. Define the password in the workflow or session parameter file, and encrypt it by using the *pmpasswd* CRYPT_DATA option. Default is disabled.
Password	Password for the database user name.
Connect String	ODBC data source to connect to Db2 Warehouse.
Database Name	Database name of Db2 Warehouse that you want to connect to.
Schema Name	The schema name in Db2 Warehouse from where you want to fetch the metadata.
Server Name	Host name of Db2 Warehouse.
Port Number	Network port number used to connect to the Db2 Warehouse server.
Driver Name	Specify the name of the IBM Data Server driver that you configured in the `odbcinst.ini` file. For example, IBM DB2 ODBC DRIVER - IBMDBCL1.
Advanced connection properties	Optional. Additional connection parameters that you want to use. Specify the connection parameters as key-value pairs in the following format, and separate each key-value pair with a semicolon: `<param1>=<value>;<param2>=<value>;<param3>=<value>....`

PowerExchange for HANA Connections

You must configure a HANA ODBC data source before you can import HANA sources.

The following table describes the HANA ODBC connection properties:

Property	Decription
Name	Name you want to use for this connection. The connection name cannot contain spaces or other special characters, except for the underscore.
Type	Read-only. Type of database. For SAP HANA, this property is set to ODBC.

Property	Decription
User Name	Database user name with the appropriate read and write database permissions to access the database. To define the user name in the parameter file, enter session parameter $ParamName as the user name, and define the value in the session or workflow parameter file. The Integration Service interprets user names that start with $Param as session parameters.
Use Parameter in Password	Indicates that the password for the database user name is a session parameter, $ParamName. Define the password in the workflow or session parameter file, and encrypt it by using the pmpasswd CRYPT_DATA option. Default is disabled.
Password	Password for the database user name. Must be in 7-bit ASCII.
Connect String	Connect string used to communicate with the SAP HANA database.
Code Page	Code page the Integration Service uses to read from a source database or write to a target database.
Connection Environment SQL	Runs an SQL command with each database connection. Default is disabled.
Transaction Environment SQL	Runs an SQL command before the initiation of each transaction. Default is disabled.
Connection Retry Period	Number of seconds the Integration Service attempts to reconnect to the database if the connection fails. If the Integration Service cannot connect to the database in the retry period, the session fails. Default value is 0.
ODBC Subtype	Type of database to which ODBC connects. Select **SAP HANA**.

INDEX

A

Amazon Redshift connection
  properties 236
Amazon S3 connection
  properties 239
application archive file 139
application connection
  configuring for Siebel sources, targets, and EIM Invoker transformations 289
application patch
  add 166
  affected 166
  affected object 166
    affected objects
      find 168
  create 171
  delete 167
    dependency
      direct 152
      indirect 152
      remote 152
  deploy 169, 171
  deployment strategy 169
  design-time 152
  direct and indirect dependencies 162
  direct dependencies 161
  direct, indirect, and remote dependencies 163
  first-degree related objects 159
  guidelines 173
  impact 165, 169
  inherit 152, 159
  modify 166
  numbers 169
  object impact 165, 166, 169
  related objects 159
  rules 173
  run-time 152
  scope 169
  selected objects 159
  type 159
applications
  connecting to a run-time application 148
  creating 144
  creating an application 134
  creation 134
  deploying to a Data Integration Service 137
  deploying to a file 139
  exporting to a file 139
  full 134
  how to create, deploy, and update 143
  incremental 135, 151
  mapping deployment properties 135
  overview 133
  properties 135
  redeploying 149
applications *(continued)*
  replacing 141
  updating 141
  validating an application 134
  viewing run-time objects 141
attributes
  relationships 114

B

bigint
  constants in expressions 204
  high precision handling 203
  using in calculations 203
  writing to flat files 205
binary datatypes
  overview 205

C

certificates
  adding untrusted certificates 79
  certificate properties 79
  managing certificates 79
  untrusted certificates 79
check in/check out objects 34
checking objects out and in 34
CI/CD
  assign objects 187, 191
  conflict detection 188
  conflict resolution 188
  continuous delivery 188
  continuous delivery example 191
  continuous integration 187
  continuous integration example 189
  deploy application 186
  deploy objects 187, 189–191
  developer collaboration 188
  infacmd 184, 186
  object deployment 187
  overview 183
  redeploy application 186
  REST API 184, 186
  team-based development 187
  test objects 190
configurations
  troubleshooting 128
Connection
  details 261
  properties 261
connection environment SQL
  configuring 47, 48
connection properties 290

connection switching
configuring 45
data types mapping 46
description 44
lookup behavior 46
postrequisites 46
prerequisites 44
connections
Connection Explorer view 41
database identifier properties 283
Salesforce Marketing Cloud 275
copy
objects 24
Cosmos DB connection
creating 263
creating
Cosmos DB connection 263
custom queries
Informatica join syntax 69
left outer join syntax 71
normal join syntax 70
outer join support 69
right outer join syntax 73
custom SQL queries
creating 74
customized data objects 62
customized data objects
adding pre- and post-mapping SQL commands 74
adding relational data objects 58
adding relational resources 57
advanced query 63
creating 57
creating a custom query 74
creating key relationships 59
creating keys 58
custom SQL queries 62
default query 63
description 54
entering source filters 66
entering user-defined joins 68
key relationships 55
pre- and post-mapping SQL commands 73
reserved words file 63
select distinct 66
simple query 63
sorted ports 67
user-defined joins 68
using select distinct 66
using sorted ports 67
write properties 56

D

data type reference
flat files 210
data types
decimal 207
double 207
IBM DB2 211
Microsoft SQL Server 215
ODBC 220
Oracle 221
overview 200
transformation 201
data viewer
configuration properties 121
configurations 121, 124
data viewer *(continued)*
creating configurations 125
troubleshooting configurations 128
database connections
identifier properties 283
database hints
entering in Developer tool 65
databases
environment SQL 47
datatypes
DB2 for i5/OS 213
Bigint 203
binary 205
Date/Time 205
DB2 for z/OS 213
implicit conversion 228
Integer 203
JDBC 214
nonrelational 218
port-to-port data conversion 228
SAP HANA 224
Small Integer 203
string 209
XML 226
Date/Time datatypes
overview 205
DDL Generation
DDL Generation Errors 62
Generate and Execute DDL 59
decimal
high precision handling 203, 207
precision loss 208
Decimal
data type conversion 228
decimal data types
overview 207
default SQL query
viewing 74
delete 167
delimited identifiers
database connections 284
delimited properties
flat file data objects 88
deployment
mapping properties 135
overview 133
replacing applications 141
to a Data Integration Service 146, 147
to an application archive file 146, 147
updating applications 141
domain
search 36
searching for objects and properties 37
domains
adding 17
description 18
double
high precision handling 207
double data types
overview 207

E

editing
application objects 149
applications 149

environment SQL
configuring 47
guidelines for entering 48
error messages
grouping 131
limiting 131
Excel
configuring flat files 98
copying to a flat file 98
editing a flat file data object 98
export
dependent objects 194
objects 195
overview 193
to an application archive file 146, 147
XML file 195

F

filtering object dependencies
object dependencies 130
fixed-width properties
flat file data objects 89
flat file data objects
configuring metadata in Excel 98
copying metadata 98
advanced properties 86
column properties 85
creating as empty 99
creating from control file 102
creating from existing flat file 100
delimited properties 88
description 84
editing in Excel 98
fixed-width properties 89
format properties 87
general properties 85
run-time read properties 90
run-time write properties 92
folders
creating 23
description 22
format properties
flat file data objects 87

G

Google BigQuery connection
properties 243
Google BigQuery connections
connection modes 244
Google Cloud Storage connections
properties 245
Greenplum connections
properties 242

H

high precision
Bigint data type 203
Decimal data type 203
hints
Query view 65
how to
create, deploy, and update an application 143

I

IBM DB2 connections
properties 246
IBM DB2 for i5/OS connections
properties 249
IBM DB2 for z/OS connections
properties 252
identifiers
delimited 284
regular 284
identifying relationships
description 114
import
application archives 148
dependent objects 194
objects 197
overview 193
XML file 195
incremental application
guidelines 173
patch 151
rules 173
update 171
incremental deployment
Incremental Deployment wizard 155
patch deployment strategy 159
patch deployment strategy properties 170
select applications 156
source object selection 157
Informatica Developer
starting 13
integers
constants in expressions 204
using in calculations 203
writing to flat files 205
intermediate calculation
in decimal 208

J

JD Edwards EnterpriseOne connection
properties 259
JDBC connections
properties 255
JDBC V2 connection
properties 257
join syntax
customized data objects 69
Informatica syntax 69
left outer join syntax 71
normal join syntax 70
right outer join syntax 73

K

key relationships
creating between relational data objects 53
creating in customized data objects 59
customized data objects 55
relational data objects 52

L

LDAP connection
  properties 260
logical data object mappings
  creating 118
  read mappings 118
  types 117
  write mappings 118
logical data object models
  creating 105
  description 105
  example 104
  importing 106
logical data objects
  attribute relationships 114
  creating 114
  description 113
  example 104
  properties 113
logical view of data
  developing 104
  overview 103
logs
  description 130

M

mappings
  configuration properties 121
  configurations 121, 126
  creating configurations 127
  deployment properties 135
  troubleshooting configurations 128
Microsoft Azure Data Lake Storage Gen2 connection
  properties 263
Model repository
  adding 17
  checking objects out and in 34
  connecting 35
  non-versioned 31
  searching for objects and properties 37
  team-based development 31
  versioned 31
  versioned objects 31
Model Repository Service
  refreshing 35
monitoring
  description 132
MS SQL Server connections
  properties 265

N

NaN
  described 204
Netezza connections
  properties 269
non-identifying relationships
  description 114
nonrelational data objects
  description 75
nonrelational data operations
  creating read, write, and lookup transformations 76

O

object dependencies
  view object dependencies 129
Object Dependencies view
  viewing data 129
objects
  copying 24
  version history 34
ODBC connections
  properties 270
Oracle connections
  properties 271
Oracle E-Business Suite
  connection properties 289
outer join support
  customized data objects 69

P

performance tuning
  creating data viewer configurations 125
  creating mapping configurations 127
  data viewer configurations 124
  mapping configurations 126
  web service configurations 127
physical data objects
  customized data objects 54
  description 49
  flat file data objects 84
  nonrelational data objects 75
  relational data objects 50
  synchronization 80
PowerExchange for Db2 Warehouse connections
  properties 293
PowerExchange for Essbase connections
  properties 291
PowerExchange for Vertica connections
  relational properties 292
pre- and post-mapping SQL commands
  adding to relational data objects 74
  customized data objects 73
precision loss
  in low precision 208
primary keys
  creating in customized data objects 58
  creating in relational data objects 53
project permissions
  allowing parent object access 21
  assigning 22
  dependent object instances 21
  external object permissions 20
  grant permission 20
  read permission 20
  showing security details 21
  write permission 20
projects
  assigning permissions 22
  creating 19
  filtering 19
  permissions 20

Q

QNAN
  converting to 1.#QNAN 204

Query view
configuring hints 65

R

regular identifiers
database connections 284
relational connections
adding to customized data objects 57
relational data objects
adding to customized data objects 58
creating key relationships 53
creating keys 53
description 50
importing 52
key relationships 52
wild card search 52
reserved words file
creating 63
customized data objects 63
resource parameters
deploying in a mapping 139
run-time application
searching for objects and properties 37
run-time read properties
flat file data objects 90
run-time write properties
flat file data objects 92

S

Salesforce Marketing Cloud
connection properties 275
SAP connections
properties 276
search
domain 36
editor 38
Model repository 36
run-time application 36
select distinct
customized data objects 66
using in customized data objects 66
self-joins
custom SQL queries 62
session
additional JDBC URL parameters 286, 287
connection properties 286, 287
database/schema 286, 287
small integers
constants in expressions 204
using in calculations 203
writing to flat files 205
Snowflake connection
properties 279
sorted ports
customized data objects 67
using in customized data objects 67
source filters
entering 66
SQL
configuring environment SQL 47
guidelines for entering environment SQL 48
SQL hints
entering in Developer tool 65
string datatypes
overview 209
synchronization
customized data objects 80
physical data objects 80

T

team-based development
troubleshooting 35
Teradata Parallel Transporter connections
properties 280
transaction environment SQL
configuring 47, 48
transformation data types
list of 201
troubleshooting
versioning 35

U

update application
objects 171
user-defined joins
customized data objects 68
entering 68
Informatica syntax 69
left outer join syntax 71
normal join syntax 70
outer join support 69
right outer join syntax 73

V

validation
configuring preferences 131
grouping error messages 131
limiting error messages 131
versioning
troubleshooting 35
view object dependencies
filtering object dependencies 130
object dependencies 129
views
Connection Explorer view 41

W

web service
configuration properties 121
configurations 127
creating configurations 127
WSDL data objects
advanced view 77
creating 77
importing 76
overview view 77
schema view 76
synchronization 78